Distance matrices, Gram matrices and multidimensional scaling (MDS)

MATH 2740 – Mathematics of Data Science – Unit U07

Julien Arino
julien.arino@umanitoba.ca

Department of Mathematics @ University of Manitoba

Fall 2026

The University of Manitoba campuses are located on original lands of Anishinaabeg, Ininew, Anisininew, Dakota and Dene peoples, and on the National Homeland of the Red River Métis. We respect the Treaties that were made on these territories, we acknowledge the harms and mistakes of the past, and we dedicate ourselves to move forward in partnership with Indigenous communities in a spirit of Reconciliation and collaboration.

Outline

- Motivation: from distances back to coordinates
- Gram matrices and Euclidean distance matrices
- The centring matrix as an orthogonal projector
- Classical multidimensional scaling (MDS)
- Algebraic duality between PCA and classical MDS

Motivation: from distances back to coordinates

Gram matrices and Euclidean distance matrices

The centring matrix as an orthogonal projector

Classical multidimensional scaling (MDS)

Algebraic duality between PCA and classical MDS

From coordinates to distances and back

In principal component analysis (Unit U05), we start with a data matrix $X \in \mathcal{M}_{np}(\mathbb{R})$ containing the coordinates of n observations $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n \in \mathbb{R}^p$ and project them onto a lower-dimensional subspace $\mathbb{R}^k$ ($k \leq p$)

From the coordinates $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n \in \mathbb{R}^p$, it is straightforward to compute all pairwise Euclidean distances

$$d_{ij} = \|\boldsymbol{x}_i - \boldsymbol{x}_j\| = \sqrt{(\boldsymbol{x}_i - \boldsymbol{x}_j) \bullet (\boldsymbol{x}_i - \boldsymbol{x}_j)}, \qquad i, j = 1, \ldots, n$$

The inverse problem: what if we do *not* know the coordinates X, and only have a table of pairwise distances or dissimilarities $D = [d_{ij}] \in \mathcal{M}_n(\mathbb{R})$ between n objects?

Where do distance-only datasets arise?

In many scientific applications, objects do not come with explicit coordinates in $\mathbb{R}^p$, or only pairwise comparisons are measured:

- **Geography and transport**: road distances, flight distances or travel times between n cities
- **Genomics and evolutionary biology**: sequence edit distances or evolutionary divergence times between n species
- **Psychology and sensory science**: human judgements of dissimilarity between colours, sounds, faces or products
- **Cheminformatics and text analysis**: structural dissimilarity between molecules or cosine/Jaccard distances between documents

The goal of multidimensional scaling (MDS)

Given an $n \times n$ matrix $D = [d_{ij}]$ of pairwise distances between n objects:

1. **Exact Euclidean embedding**: do there exist points $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n \in \mathbb{R}^p$ (for some dimension p) such that

$$\|\boldsymbol{x}_i - \boldsymbol{x}_j\| = d_{ij} \qquad \forall i, j = 1, \ldots, n?$$

2. **Dimensionality reduction and visualisation**: for a chosen small dimension k (typically $k = 1, 2$ or 3), find coordinates $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n \in \mathbb{R}^k$ whose pairwise Euclidean distances $\|\boldsymbol{x}_i - \boldsymbol{x}_j\|$ best approximate d_{ij}

This family of methods is called **multidimensional scaling** (**MDS**). When D is treated as a Euclidean distance matrix and coordinates are recovered via spectral decomposition, it is called **classical MDS** (or **principal coordinate analysis**, **PCoA**)

Non-uniqueness: rigid motions preserve distances

Before trying to recover coordinates $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n \in \mathbb{R}^p$ from pairwise distances $d_{ij} = \|\boldsymbol{x}_i - \boldsymbol{x}_j\|$, observe that coordinates can *never* be unique!

Proposition 95 (Invariance of Euclidean distances under rigid motions)

Let $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n \in \mathbb{R}^p$, let $\boldsymbol{s} \in \mathbb{R}^p$ be a translation vector and let $Q \in \mathcal{M}_p(\mathbb{R})$ be an orthogonal matrix ($Q^T Q = \mathbb{I}_p$). Define transformed points

$$\boldsymbol{y}_i = Q\boldsymbol{x}_i + \boldsymbol{s}, \qquad i = 1, \ldots, n$$

Then all pairwise Euclidean distances are preserved:

$$\|\boldsymbol{y}_i - \boldsymbol{y}_j\| = \|\boldsymbol{x}_i - \boldsymbol{x}_j\| \qquad \forall i, j = 1, \ldots, n$$

Thus, distances only determine a point configuration up to **translation**, **rotation** and **reflection** (collectively called **rigid motions** or **Euclidean isometries**)

Proof of Proposition 95

Let $\boldsymbol{y}_i = Q\boldsymbol{x}_i + \boldsymbol{s}$ for $i = 1, \ldots, n$, where $\boldsymbol{s} \in \mathbb{R}^p$ and $Q^T Q = \mathbb{I}_p$

First, taking the difference between $\boldsymbol{y}_i$ and $\boldsymbol{y}_j$ eliminates the translation vector $\boldsymbol{s}$:

$$\boldsymbol{y}_i - \boldsymbol{y}_j = (Q\boldsymbol{x}_i + \boldsymbol{s}) - (Q\boldsymbol{x}_j + \boldsymbol{s}) = Q\boldsymbol{x}_i - Q\boldsymbol{x}_j = Q(\boldsymbol{x}_i - \boldsymbol{x}_j)$$

Next, compute the squared Euclidean norm using the orthogonality of Q:

$$\begin{aligned} \|\boldsymbol{y}_i - \boldsymbol{y}_j\|^2 &= \|Q(\boldsymbol{x}_i - \boldsymbol{x}_j)\|^2 \\ &= \big(Q(\boldsymbol{x}_i - \boldsymbol{x}_j)\big)^T \big(Q(\boldsymbol{x}_i - \boldsymbol{x}_j)\big) \\ &= (\boldsymbol{x}_i - \boldsymbol{x}_j)^T Q^T Q(\boldsymbol{x}_i - \boldsymbol{x}_j) \\ &= (\boldsymbol{x}_i - \boldsymbol{x}_j)^T \mathbb{I}_p (\boldsymbol{x}_i - \boldsymbol{x}_j) = \|\boldsymbol{x}_i - \boldsymbol{x}_j\|^2 \end{aligned}$$

Taking the non-negative square root yields $\|\boldsymbol{y}_i - \boldsymbol{y}_j\| = \|\boldsymbol{x}_i - \boldsymbol{x}_j\|$ □

Removing translation ambiguity by centring at the origin

Because shifting all points by a constant vector $\boldsymbol{s} \in \mathbb{R}^p$ leaves every pairwise distance unchanged, we must fix a reference location to obtain a well-defined reconstruction

Convention in classical MDS: we require the reconstructed points $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n \in \mathbb{R}^p$ to have their **centroid** (sample mean) at the origin $\mathbf{0} \in \mathbb{R}^p$:

$$\bar{\boldsymbol{x}} = \frac{1}{n} \sum_{i=1}^{n} \boldsymbol{x}_i = \mathbf{0} \iff \sum_{i=1}^{n} \boldsymbol{x}_i = \mathbf{0}$$

- Fixing $\sum_{i=1}^{n} \boldsymbol{x}_i = \mathbf{0}$ completely removes the translation ambiguity
- Any remaining ambiguity is purely an orthogonal transformation $Q \in \mathcal{M}_p(\mathbb{R})$ (rotation or reflection about the origin)
- Algebraically, why does fixing the origin help? Because once an origin is fixed, **distances determine inner products**, and inner products give us a **Gram matrix** that we can diagonalise!

Motivation: from distances back to coordinates

Gram matrices and Euclidean distance matrices

The centring matrix as an orthogonal projector

Classical multidimensional scaling (MDS)

Algebraic duality between PCA and classical MDS

Gram matrices and Euclidean distance matrices

The Gram matrix

The squared Euclidean distance matrix

Data matrix convention: observations in rows

Throughout this unit, we represent a configuration of n points in $\mathbb{R}^p$ by a data matrix $X \in \mathcal{M}_{np}(\mathbb{R})$ whose ith row is the transpose of the column vector $\boldsymbol{x}_i \in \mathbb{R}^p$:

$$X = \begin{pmatrix} \boldsymbol{x}_1^T \\ \boldsymbol{x}_2^T \\ \vdots \\ \boldsymbol{x}_n^T \end{pmatrix} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{pmatrix} \in \mathcal{M}_{np}(\mathbb{R})$$

- Rows $i = 1, \ldots, n$ are **observations** (objects, samples, cities or individuals)
- Columns $j = 1, \ldots, p$ are **features** (also called variables or coordinate axes)

The Gram matrix of a set of vectors

Definition 96 (Gram matrix)

Let $X \in \mathcal{M}_{np}(\mathbb{R})$ have rows $\boldsymbol{x}_1^T, \ldots, \boldsymbol{x}_n^T \in \mathbb{R}^p$. The ***Gram matrix*** *(or* ***inner product matrix****) of $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n$ is the $n \times n$ matrix*

$$G = XX^T \in \mathcal{M}_n(\mathbb{R})$$

whose (i, j) entry is the standard inner product between $\boldsymbol{x}_i$ and $\boldsymbol{x}_j$:

$$g_{ij} = \boldsymbol{x}_i \bullet \boldsymbol{x}_j = \boldsymbol{x}_i^T \boldsymbol{x}_j = \sum_{m=1}^{p} x_{im} x_{jm}, \qquad i, j = 1, \ldots, n$$

Explicitly, in block row-times-column form:

$$G = XX^T = \begin{pmatrix} \boldsymbol{x}_1^T \\ \vdots \\ \boldsymbol{x}_n^T \end{pmatrix} \begin{pmatrix} \boldsymbol{x}_1 & \cdots & \boldsymbol{x}_n \end{pmatrix} = \begin{pmatrix} \boldsymbol{x}_1^T\boldsymbol{x}_1 & \boldsymbol{x}_1^T\boldsymbol{x}_2 & \cdots & \boldsymbol{x}_1^T\boldsymbol{x}_n \\ \boldsymbol{x}_2^T\boldsymbol{x}_1 & \boldsymbol{x}_2^T\boldsymbol{x}_2 & \cdots & \boldsymbol{x}_2^T\boldsymbol{x}_n \\ \vdots & \vdots & \ddots & \vdots \\ \boldsymbol{x}_n^T\boldsymbol{x}_1 & \boldsymbol{x}_n^T\boldsymbol{x}_2 & \cdots & \boldsymbol{x}_n^T\boldsymbol{x}_n \end{pmatrix}$$

Geometric meaning of the Gram matrix

Every entry of the Gram matrix $G = XX^T$ encodes geometric information relative to the origin **0**:

- **Diagonal entries** give squared lengths (squared distances from the origin):

$$g_{ii} = \boldsymbol{x}_i^T \boldsymbol{x}_i = \|\boldsymbol{x}_i\|^2 \geq 0, \qquad i = 1, \ldots, n$$

- **Off-diagonal entries** encode lengths and angles θ_{ij} between vectors $\boldsymbol{x}_i$ and $\boldsymbol{x}_j$:

$$g_{ij} = \boldsymbol{x}_i^T \boldsymbol{x}_j = \|\boldsymbol{x}_i\| \, \|\boldsymbol{x}_j\| \cos\theta_{ij}, \qquad i \neq j$$

- **Orthogonal invariance**: if we rotate or reflect all points by an orthogonal matrix $Q \in \mathcal{M}_p(\mathbb{R})$ (so $\boldsymbol{x}_i \mapsto Q\boldsymbol{x}_i$ and $X \mapsto XQ^T$), the Gram matrix is **unchanged**:

$$(XQ^T)(XQ^T)^T = XQ^TQX^T = X\mathbb{I}_pX^T = XX^T = G$$

Positive semi-definite and positive definite matrices

Definition 97 (Positive semi-definite and positive definite matrices)

Let $A \in \mathcal{M}_n(\mathbb{R})$ be a symmetric matrix ($A^T = A$).

- *A is **positive semi-definite** (**PSD**) if*

$$\boldsymbol{v}^T A \boldsymbol{v} \geq 0 \qquad \forall \boldsymbol{v} \in \mathbb{R}^n$$

- *A is **positive definite** (**PD**) if*

$$\boldsymbol{v}^T A \boldsymbol{v} > 0 \qquad \forall \boldsymbol{v} \in \mathbb{R}^n \setminus \{\mathbf{0}\}$$

Recall from Theorem **??** (Unit U04) that all eigenvalues of a real symmetric matrix $A \in \mathcal{M}_n(\mathbb{R})$ are real. Equivalently:

- A is positive semi-definite $\iff$ all eigenvalues of A satisfy $\lambda_i \geq 0$
- A is positive definite $\iff$ all eigenvalues of A satisfy $\lambda_i > 0$ (in which case A is invertible)

You have already encountered these matrices!

Even without the name, we have already proved and used these properties for matrices of the form A^TA and AA^T (with $A \in \mathcal{M}_{np}(\mathbb{R})$):

- **Unit U02 (Theorem ??)**: we proved that for any $A \in \mathcal{M}_{np}(\mathbb{R})$, both $AA^T \in \mathcal{M}_n(\mathbb{R})$ and $A^TA \in \mathcal{M}_p(\mathbb{R})$ are **symmetric**

- **Unit U03 (Least squares, invertibility of A^TA)**: for any $\boldsymbol{x} \in \mathbb{R}^p$, we wrote

$$\boldsymbol{x}^T(A^TA)\boldsymbol{x} = (A\boldsymbol{x})^T(A\boldsymbol{x}) = \|A\boldsymbol{x}\|^2 \geq 0$$

  which says precisely that A^TA is **positive semi-definite**! When A has linearly independent columns ($A\boldsymbol{x} = \boldsymbol{0} \iff \boldsymbol{x} = \boldsymbol{0}$), $\|A\boldsymbol{x}\|^2 > 0$ for all $\boldsymbol{x} \neq \boldsymbol{0}$, so A^TA is **positive definite** (and hence invertible)

- **Unit U04 (SVD, Theorem ??)**: we proved that all eigenvalues of $A^T A$ and AA^T are **real and non-negative** ($\lambda_i \geq 0$), which allowed us to define singular values $\sigma_i = \sqrt{\lambda_i}$

- **Unit U05 (PCA)**: the sample covariance matrix $S_X = \frac{1}{n-1} \widetilde{X}^T \widetilde{X}$ is symmetric positive semi-definite, and its eigenvalues $\lambda_i \geq 0$ are the variances along the principal axes

Fundamental properties of Gram matrices

Notice that for a data matrix $X \in \mathcal{M}_{np}(\mathbb{R})$, its Gram matrix $G = XX^T \in \mathcal{M}_n(\mathbb{R})$ is simply the matrix AA^T from the SVD in Unit U04 (with $A = X$), whose eigenvectors are the left singular vectors of X

Theorem 98 (Properties of the Gram matrix)

Let $X \in \mathcal{M}_{np}(\mathbb{R})$ and let $G = XX^T \in \mathcal{M}_n(\mathbb{R})$ be its Gram matrix. Then:

1. *G is symmetric ($G^T = G$) and positive semi-definite ($\boldsymbol{v}^T G \boldsymbol{v} \geq 0$ for all $\boldsymbol{v} \in \mathbb{R}^n$), with real non-negative eigenvalues $\lambda_1 \geq \cdots \geq \lambda_n \geq 0$*
2. $\text{rank}(G) = \text{rank}(X) \leq \min(n, p)$
3. $\text{tr}(G) = \sum_{i=1}^{n} \|\boldsymbol{x}_i\|^2 = \|X\|_F^2$

Proof of Theorem 98

1. Symmetry and positive semi-definiteness: Setting $A = X$ in Theorem **??** (Unit U02), $G = XX^T$ is symmetric. By Theorem **??** (Unit U04), the eigenvalues of XX^T are real and non-negative, and as in the proof of Theorem **??** (applied to X^T), for any $\boldsymbol{v} \in \mathbb{R}^n$,

$$\boldsymbol{v}^T G \boldsymbol{v} = \boldsymbol{v}^T (XX^T)\boldsymbol{v} = (X^T \boldsymbol{v})^T (X^T \boldsymbol{v}) = \|X^T \boldsymbol{v}\|^2 \geq 0$$

2. Rank: By the same nullspace argument used for $A^T A$ in Unit U03 (applied to $A = X^T$), $XX^T \boldsymbol{v} = \boldsymbol{0} \iff \|X^T \boldsymbol{v}\|^2 = 0 \iff X^T \boldsymbol{v} = \boldsymbol{0}$, so $\text{null}(XX^T) = \text{null}(X^T)$. By the rank-nullity theorem (or because $\text{rank}(X)$ in Theorem **??** equals the number r of positive eigenvalues of XX^T), $\text{rank}(G) = \text{rank}(X^T) = \text{rank}(X) \leq \min(n, p)$

3. Trace: $\text{tr}(G) = \sum_{i=1}^n g_{ii} = \sum_{i=1}^n \boldsymbol{x}_i^T \boldsymbol{x}_i = \sum_{i=1}^n \|\boldsymbol{x}_i\|^2 = \sum_{i=1}^n \sum_{m=1}^p x_{im}^2 = \|X\|_F^2$

□

Characterisation of Gram matrices via spectral factorisation

In the SVD (Theorems **??** and **??**, Unit U04), $X = U\Sigma V^T$ gives

$$XX^T = U\Sigma\Sigma^T U^T = \sum_{i=1}^{r} \sigma_i^2 \boldsymbol{u}_i \boldsymbol{u}_i^T$$

where $\lambda_i = \sigma_i^2 > 0$ are the positive eigenvalues of XX^T

Conversely, *any* symmetric PSD matrix B can be factored as a Gram matrix $B = XX^T$ by reversing this step, as shown by Theorem 99 later

Before stating that, we need to define the square root of a matrix...

Square root of a matrix

A square matrix S is called a **square root** of A if $S^2 = SS = A$, and we write $S = A^{1/2}$

For a diagonal matrix with non-negative diagonal entries $\lambda_i \geq 0$, its square root is obtained by simply taking the square root of each diagonal entry:

$$\Lambda = \text{diag}(\lambda_1, \ldots, \lambda_n) \implies \Lambda^{1/2} = \text{diag}(\sqrt{\lambda_1}, \ldots, \sqrt{\lambda_n})$$

Example (a simple 2×2 diagonal matrix):

$$\Lambda = \begin{pmatrix} 4 & 0 \\ 0 & 9 \end{pmatrix} \implies \Lambda^{1/2} = \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix}$$

since

$$\begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix} \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix} = \begin{pmatrix} 2^2 & 0 \\ 0 & 3^2 \end{pmatrix} = \begin{pmatrix} 4 & 0 \\ 0 & 9 \end{pmatrix}$$

Because a symmetric PSD matrix has eigenvalues $\lambda_i \geq 0$, the square roots $\sqrt{\lambda_i}$ are always real numbers!

Theorem 99 (Spectral factorisation of symmetric PSD matrices)

Let $B \in \mathcal{M}_n(\mathbb{R})$ be symmetric and positive semi-definite with $\operatorname{rank}(B) = r \leq n$. *Let its ordered eigenvalues be*

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_r > 0 \quad \textit{and} \quad \lambda_{r+1} = \cdots = \lambda_n = 0$$

with corresponding orthonormal eigenvectors $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n \in \mathbb{R}^n$. Define

$$V_r = \begin{pmatrix} \boldsymbol{v}_1 & \cdots & \boldsymbol{v}_r \end{pmatrix} \in \mathcal{M}_{nr}(\mathbb{R}), \qquad \Lambda_r = \operatorname{diag}(\lambda_1, \ldots, \lambda_r) \in \mathcal{M}_r(\mathbb{R})$$

Then the matrix $X = V_r \Lambda_r^{1/2} \in \mathcal{M}_{nr}(\mathbb{R})$ satisfies $XX^T = B$

Proof of Theorem 99

Since $B \in \mathcal{M}_n(\mathbb{R})$ is real and symmetric, its eigenvalues are real (Theorem **??**, Unit U04) and it has an orthonormal basis of eigenvectors $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n \in \mathbb{R}^n$ (Theorem **??**, Unit U04), giving $B = V\Lambda V^T$ with $V = [\boldsymbol{v}_1 \ \cdots \ \boldsymbol{v}_n] \in \mathcal{M}_n(\mathbb{R})$ orthogonal and $\Lambda = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$

Just as in the outer product form of the SVD (Theorem **??**, Unit U04), only the $r = \operatorname{rank}(B)$ positive eigenvalues $\lambda_1 \geq \cdots \geq \lambda_r > 0$ contribute:

$$B = \sum_{i=1}^{r} \lambda_i \boldsymbol{v}_i \boldsymbol{v}_i^T = V_r \Lambda_r V_r^T$$

Exactly as we defined singular values $\sigma_i = \sqrt{\lambda_i}$ in Unit U04 (valid since B is PSD, so $\lambda_i > 0$ for $i = 1, \ldots, r$), define $\Lambda_r^{1/2} = \operatorname{diag}(\sqrt{\lambda_1}, \ldots, \sqrt{\lambda_r})$. Since $\Lambda_r^{1/2}$ is diagonal, $(\Lambda_r^{1/2})^T = \Lambda_r^{1/2}$ and $\Lambda_r^{1/2}(\Lambda_r^{1/2})^T = \Lambda_r$. Setting $X = V_r \Lambda_r^{1/2} \in \mathcal{M}_{nr}(\mathbb{R})$ gives

$$XX^T = \left(V_r \Lambda_r^{1/2}\right)\left(V_r \Lambda_r^{1/2}\right)^T = V_r \Lambda_r^{1/2} (\Lambda_r^{1/2})^T V_r^T = V_r \Lambda_r V_r^T = B \quad \square$$

Gram matrices and Euclidean distance matrices

The Gram matrix

The squared Euclidean distance matrix

Euclidean and squared Euclidean distance matrices

Definition 100 (Euclidean distance matrix and squared distance matrix)

Let $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n \in \mathbb{R}^p$ be the rows of $X \in \mathcal{M}_{np}(\mathbb{R})$

- *The **Euclidean distance matrix** is $D = [d_{ij}] \in \mathcal{M}_n(\mathbb{R})$, where*

$$d_{ij} = \|\boldsymbol{x}_i - \boldsymbol{x}_j\| = \sqrt{\sum_{m=1}^{p} (x_{im} - x_{jm})^2}, \qquad i, j = 1, \ldots, n$$

- *The **squared Euclidean distance matrix** is $D^{(2)} = [d_{ij}^2] \in \mathcal{M}_n(\mathbb{R})$, obtained by squaring each entry of D elementwise (**Hadamard square** $D^{(2)} = D \odot D$):*

$$\left[D^{(2)}\right]_{ij} = d_{ij}^2 = \|\boldsymbol{x}_i - \boldsymbol{x}_j\|^2, \qquad i, j = 1, \ldots, n$$

Warning: $D^{(2)}$ denotes the elementwise square $[d_{ij}^2]$, *not* the matrix product DD!

Basic properties of distance matrices

For any configuration of points $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n \in \mathbb{R}^p$, the matrices $D = [d_{ij}]$ and $D^{(2)} = [d_{ij}^2]$ satisfy:

1. **Zero diagonal** (**hollow matrix**):

$$d_{ii} = \|\boldsymbol{x}_i - \boldsymbol{x}_i\| = 0 \implies \operatorname{diag}(D) = \operatorname{diag}(D^{(2)}) = \mathbf{0} \quad \text{and} \quad \operatorname{tr}(D) = \operatorname{tr}(D^{(2)}) = 0$$

2. **Symmetry**:

$$d_{ij} = \|\boldsymbol{x}_i - \boldsymbol{x}_j\| = \|\boldsymbol{x}_j - \boldsymbol{x}_i\| = d_{ji} \implies D^T = D \quad \text{and} \quad (D^{(2)})^T = D^{(2)}$$

3. **Non-negativity**: $d_{ij} \geq 0$ and $d_{ij}^2 \geq 0$ for all $i, j = 1, \ldots, n$, with $d_{ij} = 0 \iff \boldsymbol{x}_i = \boldsymbol{x}_j$
4. **Triangle inequality** (for D, though *not* necessarily for $D^{(2)}$):

$$d_{ij} \leq d_{ik} + d_{kj} \qquad \forall i, j, k = 1, \ldots, n$$

Relating squared distances to inner products

Why do we work with the *squared* distance matrix $D^{(2)}$ rather than D? Because squared Euclidean norms expand directly in terms of inner products!

For any two points $\boldsymbol{x}_i, \boldsymbol{x}_j \in \mathbb{R}^p$, using the bilinearity and symmetry of the inner product:

$$\begin{aligned} d_{ij}^2 = \|\boldsymbol{x}_i - \boldsymbol{x}_j\|^2 &= (\boldsymbol{x}_i - \boldsymbol{x}_j) \bullet (\boldsymbol{x}_i - \boldsymbol{x}_j) \\ &= (\boldsymbol{x}_i - \boldsymbol{x}_j)^T (\boldsymbol{x}_i - \boldsymbol{x}_j) \\ &= \boldsymbol{x}_i^T \boldsymbol{x}_i - \boldsymbol{x}_i^T \boldsymbol{x}_j - \boldsymbol{x}_j^T \boldsymbol{x}_i + \boldsymbol{x}_j^T \boldsymbol{x}_j \\ &= \|\boldsymbol{x}_i\|^2 - 2\boldsymbol{x}_i^T \boldsymbol{x}_j + \|\boldsymbol{x}_j\|^2 \end{aligned}$$

In terms of the entries $g_{ij} = \boldsymbol{x}_i^T \boldsymbol{x}_j$ of the Gram matrix $G = XX^T$:

$$\boxed{d_{ij}^2 = g_{ii} - 2g_{ij} + g_{jj}} \qquad \forall i, j = 1, \ldots, n$$

Notice that this is the law of cosines: $c^2 = a^2 + b^2 - 2ab\cos\theta$

Matrix formula for $D^{(2)}$ in terms of G

We can write the entrywise identity $d_{ij}^2 = g_{ii} - 2g_{ij} + g_{jj}$ as a single matrix equation

Theorem 101 (Squared distance matrix from the Gram matrix)

Let $X \in \mathcal{M}_{np}(\mathbb{R})$ have Gram matrix $G = XX^T \in \mathcal{M}_n(\mathbb{R})$. Let

$$\boldsymbol{g} = \operatorname{diag}(G) = \begin{pmatrix} g_{11} \\ g_{22} \\ \vdots \\ g_{nn} \end{pmatrix} = \begin{pmatrix} \|\boldsymbol{x}_1\|^2 \\ \|\boldsymbol{x}_2\|^2 \\ \vdots \\ \|\boldsymbol{x}_n\|^2 \end{pmatrix} \in \mathbb{R}^n \quad \textit{and} \quad \mathbb{1} = \begin{pmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{pmatrix} \in \mathbb{R}^n$$

Then the squared Euclidean distance matrix $D^{(2)} = [d_{ij}^2] \in \mathcal{M}_n(\mathbb{R})$ is given by

$$D^{(2)} = \boldsymbol{g}\mathbb{1}^T - 2G + \mathbb{1}\boldsymbol{g}^T$$

Proof of Theorem 101

Examine the (i,j) entry of the rank-one outer product matrices $\boldsymbol{g}\mathbb{1}^T$ and $\mathbb{1}\boldsymbol{g}^T$:

$$\boldsymbol{g}\mathbb{1}^T = \begin{pmatrix} g_{11} \\ g_{22} \\ \vdots \\ g_{nn} \end{pmatrix} \begin{pmatrix} 1 & 1 & \cdots & 1 \end{pmatrix} = \begin{pmatrix} g_{11} & g_{11} & \cdots & g_{11} \\ g_{22} & g_{22} & \cdots & g_{22} \\ \vdots & \vdots & \ddots & \vdots \\ g_{nn} & g_{nn} & \cdots & g_{nn} \end{pmatrix} \implies \left[\boldsymbol{g}\mathbb{1}^T\right]_{ij} = g_{ii}$$

Similarly, taking the transpose:

$$\mathbb{1}\boldsymbol{g}^T = (\boldsymbol{g}\mathbb{1}^T)^T = \begin{pmatrix} g_{11} & g_{22} & \cdots & g_{nn} \\ g_{11} & g_{22} & \cdots & g_{nn} \\ \vdots & \vdots & \ddots & \vdots \\ g_{11} & g_{22} & \cdots & g_{nn} \end{pmatrix} \implies \left[\mathbb{1}\boldsymbol{g}^T\right]_{ij} = g_{jj}$$

Therefore, for every $(i,j) \in \{1,\ldots,n\}^2$,

$$\left[\boldsymbol{g}\mathbb{1}^T - 2G + \mathbb{1}\boldsymbol{g}^T\right]_{ij} = g_{ii} - 2g_{ij} + g_{jj} = \|\boldsymbol{x}_i\|^2 - 2\boldsymbol{x}_i^T\boldsymbol{x}_j + \|\boldsymbol{x}_j\|^2 = \|\boldsymbol{x}_i - \boldsymbol{x}_j\|^2 = d_{ij}^2 \quad \square$$

Example – Computing G and $D^{(2)}$ from X (1/2)

Consider $n = 3$ points in $\mathbb{R}^2$ given by the rows of

$$X = \begin{pmatrix} 1 & 2 \\ 3 & 1 \\ -1 & 0 \end{pmatrix} \in \mathcal{M}_{32}(\mathbb{R}), \qquad \boldsymbol{x}_1 = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \quad \boldsymbol{x}_2 = \begin{pmatrix} 3 \\ 1 \end{pmatrix}, \quad \boldsymbol{x}_3 = \begin{pmatrix} -1 \\ 0 \end{pmatrix}$$

First, compute the Gram matrix $G = XX^T$:

$$G = \begin{pmatrix} 1 & 2 \\ 3 & 1 \\ -1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 3 & -1 \\ 2 & 1 & 0 \end{pmatrix} = \begin{pmatrix} 5 & 5 & -1 \\ 5 & 10 & -3 \\ -1 & -3 & 1 \end{pmatrix}$$

The vector of diagonal entries of G (squared norms of the points) is

$$\boldsymbol{g} = \operatorname{diag}(G) = \begin{pmatrix} 5 \\ 10 \\ 1 \end{pmatrix}$$

Example – Computing G and $D^{(2)}$ from X (2/2)

Using Theorem 101, we form $\boldsymbol{g}\mathbb{1}^T$, $\mathbb{1}\boldsymbol{g}^T$ and $D^{(2)} = \boldsymbol{g}\mathbb{1}^T - 2G + \mathbb{1}\boldsymbol{g}^T$:

$$\boldsymbol{g}\mathbb{1}^T = \begin{pmatrix} 5 & 5 & 5 \\ 10 & 10 & 10 \\ 1 & 1 & 1 \end{pmatrix}, \qquad \mathbb{1}\boldsymbol{g}^T = \begin{pmatrix} 5 & 10 & 1 \\ 5 & 10 & 1 \\ 5 & 10 & 1 \end{pmatrix}$$

Hence:

$$D^{(2)} = \begin{pmatrix} 5 & 5 & 5 \\ 10 & 10 & 10 \\ 1 & 1 & 1 \end{pmatrix} - 2\begin{pmatrix} 5 & 5 & -1 \\ 5 & 10 & -3 \\ -1 & -3 & 1 \end{pmatrix} + \begin{pmatrix} 5 & 10 & 1 \\ 5 & 10 & 1 \\ 5 & 10 & 1 \end{pmatrix} = \begin{pmatrix} 0 & 5 & 8 \\ 5 & 0 & 17 \\ 8 & 17 & 0 \end{pmatrix}$$

Check directly:

- $d_{12}^2 = \|(1,2)^T - (3,1)^T\|^2 = (-2)^2 + 1^2 = 5$
- $d_{13}^2 = \|(1,2)^T - (-1,0)^T\|^2 = 2^2 + 2^2 = 8$
- $d_{23}^2 = \|(3,1)^T - (-1,0)^T\|^2 = 4^2 + 1^2 = 17$

Motivation: from distances back to coordinates

Gram matrices and Euclidean distance matrices

The centring matrix as an orthogonal projector

Classical multidimensional scaling (MDS)

Algebraic duality between PCA and classical MDS

The all-ones vector $\mathbb{1}$ and the all-ones matrix $\mathbb{J}_n$

Two simple building blocks in $\mathbb{R}^n$ and $\mathcal{M}_n(\mathbb{R})$:

- The **all-ones vector** $\mathbb{1} \in \mathbb{R}^n$:

$$\mathbb{1} = \begin{pmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{pmatrix} \in \mathbb{R}^n \implies \mathbb{1}^T\mathbb{1} = \sum_{i=1}^{n} 1^2 = n \quad \text{and} \quad \|\mathbb{1}\| = \sqrt{n}$$

- The **all-ones matrix** $\mathbb{J}_n = \mathbb{1}\mathbb{1}^T \in \mathcal{M}_n(\mathbb{R})$:

$$\mathbb{J}_n = \begin{pmatrix} 1 & 1 & \cdots & 1 \\ 1 & 1 & \cdots & 1 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & \cdots & 1 \end{pmatrix} \in \mathcal{M}_n(\mathbb{R}) \implies \mathbb{J}_n^2 = (\mathbb{1}\mathbb{1}^T)(\mathbb{1}\mathbb{1}^T) = \mathbb{1}(\mathbb{1}^T\mathbb{1})\mathbb{1}^T = n\mathbb{J}_n$$

Definition of the centring matrix

Definition 102 (Centring matrix)

*For $n \geq 1$, the **centring matrix** of order n is the $n \times n$ matrix*

$$C_n = \mathbb{I}_n - \frac{1}{n}\mathbb{1}\mathbb{1}^T = \mathbb{I}_n - \frac{1}{n}\mathbb{J}_n \in \mathcal{M}_n(\mathbb{R})$$

where $\mathbb{I}_n$ is the $n \times n$ identity matrix, $\mathbb{1} = (1, \ldots, 1)^T \in \mathbb{R}^n$ and $\mathbb{J}_n = \mathbb{1}\mathbb{1}^T$ is the $n \times n$ matrix of all ones

Entrywise, the (i, j) entry of C_n is

$$[C_n]_{ij} = \delta_{ij} - \frac{1}{n} = \begin{cases} 1 - \dfrac{1}{n} = \dfrac{n-1}{n} & \text{if } i = j \\ -\dfrac{1}{n} & \text{if } i \neq j \end{cases}$$

Explicit examples of C_n for small n

Writing out $C_n = \mathbb{I}_n - \frac{1}{n}\mathbb{J}_n$ explicitly for $n = 2, 3, 4$:

$$C_2 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} - \frac{1}{2}\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} = \frac{1}{2}\begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}$$

$$C_3 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} - \frac{1}{3}\begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix} = \frac{1}{3}\begin{pmatrix} 2 & -1 & -1 \\ -1 & 2 & -1 \\ -1 & -1 & 2 \end{pmatrix}$$

$$C_4 = \frac{1}{4}\begin{pmatrix} 3 & -1 & -1 & -1 \\ -1 & 3 & -1 & -1 \\ -1 & -1 & 3 & -1 \\ -1 & -1 & -1 & 3 \end{pmatrix}$$

Notice that in every C_n, each row and each column sums to zero!

Action of C_n on a vector: subtracting the mean

Why is C_n called the *centring matrix*? Let $\boldsymbol{v} = (v_1, \ldots, v_n)^T \in \mathbb{R}^n$ be a column vector of n observations of a feature

First note that the inner product $\mathbb{1}^T \boldsymbol{v}$ is the sum of the entries of $\boldsymbol{v}$:

$$\mathbb{1}^T \boldsymbol{v} = \sum_{i=1}^{n} v_i = n\bar{v}, \qquad \text{where} \quad \bar{v} = \frac{1}{n} \sum_{i=1}^{n} v_i$$

Multiplying $\boldsymbol{v}$ on the left by C_n gives:

$$C_n \boldsymbol{v} = \left(\mathbb{I}_n - \frac{1}{n} \mathbb{1}\mathbb{1}^T \right) \boldsymbol{v} = \boldsymbol{v} - \frac{1}{n} \mathbb{1}(\mathbb{1}^T \boldsymbol{v}) = \boldsymbol{v} - \bar{v}\mathbb{1} = \begin{pmatrix} v_1 - \bar{v} \\ v_2 - \bar{v} \\ \vdots \\ v_n - \bar{v} \end{pmatrix}$$

Thus, $C_n \boldsymbol{v}$ subtracts the arithmetic mean $\bar{v}$ from every component of $\boldsymbol{v}$, producing a **zero-mean** (centred) vector!

Action of C_n on a data matrix and link to covariance

Now let $X \in \mathcal{M}_{np}(\mathbb{R})$ be a data matrix with columns $\boldsymbol{c}_1, \ldots, \boldsymbol{c}_p \in \mathbb{R}^n$ (the p features) and rows $\boldsymbol{x}_1^T, \ldots, \boldsymbol{x}_n^T \in \mathbb{R}^p$ (the n observations)

- **Column-wise view**: left-multiplying X by C_n centres every column of X simultaneously:

$$\widetilde{X} = C_n X = \begin{pmatrix} C_n \boldsymbol{c}_1 & C_n \boldsymbol{c}_2 & \cdots & C_n \boldsymbol{c}_p \end{pmatrix}$$

- **Row-wise view**: if $\bar{\boldsymbol{x}} = \frac{1}{n}\sum_{i=1}^n \boldsymbol{x}_i = \frac{1}{n} X^T \mathbb{1} \in \mathbb{R}^p$ is the centroid of the n points, then

$$\widetilde{X} = C_n X = X - \frac{1}{n}\mathbb{1}\mathbb{1}^T X = X - \mathbb{1}\bar{\boldsymbol{x}}^T = \begin{pmatrix} (\boldsymbol{x}_1 - \bar{\boldsymbol{x}})^T \\ \vdots \\ (\boldsymbol{x}_n - \bar{\boldsymbol{x}})^T \end{pmatrix}$$

  shifts the centroid of the n points to the origin $\mathbf{0} \in \mathbb{R}^p$

- **Link to Unit U05**: the sample covariance matrix $S_X \in \mathcal{M}_p(\mathbb{R})$ of X can be written compactly as

$$S_X = \frac{1}{n-1}\widetilde{X}^T\widetilde{X} = \frac{1}{n-1}(C_n X)^T (C_n X) = \frac{1}{n-1} X^T C_n^T C_n X = \frac{1}{n-1} X^T C_n X$$

Algebraic and geometric properties of the centring matrix

Theorem 103 (Properties of the centring matrix C_n)

Let $C_n = \mathbb{I}_n - \frac{1}{n}\mathbb{1}\mathbb{1}^T \in \mathcal{M}_n(\mathbb{R})$. Then:

1. ***Symmetry**: $C_n^T = C_n$*
2. ***Idempotence**: $C_n^2 = C_n$ (and more generally $C_n^k = C_n$ for all integers $k \geq 1$)*
3. ***Null space**: $C_n\mathbb{1} = \mathbf{0}$ and $\mathbb{1}^T C_n = \mathbf{0}^T$; in fact, $\text{null}(C_n) = \text{span}(\mathbb{1})$*
4. ***Orthogonal projector**: C_n is the orthogonal projection matrix onto the $(n-1)$-dimensional subspace of zero-mean vectors*

$$W = \text{span}(\mathbb{1})^\perp = \left\{ \boldsymbol{v} \in \mathbb{R}^n : \mathbb{1}^T \boldsymbol{v} = \sum_{i=1}^{n} v_i = 0 \right\}$$

5. ***Spectrum, trace and rank**: the eigenvalues of C_n are $\lambda = 0$ (with multiplicity 1, eigenspace $\text{span}(\mathbb{1})$) and $\lambda = 1$ (with multiplicity $n-1$, eigenspace W). Consequently,*

$$\text{rank}(C_n) = \text{tr}(C_n) = n - 1$$

Proof of Theorem 103 – Parts 1 and 2

1. C_n **is symmetric (**$C_n^T = C_n$**)**: Using linearity of the transpose, $\mathbb{I}_n^T = \mathbb{I}_n$ and $(\mathbb{1}\mathbb{1}^T)^T = (\mathbb{1}^T)^T\mathbb{1}^T = \mathbb{1}\mathbb{1}^T$:

$$C_n^T = \left(\mathbb{I}_n - \frac{1}{n}\mathbb{1}\mathbb{1}^T\right)^T = \mathbb{I}_n^T - \frac{1}{n}(\mathbb{1}\mathbb{1}^T)^T = \mathbb{I}_n - \frac{1}{n}\mathbb{1}\mathbb{1}^T = C_n \quad \square$$

2. C_n **is idempotent (**$C_n^2 = C_n$**)**: Expanding the square and using the associativity of matrix multiplication together with $\mathbb{1}^T\mathbb{1} = n$:

$$\begin{aligned} C_n^2 &= \left(\mathbb{I}_n - \frac{1}{n}\mathbb{1}\mathbb{1}^T\right)\left(\mathbb{I}_n - \frac{1}{n}\mathbb{1}\mathbb{1}^T\right) \\ &= \mathbb{I}_n^2 - \frac{1}{n}\mathbb{1}\mathbb{1}^T - \frac{1}{n}\mathbb{1}\mathbb{1}^T + \frac{1}{n^2}(\mathbb{1}\mathbb{1}^T)(\mathbb{1}\mathbb{1}^T) \\ &= \mathbb{I}_n - \frac{2}{n}\mathbb{1}\mathbb{1}^T + \frac{1}{n^2}\mathbb{1}(\underbrace{\mathbb{1}^T\mathbb{1}}_{=n})\mathbb{1}^T \\ &= \mathbb{I}_n - \frac{2}{n}\mathbb{1}\mathbb{1}^T + \frac{1}{n}\mathbb{1}\mathbb{1}^T = \mathbb{I}_n - \frac{1}{n}\mathbb{1}\mathbb{1}^T = C_n \quad \square \end{aligned}$$

Proof of Theorem 103 – Part 3

3. $C_n\mathbb{1} = \mathbf{0}$, $\mathbb{1}^T C_n = \mathbf{0}^T$ **and** $\text{null}(C_n) = \text{span}(\mathbb{1})$:
First, multiply C_n on the right by $\mathbb{1}$:

$$C_n\mathbb{1} = \left(\mathbb{I}_n - \frac{1}{n}\mathbb{1}\mathbb{1}^T\right)\mathbb{1} = \mathbb{1} - \frac{1}{n}\mathbb{1}(\mathbb{1}^T\mathbb{1}) = \mathbb{1} - \frac{1}{n}\mathbb{1}(n) = \mathbb{1} - \mathbb{1} = \mathbf{0}$$

Taking transposes and using $C_n^T = C_n$ immediately gives

$$\mathbb{1}^T C_n = \mathbb{1}^T C_n^T = (C_n\mathbb{1})^T = \mathbf{0}^T$$

To show $\text{null}(C_n) = \text{span}(\mathbb{1})$, let $\boldsymbol{v} \in \mathbb{R}^n$. Then

$$\boldsymbol{v} \in \text{null}(C_n) \iff C_n\boldsymbol{v} = \mathbf{0} \iff \boldsymbol{v} - \left(\frac{1}{n}\mathbb{1}^T\boldsymbol{v}\right)\mathbb{1} = \mathbf{0} \iff \boldsymbol{v} = \bar{v}\mathbb{1} \iff \boldsymbol{v} \in \text{span}(\mathbb{1})$$

Thus $\text{null}(C_n) = \text{span}(\mathbb{1})$, which is a 1-dimensional subspace of $\mathbb{R}^n$
(nullity$(C_n) = 1$) □

Proof of Theorem 103 – Part 4

4. C_n is the orthogonal projection matrix onto $W = \text{span}(\mathbb{1})^\perp$:
Let $L = \text{span}(\mathbb{1})$. Since $\{\mathbb{1}\}$ is an orthogonal basis of the 1-dimensional subspace L with $\|\mathbb{1}\|^2 = \mathbb{1}^T\mathbb{1} = n$, the orthogonal projection of any vector $\boldsymbol{v} \in \mathbb{R}^n$ onto L is

$$\text{proj}_L(\boldsymbol{v}) = \frac{\mathbb{1} \bullet \boldsymbol{v}}{\|\mathbb{1}\|^2}\mathbb{1} = \frac{\mathbb{1}^T\boldsymbol{v}}{n}\mathbb{1} = \left(\frac{1}{n}\mathbb{1}\mathbb{1}^T\right)\boldsymbol{v}$$

By definition of the orthogonal complement $W = L^\perp = \{\boldsymbol{v} \in \mathbb{R}^n : \mathbb{1}^T\boldsymbol{v} = 0\}$, the orthogonal projection of $\boldsymbol{v}$ onto W is the component of $\boldsymbol{v}$ orthogonal to L:

$$\text{proj}_W(\boldsymbol{v}) = \text{perp}_L(\boldsymbol{v}) = \boldsymbol{v} - \text{proj}_L(\boldsymbol{v}) = \boldsymbol{v} - \left(\frac{1}{n}\mathbb{1}\mathbb{1}^T\right)\boldsymbol{v} = \left(\mathbb{I}_n - \frac{1}{n}\mathbb{1}\mathbb{1}^T\right)\boldsymbol{v} = C_n\boldsymbol{v}$$

Note also that for any $\boldsymbol{v} \in \mathbb{R}^n$, the centred vector $C_n\boldsymbol{v}$ and the mean vector $\bar{v}\mathbb{1} = (\mathbb{I}_n - C_n)\boldsymbol{v}$ are orthogonal:

$$(C_n\boldsymbol{v}) \bullet (\bar{v}\mathbb{1}) = \bar{v}\boldsymbol{v}^T C_n^T\mathbb{1} = \bar{v}\boldsymbol{v}^T(C_n\mathbb{1}) = 0 \quad \square$$

Proof of Theorem 103 – Part 5

5. Eigenvalues, trace and rank of C_n:

Let $(\lambda, \boldsymbol{v})$ be an eigenpair of C_n ($\boldsymbol{v} \neq \mathbf{0}$ and $C_n\boldsymbol{v} = \lambda\boldsymbol{v}$). Applying C_n to both sides and using idempotence $C_n^2 = C_n$:

$$C_n\boldsymbol{v} = C_n^2\boldsymbol{v} = C_n(C_n\boldsymbol{v}) = C_n(\lambda\boldsymbol{v}) = \lambda(C_n\boldsymbol{v}) = \lambda^2\boldsymbol{v}$$

Since $\boldsymbol{v} \neq \mathbf{0}$, $(\lambda^2 - \lambda)\boldsymbol{v} = \mathbf{0} \implies \lambda(\lambda - 1) = 0$, so every eigenvalue of C_n is either $\lambda = 0$ or $\lambda = 1$:

- For $\lambda = 0$: the eigenspace is $\text{null}(C_n) = \text{span}(\mathbb{1})$, which has dimension 1 (multiplicity 1)
- For $\lambda = 1$: $\boldsymbol{v} \in W = \text{span}(\mathbb{1})^\perp \iff \mathbb{1}^T\boldsymbol{v} = 0 \implies C_n\boldsymbol{v} = \boldsymbol{v} - \frac{1}{n}\mathbb{1}(\mathbb{1}^T\boldsymbol{v}) = \boldsymbol{v}$, so the eigenspace for $\lambda = 1$ is W, which has dimension $n - 1$ (multiplicity $n - 1$)

Finally, the trace equals the sum of the diagonal entries (and the sum of the eigenvalues):

$$\text{tr}(C_n) = \sum_{i=1}^{n}\left(1 - \frac{1}{n}\right) = n\left(\frac{n-1}{n}\right) = n - 1 = \text{rank}(C_n) \quad \square$$

Motivation: from distances back to coordinates

Gram matrices and Euclidean distance matrices

The centring matrix as an orthogonal projector

Classical multidimensional scaling (MDS)

Algebraic duality between PCA and classical MDS

Classical multidimensional scaling (MDS)

- **Double centring: recovering the Gram matrix from distances**
- Coordinate recovery via spectral decomposition
- Worked exam-style examples

From $D^{(2)}$ back to G: why double centring works

Recall Theorem 101: given the Gram matrix $G = XX^T$, the squared distance matrix is

$$D^{(2)} = \boldsymbol{g}\mathbb{1}^T - 2G + \mathbb{1}\boldsymbol{g}^T$$

We now want to **invert** this relationship: given only $D^{(2)}$, how can we isolate G?

Look closely at the two nuisance terms $\boldsymbol{g}\mathbb{1}^T$ and $\mathbb{1}\boldsymbol{g}^T$:

- $\boldsymbol{g}\mathbb{1}^T$ has $\mathbb{1}^T$ on the **right**, and we know that $\mathbb{1}^T C_n = \mathbf{0}^T$!
- $\mathbb{1}\boldsymbol{g}^T$ has $\mathbb{1}$ on the **left**, and we know that $C_n\mathbb{1} = \mathbf{0}$!

Therefore, if we multiply $D^{(2)}$ on the **left** by C_n and on the **right** by C_n, both rank-one nuisance terms $\boldsymbol{g}\mathbb{1}^T$ and $\mathbb{1}\boldsymbol{g}^T$ are completely annihilated!

This operation $D^{(2)} \mapsto -\frac{1}{2} C_n D^{(2)} C_n$ is called **double centring**

The double centring theorem

Theorem 104 (Double centring formula)

Let $X \in \mathcal{M}_{np}(\mathbb{R})$ be a data matrix with squared Euclidean distance matrix $D^{(2)} = [d_{ij}^2] \in \mathcal{M}_n(\mathbb{R})$, and let $\widetilde{X} = C_n X$ be the column-centred data matrix with centred Gram matrix $B = \widetilde{X}\widetilde{X}^T = C_n X X^T C_n$. Then

$$B = -\frac{1}{2} C_n D^{(2)} C_n = \widetilde{X}\widetilde{X}^T$$

In particular, if the points in X are already centred at the origin ($C_n X = X$, so $\sum_{i=1}^n \boldsymbol{x}_i = \mathbf{0}$), then $B = -\frac{1}{2} C_n D^{(2)} C_n = G = XX^T$ recovers the exact Gram matrix of X

Proof of Theorem 104

By Theorem 101, $D^{(2)} = \boldsymbol{g}\mathbb{1}^T - 2G + \mathbb{1}\boldsymbol{g}^T$, where $G = XX^T$ and $\boldsymbol{g} = \text{diag}(G)$
Multiply $D^{(2)}$ on the left by C_n and on the right by C_n:

$$C_n D^{(2)} C_n = C_n(\boldsymbol{g}\mathbb{1}^T)C_n - 2C_nGC_n + C_n(\mathbb{1}\boldsymbol{g}^T)C_n$$

Using associativity and Part 3 of Theorem 103 ($C_n\mathbb{1} = \mathbf{0}$ and $\mathbb{1}^T C_n = \mathbf{0}^T$):

$$C_n(\boldsymbol{g}\mathbb{1}^T)C_n = (C_n\boldsymbol{g})(\underbrace{\mathbb{1}^T C_n}_{=\mathbf{0}^T}) = (C_n\boldsymbol{g})\mathbf{0}^T = \mathbf{0}_{n\times n}$$

$$C_n(\mathbb{1}\boldsymbol{g}^T)C_n = (\underbrace{C_n\mathbb{1}}_{=\mathbf{0}})(\boldsymbol{g}^T C_n) = \mathbf{0}(\boldsymbol{g}^T C_n) = \mathbf{0}_{n\times n}$$

Thus both outer-product terms vanish, leaving

$$C_n D^{(2)} C_n = -2C_nGC_n = -2C_n(XX^T)C_n^T = -2(C_nX)(C_nX)^T = -2\widetilde{X}\widetilde{X}^T$$

Dividing both sides by -2 gives $-\frac{1}{2}C_nD^{(2)}C_n = C_nGC_n = \widetilde{X}\widetilde{X}^T$. If $C_nX = X$, then $\widetilde{X} = X$ and $-\frac{1}{2}C_nD^{(2)}C_n = XX^T = G$ □

Scalar form of double centring

Why is $B = -\frac{1}{2} C_n D^{(2)} C_n$ called *double* centring?

- Left-multiplying $D^{(2)}$ by C_n subtracts the **column means** of $D^{(2)}$ from each column
- Right-multiplying by $C_n = C_n^T$ subtracts the **row means** from each row

Let $\bar{d}^2_{i\cdot} = \frac{1}{n}\sum_{j=1}^n d^2_{ij}$ (mean of row i), $\bar{d}^2_{\cdot j} = \frac{1}{n}\sum_{i=1}^n d^2_{ij}$ (mean of column j) and $\bar{d}^2_{\cdot\cdot} = \frac{1}{n^2}\sum_{i=1}^n\sum_{j=1}^n d^2_{ij}$ (grand mean of $D^{(2)}$). Expanding $B = -\frac{1}{2}\left(\mathbb{I}_n - \frac{1}{n}\mathbb{J}_n\right) D^{(2)} \left(\mathbb{I}_n - \frac{1}{n}\mathbb{J}_n\right)$ gives the entrywise formula:

$$b_{ij} = -\frac{1}{2}\left(d^2_{ij} - \bar{d}^2_{i\cdot} - \bar{d}^2_{\cdot j} + \bar{d}^2_{\cdot\cdot}\right), \qquad i, j = 1, \ldots, n$$

Consequently, every row sum and every column sum of B is zero: $B\mathbb{1} = \mathbf{0}$ and $\mathbb{1}^T B = \mathbf{0}^T$, so $\lambda = 0$ is always an eigenvalue of B with eigenvector $\mathbb{1}$!

Classical multidimensional scaling (MDS)

- Double centring: recovering the Gram matrix from distances
- **Coordinate recovery via spectral decomposition**
- Worked exam-style examples

The classical MDS algorithm

Definition 105 (Classical MDS algorithm)

Given an $n \times n$ distance or dissimilarity matrix $D = [d_{ij}] \in \mathcal{M}_n(\mathbb{R})$ and a target embedding dimension $k \in \{1, \dots, n-1\}$:

1. ***Square the distances**: form $D^{(2)} = [d_{ij}^2] \in \mathcal{M}_n(\mathbb{R})$*
2. ***Double-centre**: compute the inner product matrix $B = -\frac{1}{2} C_n D^{(2)} C_n$*
3. ***Spectral decomposition**: compute the eigenvalues $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n$ and orthonormal eigenvectors $\boldsymbol{v}_1, \boldsymbol{v}_2, \dots, \boldsymbol{v}_n \in \mathbb{R}^n$ of B*
4. ***Construct k-dimensional coordinates**: assuming $\lambda_1 \geq \cdots \geq \lambda_k > 0$, let $V_k = [\boldsymbol{v}_1 \ \cdots \ \boldsymbol{v}_k] \in \mathcal{M}_{nk}(\mathbb{R})$ and $\Lambda_k = \operatorname{diag}(\lambda_1, \dots, \lambda_k) \in \mathcal{M}_k(\mathbb{R})$. The k-dimensional MDS coordinate matrix is*

$$X_k = V_k \Lambda_k^{1/2} = \begin{pmatrix} \sqrt{\lambda_1}\, \boldsymbol{v}_1 & \sqrt{\lambda_2}\, \boldsymbol{v}_2 & \cdots & \sqrt{\lambda_k}\, \boldsymbol{v}_k \end{pmatrix} \in \mathcal{M}_{nk}(\mathbb{R})$$

Schoenberg's theorem: when is a distance matrix Euclidean?

What if the input matrix $D = [d_{ij}]$ did *not* come from Euclidean points in $\mathbb{R}^p$, or came from geodesic/road distances on a curved surface?

Theorem 106 (Schoenberg, 1935; Young and Householder, 1938)

Let $D = [d_{ij}] \in \mathcal{M}_n(\mathbb{R})$ be a symmetric hollow matrix ($d_{ii} = 0$, $d_{ij} = d_{ji} \geq 0$) and let $B = -\frac{1}{2} C_n D^{(2)} C_n$. Then:

1. *There exist points $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n \in \mathbb{R}^r$ whose pairwise Euclidean distances equal d_{ij} if and only if B is **positive semi-definite** with $\text{rank}(B) \leq r$*
2. *The smallest dimension r in which the n points can be isometrically embedded in $\mathbb{R}^r$ is $r = \text{rank}(B)$ (the number of strictly positive eigenvalues of B), and $r \leq n - 1$*

Dimensionality reduction, strain and goodness of fit

When $B = -\frac{1}{2} C_n D^{(2)} C_n$ has rank $r > k$ (or small negative eigenvalues due to non-Euclidean noise) and we keep only the top k positive eigenpairs:

- By the **Eckart-Young-Mirsky theorem** (Unit U04), $B_k = X_k X_k^T = \sum_{i=1}^{k} \lambda_i \boldsymbol{v}_i \boldsymbol{v}_i^T$ is the best rank-k positive semi-definite approximation to B in the Frobenius norm, minimising the **classical MDS strain**:

$$\text{Strain}(X_k) = \|B - X_k X_k^T\|_F^2 = \sum_{i=k+1}^{n} \lambda_i^2$$

- The **goodness of fit** (proportion of variance or dispersion explained) by the first k MDS dimensions:
  - When B is positive semi-definite ($\lambda_i \geq 0$ for all i):

$$\text{GOF}_1(k) = \frac{\sum_{i=1}^{k} \lambda_i}{\sum_{i=1}^{n} \lambda_i} = \frac{\sum_{i=1}^{k} \lambda_i}{\text{tr}(B)}$$

  - When B has some negative eigenvalues (two standard conventions in `cmdscale`):

$$\text{GOF}_1(k) = \frac{\sum_{i=1}^{k} \lambda_i}{\sum_{i=1}^{n} |\lambda_i|} \quad \text{or} \quad \text{GOF}_2(k) = \frac{\sum_{i=1}^{k} \lambda_i}{\sum_{i=1}^{n} \max(\lambda_i, 0)}$$

Classical multidimensional scaling (MDS)

- Double centring: recovering the Gram matrix from distances
- Coordinate recovery via spectral decomposition
- **Worked exam-style examples**

Example 1 – Recovering 1D coordinates from $D^{(2)}$ (1/4)

Suppose $n = 3$ objects have the squared Euclidean distance matrix

$$D^{(2)} = \begin{pmatrix} 0 & 4 & 16 \\ 4 & 0 & 4 \\ 16 & 4 & 0 \end{pmatrix}$$

(i.e. $d_{12} = 2$, $d_{23} = 2$ and $d_{13} = 4$)

Problem:

1. Compute the double-centred Gram matrix $B = -\frac{1}{2} C_3 D^{(2)} C_3$
2. Find the eigenvalues and orthonormal eigenvectors of B
3. Determine the minimal embedding dimension $r = \text{rank}(B)$ and recover the centred coordinates X_r

Example 1 – Computing $B = -\frac{1}{2} C_3 D^{(2)} C_3$ (2/4)

Recall $C_3 = \frac{1}{3}\begin{pmatrix} 2 & -1 & -1 \\ -1 & 2 & -1 \\ -1 & -1 & 2 \end{pmatrix}$. First compute $C_3 D^{(2)}$:

$$C_3 D^{(2)} = \frac{1}{3}\begin{pmatrix} 2 & -1 & -1 \\ -1 & 2 & -1 \\ -1 & -1 & 2 \end{pmatrix}\begin{pmatrix} 0 & 4 & 16 \\ 4 & 0 & 4 \\ 16 & 4 & 0 \end{pmatrix} = \frac{1}{3}\begin{pmatrix} -20 & 4 & 28 \\ -16 & -8 & -16 \\ 36 & 4 & -12 \end{pmatrix} = \frac{4}{3}\begin{pmatrix} -5 & 1 & 7 \\ -4 & -2 & -4 \\ 9 & 1 & -3 \end{pmatrix}$$

Now right-multiply by C_3 and multiply by $-\frac{1}{2}$:

$$C_3 D^{(2)} C_3 = \frac{4}{9}\begin{pmatrix} -5 & 1 & 7 \\ -4 & -2 & -4 \\ 9 & 1 & -3 \end{pmatrix}\begin{pmatrix} 2 & -1 & -1 \\ -1 & 2 & -1 \\ -1 & -1 & 2 \end{pmatrix} = \frac{4}{9}\begin{pmatrix} -18 & 0 & 18 \\ 0 & 0 & 0 \\ 18 & 0 & -18 \end{pmatrix} = \begin{pmatrix} -8 & 0 & 8 \\ 0 & 0 & 0 \\ 8 & 0 & -8 \end{pmatrix}$$

$$\implies B = -\frac{1}{2} C_3 D^{(2)} C_3 = \begin{pmatrix} 4 & 0 & -4 \\ 0 & 0 & 0 \\ -4 & 0 & 4 \end{pmatrix}$$

Example 1 – Eigenpairs and coordinate recovery (3/4)

We have

$$B = \begin{pmatrix} 4 & 0 & -4 \\ 0 & 0 & 0 \\ -4 & 0 & 4 \end{pmatrix}$$

- Notice that row 1 and row 3 are opposites and row 2 is zero, so $\text{rank}(B) = 1$
- Since $\text{tr}(B) = 4 + 0 + 4 = 8$ and $\text{rank}(B) = 1$, the eigenvalues of B are immediately

$$\lambda_1 = 8, \qquad \lambda_2 = 0, \qquad \lambda_3 = 0$$

- For $\lambda_1 = 8$, $(B - 8\mathbb{I}_3)\boldsymbol{v}_1 = \mathbf{0} \implies v_{11} + v_{13} = 0$ and $v_{12} = 0$. Normalising gives the unit eigenvector

$$\boldsymbol{v}_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} -1 \\ 0 \\ 1 \end{pmatrix} \qquad (\text{or } -\boldsymbol{v}_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 0 \\ -1 \end{pmatrix})$$

Example 1 – Verification of recovered 1D coordinates (4/4)

Since B is positive semi-definite with $r = \text{rank}(B) = 1$, the 3 points lie on a 1-dimensional line ($\mathbb{R}^1$)!

Recovering the 1-dimensional coordinate vector $X_1 = \sqrt{\lambda_1}\, \boldsymbol{v}_1 \in \mathcal{M}_{31}(\mathbb{R})$:

$$X_1 = \sqrt{8}\left(\frac{1}{\sqrt{2}}\begin{pmatrix}-1\\0\\1\end{pmatrix}\right) = \frac{2\sqrt{2}}{\sqrt{2}}\begin{pmatrix}-1\\0\\1\end{pmatrix} = \begin{pmatrix}-2\\0\\2\end{pmatrix}$$

Check:

- Centred at the origin: $x_1 + x_2 + x_3 = -2 + 0 + 2 = 0$
- Pairwise squared distances:

$$(x_1 - x_2)^2 = (-2-0)^2 = 4, \quad (x_2 - x_3)^2 = (0-2)^2 = 4, \quad (x_1 - x_3)^2 = (-2-2)^2 = 16$$

which reproduces $D^{(2)}$ exactly! (Choosing $-\boldsymbol{v}_1$ gives $X_1 = (2, 0, -2)^T$, the reflection across the origin)

Example 2 – Recovering a 2D triangle from distances (1/4)

Now consider $n = 3$ points with squared Euclidean distance matrix

$$D^{(2)} = \begin{pmatrix} 0 & 6 & 6 \\ 6 & 0 & 12 \\ 6 & 12 & 0 \end{pmatrix}$$

(an isosceles right triangle with leg length $\sqrt{6}$ and hypotenuse $\sqrt{12}$, since $6 + 6 = 12$)

Let us compute $B = -\frac{1}{2} C_3 D^{(2)} C_3$ using the row/column mean formula $b_{ij} = -\frac{1}{2}(d_{ij}^2 - \bar{d}_{i\cdot}^2 - \bar{d}_{\cdot j}^2 + \bar{d}_{\cdot\cdot}^2)$:

- Row (and column) means of $D^{(2)}$:

$$\bar{d}_{1\cdot}^2 = \frac{0 + 6 + 6}{3} = 4, \qquad \bar{d}_{2\cdot}^2 = \frac{6 + 0 + 12}{3} = 6, \qquad \bar{d}_{3\cdot}^2 = \frac{6 + 12 + 0}{3} = 6$$

- Grand mean of $D^{(2)}$:

$$\bar{d}_{\cdot\cdot}^2 = \frac{4 + 6 + 6}{3} = \frac{16}{3}$$

Example 2 – Forming B and finding its eigenpairs (2/4)

Computing each entry $b_{ij} = -\frac{1}{2}(d_{ij}^2 - \bar{d}_{i\cdot}^2 - \bar{d}_{\cdot j}^2 + \frac{16}{3})$ (or multiplying $-\frac{1}{2}C_3 D^{(2)} C_3$ directly):

$$B = -\frac{1}{2}C_3 D^{(2)} C_3 = \frac{1}{3}\begin{pmatrix} 4 & -2 & -2 \\ -2 & 10 & -8 \\ -2 & -8 & 10 \end{pmatrix} = \begin{pmatrix} 4/3 & -2/3 & -2/3 \\ -2/3 & 10/3 & -8/3 \\ -2/3 & -8/3 & 10/3 \end{pmatrix}$$

Let us find the eigenvalues and orthonormal eigenvectors of B:

- We always know $\boldsymbol{v}_3 = \frac{1}{\sqrt{3}}(1,1,1)^T$ is an eigenvector with eigenvalue $\lambda_3 = 0$
- By symmetry between points 2 and 3, try $\boldsymbol{v}_1 = \frac{1}{\sqrt{2}}(0,1,-1)^T$:

$$B\boldsymbol{v}_1 = \frac{1}{3\sqrt{2}}\begin{pmatrix} 4 & -2 & -2 \\ -2 & 10 & -8 \\ -2 & -8 & 10 \end{pmatrix}\begin{pmatrix} 0 \\ 1 \\ -1 \end{pmatrix} = \frac{1}{3\sqrt{2}}\begin{pmatrix} 0 \\ 18 \\ -18 \end{pmatrix} = 6\boldsymbol{v}_1 \implies \lambda_1 = 6$$

Example 2 – Second eigenpair and 2D coordinates (3/4)

- Since $\text{tr}(B) = \frac{4+10+10}{3} = 8 = \lambda_1 + \lambda_2 + \lambda_3$ and $\lambda_1 = 6, \lambda_3 = 0$, the remaining eigenvalue is
$$\lambda_2 = 8 - 6 - 0 = 2$$
- The unit eigenvector $\boldsymbol{v}_2$ must be orthogonal to both $\boldsymbol{v}_3 = \frac{1}{\sqrt{3}}(1,1,1)^T$ and $\boldsymbol{v}_1 = \frac{1}{\sqrt{2}}(0,1,-1)^T$, which gives
$$\boldsymbol{v}_2 = \frac{1}{\sqrt{6}} \begin{pmatrix} 2 \\ -1 \\ -1 \end{pmatrix} \qquad \left(\text{check: } B\boldsymbol{v}_2 = \frac{1}{3\sqrt{6}} \begin{pmatrix} 12 \\ -6 \\ -6 \end{pmatrix} = 2\boldsymbol{v}_2 \right)$$

Since $\lambda_1 = 6 > 0$, $\lambda_2 = 2 > 0$ and $\lambda_3 = 0$, $\text{rank}(B) = 2$, so the 3 points embed isometrically in $\mathbb{R}^2$!

Example 2 – Constructing X_2 and verifying distances (4/4)

Using $X_2 = V_2 \Lambda_2^{1/2} = \begin{pmatrix} \sqrt{\lambda_1}\,\boldsymbol{v}_1 & \sqrt{\lambda_2}\,\boldsymbol{v}_2 \end{pmatrix}$ with $\sqrt{\lambda_1} = \sqrt{6}$ and $\sqrt{\lambda_2} = \sqrt{2}$:

$$X_2 = \begin{pmatrix} \sqrt{6} \cdot 0 & \sqrt{2} \cdot \frac{2}{\sqrt{6}} \\ \sqrt{6} \cdot \frac{1}{\sqrt{2}} & \sqrt{2} \cdot \frac{-1}{\sqrt{6}} \\ \sqrt{6} \cdot \frac{-1}{\sqrt{2}} & \sqrt{2} \cdot \frac{-1}{\sqrt{6}} \end{pmatrix} = \begin{pmatrix} 0 & \frac{2}{\sqrt{3}} \\ \sqrt{3} & -\frac{1}{\sqrt{3}} \\ -\sqrt{3} & -\frac{1}{\sqrt{3}} \end{pmatrix} \in \mathcal{M}_{32}(\mathbb{R})$$

Verification:

- Column sums are zero: $0 + \sqrt{3} - \sqrt{3} = 0$ and $\frac{2}{\sqrt{3}} - \frac{1}{\sqrt{3}} - \frac{1}{\sqrt{3}} = 0$
- Pairwise squared distances between rows $\boldsymbol{x}_1^T, \boldsymbol{x}_2^T, \boldsymbol{x}_3^T$ of X_2:

$$\|\boldsymbol{x}_1 - \boldsymbol{x}_2\|^2 = (0 - \sqrt{3})^2 + \left(\frac{2}{\sqrt{3}} + \frac{1}{\sqrt{3}}\right)^2 = 3 + 3 = 6 = d_{12}^2$$

$$\|\boldsymbol{x}_1 - \boldsymbol{x}_3\|^2 = (0 + \sqrt{3})^2 + \left(\frac{2}{\sqrt{3}} + \frac{1}{\sqrt{3}}\right)^2 = 3 + 3 = 6 = d_{13}^2$$

$$\|\boldsymbol{x}_2 - \boldsymbol{x}_3\|^2 = (\sqrt{3} + \sqrt{3})^2 + 0^2 = (2\sqrt{3})^2 = 12 = d_{23}^2$$

Motivation: from distances back to coordinates

Gram matrices and Euclidean distance matrices

The centring matrix as an orthogonal projector

Classical multidimensional scaling (MDS)

Algebraic duality between PCA and classical MDS

Algebraic duality between PCA and classical MDS

Equivalence via the singular value decomposition

Application in R: reconstructing maps from distances

Two $n \times p$ products: $\widetilde{X}^T\widetilde{X}$ versus $\widetilde{X}\widetilde{X}^T$

Suppose we have a data matrix $X \in \mathcal{M}_{np}(\mathbb{R})$ (n observations and p features) and its column-centred data matrix $\widetilde{X} = C_n X \in \mathcal{M}_{np}(\mathbb{R})$. From $\widetilde{X}$, we can form two symmetric positive semi-definite matrices:

Feature scatter / covariance (PCA, Unit U05)

$$\widetilde{X}^T\widetilde{X} \in \mathcal{M}_p(\mathbb{R}) \quad \left(\text{or } S_X = \frac{1}{n-1}\widetilde{X}^T\widetilde{X}\right)$$

- Size: $p \times p$ (features $\times$ features)
- (j, ℓ) entry: inner product between centred column j and centred column ℓ
- Diagonalising $\widetilde{X}^T\widetilde{X}$ gives the principal directions (loadings) in $\mathbb{R}^p$

Centred Gram matrix (Classical MDS, Unit U07)

$$B = \widetilde{X}\widetilde{X}^T = -\frac{1}{2}C_n D^{(2)} C_n \in \mathcal{M}_n(\mathbb{R})$$

- Size: $n \times n$ (observations $\times$ observations)
- (i, j) entry: inner product $\tilde{\boldsymbol{x}}_i^T \tilde{\boldsymbol{x}}_j$ between centred observation i and centred observation j
- Diagonalising $\widetilde{X}\widetilde{X}^T$ gives the principal coordinates in $\mathbb{R}^n$

Connecting PCA and classical MDS via the SVD

Let $\widetilde{X} = C_n X \in \mathcal{M}_{np}(\mathbb{R})$ be the column-centred data matrix of rank $r \leq \min(n, p)$, and write its thin singular value decomposition (Unit U04):

$$\widetilde{X} = U_r \Sigma_r V_r^T$$

where:

- $U_r = [\boldsymbol{u}_1 \ \cdots \ \boldsymbol{u}_r] \in \mathcal{M}_{nr}(\mathbb{R})$ has orthonormal columns ($U_r^T U_r = \mathbb{I}_r$)
- $\Sigma_r = \text{diag}(\sigma_1, \ldots, \sigma_r) \in \mathcal{M}_r(\mathbb{R})$ contains the positive singular values $\sigma_1 \geq \cdots \geq \sigma_r > 0$
- $V_r = [\boldsymbol{v}_1 \ \cdots \ \boldsymbol{v}_r] \in \mathcal{M}_{pr}(\mathbb{R})$ has orthonormal columns ($V_r^T V_r = \mathbb{I}_r$)

Now express both $\widetilde{X}^T \widetilde{X}$ and $B = \widetilde{X} \widetilde{X}^T$ in terms of the SVD of $\widetilde{X}$:

$$\widetilde{X}^T \widetilde{X} = V_r \Sigma_r^2 V_r^T \in \mathcal{M}_p(\mathbb{R}) \qquad \text{and} \qquad B = \widetilde{X} \widetilde{X}^T = U_r \Sigma_r^2 U_r^T \in \mathcal{M}_n(\mathbb{R})$$

Duality theorem: PCA scores equal classical MDS coordinates

Theorem 107 (Equivalence of PCA and Euclidean classical MDS)

Let $X \in \mathcal{M}_{np}(\mathbb{R})$ be a data matrix with column-centred data matrix $\widetilde{X} = C_n X$ having thin SVD $\widetilde{X} = U_r \Sigma_r V_r^T$, and let $D = [\|\boldsymbol{x}_i - \boldsymbol{x}_j\|] \in \mathcal{M}_n(\mathbb{R})$ be the Euclidean distance matrix between the rows of X. Then for any $k \leq r$:

1. *The non-zero eigenvalues of $\widetilde{X}^T \widetilde{X} \in \mathcal{M}_p(\mathbb{R})$ and $B = -\frac{1}{2} C_n D^{(2)} C_n = \widetilde{X} \widetilde{X}^T \in \mathcal{M}_n(\mathbb{R})$ are identical:*

$$\lambda_i = \sigma_i^2 > 0, \qquad i = 1, \ldots, r$$

2. *The k-dimensional principal component score matrix $Y_k = \widetilde{X} V_k \in \mathcal{M}_{nk}(\mathbb{R})$ from PCA and the k-dimensional coordinate matrix $X_k = U_k \Lambda_k^{1/2} \in \mathcal{M}_{nk}(\mathbb{R})$ from classical MDS on D are **identical** (up to sign choices of the eigenvectors):*

$$Y_k = \widetilde{X} V_k = U_k \Sigma_k = U_k \Lambda_k^{1/2} = X_k$$

Proof of Theorem 107

Let $\widetilde{X} = U_r \Sigma_r V_r^T$, where $U_r = [\boldsymbol{u}_1 \ \cdots \ \boldsymbol{u}_r]$, $V_r = [\boldsymbol{v}_1 \ \cdots \ \boldsymbol{v}_r]$ and $\Sigma_r = \text{diag}(\sigma_1, \ldots, \sigma_r)$

1. PCA projection onto top k right singular vectors: The columns of $V_k = [\boldsymbol{v}_1 \ \cdots \ \boldsymbol{v}_k] \in \mathcal{M}_{pk}(\mathbb{R})$ are the top k orthonormal eigenvectors of $\widetilde{X}^T \widetilde{X}$ (and of $S_X = \frac{1}{n-1} \widetilde{X}^T \widetilde{X}$). Using $V_r^T V_k = \begin{pmatrix} \mathbb{I}_k \\ \mathbf{0}_{(r-k)\times k} \end{pmatrix}$, the PCA score matrix is

$$Y_k = \widetilde{X} V_k = (U_r \Sigma_r V_r^T) V_k = U_k \Sigma_k$$

2. Classical MDS on Euclidean distances of X: By Theorem 104, double centring $D^{(2)}$ yields $B = -\frac{1}{2} C_n D^{(2)} C_n = \widetilde{X} \widetilde{X}^T = U_r \Sigma_r^2 U_r^T$. Thus the top k eigenvalues of B are $\lambda_i = \sigma_i^2$ ($\Lambda_k^{1/2} = \Sigma_k$) with orthonormal eigenvectors $U_k = [\boldsymbol{u}_1 \ \cdots \ \boldsymbol{u}_k] \in \mathcal{M}_{nk}(\mathbb{R})$. The classical MDS coordinate matrix is therefore

$$X_k = U_k \Lambda_k^{1/2} = U_k \Sigma_k = Y_k \quad \square$$

Computational and practical payoff of PCA–MDS duality

Why is Theorem 107 so useful in practice?

1. **Tall-and-skinny data ($n \gg p$, many observations, few features)**:
   - $\widetilde{X}^T\widetilde{X}$ is a small $p \times p$ matrix, whereas $\widetilde{X}\widetilde{X}^T$ is a huge $n \times n$ matrix
   - Computing PCA via $\widetilde{X}^T\widetilde{X}$ costs $O(np^2 + p^3)$, which is much faster than $O(n^3)$ for $\widetilde{X}\widetilde{X}^T$
2. **Short-and-wide data ($p \gg n$, few observations, huge number of features)**:
   - E.g. gene expression ($n = 50$ patients, $p = 20{,}000$ genes) or high-resolution images ($n = 100$ faces, $p = 10^6$ pixels)
   - $\widetilde{X}^T\widetilde{X}$ is $20{,}000 \times 20{,}000$, whereas the Gram matrix $B = \widetilde{X}\widetilde{X}^T$ is only 50×50!
   - Diagonalising the $n \times n$ Gram matrix $B = \widetilde{X}\widetilde{X}^T$ (classical MDS) gives the exact PCA scores in $O(n^2p + n^3)$ time!
3. **Coordinate-free and kernel extensions**:
   - Classical MDS works when X is unknown and only distances D are given
   - Replacing $\boldsymbol{x}_i^T\boldsymbol{x}_j$ by a general similarity kernel $k(\boldsymbol{x}_i, \boldsymbol{x}_j)$ leads directly to **kernel PCA**!

Algebraic duality between PCA and classical MDS

Equivalence via the singular value decomposition

Application in R: reconstructing maps from distances

Verifying PCA and classical MDS equivalence in R

Let us first verify Theorem 107 numerically in R on a synthetic 5×3 centred data matrix $\widetilde{X}$:

```
set.seed(2740)
X_tilde = scale(matrix(rnorm(15), nrow = 5, ncol = 3), center = TRUE, scale
pca_scores = prcomp(X_tilde, center = FALSE)$x[, 1:2]
mds_coords = cmdscale(dist(X_tilde), k = 2)
# Compare absolute values (eigenvectors are defined up to a sign +/- 1)
max(abs(abs(pca_scores) - abs(mds_coords)))

## [1] 2.442491e-15
```

Up to machine precision ($\approx 10^{-16}$) and column signs, the PCA scores `prcomp(X_tilde)$x[,1:2]` and the classical MDS coordinates `cmdscale(dist(X_tilde), k=2)` are identical!

Geographical example: Canadian cities distance matrix

Now suppose we are only given approximate great-circle distances (in kilometres) between $n = 8$ Canadian cities, with **no latitude or longitude coordinates**:

```
cities = c("Victoria", "Vancouver", "Calgary", "Regina",
           "Winnipeg", "Toronto", "Montreal", "Halifax")
D_can = matrix(c(
     0,   90,  725, 1215, 1690, 3390, 3760, 4490,
    90,    0,  675, 1165, 1640, 3355, 3685, 4430,
   725,  675,    0,  505,  990, 2705, 3020, 3770,
  1215, 1165,  505,    0,  500, 2030, 2360, 3125,
  1690, 1640,  990,  500,    0, 1515, 1825, 2590,
  3390, 3355, 2705, 2030, 1515,    0,  505, 1265,
  3760, 3685, 3020, 2360, 1825,  505,    0,  790,
  4490, 4430, 3770, 3125, 2590, 1265,  790,    0
), nrow = 8, byrow = TRUE, dimnames = list(cities, cities))
```

Step-by-step manual classical MDS vs `cmdscale`

Let us implement the classical MDS algorithm directly from $B = -\frac{1}{2} C_n D^{(2)} C_n$ and compare with R's built-in `cmdscale` function:

```
n = nrow(D_can)
C_n = diag(n) - matrix(1, n, n) / n
B = -0.5 * C_n %*% (D_can^2) %*% C_n
ev = eigen(B, symmetric = TRUE)
round(ev$values, 1)
```

```
## [1] 20985217.2   224941.6    67722.5    17544.1     3459.2        0.0    -
## [8]  -482729.4
```

```
X_manual = ev$vectors[, 1:2] %*% diag(sqrt(ev$values[1:2]))
rownames(X_manual) = cities
mds_builtin = cmdscale(as.dist(D_can), k = 2, eig = TRUE)
max(abs(abs(X_manual) - abs(mds_builtin$points)))
```

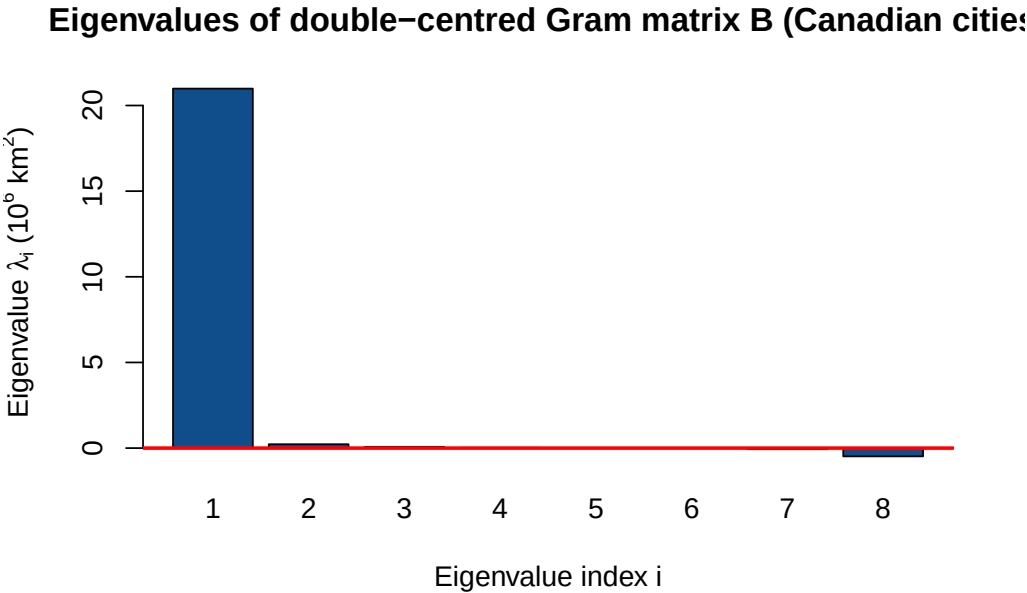
```
## [1] 0
```

Eigenvalue spectrum and goodness of fit

Notice how the first two eigenvalues dominate the spectrum of B:

```
gof_top2 = sum(ev$values[1:2]) / sum(ev$values[ev$values > 0])
gof_1d   = ev$values[1] / sum(ev$values[ev$values > 0])
c(GOF_1D = round(gof_1d, 4), GOF_2D = round(gof_top2, 4))

## GOF_1D GOF_2D
## 0.9853 0.9958
```

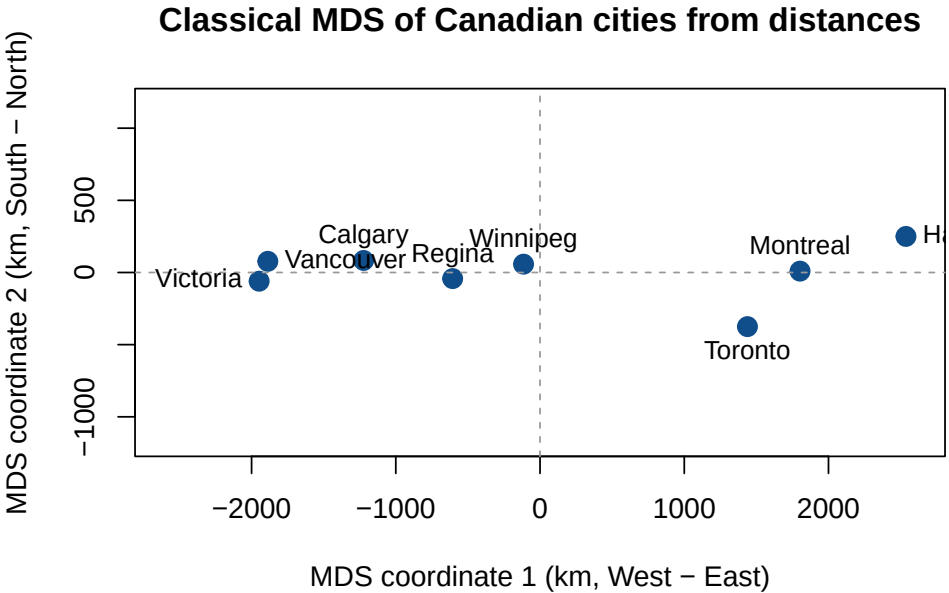
The first dimension alone (East–West axis across Canada) explains 98.5% of the positive trace, and 2 dimensions explain 99.58%!


Eigenvalues of double−centred Gram matrix B (Canadian cities
Eigenvalue λ_i (10^6 km^2)
0
5
10
15
20
1
2
3
4
5
6
7
8
Eigenvalue index i

Plotting the reconstructed 2D map of Canadian cities

Recall that classical MDS only determines coordinates up to orthogonal transformations (rotations and reflections). We plot the raw MDS coordinates and then flip signs if needed so West is left and North is up:

```
coords = X_manual
# Ensure West < 0 and Winnipeg is North of Toronto
if (coords["Vancouver", 1] > 0) coords[, 1] = -coords[, 1]
if (coords["Winnipeg", 2] < coords["Toronto", 2]) coords[, 2] = -coords[, 2]
plot(coords[, 1], coords[, 2], pch = 19, cex = 1.5, col = "dodgerblue4",
     asp = 1, xlim = c(-2600, 2600), ylim = c(-900, 900),
     xlab = "MDS coordinate 1 (km, West - East)",
     ylab = "MDS coordinate 2 (km, South - North)",
     main = "Classical MDS of Canadian cities from distances")
abline(h = 0, v = 0, lty = 2, col = "grey60")
```

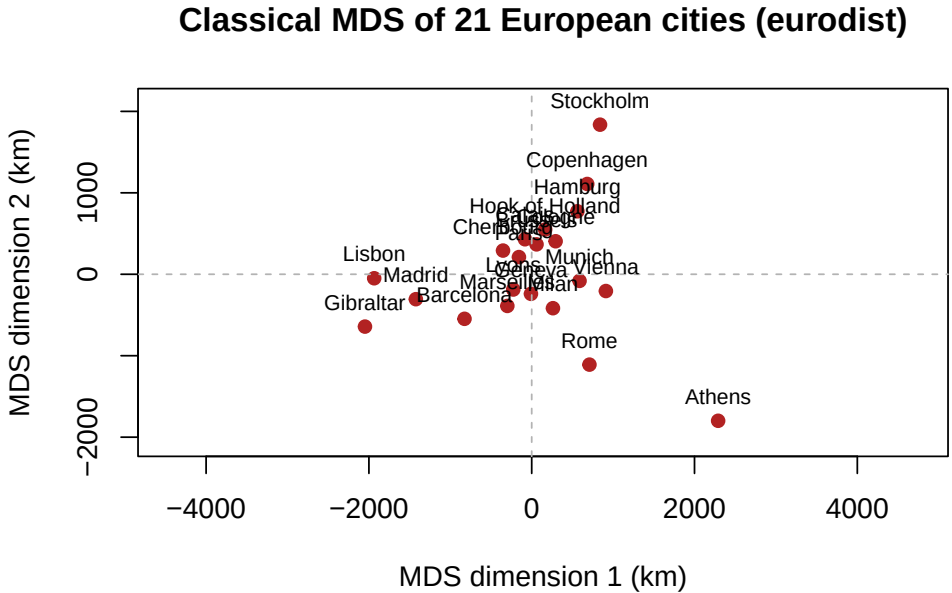

Classical MDS of Canadian cities from distances
MDS coordinate 2 (km, South − North)
500
0
−1000
Calgary
Winnipeg
Vancouver
Regina
Victoria
Montreal
Toronto
−2000
−1000
0
1000
2000
MDS coordinate 1 (km, West − East)

Classical example: European road distances (`eurodist`)

R's built-in `datasets` package includes `eurodist`, a table of road distances (in km) between 21 European cities:

```
D_euro = as.matrix(eurodist)
mds_euro = cmdscale(eurodist, k = 2, eig = TRUE)
x_eu = mds_euro$points[, 1]
y_eu = -mds_euro$points[, 2] # Flip North-South axis
plot(x_eu, y_eu, pch = 19, col = "firebrick", asp = 1,
     xlim = range(x_eu) * 1.15, ylim = range(y_eu) * 1.15,
     xlab = "MDS dimension 1 (km)", ylab = "MDS dimension 2 (km)",
     main = "Classical MDS of 21 European cities (eurodist)")
abline(h = 0, v = 0, lty = 2, col = "grey70")
```

Even though road distances include detours around mountains and seas (so B has small negative eigenvalues), the top 2 eigenvectors of $-\frac{1}{2}C_{21}D^{(2)}C_{21}$ faithfully reconstruct the map of Europe!


Classical MDS of 21 European cities (eurodist)
Stockholm
Copenhagen
Hamburg
Hook of Holland
Lisbon
Madrid
Gibraltar
Barcelona
Munich
Vienna
Rome
Athens
MDS dimension 2 (km)
1000
0
−2000
−4000
−2000
0
2000
4000
MDS dimension 1 (km)

Summary of Unit U07

- **Gram matrix**: for $X \in \mathcal{M}_{np}(\mathbb{R})$, $G = XX^T \in \mathcal{M}_n(\mathbb{R})$ has entries $g_{ij} = \boldsymbol{x}_i^T \boldsymbol{x}_j$, is symmetric positive semi-definite and satisfies $\operatorname{rank}(G) = \operatorname{rank}(X)$
- **From Gram to squared distances**: with $\boldsymbol{g} = \operatorname{diag}(G)$,

$$D^{(2)} = \boldsymbol{g}\mathbb{1}^T - 2G + \mathbb{1}\boldsymbol{g}^T$$

- **Centring matrix**: $C_n = \mathbb{I}_n - \frac{1}{n}\mathbb{1}\mathbb{1}^T$ is the symmetric, idempotent orthogonal projection matrix onto $\operatorname{span}(\mathbb{1})^\perp$ with $\operatorname{rank}(C_n) = \operatorname{tr}(C_n) = n - 1$ and $C_n\mathbb{1} = \mathbf{0}$
- **Double centring and classical MDS**: $B = -\frac{1}{2}C_n D^{(2)} C_n = \widetilde{X}\widetilde{X}^T$ recovers the centred Gram matrix from distances alone, and $X_k = V_k \Lambda_k^{1/2}$ recovers k-dimensional coordinates
- **PCA–MDS duality**: via the SVD $\widetilde{X} = U\Sigma V^T$, classical MDS on Euclidean distances of X produces the exact same low-dimensional coordinates $U_k\Sigma_k$ as PCA on X

Non-negative matrices, digraphs and regular Markov chains

MATH 2740 – Mathematics of Data Science – Unit U08

Julien Arino
julien.arino@umanitoba.ca

Department of Mathematics @ University of Manitoba

Fall 2026

The University of Manitoba campuses are located on original lands of Anishinaabeg, Ininew, Anisininew, Dakota and Dene peoples, and on the National Homeland of the Red River Métis. We respect the Treaties that were made on these territories, we acknowledge the harms and mistakes of the past, and we dedicate ourselves to move forward in partnership with Indigenous communities in a spirit of Reconciliation and collaboration.

Outline

- Non-negative matrices and their digraphs
- Matrix powers, walks and connectivity
- The Perron-Frobenius theorem
- Regular discrete-time Markov chains

Non-negative matrices and their digraphs

Matrix powers, walks and connectivity

The Perron-Frobenius theorem

Regular discrete-time Markov chains

Why study non-negative matrices and digraphs together?

Many matrices in data science have entries that represent quantities that cannot be negative:

- counts, weights, distances or similarity scores
- transition probabilities between states in a stochastic process
- hyperlinks between web pages or flows in a network

Such matrices have a rich geometric and spectral theory (Perron-Frobenius theory)

Their zero/non-zero sparsity pattern is naturally encoded by a **directed graph** (**digraph**), turning algebraic questions about matrix powers into visual questions about walks on a graph

Non-negative and positive vectors and matrices

Definition 107 (Non-negative and positive matrices)

Let $A = [a_{ij}] \in \mathcal{M}_{np}(\mathbb{R})$ be a real matrix

- *A is **non-negative**, written $A \geq 0$ (or $A \in \mathcal{M}_{np}(\mathbb{R}_{\geq 0})$), if*

$$a_{ij} \geq 0 \quad \text{for all } i = 1, \ldots, n \text{ and } j = 1, \ldots, p$$

- *A is **positive** (or **strictly positive**), written $A > 0$, if*

$$a_{ij} > 0 \quad \text{for all } i = 1, \ldots, n \text{ and } j = 1, \ldots, p$$

The same notation applies to vectors $\boldsymbol{x} = (x_1, \ldots, x_n)^T \in \mathbb{R}^n$:

- $\boldsymbol{x} \geq \mathbf{0}$ means $x_i \geq 0$ for all $i = 1, \ldots, n$
- $\boldsymbol{x} > \mathbf{0}$ means $x_i > 0$ for all $i = 1, \ldots, n$

Componentwise inequalities and basic properties

For two matrices $A, B \in \mathcal{M}_{np}(\mathbb{R})$, we write $A \geq B$ if $A - B \geq 0$, and $A > B$ if $A - B > 0$

Proposition 108 (Arithmetic of non-negative matrices)

Let $A, B \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ and $\boldsymbol{x} \in \mathbb{R}^n_{\geq 0}$

1. *$A + B \geq 0$ and $AB \geq 0$*
2. *For any integer $k \geq 1$, $A^k \geq 0$*
3. *If $A > 0$ and $\boldsymbol{x} \geq \mathbf{0}$ with $\boldsymbol{x} \neq \mathbf{0}$, then $A\boldsymbol{x} > \mathbf{0}$*
4. *If $A \geq 0$ and $\boldsymbol{x} > \mathbf{0}$, then $A\boldsymbol{x} > \mathbf{0}$ if and only if A has no zero row*

Why does property 3 hold? Because $\boldsymbol{x} \neq \mathbf{0}$ has at least one entry $x_{j_0} > 0$, so for every row i,

$$(A\boldsymbol{x})_i = \sum_{j=1}^{n} a_{ij} x_j \geq a_{i,j_0} x_{j_0} > 0$$

Directed graphs (digraphs)

Definition 109 (Directed graph)

*A **directed graph** (or **digraph**) is a pair $G = (V, A)$ where:*

- $V = \{1, 2, \ldots, n\}$ *is a finite set of elements called **vertices** (or **nodes**)*
- $A \subseteq V \times V$ *is a set of ordered pairs of vertices called **arcs** (or **directed edges**)*

Definition 110 (Arc, tail, head and loop)

An **arc** $e = (u, v) \in A$ represents a directed connection from vertex u to vertex v

- u is the **initial endpoint** (or **tail**) and v is the **terminal endpoint** (or **head**)
- u is a **predecessor** of v and v is a **successor** of u
- An arc of the form $(u, u) \in A$ from a vertex to itself is called a **loop** (or **self-loop**)

Indegree, outdegree, sources and sinks

Let $G = (V, A)$ be a digraph and $v \in V$

Definition 111 (Outdegree and indegree)

- The **outdegree** $d_G^+(v)$ is the number of arcs directed away from v:

$$d_G^+(v) = |\{w \in V : (v, w) \in A\}|$$

- The **indegree** $d_G^-(v)$ is the number of arcs directed towards v:

$$d_G^-(v) = |\{u \in V : (u, v) \in A\}|$$

- A vertex with $d_G^-(v) = 0$ is a **source**; a vertex with $d_G^+(v) = 0$ is a **sink**

Since every arc has one tail and one head,

$$\sum_{v \in V} d_G^+(v) = \sum_{v \in V} d_G^-(v) = |A|$$

The digraph associated with a non-negative matrix

Definition 112 (Digraph of a matrix and adjacency matrix)

- *Given a square non-negative matrix $M = [m_{ij}] \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$, its associated* ***digraph*** *is $\mathcal{G}(M) = (V, A)$ with vertex set $V = \{1, \ldots, n\}$ and arc set*

$$A = \{(i, j) \in V \times V : m_{ij} > 0\}$$

  If we attach weight m_{ij} to each arc $(i, j) \in A$, we obtain a ***weighted digraph*** *(or* ***directed network****)*
- *Conversely, given a digraph $G = (V, A)$ on $V = \{1, \ldots, n\}$, its (binary)* ***adjacency matrix*** *is $A(G) = [a_{ij}] \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ defined by*

$$a_{ij} = \begin{cases} 1 & \text{if } (i, j) \in A \\ 0 & \text{otherwise} \end{cases}$$

Note that diagonal entries $m_{ii} > 0$ correspond to self-loops (i, i) at vertex i

Orientation convention: general matrices vs Markov chains

There are two natural conventions for drawing arcs from a matrix $M = [m_{ij}]$:

1. **Row-to-column (adjacency) convention**: arc (i, j) goes from i to j when $m_{ij} > 0$
   - Row i lists outgoing arcs from i; column j lists incoming arcs into j

2. **Column-to-row (transition) convention**: in column-stochastic Markov chains, $p_{ij} = \mathbb{P}(S_i(t+1) \mid S_j(t))$ is the probability of moving **from** j **to** i, so we draw an arc from j to i when $p_{ij} > 0$
   - Column j lists outgoing transitions from state j (summing to 1); row i lists incoming transitions into state i
   - This transition digraph is simply $\mathcal{G}(P^T)$, obtained by reversing all arrows of $\mathcal{G}(P)$

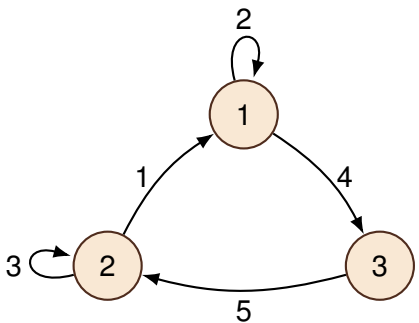
Crucial fact: Reversing all arrows preserves closed walks, cycle lengths and strong connectedness, so irreducibility and primitivity of M and M^T are identical

Example: 3×3 non-negative matrix and its digraph $\mathcal{G}(M)$

Consider the 3×3 non-negative matrix M and its binary sparsity pattern $A(\mathcal{G}(M))$:

$$M = \begin{pmatrix} 2 & 0 & 4 \\ 1 & 3 & 0 \\ 0 & 5 & 0 \end{pmatrix}, \qquad A(\mathcal{G}(M)) = \begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 0 \end{pmatrix}$$

Weighted digraph $\mathcal{G}(M)$

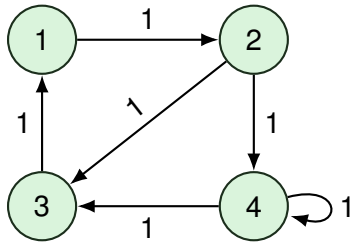


Reading $\mathcal{G}(M)$ directly from M:

- $m_{11} = 2 > 0 \implies$ self-loop $(1, 1)$
- $m_{13} = 4 > 0 \implies$ arc $(1, 3)$
- $m_{21} = 1 > 0 \implies$ arc $(2, 1)$
- $m_{22} = 3 > 0 \implies$ self-loop $(2, 2)$
- $m_{32} = 5 > 0 \implies$ arc $(3, 2)$
- All other entries are 0 (no other arcs)

Example: 4×4 sparse matrix and its digraph $\mathcal{G}(M)$

$$M = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{pmatrix}$$



- Row sums of $A(\mathcal{G}(M))$ give the outdegrees:

$$d^+ = (1, 2, 1, 2)^T$$

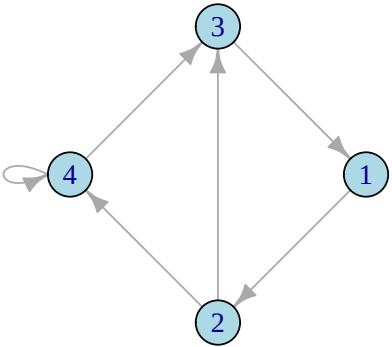
- Column sums of $A(\mathcal{G}(M))$ give the indegrees:

$$d^- = (1, 1, 2, 2)^T$$

Building a digraph from a matrix in igraph

```
M <- matrix(c(0, 1, 0, 0,
              0, 0, 1, 1,
              1, 0, 0, 0,
              0, 0, 1, 1), nrow = 4, byrow = TRUE)
G <- graph_from_adjacency_matrix(M, mode = "directed")
degree(G, mode = "out")

## [1] 1 2 1 2

degree(G, mode = "in")

## [1] 1 1 2 2

plot(G, edge.arrow.size = 0.7, vertex.color = "lightblue",
     vertex.size = 30, main = "Digraph G(M) from adjacency matrix M")
```

Digraph G(M) from adjacency matrix M



Non-negative matrices and their digraphs

Matrix powers, walks and connectivity

The Perron-Frobenius theorem

Regular discrete-time Markov chains

Matrix powers, walks and connectivity

Walks and powers of the adjacency matrix

Irreducibility and strong connectedness

Primitivity and aperiodicity

Walks, trails, paths and cycles in a digraph

Let $G = (V, A)$ be a digraph

Definition 113 (Directed walk, length and closed walk)

- *A **directed walk** from vertex v_0 to vertex v_k in G is a sequence of vertices $v_0, v_1, \ldots, v_k$ such that $(v_{r-1}, v_r) \in A$ is an arc in G for all $r = 1, \ldots, k$*
- *The **length** of the directed walk is k, the number of arcs traversed*
- *The walk is **closed** if its initial and terminal vertices coincide ($v_0 = v_k$)*

Definition 114 (Directed trail, path and cycle)

- A **directed trail** is a directed walk in which all arcs are distinct
- A **directed path** is a directed walk in which all vertices $v_0, v_1, \ldots, v_k$ are distinct
- A **directed cycle** (of length $k \geq 1$) is a closed walk $v_0, v_1, \ldots, v_k = v_0$ in which $v_0, v_1, \ldots, v_{k-1}$ are distinct (a cycle of length 1 is a self-loop)

Powers of a matrix and walks in its digraph

Theorem 115 (Matrix powers count walks)

Let $A = [a_{ij}] \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ and denote the (i,j) entry of A^k by $a_{ij}^{(k)}$ for $k \in \mathbb{N}_+$. Then

$$a_{ij}^{(k)} = \sum_{r_1=1}^{n} \sum_{r_2=1}^{n} \cdots \sum_{r_{k-1}=1}^{n} a_{i,r_1} a_{r_1,r_2} \cdots a_{r_{k-1},j}$$

In particular:

1. $a_{ij}^{(k)} > 0$ *if and only if there is at least one directed walk of length k from i to j in $\mathcal{G}(A)$*
2. *If $A \in \{0,1\}^{n \times n}$ is a binary adjacency matrix, then $a_{ij}^{(k)}$ equals the exact number of directed walks of length k from vertex i to vertex j in $\mathcal{G}(A)$*

Proof of Theorem 115

We prove the formula by mathematical induction on the walk length $k \geq 1$

Base case ($k = 1$): $A^1 = A$, and $a_{ij}^{(1)} = a_{ij}$ is the weight of the single arc (i, j) (a walk of length 1 from i to j), which equals 1 if (i, j) is an arc in the binary case and 0 otherwise

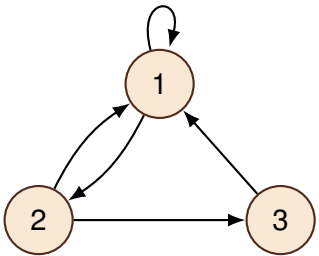
Inductive step: Suppose the result holds for $k \geq 1$. Since $A^{k+1} = A^k A$, the definition of matrix multiplication gives

$$a_{ij}^{(k+1)} = \sum_{r=1}^{n} a_{ir}^{(k)} a_{rj}$$

In the binary case $A \in \{0, 1\}^{n \times n}$, any walk of length $k + 1$ from i to j decomposes uniquely into a walk of length k from i to some penultimate vertex r, followed by the arc (r, j). For a fixed r, there are $a_{ir}^{(k)} a_{rj}$ such walks. Summing over all $r \in \{1, \ldots, n\}$ gives $a_{ij}^{(k+1)}$, completing the induction □

Example: counting walks of length 2 and 3

Consider the binary adjacency matrix A and its digraph $\mathcal{G}(A)$:



$$A = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}, \quad A^2 = \begin{pmatrix} 2 & 1 & 1 \\ 2 & 1 & 0 \\ 1 & 1 & 0 \end{pmatrix}$$

$$A^3 = A^2 A = \begin{pmatrix} 4 & 2 & 1 \\ 3 & 2 & 1 \\ 2 & 1 & 1 \end{pmatrix}$$

Let us verify individual entries directly from $\mathcal{G}(A)$:

- $(A^2)_{11} = 2$: the two walks of length 2 from 1 to 1 are $1 \to 1 \to 1$ and $1 \to 2 \to 1$
- $(A^2)_{23} = 0$: from 2, length-1 steps reach 1 or 3, neither of which has an arc to 3
- $(A^3)_{11} = 4$: the four walks of length 3 from 1 to 1 are

$$1 \to 1 \to 1 \to 1, \quad 1 \to 1 \to 2 \to 1, \quad 1 \to 2 \to 1 \to 1, \quad 1 \to 2 \to 3 \to 1$$

Matrix powers, walks and connectivity

- Walks and powers of the adjacency matrix
- **Irreducibility and strong connectedness**
- Primitivity and aperiodicity

Strong connectedness and strongly connected components

Definition 116 (Strongly connected digraph)

*A digraph $G = (V, A)$ is **strongly connected** if for every ordered pair of vertices $(i, j) \in V \times V$ with $i \neq j$, there exists a directed walk (and hence a directed path) in G from i to j and from j to i*

Define the mutual reachability relation $i \equiv j$ on V by:

$$i \equiv j \iff i = j \text{ or there is a directed walk from } i \text{ to } j \text{ and from } j \text{ to } i$$

- $\equiv$ is reflexive, symmetric and transitive, so it is an **equivalence relation** on V
- Its equivalence classes partition V into maximal strongly connected subdigraphs called the **strongly connected components** (SCCs) of G
- G is strongly connected if and only if it has a single strongly connected component

Reducible and irreducible matrices

Definition 117 (Reducible and irreducible matrices)

A matrix $A \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ ($n \geq 2$) is ***reducible*** *if there exists a permutation matrix $P_\pi \in \mathcal{M}_n$ such that*

$$P_\pi^T A P_\pi = \begin{pmatrix} A_{11} & A_{12} \\ \mathbf{0} & A_{22} \end{pmatrix}$$

where $A_{11} \in \mathcal{M}_r$ and $A_{22} \in \mathcal{M}_{n-r}$ are square blocks with $1 \leq r < n$, and $\mathbf{0}$ is the $(n-r) \times r$ zero matrix

The matrix A is ***irreducible*** *if no such permutation matrix exists (by convention, any 1×1 matrix is irreducible)*

Note that $P_\pi^T A P_\pi$ simply relabels the rows and columns of A using the same permutation π of the vertices $\{1, \ldots, n\}$

Why block upper-triangular form prevents strong connectedness

Suppose A has been relabelled into the reducible block form

$$\tilde{A} = P_\pi^T A P_\pi = \begin{pmatrix} A_{11} & A_{12} \\ \mathbf{0} & A_{22} \end{pmatrix} \begin{matrix} \} \ V_1 = \{1, \ldots, r\} \\ \} \ V_2 = \{r+1, \ldots, n\} \end{matrix}$$

- The bottom-left block is $\mathbf{0}$, which means $\tilde{a}_{ij} = 0$ for all $i \in V_2$ and $j \in V_1$: there is **no arc** from any vertex in V_2 to any vertex in V_1
- Furthermore, for every power $k \geq 1$, block multiplication yields

$$\tilde{A}^k = \begin{pmatrix} A_{11}^k & * \\ \mathbf{0} & A_{22}^k \end{pmatrix}$$

  so $(\tilde{A}^k)_{ij} = 0$ for all $i \in V_2$ and $j \in V_1$
- Thus no walk of any length $k \geq 1$ can ever go from V_2 to V_1: the vertex set V_2 is a **closed trap** under the row-to-column walk dynamics

Characterisations of irreducibility

Theorem 118 (Characterisations of irreducible matrices)

For a non-negative matrix $A \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$, the following statements are equivalent:

1. *A is irreducible*
2. *The associated digraph $\mathcal{G}(A)$ is strongly connected*
3. $\mathbb{I}_n + A + A^2 + \cdots + A^{n-1} > 0$ *(strictly positive entrywise)*
4. $(\mathbb{I}_n + A)^{n-1} > 0$ *(strictly positive entrywise)*

Items 3 and 4 provide a purely algebraic test for irreducibility without searching over all $n!$ permutation matrices

Proof of Theorem 118

(1) $\iff$ **(2)**: A is reducible $\iff$ vertices can be partitioned into non-empty sets V_1, V_2 with no arc from V_2 to V_1 $\iff$ no walk can go from V_2 to V_1 $\iff$ $\mathcal{G}(A)$ is not strongly connected

(2) $\iff$ **(3)**: By Theorem 115, $(A^k)_{ij} > 0$ $\iff$ there is a walk of length k from i to j. Any shortest walk between two distinct vertices $i \neq j$ is a simple path visiting at most n vertices, so its length is $k \in \{1, \ldots, n-1\}$. With $(\mathbb{I}_n)_{ii} = 1 > 0$ covering $i = j$, $\mathcal{G}(A)$ is strongly connected $\iff$ for every pair (i, j) there is some $k \in \{0, 1, \ldots, n-1\}$ with $(A^k)_{ij} > 0$ $\iff$ $\sum_{k=0}^{n-1} A^k > 0$

(3) $\iff$ **(4)**: By the binomial expansion with positive coefficients $\binom{n-1}{k} > 0$ and $A^k \geq 0$,

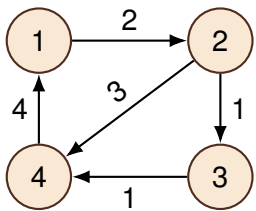
$$(\mathbb{I}_n + A)^{n-1} = \sum_{k=0}^{n-1} \binom{n-1}{k} A^k$$

A sum of non-negative terms with positive weights is strictly positive if and only if at least one term is strictly positive, so $(\mathbb{I}_n + A)^{n-1} > 0$ $\iff$ $\sum_{k=0}^{n-1} A^k > 0$ □

Exam-style example 1: testing irreducibility via $\mathcal{G}(A)$

Problem: Determine whether the following 4×4 matrix is irreducible:

$$A = \begin{pmatrix} 0 & 2 & 0 & 0 \\ 0 & 0 & 1 & 3 \\ 0 & 0 & 0 & 1 \\ 4 & 0 & 0 & 0 \end{pmatrix}$$



Solution:

- Draw the 4-node digraph $\mathcal{G}(A)$
- Observe the directed Hamiltonian cycle

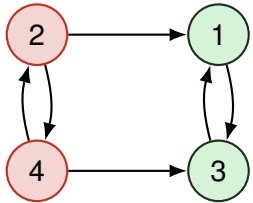
$$1 \to 2 \to 3 \to 4 \to 1$$

- Since this cycle visits all 4 vertices, every vertex can reach every other vertex along the cycle
- Therefore $\mathcal{G}(A)$ is **strongly connected**, so A is **irreducible**

Exam-style example 2: reducible matrix and block triangular form

Problem: Show that B is reducible, find its SCCs and write $P_\pi^T B P_\pi$ in block upper-triangular form:

$$B = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \end{pmatrix}$$



- SCCs are $C_1 = \{2, 4\}$ and $C_2 = \{1, 3\}$
- Arcs go from C_1 to C_2, but **no arc goes from C_2 to C_1**, so $\mathcal{G}(B)$ is not strongly connected (B is **reducible**)
- Ordering vertices as $(2, 4, 1, 3)$ gives

$$P_\pi^T B P_\pi = \left(\begin{array}{cc|cc} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ \hline 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{array}\right) = \begin{pmatrix} B_{11} & B_{12} \\ \mathbf{0} & B_{22} \end{pmatrix}$$

Checking strong connectedness and $(\mathbb{I}_n + A)^{n-1}$ in R

```
A <- matrix(c(0, 2, 0, 0,
              0, 0, 1, 3,
              0, 0, 0, 1,
              4, 0, 0, 0), nrow = 4, byrow = TRUE)
is_connected(graph_from_adjacency_matrix(A > 0), mode = "strong")

## [1] TRUE

# Compute (I_4 + A)^3
I4_plus_A <- diag(4) + A
I4_plus_A %*% I4_plus_A %*% I4_plus_A

##      [,1] [,2] [,3] [,4]
## [1,]   25    6    6   20
## [2,]   40   25    3   12
## [3,]   12    8    1    3
## [4,]   12   24    8   25
```

Matrix powers, walks and connectivity

Walks and powers of the adjacency matrix

Irreducibility and strong connectedness

Primitivity and aperiodicity

Primitive matrices

Definition 119 (Primitive matrix)

A non-negative matrix $A \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ is ***primitive*** *if there exists a positive integer $m \in \mathbb{N}_+$ such that*

$$A^m > 0 \quad (\textit{every entry of } A^m \textit{ is strictly positive})$$

The smallest positive integer m such that $A^m > 0$ is called the ***index of primitivity*** *of A, denoted $\gamma(A)$*

An irreducible non-negative matrix that is not primitive is called ***imprimitive*** *(or* ***periodic****)*

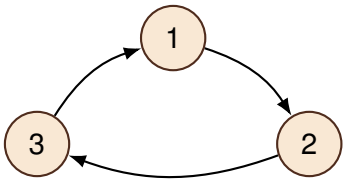
Proposition 120

1. *Every primitive matrix is irreducible*
2. *If $A^m > 0$ for some $m \in \mathbb{N}_+$, then $A^k > 0$ for all $k \geq m$*

Irreducible does not imply primitive!

Consider the 2×2 and 3×3 cyclic permutation matrices:

$$C_2 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad C_3 = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}$$



- Both $\mathcal{G}(C_2)$ and $\mathcal{G}(C_3)$ are single directed cycles, so C_2 and C_3 are **irreducible**
- Yet for C_2: $C_2^{2k} = \mathbb{I}_2$ and $C_2^{2k+1} = C_2$
- And for C_3: $C_3^2 = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}$, $C_3^3 = \mathbb{I}_3$, $C_3^4 = C_3, \ldots$
- Every power C_2^m and C_3^m contains zeros, so neither matrix is primitive!

Why? Because walks from i to j only occur at lengths m belonging to a fixed congruence class modulo 2 (for C_2) or modulo 3 (for C_3)

Period of a digraph and characterisation of primitivity

Definition 121 (Period and aperiodicity)

*Let $G = (V, A)$ be a strongly connected digraph. The **period** (or **index of imprimitivity**) $d(G)$ of G is the greatest common divisor of the lengths of all directed cycles (or closed walks) in G:*

$$d(G) = \gcd\{\mathit{length}(C) : C \text{ is a directed cycle in } G\}$$

- *If $d(G) = 1$, we say that G (and its matrix A) is **aperiodic***
- *If $d(G) > 1$, we say that G (and its matrix A) is **periodic** with period $d(G)$*

Theorem 122 (Graphical characterisation of primitive matrices)

*A non-negative matrix $A \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ is **primitive** if and only if $\mathcal{G}(A)$ is **strongly connected** and **aperiodic** ($d(\mathcal{G}(A)) = 1$)*

Practical corollaries for exams

Checking gcd of cycle lengths is immediate in two common situations:

Corollary 123 (Self-loop test for primitivity)

*If $A \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ is irreducible ($\mathcal{G}(A)$ is strongly connected) and has **at least one positive diagonal entry** $a_{ii} > 0$ for some $i \in \{1, \ldots, n\}$, then A is **primitive***

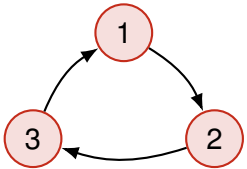
Reason: $a_{ii} > 0$ gives a self-loop $i \to i$ of length 1 in $\mathcal{G}(A)$, and $\gcd(1, \ldots) = 1$

Corollary 124 (Coprime cycle lengths)

*If $\mathcal{G}(A)$ is strongly connected and contains two directed cycles of coprime lengths ℓ_1 and ℓ_2 (i.e. $\gcd(\ell_1, \ell_2) = 1$, such as a 2-cycle and a 3-cycle), then A is **primitive***

Visualising periodic vs primitive digraphs

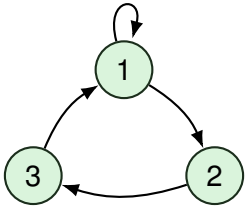
Case 1: Irreducible, periodic ($d = 3$)



$$A_1 = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}$$

Cycle length: $\{3\}$
$\gcd(3) = 3 \implies$ **not primitive**

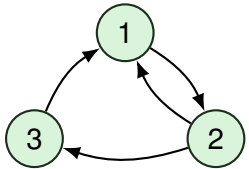
Case 2: Adding one self-loop ($d = 1$)



$$A_2 = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}$$

Cycle lengths: $\{1, 3\}$
$\gcd(1, 3) = 1 \implies$ **primitive** ($A_2^4 > 0$)

Case 3: Zero trace, coprime cycles ($d = 1$)



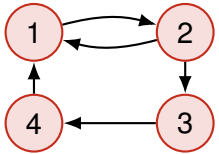
$$A_3 = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}$$

Cycles: $1 \leftrightarrow 2$ (2),
$1 \to 2 \to 3 \to 1$ (3)
$\gcd(2, 3) = 1 \implies$ **primitive** ($A_3^5 > 0$)

Exam-style comparison: two 4×4 irreducible matrices

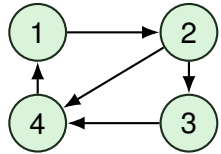
Both matrices below are irreducible with zero diagonal ($\mathrm{tr} = 0$). Which one is primitive?

$$M_1 = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{pmatrix}$$



- Directed cycles: $1 \to 2 \to 1$ (length 2) and $1 \to 2 \to 3 \to 4 \to 1$ (length 4)
- $d = \gcd(2, 4) = 2 > 1$:

$$M_2 = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{pmatrix}$$



- Directed cycles: $1 \to 2 \to 4 \to 1$ (length 3) and $1 \to 2 \to 3 \to 4 \to 1$ (length 4)
- $d = \gcd(3, 4) = 1$: **primitive**

Wielandt's bound on the index of primitivity

How large can the smallest exponent $\gamma(A)$ with $A^{\gamma(A)} > 0$ be for an $n \times n$ primitive matrix?

Theorem 125 (Wielandt's theorem)

Let $A \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ with $n \geq 2$. Then A is primitive if and only if

$$A^{n^2-2n+2} > 0$$

Moreover, if A is irreducible and has $d \geq 1$ positive diagonal entries, then

$$\gamma(A) \leq 2n - d - 1$$

- For $n = 3$, $n^2 - 2n + 2 = 5$: the matrix A_3 has $A_3^4 \not> 0$ and $A_3^5 > 0$
- For $n = 4$, $n^2 - 2n + 2 = 10$: the matrix M_2 on the previous slide has $M_2^9 \not> 0$ and $M_2^{10} > 0$, showing that Wielandt's bound $n^2 - 2n + 2$ is sharp!

Non-negative matrices and their digraphs

Matrix powers, walks and connectivity

The Perron-Frobenius theorem

Regular discrete-time Markov chains

Spectrum and spectral radius

Definition 126 (Spectrum and spectral radius)

Let $A \in \mathcal{M}_n(\mathbb{C})$ be a square matrix

- *The **spectrum** of A, denoted $\sigma(A)$, is the set of all eigenvalues of A:*

$$\sigma(A) = \{\lambda \in \mathbb{C} : \det(A - \lambda \mathbb{I}_n) = 0\}$$

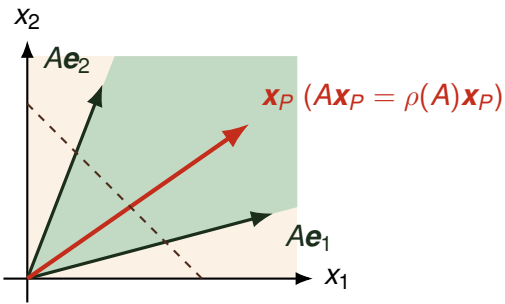
- *The **spectral radius** of A, denoted $\rho(A)$, is the maximum modulus of its eigenvalues:*

$$\rho(A) = \max_{\lambda \in \sigma(A)} |\lambda|$$

Geometrically, in the complex plane $\mathbb{C}$, all eigenvalues of A lie inside or on the closed disc $\{z \in \mathbb{C} : |z| \leq \rho(A)\}$ of radius $\rho(A)$ centred at the origin

Geometric intuition in $\mathbb{R}^2$: trapping an invariant ray

Why must a positive matrix $A > 0$ have a positive eigenvalue with a positive eigenvector?



- Consider the non-negative quadrant $\mathbb{R}^2_{\geq 0} = \{\boldsymbol{x} \geq \mathbf{0}\}$
- Since $A > 0$, both columns $A\boldsymbol{e}_1 > \mathbf{0}$ and $A\boldsymbol{e}_2 > \mathbf{0}$ lie **strictly inside** the open first quadrant
- Thus A maps the entire quadrant $\mathbb{R}^2_{\geq 0} \setminus \{\mathbf{0}\}$ into a narrower cone strictly inside the first quadrant
- By Brouwer's fixed-point theorem (or the intermediate value theorem on the angle of $\boldsymbol{x}$), A must map a unique ray in the first quadrant onto itself!

Perron's theorem for positive matrices

Theorem 127 (Perron's theorem, 1907)

Let $A \in \mathcal{M}_n(\mathbb{R})$ be a strictly positive matrix ($A > 0$). Then:

1. $\rho(A) > 0$ *is an eigenvalue of A (the* **Perron root** *or* **dominant eigenvalue***)*
2. $\rho(A)$ *is algebraically and geometrically simple (multiplicity* 1*)*
3. $\rho(A)$ *strictly dominates every other eigenvalue:* $|\lambda| < \rho(A)$ *for all* $\lambda \in \sigma(A) \setminus \{\rho(A)\}$
4. *There exists a* ***unique*** *strictly positive right eigenvector* $\boldsymbol{x} > \mathbf{0}$ *such that*

$$A\boldsymbol{x} = \rho(A)\boldsymbol{x} \quad \text{and} \quad \mathbb{1}^T\boldsymbol{x} = \sum_{i=1}^{n} x_i = 1$$

   (the **right Perron vector***, and similarly a unique left Perron vector* $\boldsymbol{y} > \mathbf{0}$*)*
5. *No other eigenvalue of A has a non-negative eigenvector*

The Perron-Frobenius theorem for primitive and irreducible matrices

Frobenius (1912) extended Perron's theorem to irreducible non-negative matrices:

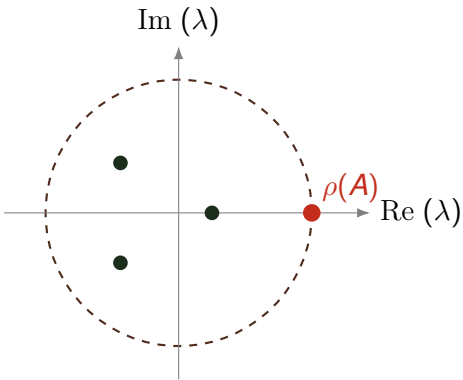
Theorem 128 (Perron-Frobenius theorem)

*Let $A \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ be an **irreducible** non-negative matrix*

1. *$\rho(A) > 0$ is a simple eigenvalue of A (algebraic and geometric multiplicity 1)*
2. *There exists a unique strictly positive right eigenvector $\boldsymbol{x} > \mathbf{0}$ with $A\boldsymbol{x} = \rho(A)\boldsymbol{x}$ and $\mathbb{1}^T\boldsymbol{x} = 1$ (and no other eigenvalue has a non-negative eigenvector)*
3. ***If** A **is primitive** ($d = 1$): $|\lambda| < \rho(A)$ for all $\lambda \in \sigma(A)$ with $\lambda \neq \rho(A)$*
4. ***If** A **is imprimitive with period** $d > 1$: A has exactly d eigenvalues of modulus $\rho(A)$, given by $\lambda_k = \rho(A)e^{2\pi ik/d}$ ($k = 0, 1, \ldots, d-1$), and $|\lambda| < \rho(A)$ for all others*

Spectral circle: primitive ($d = 1$) vs periodic ($d = 3$)

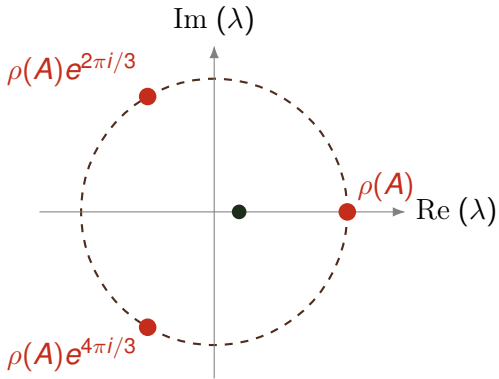
Primitive matrix ($d = 1$)



Strict spectral gap $|\lambda_j| < \rho(A)$ for $j \geq 2$

$\Longrightarrow \left(\frac{1}{\rho(A)}A\right)^k$ **converges** as $k \to \infty$

Irreducible periodic matrix ($d = 3$)

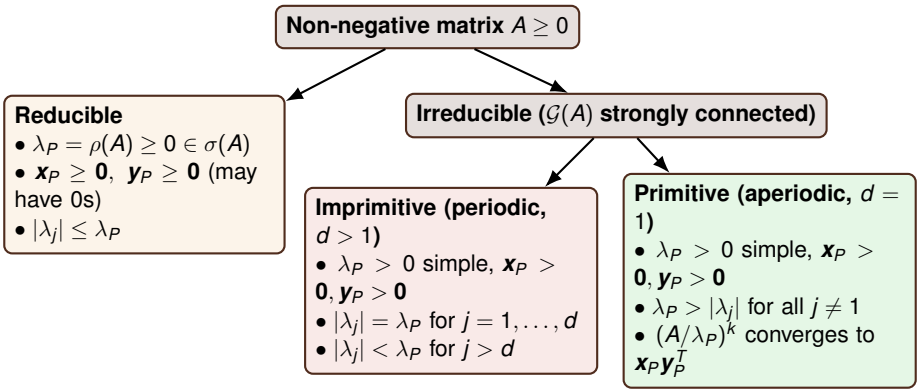


$d = 3$ eigenvalues on the circle $|\lambda| = \rho(A)$

$\Longrightarrow \left(\frac{1}{\rho(A)}A\right)^k$ **oscillates** with period 3

Summary of Perron-Frobenius across all non-negative matrices

Let $A \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ have Perron root $\lambda_P = \rho(A)$ and right/left Perron vectors $\boldsymbol{x}_P, \boldsymbol{y}_P$:



Asymptotic behaviour of powers of a primitive matrix

Theorem 129 (Asymptotics of primitive matrices)

Let $A \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ be a primitive matrix with Perron root $\rho(A) > 0$. Let $\boldsymbol{x} > \mathbf{0}$ and $\boldsymbol{y} > \mathbf{0}$ be the unique positive right and left eigenvectors satisfying $A\boldsymbol{x} = \rho(A)\boldsymbol{x}$, $\boldsymbol{y}^T A = \rho(A)\boldsymbol{y}^T$, $\mathbb{1}^T \boldsymbol{x} = 1$ and $\boldsymbol{y}^T \boldsymbol{x} = 1$. Then

$$\lim_{k\to\infty} \left(\frac{1}{\rho(A)} A\right)^k = \boldsymbol{x}\boldsymbol{y}^T > 0$$

Why this holds: Write $\boldsymbol{z} = (\boldsymbol{y}^T \boldsymbol{z})\boldsymbol{x} + \sum_{j=2}^n c_j \boldsymbol{v}_j$. Since $|\lambda_j/\rho(A)| < 1$ for all $j \geq 2$,

$$\left(\frac{1}{\rho(A)} A\right)^k \boldsymbol{z} = (\boldsymbol{y}^T \boldsymbol{z})\boldsymbol{x} + \sum_{j=2}^{n} c_j \left(\frac{\lambda_j}{\rho(A)}\right)^k \boldsymbol{v}_j \xrightarrow[k\to\infty]{} (\boldsymbol{y}^T \boldsymbol{z})\boldsymbol{x} = (\boldsymbol{x}\boldsymbol{y}^T)\boldsymbol{z}$$

Inspecting the Perron-Frobenius spectrum in R

Compare the eigenvalues of the periodic matrix A_1 ($d = 3$) and primitive matrix A_2 ($d = 1$):

```
A1 <- matrix(c(0, 1, 0,  0, 0, 1,  1, 0, 0), nrow = 3, byrow = TRUE)
A2 <- matrix(c(1, 1, 0,  0, 0, 1,  1, 0, 0), nrow = 3, byrow = TRUE)
# Moduli of eigenvalues of A1 (all 3 have modulus 1)
round(Mod(eigen(A1)$values), 4)

## [1] 1 1 1

# Eigenvalues and moduli for primitive A2 (unique dominant Perron root)
round(eigen(A2)$values, 4)

## [1]  1.4656+0.0000i -0.2328+0.7926i -0.2328-0.7926i

round(Mod(eigen(A2)$values), 4)

## [1] 1.4656 0.8260 0.8260
```

Non-negative matrices and their digraphs

Matrix powers, walks and connectivity

The Perron-Frobenius theorem

Regular discrete-time Markov chains

Regular discrete-time Markov chains

Markov chains and column-stochastic matrices

The fundamental limit theorem for regular Markov chains

Examples and running example: orange cats

Discrete-time Markov chains

A **Markov chain** is a discrete-time stochastic process whose evolution depends **only on the current state** of the system (the **memoryless** or **Markov property**):

- Consider a system that can be in one of n finite states $S = \{S_1, \ldots, S_n\}$ at discrete times $t = 0, 1, 2, \ldots$
- The probability of moving to state S_i at time $t + 1$ given that the system is in state S_j at time t is

$$p_{ij} = \mathbb{P}(S_i(t + 1) \mid S_j(t))$$

- Note the index order carefully: p_{ij} is the transition probability **from state S_j (column j) to state S_i (row i)**

Markov chains are a cornerstone of data science: they underpin PageRank, Hidden Markov Models (HMMs), Markov Chain Monte Carlo (MCMC) sampling and reinforcement learning

The transition matrix P

Definition 130 (Discrete-time Markov chain and transition matrix)

A sequence of random states from $S = \{S_1, \ldots, S_n\}$ is a (time-homogeneous) ***discrete-time Markov chain*** *(DTMC) if for all $t = 0, 1, 2, \ldots$ and all $1 \leq i, j \leq n$, the transition probability*

$$p_{ij} = \mathbb{P}(S_i(t+1) \mid S_j(t))$$

depends only on i and j. The matrix $P = [p_{ij}] \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ is called the ***transition matrix*** *of the Markov chain*

If the system is in state S_j at time t, it must transition to one of $S_1, \ldots, S_n$ at time $t+1$, so the entries in **column** j of P must sum to 1:

$$p_{1j} + p_{2j} + \cdots + p_{nj} = \sum_{i=1}^{n} p_{ij} = 1 \quad \text{for all } j = 1, \ldots, n$$

How do we get to state S_i at time $t+1$?

For $1 \leq i \leq n$, let $p_i(t) = \mathbb{P}(S_i(t))$ be the probability that the system is in state S_i at time t

List all n mutually exclusive ways to arrive in state S_i at time $t+1$:

1. We were in S_1 at time t (probability $p_1(t)$) and transitioned from S_1 to S_i (probability p_{i1}), contributing $p_{i1}p_1(t)$
2. We were in S_2 at time t (probability $p_2(t)$) and transitioned from S_2 to S_i (probability p_{i2}), contributing $p_{i2}p_2(t)$

⋮

n. We were in S_n at time t (probability $p_n(t)$) and transitioned from S_n to S_i (probability p_{in}), contributing $p_{in}p_n(t)$

Summing over all $j = 1, \ldots, n$ gives the law of total probability:

$$p_i(t+1) = \sum_{j=1}^{n} p_{ij}p_j(t) = p_{i1}p_1(t) + p_{i2}p_2(t) + \cdots + p_{in}p_n(t)$$

State distribution update in matrix form

Collecting the state probabilities into the column **probability vector**

$$\boldsymbol{p}(t) = \begin{pmatrix} p_1(t) \\ \vdots \\ p_n(t) \end{pmatrix} \geq \mathbf{0}, \qquad \mathbb{1}^T \boldsymbol{p}(t) = \sum_{i=1}^{n} p_i(t) = 1$$

the update equations for $i = 1, \ldots, n$ take the compact matrix form

$$\begin{pmatrix} p_1(t+1) \\ \vdots \\ p_n(t+1) \end{pmatrix} = \begin{pmatrix} p_{11} & p_{12} & \cdots & p_{1n} \\ \vdots & \vdots & \ddots & \vdots \\ p_{n1} & p_{n2} & \cdots & p_{nn} \end{pmatrix} \begin{pmatrix} p_1(t) \\ \vdots \\ p_n(t) \end{pmatrix} \qquad \Longleftrightarrow \qquad \boldsymbol{p}(t+1) = P\boldsymbol{p}(t)$$

Iterating from the initial distribution $\boldsymbol{p}(0)$ gives $\boldsymbol{p}(1) = P\boldsymbol{p}(0)$, $\boldsymbol{p}(2) = P\boldsymbol{p}(1) = P^2\boldsymbol{p}(0)$ and for any $t \in \mathbb{N}$:

$$\boldsymbol{p}(t) = P^t \boldsymbol{p}(0)$$

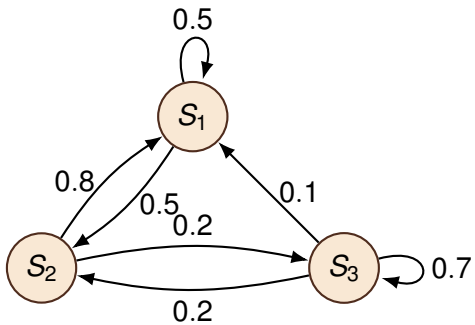
Transition matrix and transition digraph

For a column-stochastic transition matrix $P = [p_{ij}]$, recall that $p_{ij} > 0$ represents a transition **from state S_j to state S_i**:

Column-stochastic matrix P

$$P = \begin{pmatrix} 0.5 & 0.8 & 0.1 \\ 0.5 & 0 & 0.2 \\ 0 & 0.2 & 0.7 \end{pmatrix}$$

Transition digraph



- Column 1 (from S_1): $0.5 + 0.5 + 0 = 1$
- Column 2 (from S_2): $0.8 + 0 + 0.2 = 1$
- Column 3 (from S_3): $0.1 + 0.2 + 0.7 = 1$

Out-weights from each vertex S_j in the transition digraph correspond to **column** j of P and always sum to 1

Stochastic matrices

Definition 131 (Column-stochastic, row-stochastic and doubly stochastic matrices)

Let $M = [m_{ij}] \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ be a non-negative matrix and $\mathbb{1} = (1, \ldots, 1)^T \in \mathbb{R}^n$

- *M is **column-stochastic** if every column sums to 1:*

$$\sum_{i=1}^{n} m_{ij} = 1 \quad \text{for all } j = 1, \ldots, n \qquad \Longleftrightarrow \qquad \mathbb{1}^T M = \mathbb{1}^T$$

- *M is **row-stochastic** if every row sums to 1:*

$$\sum_{j=1}^{n} m_{ij} = 1 \quad \text{for all } i = 1, \ldots, n \qquad \Longleftrightarrow \qquad M\mathbb{1} = \mathbb{1}$$

- *M is **doubly stochastic** if it is both column-stochastic and row-stochastic ($\mathbb{1}^T M = \mathbb{1}^T$ and $M\mathbb{1} = \mathbb{1}$)*

Products and powers of column-stochastic matrices

Theorem 132

1. *If $M, N \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ are column-stochastic matrices, then their product MN is also column-stochastic*
2. *For any column-stochastic matrix $P \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ and any integer $t \geq 1$, P^t is column-stochastic, and if $\boldsymbol{p}(0)$ is a probability vector, then $\boldsymbol{p}(t) = P^t\boldsymbol{p}(0)$ is a probability vector*

Proof:

1. Since $M \geq 0$ and $N \geq 0$, their product satisfies $MN \geq 0$. Furthermore, using the associativity of matrix multiplication and $\mathbb{1}^T M = \mathbb{1}^T$, $\mathbb{1}^T N = \mathbb{1}^T$:

   $$\mathbb{1}^T(MN) = (\mathbb{1}^T M)N = \mathbb{1}^T N = \mathbb{1}^T$$

   Thus MN is non-negative and its columns sum to 1, so MN is column-stochastic
2. By induction on $t \geq 1$, P^t is column-stochastic, and $\boldsymbol{p}(t) = P^t\boldsymbol{p}(0) \geq \mathbf{0}$ with $\mathbb{1}^T\boldsymbol{p}(t) = (\mathbb{1}^T P^t)\boldsymbol{p}(0) = \mathbb{1}^T\boldsymbol{p}(0) = 1$

□

Spectral radius of a column-stochastic matrix

Theorem 133 (Spectrum of a stochastic matrix)

Let $P \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ be a column-stochastic matrix. Then:

1. $\lambda_1 = 1$ *is an eigenvalue of* P
2. *Every eigenvalue* $\lambda \in \sigma(P)$ *satisfies* $|\lambda| \leq 1$*, so the spectral radius is* $\rho(P) = 1$

Proof:

1. Since P is column-stochastic, $\mathbb{1}^T P = \mathbb{1}^T$. Taking the transpose gives $P^T \mathbb{1} = \mathbb{1}$ with $\mathbb{1} \neq \mathbf{0}$, so $\lambda = 1$ is an eigenvalue of P^T. Since $\det(P^T - \lambda \mathbb{I}_n) = \det(P - \lambda \mathbb{I}_n)$, P and P^T have the same eigenvalues ($\sigma(P) = \sigma(P^T)$), so $1 \in \sigma(P)$
2. Let $\lambda \in \sigma(P) = \sigma(P^T)$ and let $\boldsymbol{x} \neq \mathbf{0}$ be a right eigenvector of P^T: $P^T \boldsymbol{x} = \lambda \boldsymbol{x}$. Pick an index k maximising $|x_k| = \max_{1 \leq j \leq n} |x_j| > 0$. Then from row k of $P^T \boldsymbol{x} = \lambda \boldsymbol{x}$:

$$|\lambda|\,|x_k| = |\lambda x_k| = \left| \sum_{i=1}^{n} p_{ik} x_i \right| \leq \sum_{i=1}^{n} p_{ik} |x_i| \leq \left(\sum_{i=1}^{n} p_{ik} \right) |x_k| = 1 \cdot |x_k|$$

Regular discrete-time Markov chains

- Markov chains and column-stochastic matrices
- **The fundamental limit theorem for regular Markov chains**
- Examples and running example: orange cats

Regular Markov chains

Definition 134 (Regular Markov chain)

*A discrete-time Markov chain with column-stochastic transition matrix $P \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ is **regular** if P is **primitive**, i.e. there exists a positive integer $k \geq 1$ such that*

$$P^k > 0 \quad (\textit{every entry of } P^k \textit{ is strictly positive})$$

Interpretation:

- The entry $(P^k)_{ij}$ is the k-step transition probability of moving from state S_j to state S_i in exactly k steps
- Thus a Markov chain is regular if and only if there is some time horizon $k \geq 1$ such that from **any** starting state S_j, there is a strictly positive probability of being in **any** target state S_i after k steps
- By Theorem 122, a Markov chain is regular $\iff$ its transition digraph is **strongly connected** (irreducible) and **aperiodic** ($d = 1$, e.g. at least one state has a self-loop $p_{ii} > 0$)

The fundamental limit theorem for regular Markov chains

Applying Theorem 129 with $\rho(P) = 1$ and left Perron vector $\boldsymbol{y} = \mathbb{1}$ yields:

Theorem 135 (Fundamental limit theorem for regular Markov chains)

*Let $P \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ be the column-stochastic transition matrix of a **regular** Markov chain. Then:*

1. *There exists a **unique** strictly positive probability vector $\boldsymbol{v} = (v_1, \ldots, v_n)^T > \mathbf{0}$ satisfying $P\boldsymbol{v} = \boldsymbol{v}$ and $\mathbb{1}^T\boldsymbol{v} = \sum_{i=1}^n v_i = 1$, called the **stationary distribution** (or **equilibrium** / **steady-state distribution**) of the Markov chain*
2. *As $t \to \infty$, the matrix powers P^t converge to the rank-one column-stochastic matrix*
$$\lim_{t\to\infty} P^t = W = \begin{pmatrix} \boldsymbol{v} & \boldsymbol{v} & \cdots & \boldsymbol{v} \end{pmatrix} = \boldsymbol{v}\mathbb{1}^T > 0$$
3. *For **any** initial probability distribution $\boldsymbol{p}(0)$, the state vector $\boldsymbol{p}(t) = P^t\boldsymbol{p}(0)$ converges to $\boldsymbol{v}$:*
$$\lim_{t\to\infty} \boldsymbol{p}(t) = W\boldsymbol{p}(0) = \boldsymbol{v}(\mathbb{1}^T\boldsymbol{p}(0)) = \boldsymbol{v}$$

Finding the stationary distribution $\boldsymbol{v}$ in practice

How do we compute the unique stationary distribution $\boldsymbol{v}$ of a regular transition matrix $P \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ on an exam or by hand?

1. **Write the fixed-point equation $P\boldsymbol{v} = \boldsymbol{v}$ as a homogeneous linear system**:

$$(P - \mathbb{I}_n)\boldsymbol{v} = \mathbf{0} \qquad (\text{so } \boldsymbol{v} \in \operatorname{null}(P - \mathbb{I}_n))$$

   - Since $\lambda_1 = 1$ is a simple eigenvalue of P, $\dim \operatorname{null}(P - \mathbb{I}_n) = 1$
   - Because $\mathbb{1}^T(P - \mathbb{I}_n) = \mathbf{0}^T$, the rows of $P - \mathbb{I}_n$ always sum to zero (one row is redundant)

2. **Row-reduce** $P - \mathbb{I}_n$ (or solve the first $n - 1$ equations) to express $v_1, \ldots, v_n$ in terms of a single free parameter, giving a positive basis vector $\boldsymbol{w} \in \operatorname{null}(P - \mathbb{I}_n)$
3. **Normalise to a probability vector** using $\mathbb{1}^T\boldsymbol{v} = v_1 + \cdots + v_n = 1$:

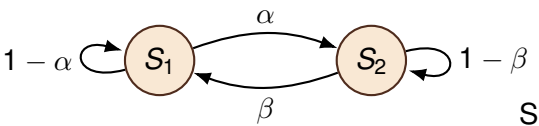
$$\boldsymbol{v} = \frac{\boldsymbol{w}}{\|\boldsymbol{w}\|_1} = \frac{\boldsymbol{w}}{w_1 + \cdots + w_n}$$

Regular discrete-time Markov chains

- Markov chains and column-stochastic matrices
- The fundamental limit theorem for regular Markov chains
- **Examples and running example: orange cats**

Exam-style example 1: general 2×2 regular Markov chain

Consider a two-state Markov chain with transition probabilities $\alpha = \mathbb{P}(S_2 \mid S_1)$ and $\beta = \mathbb{P}(S_1 \mid S_2)$, where $0 < \alpha, \beta < 1$:



$$P = \begin{pmatrix} 1-\alpha & \beta \\ \alpha & 1-\beta \end{pmatrix} > 0$$

Since $P > 0$, the chain is regular ($P^1 > 0$)

Finding the stationary distribution $\boldsymbol{v} = (v_1, v_2)^T$:

$$(P - \mathbb{I}_2)\boldsymbol{v} = \begin{pmatrix} -\alpha & \beta \\ \alpha & -\beta \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \iff -\alpha v_1 + \beta v_2 = 0 \iff v_2 = \frac{\alpha}{\beta} v_1$$

Imposing $v_1 + v_2 = 1$ gives $v_1 \left(1 + \frac{\alpha}{\beta}\right) = 1$, hence:

$$\boldsymbol{v} = \frac{1}{\alpha + \beta} \begin{pmatrix} \beta \\ \alpha \end{pmatrix}, \qquad \lim_{t \to \infty} P^t = W = \frac{1}{\alpha + \beta} \begin{pmatrix} \beta & \beta \\ \alpha & \alpha \end{pmatrix}$$

Exam-style example 2: 3×3 regular chain via row reduction

Problem: Show that the Markov chain with transition matrix P is regular and find its stationary distribution $\boldsymbol{v}$ and $\lim_{t\to\infty} P^t$:

$$P = \begin{pmatrix} 0.5 & 0.4 & 0 \\ 0.5 & 0.2 & 0.5 \\ 0 & 0.4 & 0.5 \end{pmatrix}$$

Step 1 (Regularity):

- From P, we have transitions $S_1 \to S_2 \to S_3 \to S_2 \to S_1$, so all 3 states communicate ($\mathcal{G}(P^T)$ is strongly connected, P is irreducible)
- Diagonal entries $p_{11} = 0.5 > 0$, $p_{22} = 0.2 > 0$, $p_{33} = 0.5 > 0$ give self-loops, so by Corollary 123, P is **primitive** (in fact $P^2 > 0$) and the chain is **regular**

Exam-style example 2 (continued): solving $(P - \mathbb{I}_3)\boldsymbol{v} = \mathbf{0}$

Step 2 (Nullspace of $P - \mathbb{I}_3$): Form $P - \mathbb{I}_3$ (notice the columns sum to 0):

$$P - \mathbb{I}_3 = \begin{pmatrix} -0.5 & 0.4 & 0 \\ 0.5 & -0.8 & 0.5 \\ 0 & 0.4 & -0.5 \end{pmatrix} \xrightarrow{R_2 + R_1} \begin{pmatrix} -0.5 & 0.4 & 0 \\ 0 & -0.4 & 0.5 \\ 0 & 0.4 & -0.5 \end{pmatrix} \xrightarrow{R_3 + R_2} \begin{pmatrix} -0.5 & 0.4 & 0 \\ 0 & -0.4 & 0.5 \\ 0 & 0 & 0 \end{pmatrix}$$

From the two pivot rows with v_3 as free variable:

- Row 2: $-0.4v_2 + 0.5v_3 = 0 \implies v_2 = \frac{5}{4}v_3$
- Row 1: $-0.5v_1 + 0.4v_2 = 0 \implies v_1 = \frac{4}{5}v_2 = \frac{4}{5} \cdot \frac{5}{4}v_3 = v_3$

Step 3 (Normalisation): Impose $v_1 + v_2 + v_3 = 1$, i.e. $\left(1 + \frac{5}{4} + 1\right) v_3 = \frac{13}{4}v_3 = 1 \implies v_3 = \frac{4}{13}$, giving:

$$\boldsymbol{v} = \begin{pmatrix} 4/13 \\ 5/13 \\ 4/13 \end{pmatrix}, \qquad \lim_{t \to \infty} P^t = W = \begin{pmatrix} 4/13 & 4/13 & 4/13 \\ 5/13 & 5/13 & 5/13 \\ 4/13 & 4/13 & 4/13 \end{pmatrix}$$

Running example: the “orange” gene in cats

A cat’s coat colour is determined by many genes; the “orange” trait comes from a specific gene called the **orange locus**

It has two *alleles* (versions):

- **O** $\implies$ produces *phaeomelanin* (red/orange pigment)
- **o** $\implies$ produces *eumelanin* (black/brown pigment)

This gene is *sex-linked*: it is located on the **X chromosome**

- Females have two X chromosomes (**XX**), receiving one allele from each parent: genotypes $X^O X^O$, $X^o X^o$ or $X^O X^o$
- Males have one X and one Y chromosome (**XY**), receiving only one allele (always from the mother): genotypes $X^O Y$ or $X^o Y$

Genotype vs phenotype in cats

Males (simple):

- $X^O Y \implies$ **orange cat**
- $X^o Y \implies$ **non-orange cat** (e.g. black)

Females (the special case):

- $X^O X^O \implies$ **orange cat** (S_1)
- $X^o X^o \implies$ **non-orange cat** (S_2, e.g. black)
- $X^O X^o \implies$ **tortoiseshell cat** (S_3)

A “tortie” is not a simple blend: because of random X-inactivation in female cells, both alleles (O and o) are active in different patches of skin, creating the mottled orange and black pattern (male tortoiseshells are extremely rare genetic anomalies with **XXY** genotype $X^O X^o Y$)

Punnett square warm-up: crossing two parents

Cross 1: orange dad ($X^O Y$) + black mum ($X^o X^o$)

		Father	
		X^O	Y
Mother	X^o	$X^O X^o$	$X^o Y$
	X^o	$X^O X^o$	$X^o Y$

- All daughters ($X^O X^o$) are **tortoiseshell**
- All sons ($X^o Y$) are **black**

Cross 2: black dad ($X^o Y$) + tortie mum ($X^O X^o$)

		Father	
		X^o	Y
Mother	X^O	$X^O X^o$	$X^O Y$
	X^o	$X^o X^o$	$X^o Y$

- Daughters: 50% tortie ($X^O X^o$), 50% black ($X^o X^o$)
- Sons: 50% orange ($X^O Y$), 50% black ($X^o Y$)

Setting up the regular Markov chain for female genotypes

Track the 3 female genotypes across generations:

- S_1: $X^O X^O$ (orange female)
- S_2: $X^o X^o$ (black female)
- S_3: $X^O X^o$ (tortoiseshell female)

In each generation, we mate the current female with a male chosen at random from a **fixed population** consisting of:

- 50% orange males ($X^O Y$)
- 50% black males ($X^o Y$)

and select one of her daughters at random as the next generation's female

State 1: orange female ($X^O X^O$)

The mother is $X^O X^O$. We pick a father with 50/50 probability:

Case 1: father is $X^O Y$

Mother	X^O	Y
X^O	$X^O X^O$ (S_1)	Male
X^O	$X^O X^O$ (S_1)	Male

Daughters: 100% S_1

Case 2: father is $X^o Y$

Mother	X^o	Y
X^O	$X^O X^o$ (S_3)	Male
X^O	$X^O X^o$ (S_3)	Male

Daughters: 100% S_3

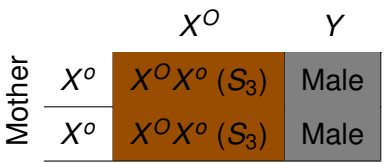
Transitions from S_1 (column 1 of P):

- $p_{11} = \mathbb{P}(S_1 \mid S_1) = 0.5 \times 1.0 = \mathbf{0.5}$
- $p_{21} = \mathbb{P}(S_2 \mid S_1) = \mathbf{0}$
- $p_{31} = \mathbb{P}(S_3 \mid S_1) = 0.5 \times 1.0 = \mathbf{0.5}$

State 2: black female (X^oX^o)

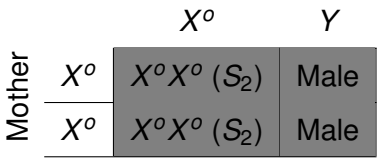
The mother is X^oX^o. We pick a father with 50/50 probability:

Case 1: father is X^OY



Daughters: 100% S_3

Case 2: father is X^oY



Daughters: 100% S_2

Transitions from S_2 (column 2 of P):

- $p_{12} = \mathbb{P}(S_1 \mid S_2) = \mathbf{0}$
- $p_{22} = \mathbb{P}(S_2 \mid S_2) = 0.5 \times 1.0 = \mathbf{0.5}$
- $p_{32} = \mathbb{P}(S_3 \mid S_2) = 0.5 \times 1.0 = \mathbf{0.5}$

State 3: tortoiseshell female ($X^O X^o$)

The mother is $X^O X^o$. We pick a father with 50/50 probability:

Case 1: father is $X^O Y$

Mother	X^O	Y
X^O	$X^O X^O$ (S_1)	Male
X^o	$X^O X^o$ (S_3)	Male

Daughters: 50% S_1, 50% S_3

Case 2: father is $X^o Y$

Mother	X^o	Y
X^O	$X^O X^o$ (S_3)	Male
X^o	$X^o X^o$ (S_2)	Male

Daughters: 50% S_2, 50% S_3

Transitions from S_3 (column 3 of P):

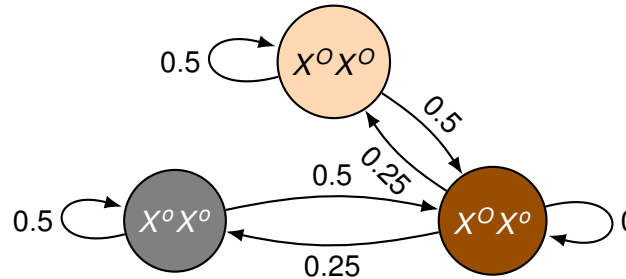
- $p_{13} = \mathbb{P}(S_1 \mid S_3) = 0.5 \times 0.5 = \mathbf{0.25}$
- $p_{23} = \mathbb{P}(S_2 \mid S_3) = 0.5 \times 0.5 = \mathbf{0.25}$
- $p_{33} = \mathbb{P}(S_3 \mid S_3) = 0.5 \times 0.5 + 0.5 \times 0.5 = \mathbf{0.5}$

Transition matrix and digraph of the orange cat chain

$$P = \begin{pmatrix} 0.5 & 0 & 0.25 \\ 0 & 0.5 & 0.25 \\ 0.5 & 0.5 & 0.5 \end{pmatrix}$$

Is this Markov chain regular?

- **Strongly connected**: $S_1 \to S_3 \to S_2$ and $S_2 \to S_3 \to S_1$, so all states communicate (P is irreducible)
- **Aperiodic**: $p_{11}, p_{22}, p_{33} > 0$ (self-loops), so P is **primitive** ($P^2 > 0$) and the chain is **regular**



Stationary distribution of the orange cat chain

We solve $(P - \mathbb{I}_3)\boldsymbol{v} = \mathbf{0}$ with $v_1 + v_2 + v_3 = 1$:

$$(P - \mathbb{I}_3)\boldsymbol{v} = \begin{pmatrix} -0.5 & 0 & 0.25 \\ 0 & -0.5 & 0.25 \\ 0.5 & 0.5 & -0.5 \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

From the first two equations:

- $-0.5v_1 + 0.25v_3 = 0 \implies v_1 = 0.5v_3$
- $-0.5v_2 + 0.25v_3 = 0 \implies v_2 = 0.5v_3$

Substituting into the normalisation condition $v_1 + v_2 + v_3 = 1$:

$$0.5v_3 + 0.5v_3 + v_3 = 2v_3 = 1 \implies v_3 = 0.5, \quad v_1 = 0.25, \quad v_2 = 0.25$$

Thus the unique stationary distribution is $\boldsymbol{v} = (0.25, 0.25, 0.5)^T$: in the long run, regardless of the initial female's genotype, 25% of females are orange, 25% are black and 50% are tortoiseshell

Computing matrix powers and stationary distributions in R

Let us verify the convergence of P^t and the eigenpair $P\boldsymbol{v} = \boldsymbol{v}$ in R:

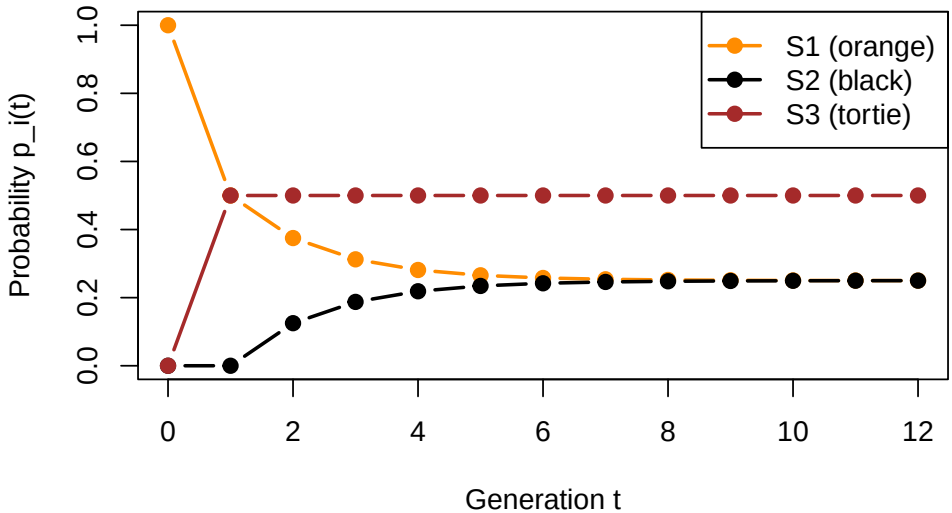
```
P_reg <- matrix(c(0.5, 0,   0.25,
                  0,   0.5, 0.25,
                  0.5, 0.5, 0.5),
                nrow = 3, byrow = TRUE)
# Dominant right eigenvector corresponding to eigenvalue 1
ev <- eigen(P_reg)
v_stat <- Re(ev$vectors[, 1])
v_stat <- v_stat / sum(v_stat)
v_stat
```

```
## [1] 0.25 0.25 0.50
```

```
# Compare with P^20 computed by repeated multiplication
P_20 <- Reduce(`%*%`, replicate(20, P_reg, simplify = FALSE))
round(P_20, 6)
```

Convergence of $\boldsymbol{p}(t) = P^t \boldsymbol{p}(0)$ from an initial state

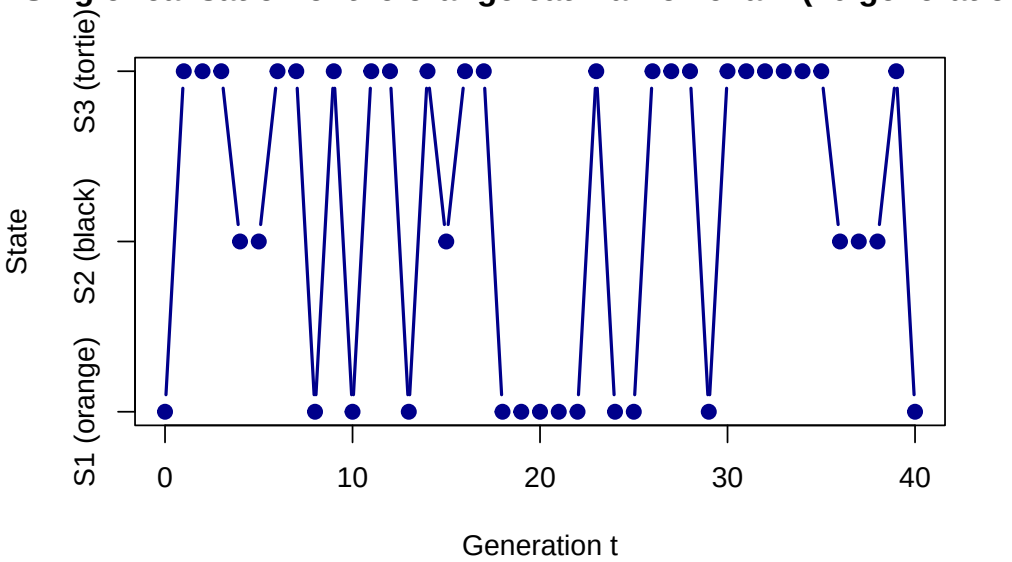
```r
t_max <- 12
p_hist <- matrix(0, nrow = 3, ncol = t_max + 1)
p_hist[, 1] <- c(1, 0, 0) # Start with 100% orange female (S_1)
for (t in 1:t_max) {
  p_hist[, t + 1] <- P_reg %*% p_hist[, t]
}
matplot(0:t_max, t(p_hist), type = "b", pch = 19, lty = 1, lwd = 2,
        col = c("darkorange", "black", "brown"),
        xlab = "Generation t", ylab = "Probability p_i(t)",
        ylim = c(0, 1), main = "Convergence to stationary distribution v = (
legend("topright", legend = c("S1 (orange)", "S2 (black)", "S3 (tortie)"),
       col = c("darkorange", "black", "brown"), lty = 1, pch = 19, lwd = 2)
```


Convergence to stationary distribution v = (0.25, 0.25, 0.5)
Probability p_i(t)
0.0
0.2
0.4
0.6
0.8
1.0
0
2
4
6
8
10
12
Generation t
S1 (orange)
S2 (black)
S3 (tortie)

Simulating a single realisation of the regular Markov chain

In a column-stochastic chain, when in state j at step t, the next state at step $t+1$ is drawn with probabilities given by **column** j of P:

```
set.seed(2740)
n_steps <- 40
states <- numeric(n_steps + 1)
states[1] <- 1 # Start in state S_1 (orange female)
for (t in 1:n_steps) {
  states[t + 1] <- sample(1:3, size = 1, prob = P_reg[, states[t]])
}
plot(0:n_steps, states, type = "b", pch = 19, lwd = 1.8, col = "darkblue",
     yaxt = "n", xlab = "Generation t", ylab = "State",
     main = "Single realisation of the orange cat Markov chain (40 generatio
axis(2, at = 1:3, labels = c("S1 (orange)", "S2 (black)", "S3 (tortie)"))
```


Single realisation of the orange cat Markov chain (40 generation
State
S3 (tortie)
S2 (black)
S1 (orange)
0
10
20
30
40
Generation t

Empirical state frequencies vs stationary distribution $\boldsymbol{v}$

By the ergodic theorem for regular Markov chains, the long-run fraction of time spent in each state along a single trajectory converges to $\boldsymbol{v} = (0.25, 0.25, 0.5)^T$:

```r
set.seed(2740)
n_long <- 10000
traj <- numeric(n_long)
traj[1] <- 1
for (t in 1:(n_long - 1)) {
  traj[t + 1] <- sample(1:3, size = 1, prob = P_reg[, traj[t]])
}
# Empirical frequencies over 10,000 generations vs exact v
rbind(empirical = prop.table(table(factor(traj, levels = 1:3))),
      stationary = v_stat)
```

```
##                 1      2      3
## empirical  0.2414 0.2505 0.5081
## stationary 0.2500 0.2500 0.5000
```

Absorbing Markov chains, spectral centrality and PageRank

MATH 2740 – Mathematics of Data Science – Unit U09

Julien Arino
julien.arino@umanitoba.ca

Department of Mathematics @ University of Manitoba

Fall 2026

The University of Manitoba campuses are located on original lands of Anishinaabeg, Ininew, Anisininew, Dakota and Dene peoples, and on the National Homeland of the Red River Métis. We respect the Treaties that were made on these territories, we acknowledge the harms and mistakes of the past, and we dedicate ourselves to move forward in partnership with Indigenous communities in a spirit of Reconciliation and collaboration.

Outline

Absorbing Markov chains

Spectral centrality in networks

PageRank and the Google matrix

From Unit 08 to Unit 09

In Unit 08, we studied irreducible and primitive non-negative matrices and saw that a regular Markov chain converges to a unique positive stationary distribution $\boldsymbol{w} = P\boldsymbol{w}$

Two natural questions arise when a directed network is *not* regular:

1. What happens when a Markov chain has **absorbing states** (states that can be entered but never left), making P reducible?
2. How do we rank the importance of vertices in large directed networks (citations, social networks or the World Wide Web) using eigenvectors and linear systems?

Both questions are answered by the same two linear algebra tools:
Perron-Frobenius theory and the **Neumann series** $(\mathbb{I} - M)^{-1} = \sum_{k=0}^{\infty} M^k$

Absorbing Markov chains

Spectral centrality in networks

PageRank and the Google matrix

Absorbing Markov chains

Absorbing states and motivating examples

Canonical block form and reducibility

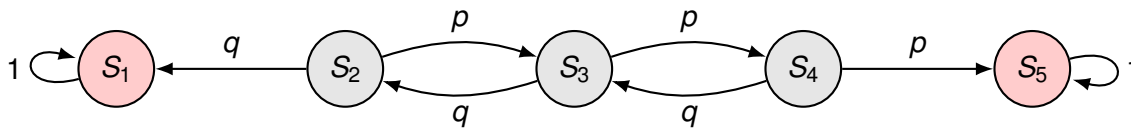
The Neumann series and fundamental matrix

Examples and R implementation

Motivating example 1 – 1D random walk with absorbing walls

Consider a particle (or gambler's fortune) moving on the states $\{S_1, S_2, S_3, S_4, S_5\}$:

- From an interior state S_j ($j \in \{2, 3, 4\}$), the particle moves left to S_{j-1} with probability $q = 1 - p$ and right to S_{j+1} with probability $p \in (0, 1)$
- States S_1 (ruin) and S_5 (target jackpot) are **absorbing boundaries**: once reached, the process stays there forever



Motivating example 2 – Modifying the breeding experiment

We modify the sex-linked orange cat breeding setup from Unit 08 by repeatedly mating a female offspring with an **orange male** ($X^O Y$) at every generation

The state of the chain is the genotype of the female offspring:

- S_1: $X^O X^O$ (orange female)
- S_2: $X^o X^o$ (black female)
- S_3: $X^O X^o$ (tortoiseshell female)

Let us compute the transition probabilities $p_{ij} = \mathbb{P}(S_i(t+1) \mid S_j(t))$ when mating each female genotype with an $X^O Y$ male

Transitions for the modified breeding experiment

Current state S_1 (X^OX^O)

Mom	X^O	Y
X^O	S_1	M
X^O	S_1	M

100% X^OX^O (S_1)

$\Longrightarrow$ $p_{11} = 1$

Current state S_2 (X^oX^o)

Mom	X^O	Y
X^o	S_3	M
X^o	S_3	M

100% X^OX^o (S_3)

$\Longrightarrow$ $p_{32} = 1$

Current state S_3 (X^OX^o)

Mom	X^O	Y
X^O	S_1	M
X^o	S_3	M

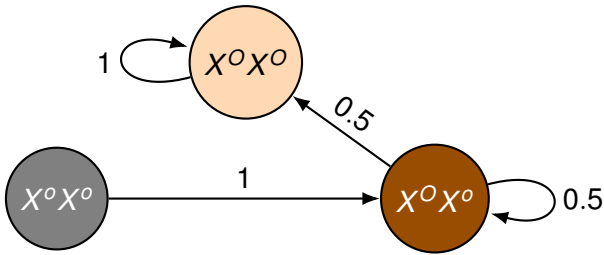
50% S_1, 50% S_3

$\Longrightarrow$ $p_{13} = 1/2$, $p_{33} = 1/2$

Summary of the modified breeding chain

In our column-stochastic convention ($p_{ij} = \mathbb{P}(S_i(t+1) \mid S_j(t))$), the transition matrix is

$$P = \begin{pmatrix} 1 & 0 & 1/2 \\ 0 & 0 & 0 \\ 0 & 1 & 1/2 \end{pmatrix}$$



Absorbing states and absorbing Markov chains

Definition 136 (Absorbing state)

A state S_i in a Markov chain with column-stochastic transition matrix $P = [p_{ij}]$ is ***absorbing*** *if once the chain enters S_i, it never leaves:*

$$p_{ii} = 1 \quad \text{and} \quad p_{ji} = 0 \text{ for all } j \neq i$$

Equivalently, column i of P is the standard basis vector $\mathbf{e}_i$, and in $\mathcal{G}(P)$ vertex S_i has a self-loop of weight 1 and no outgoing arcs to any other vertex

Definition 137 (Absorbing Markov chain)

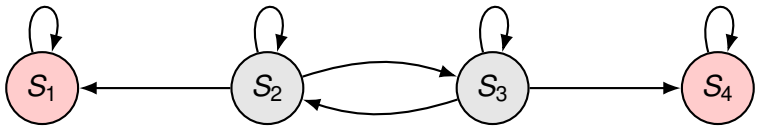
A Markov chain is ***absorbing*** *if:*

1. *It has at least one absorbing state*
2. *From every non-absorbing state, it is possible to reach at least one absorbing state in a finite number of steps (there is a directed walk in $\mathcal{G}(P)$ to an absorbing state)*

In an absorbing Markov chain, a state that is not absorbing is called ***transient***

Key questions about absorbing Markov chains

Suppose we have an absorbing chain such as a random walk with two absorbing boundaries S_1 and S_4:



1. Does the process eventually reach an absorbing state with probability 1?
2. Starting in transient state S_j, what is the expected number of visits to transient state S_i?
3. Starting in transient state S_j, what is the expected number of steps before absorption?
4. When there are multiple absorbing states, what is the probability of being absorbed in absorbing state S_i when starting in transient state S_j?

Absorbing Markov chains

- Absorbing states and motivating examples
- **Canonical block form and reducibility**
- The Neumann series and fundamental matrix
- Examples and R implementation

Absorbing chains are reducible

Suppose a Markov chain on n states has $r \geq 1$ absorbing states and $m = n - r \geq 1$ transient states

- Because an absorbing state S_i has no outgoing arcs to any other vertex ($p_{ji} = 0$ for all $j \neq i$), there is no directed path from S_i to any transient state
- Therefore, the transition digraph $\mathcal{G}(P)$ is **not strongly connected**
- By the characterisation from Unit 08, the transition matrix P is **reducible**

Relabelling the states of the chain corresponds to applying a permutation matrix $\Pi \in \mathcal{M}_n$:

$$\tilde{P} = \Pi^T P \Pi$$

We always relabel the states so that the r **absorbing states come first** $(S_1, \ldots, S_r)$ and the $m = n - r$ **transient states come last** $(S_{r+1}, \ldots, S_n)$

Example – Reordering the 5-state random walk into canonical form

In Motivating example 1 on $(S_1, S_2, S_3, S_4, S_5)$, states S_1 and S_5 are absorbing while S_2, S_3, S_4 are transient. In the original state ordering, the transition matrix is

$$P_{\text{orig}} = \begin{array}{c} \\ S_1 \\ S_2 \\ S_3 \\ S_4 \\ S_5 \end{array} \begin{array}{c} \begin{array}{ccccc} S_1 & S_2 & S_3 & S_4 & S_5 \end{array} \\ \begin{pmatrix} 1 & q & 0 & 0 & 0 \\ 0 & 0 & q & 0 & 0 \\ 0 & p & 0 & q & 0 \\ 0 & 0 & p & 0 & 0 \\ 0 & 0 & 0 & p & 1 \end{pmatrix} \end{array}$$

Reordering the states as $(S_1, S_5, S_2, S_3, S_4)$ via the permutation matrix $\Pi = [\boldsymbol{e}_1\ \boldsymbol{e}_5\ \boldsymbol{e}_2\ \boldsymbol{e}_3\ \boldsymbol{e}_4]$ gives the canonical block form $\tilde{P} = \Pi^T P_{\text{orig}} \Pi$:

$$\tilde{P} = \begin{array}{c} \\ S_1 \\ S_5 \\ \hline S_2 \\ S_3 \\ S_4 \end{array} \begin{array}{c} \begin{array}{cc|ccc} S_1 & S_5 & S_2 & S_3 & S_4 \end{array} \\ \left(\begin{array}{cc|ccc} 1 & 0 & q & 0 & 0 \\ 0 & 1 & 0 & 0 & p \\ \hline 0 & 0 & 0 & q & 0 \\ 0 & 0 & p & 0 & q \\ 0 & 0 & 0 & p & 0 \end{array}\right) \end{array} = \begin{pmatrix} \mathbb{I}_2 & R \\ \mathbf{0}_{3,2} & Q \end{pmatrix}$$

Canonical block form of an absorbing chain

Definition 138 (Canonical block form)

For an absorbing Markov chain with r absorbing states (listed first) and $m = n - r$ transient states (listed last), the column-stochastic transition matrix takes the ***canonical block form*** *(or* ***standard form****)*

$$P = \begin{pmatrix} \mathbb{I}_r & R \\ \mathbf{0}_{m,r} & Q \end{pmatrix}$$

where:

- $\mathbb{I}_r \in \mathcal{M}_r$ *is the $r \times r$ identity matrix (absorbing states stay put)*
- $\mathbf{0}_{m,r} \in \mathcal{M}_{mr}$ *is the $m \times r$ zero matrix (absorbing states never move to transient states)*
- $R \in \mathcal{M}_{rm}$ *contains the one-step transition probabilities from transient states to absorbing states*
- $Q \in \mathcal{M}_m$ *contains the one-step transition probabilities among transient states*

Visualising the canonical blocks

In our column-stochastic convention, columns correspond to the **from** state (at time t) and rows correspond to the **to** state (at time $t+1$):

	From absorbing (r cols)	**From transient** (m cols)
To absorbing (r rows)	$\mathbb{I}_r \in \mathcal{M}_r$	$R \in \mathcal{M}_{rm}$
To transient (m rows)	$\mathbf{0}_{m,r} \in \mathcal{M}_{mr}$	$Q \in \mathcal{M}_m$

Notice that:

- Each column of $\begin{pmatrix} R \\ Q \end{pmatrix}$ sums to 1: $\mathbb{1}_r^T R + \mathbb{1}_m^T Q = \mathbb{1}_m^T$
- Q is non-negative and **column-substochastic**: $\mathbb{1}_m^T Q \leq \mathbb{1}_m^T$, with strict inequality in every column j that has a direct transition to an absorbing state ($\sum_{i=1}^{r} r_{ij} > 0$)

Powers of P in canonical form

To understand the long-term behaviour $\boldsymbol{p}(k) = P^k \boldsymbol{p}(0)$, we compute the kth power of P in block form

Proposition 139 (Powers of the canonical transition matrix)

Let $P = \begin{pmatrix} \mathbb{I}_r & R \\ \mathbf{0}_{m,r} & Q \end{pmatrix}$ *be in canonical block form. For every integer* $k \geq 1$,

$$P^k = \begin{pmatrix} \mathbb{I}_r & R(\mathbb{I}_m + Q + Q^2 + \cdots + Q^{k-1}) \\ \mathbf{0}_{m,r} & Q^k \end{pmatrix} = \begin{pmatrix} \mathbb{I}_r & R\sum_{j=0}^{k-1} Q^j \\ \mathbf{0}_{m,r} & Q^k \end{pmatrix}$$

The (i, j) entry of Q^k is the probability that the chain is in the ith transient state after k steps, given that it started in the jth transient state

Proof of Proposition 139

We proceed by mathematical induction on $k \geq 1$

Base case ($k = 1$): Since $Q^0 = \mathbb{I}_m$, the formula gives $\begin{pmatrix} \mathbb{I}_r & R\mathbb{I}_m \\ \mathbf{0}_{m,r} & Q^1 \end{pmatrix} = P$, which holds

Inductive step: Suppose the formula holds for $k \geq 1$. Using block matrix multiplication for $P^{k+1} = P^k P$:

$$\begin{aligned} P^{k+1} &= \begin{pmatrix} \mathbb{I}_r & R(\mathbb{I}_m + Q + \cdots + Q^{k-1}) \\ \mathbf{0}_{m,r} & Q^k \end{pmatrix} \begin{pmatrix} \mathbb{I}_r & R \\ \mathbf{0}_{m,r} & Q \end{pmatrix} \\ &= \begin{pmatrix} \mathbb{I}_r\mathbb{I}_r + R(\cdots)\mathbf{0}_{m,r} & \mathbb{I}_r R + R(\mathbb{I}_m + Q + \cdots + Q^{k-1})Q \\ \mathbf{0}_{m,r}\mathbb{I}_r + Q^k\mathbf{0}_{m,r} & \mathbf{0}_{m,r}R + Q^k Q \end{pmatrix} \\ &= \begin{pmatrix} \mathbb{I}_r & R(\mathbb{I}_m + Q + Q^2 + \cdots + Q^k) \\ \mathbf{0}_{m,r} & Q^{k+1} \end{pmatrix} \end{aligned}$$

This completes the induction ■

Absorbing Markov chains

- Absorbing states and motivating examples
- Canonical block form and reducibility
- **The Neumann series and fundamental matrix**
- Examples and R implementation

Spectral radius of the transient block Q

Theorem 140 (Contraction of the transient block)

In any absorbing Markov chain with canonical transient block $Q \in \mathcal{M}_m$, the spectral radius satisfies

$$\rho(Q) < 1 \quad \textit{and consequently} \quad \lim_{k \to \infty} Q^k = \mathbf{0}_m$$

Why does this hold?

- By definition of an absorbing chain, from every transient state $j \in \{1, \ldots, m\}$ there is a directed walk to some absorbing state of length at most m
- Thus, starting from transient state j, the probability of being absorbed within m steps is strictly positive (> 0), so the probability of remaining in the transient states after m steps is strictly less than 1
- This means every column sum of Q^m is strictly less than 1: $\|Q^m\|_1 = \max_j \sum_{i=1}^{m} (Q^m)_{ij} < 1$
- Therefore $\rho(Q)^m = \rho(Q^m) \leq \|Q^m\|_1 < 1$, which implies $\rho(Q) < 1$ and $\lim_{k \to \infty} Q^k = \mathbf{0}$

Absorption occurs with probability 1

Because $(Q^k)_{ij}$ is the probability of being in transient state i at step k starting from transient state j, Theorem 140 immediately answers our first question

Theorem 141 (Eventual absorption)

In an absorbing Markov chain, the probability that the process eventually reaches an absorbing state is 1*, regardless of the initial state*

Indeed, starting from transient state j, the probability of still being in *some* transient state after k steps is the jth column sum $\sum_{i=1}^{m}(Q^k)_{ij} = (\mathbb{1}_m^T Q^k)_j \to 0$ as $k \to \infty$

The Neumann series for matrices

Recall the scalar geometric series: for $q \in \mathbb{R}$ with $|q| < 1$, $(1-q)^{-1} = \sum_{k=0}^{\infty} q^k$. The exact same identity holds for square matrices!

Theorem 142 (Neumann series)

Let $Q \in \mathcal{M}_m$ be a matrix with spectral radius $\rho(Q) < 1$ (equivalently, $\lim_{k\to\infty} Q^k = \mathbf{0}_m$). Then $\mathbb{I}_m - Q$ is invertible and its inverse is given by the convergent series

$$(\mathbb{I}_m - Q)^{-1} = \sum_{k=0}^{\infty} Q^k = \mathbb{I}_m + Q + Q^2 + Q^3 + \cdots$$

In particular, since $Q \geq 0$ for an absorbing Markov chain, every term $Q^k \geq 0$ is non-negative, so $(\mathbb{I}_m - Q)^{-1} \geq \mathbb{I}_m \geq 0$ has strictly non-negative entries (and diagonal entries ≥ 1)

Proof of Theorem 142 (Neumann series)

Consider the partial sum $S_k = \mathbb{I}_m + Q + Q^2 + \cdots + Q^{k-1}$ for $k \geq 1$. Multiplying by $\mathbb{I}_m - Q$ gives a telescoping sum:

$$\begin{aligned}(\mathbb{I}_m - Q)S_k &= (\mathbb{I}_m - Q)(\mathbb{I}_m + Q + Q^2 + \cdots + Q^{k-1}) \\ &= (\mathbb{I}_m + Q + Q^2 + \cdots + Q^{k-1}) - (Q + Q^2 + \cdots + Q^{k-1} + Q^k) \\ &= \mathbb{I}_m - Q^k\end{aligned}$$

First, suppose $(\mathbb{I}_m - Q)\boldsymbol{x} = \mathbf{0}$. Then $\boldsymbol{x} = Q\boldsymbol{x} = Q^2\boldsymbol{x} = \cdots = Q^k\boldsymbol{x} \to \mathbf{0}$ as $k \to \infty$ since $\rho(Q) < 1$. Hence $\text{null}(\mathbb{I}_m - Q) = \{\mathbf{0}\}$, so $\mathbb{I}_m - Q$ is invertible
Multiplying $(\mathbb{I}_m - Q)S_k = \mathbb{I}_m - Q^k$ on the left by $(\mathbb{I}_m - Q)^{-1}$ gives

$$S_k = (\mathbb{I}_m - Q)^{-1}(\mathbb{I}_m - Q^k) = (\mathbb{I}_m - Q)^{-1} - (\mathbb{I}_m - Q)^{-1}Q^k$$

Taking the limit as $k \to \infty$ and using $\lim_{k\to\infty} Q^k = \mathbf{0}_m$ yields

$$\sum_{j=0}^{\infty} Q^j = \lim_{k\to\infty} S_k = (\mathbb{I}_m - Q)^{-1} \qquad \blacksquare$$

The fundamental matrix and absorption quantities

Definition 143 (Fundamental matrix)

For an absorbing Markov chain with canonical transient block $Q \in \mathcal{M}_m$, the matrix

$$N = (\mathbb{I}_m - Q)^{-1} = \mathbb{I}_m + Q + Q^2 + \cdots \in \mathcal{M}_m$$

is called the ***fundamental matrix*** *of the absorbing Markov chain*

Theorem 144 (Quantitative analysis of absorbing Markov chains)

Let $P = \begin{pmatrix} \mathbb{I}_r & R \\ \mathbf{0}_{m,r} & Q \end{pmatrix}$ *be in canonical form with fundamental matrix* $N = [n_{ij}] = (\mathbb{I}_m - Q)^{-1}$. *Then:*

1. ***Expected visits to transient states:*** *n_{ij} is the expected total number of times the chain is in the ith transient state, given that it starts in the jth transient state*
2. ***Expected time to absorption:*** *Let $\boldsymbol{t}^T = \mathbb{1}_m^T N = (t_1, \ldots, t_m)$. Then $t_j = \sum_{i=1}^{m} n_{ij}$ (the jth column sum of N) is the expected number of steps before*

Proof of Theorem 144

1. Expected visits $N = (\mathbb{I}_m - Q)^{-1}$: Starting in transient state j, let $X_{ij}^{(k)} \in \{0, 1\}$ be the indicator random variable that equals 1 if the chain is in transient state i at step $k \geq 0$, and 0 otherwise. Then $\mathbb{E}[X_{ij}^{(k)}] = \mathbb{P}(\text{in transient state } i \text{ at step } k \mid \text{start in } j) = (Q^k)_{ij}$. By linearity of expectation, the expected total number of visits to transient state i is

$$\mathbb{E}\left[\sum_{k=0}^{\infty} X_{ij}^{(k)}\right] = \sum_{k=0}^{\infty} (Q^k)_{ij} = \left(\sum_{k=0}^{\infty} Q^k\right)_{ij} = n_{ij}$$

2. Time to absorption $t^T = \mathbb{1}_m^T N$: The total number of steps spent in *all* transient states before absorption starting from j is the sum over all $i = 1, \ldots, m$ of visits to transient state i:

$$t_j = \sum_{i=1}^{m} n_{ij} = (\mathbb{1}_m^T N)_j$$

3. Absorption probabilities $B = RN$: Taking $k \to \infty$ in Proposition 139 gives

$$\lim_{k\to\infty} P^k = \begin{pmatrix} \mathbb{I}_r & R\sum_{j=0}^{\infty} Q^j \\ \mathbf{0}_{m,r} & \lim_{k\to\infty} Q^k \end{pmatrix} = \begin{pmatrix} \mathbb{I}_r & RN \\ \mathbf{0}_{m,r} & \mathbf{0}_m \end{pmatrix} = \begin{pmatrix} \mathbb{I}_r & B \\ \mathbf{0}_{m,r} & \mathbf{0}_m \end{pmatrix} \quad \blacksquare$$

Absorbing Markov chains

- Absorbing states and motivating examples
- Canonical block form and reducibility
- The Neumann series and fundamental matrix
- **Examples and R implementation**

Example 1 – Back to the genetic example

In the modified orange cat breeding chain ($S_1 = X^O X^O$, $S_2 = X^o X^o$, $S_3 = X^O X^o$), state S_1 is absorbing ($r = 1$) and S_2, S_3 are transient ($m = 2$). The matrix is already in canonical form:

$$P = \begin{pmatrix} 1 & 0 & 1/2 \\ 0 & 0 & 0 \\ 0 & 1 & 1/2 \end{pmatrix} = \begin{pmatrix} \mathbb{I}_1 & R \\ \mathbf{0}_{2,1} & Q \end{pmatrix} \quad \text{with} \quad R = \begin{pmatrix} 0 & 1/2 \end{pmatrix}, \quad Q = \begin{pmatrix} 0 & 0 \\ 1 & 1/2 \end{pmatrix}$$

We compute $\mathbb{I}_2 - Q$ and invert it using $\begin{pmatrix} a & b \\ c & d \end{pmatrix}^{-1} = \frac{1}{ad-bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$:

$$\mathbb{I}_2 - Q = \begin{pmatrix} 1 & 0 \\ -1 & 1/2 \end{pmatrix} \implies N = (\mathbb{I}_2 - Q)^{-1} = \frac{1}{1/2} \begin{pmatrix} 1/2 & 0 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 2 & 2 \end{pmatrix}$$

Interpreting Example 1

The fundamental matrix for transient states $(S_2, S_3) = (X^o X^o, X^O X^o)$ is

$$N = \begin{pmatrix} 1 & 0 \\ 2 & 2 \end{pmatrix}$$

- Starting with a black female (S_2, column 1 of N): expected visits are $n_{11} = 1$ generation in S_2 (at $t = 0$) and $n_{21} = 2$ generations in S_3 (tortoiseshell)
- Starting with a tortoiseshell female (S_3, column 2 of N): $n_{12} = 0$ generations in S_2 and $n_{22} = 2$ generations in S_3

Expected number of generations before absorption into S_1 ($X^O X^O$):

$$\boldsymbol{t}^T = \mathbb{1}_2^T N = \begin{pmatrix} 1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 2 & 2 \end{pmatrix} = \begin{pmatrix} 3 & 2 \end{pmatrix}$$

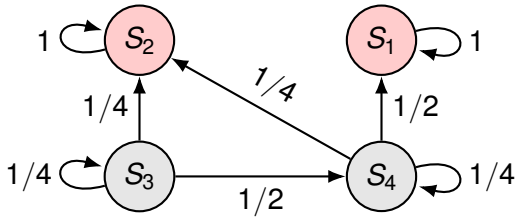
Absorption probability matrix (into the sole absorbing state S_1):

$$B = RN = \begin{pmatrix} 0 & 1/2 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 2 & 2 \end{pmatrix} = \begin{pmatrix} 1 & 1 \end{pmatrix}$$

Example 2 – 4-state chain with 2 absorbing states

Consider a 4-state absorbing chain where S_1 (pass/graduated) and S_2 (withdraw) are absorbing ($r = 2$), and S_3 (Year 1) and S_4 (Year 2) are transient ($m = 2$):

$$P = \begin{array}{c} \\ S_1 \\ S_2 \\ S_3 \\ S_4 \end{array} \begin{array}{c} \begin{array}{cccc} S_1 & S_2 & S_3 & S_4 \end{array} \\ \left(\begin{array}{cc|cc} 1 & 0 & 0 & 1/2 \\ 0 & 1 & 1/4 & 1/4 \\ \hline 0 & 0 & 1/4 & 0 \\ 0 & 0 & 1/2 & 1/4 \end{array}\right) \end{array}$$



Here the canonical blocks are

$$R = \begin{pmatrix} 0 & 1/2 \\ 1/4 & 1/4 \end{pmatrix} \in \mathcal{M}_2, \qquad Q = \begin{pmatrix} 1/4 & 0 \\ 1/2 & 1/4 \end{pmatrix} \in \mathcal{M}_2$$

Example 2 – Hand calculation of N, $\boldsymbol{t}^T$ and B

Step 1 (Fundamental matrix N): Compute $\mathbb{I}_2 - Q$ and its determinant:

$$\mathbb{I}_2 - Q = \begin{pmatrix} 3/4 & 0 \\ -1/2 & 3/4 \end{pmatrix}, \qquad \det(\mathbb{I}_2 - Q) = \frac{3}{4} \cdot \frac{3}{4} - 0 = \frac{9}{16}$$

Inverting via the 2×2 inverse formula gives

$$N = (\mathbb{I}_2 - Q)^{-1} = \frac{16}{9} \begin{pmatrix} 3/4 & 0 \\ 1/2 & 3/4 \end{pmatrix} = \begin{pmatrix} 4/3 & 0 \\ 8/9 & 4/3 \end{pmatrix}$$

Step 2 (Expected steps to absorption $\boldsymbol{t}^T = \mathbb{1}_2^T N$):

$$\boldsymbol{t}^T = \begin{pmatrix} 1 & 1 \end{pmatrix} \begin{pmatrix} 4/3 & 0 \\ 8/9 & 4/3 \end{pmatrix} = \begin{pmatrix} 20/9 & 4/3 \end{pmatrix} \quad \text{(starting from } S_3 \text{ and } S_4\text{)}$$

Step 3 (Absorption probabilities $B = RN$):

$$B = RN = \begin{pmatrix} 0 & 1/2 \\ 1/4 & 1/4 \end{pmatrix} \begin{pmatrix} 4/3 & 0 \\ 8/9 & 4/3 \end{pmatrix} = \begin{pmatrix} 4/9 & 2/3 \\ 5/9 & 1/3 \end{pmatrix}$$

Notice each column of B sums to 1: starting in S_3, a student graduates (S_1) with probability $4/9$ and withdraws (S_2) with probability $5/9$

Example 3 – Symmetric 5-state random walk

Return to Motivating example 1 on $\{S_1, S_2, S_3, S_4, S_5\}$ with $p = q = 1/2$ and absorbing walls S_1, S_5. Reordering states as $(S_1, S_5, S_2, S_3, S_4)$ puts P in canonical form with $r = 2$ and $m = 3$:

$$R = \begin{array}{c} \\ S_1 \\ S_5 \end{array} \begin{array}{c} \begin{array}{ccc} S_2 & S_3 & S_4 \end{array} \\ \begin{pmatrix} 1/2 & 0 & 0 \\ 0 & 0 & 1/2 \end{pmatrix} \end{array}, \qquad Q = \begin{array}{c} \\ S_2 \\ S_3 \\ S_4 \end{array} \begin{array}{c} \begin{array}{ccc} S_2 & S_3 & S_4 \end{array} \\ \begin{pmatrix} 0 & 1/2 & 0 \\ 1/2 & 0 & 1/2 \\ 0 & 1/2 & 0 \end{pmatrix} \end{array}$$

Inverting $\mathbb{I}_3 - Q = \begin{pmatrix} 1 & -1/2 & 0 \\ -1/2 & 1 & -1/2 \\ 0 & -1/2 & 1 \end{pmatrix}$ gives

$$N = (\mathbb{I}_3 - Q)^{-1} = \begin{pmatrix} 3/2 & 1 & 1/2 \\ 1 & 2 & 1 \\ 1/2 & 1 & 3/2 \end{pmatrix}, \quad \boldsymbol{t}^T = \mathbb{1}_3^T N = \begin{pmatrix} 3 & 4 & 3 \end{pmatrix}, \quad B = RN = \begin{pmatrix} 3/4 & 1/2 \\ 1/4 & 1/2 \end{pmatrix}$$

R implementation – Direct matrix calculation

We can verify Example 3 directly in base R using `solve`:

```
R_mat <- matrix(c(0.5, 0,   0,
                  0,   0,   0.5), nrow = 2, byrow = TRUE)
Q_mat <- matrix(c(0,   0.5, 0,
                  0.5, 0,   0.5,
                  0,   0.5, 0),   nrow = 3, byrow = TRUE)
colnames(R_mat) <- colnames(Q_mat) <- rownames(Q_mat) <- c("S2", "S3", "S4")
rownames(R_mat) <- c("S1", "S5")

N_mat <- solve(diag(3) - Q_mat)
t_vec <- colSums(N_mat)
B_mat <- R_mat %*% N_mat
N_mat

##     S2 S3  S4
## S2 1.5  1 0.5
## S3 1.0  2 1.0
```

Using the `markovchain` package

Note that `markovchain` uses row-stochastic matrices, so we pass `t(P_abs)`

```r
library(markovchain)
P_abs <- matrix(c(1, 0, 0.5,
                  0, 0, 0,
                  0, 1, 0.5), nrow = 3, byrow = TRUE)
mc_abs <- new("markovchain",
              states = sprintf("S_%d", 1:3),
              transitionMatrix = t(P_abs),
              name = "orange_abs")
summary(mc_abs)
```

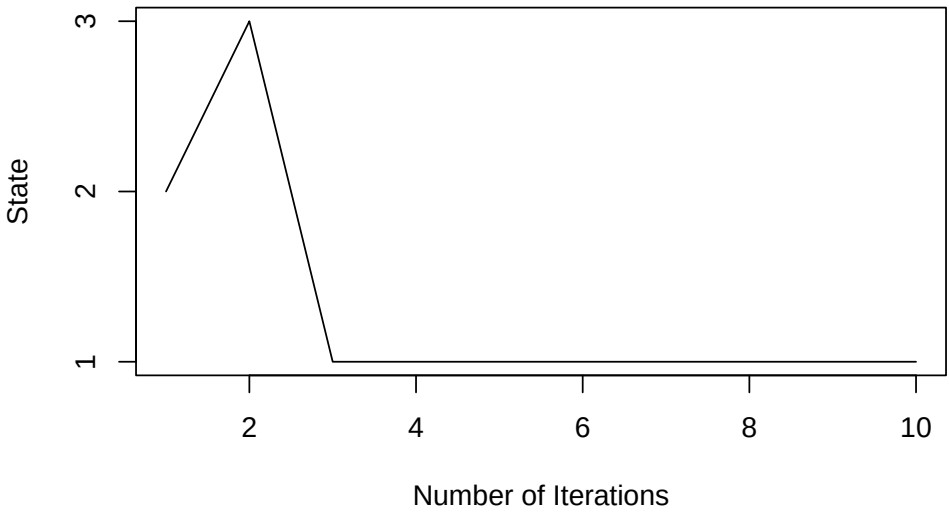
```
## orange_abs  Markov chain that is composed by:
## Closed classes:
## S_1
## Recurrent classes:
## {S_1}
## Transient classes:
```

Canonical form, absorption times and probabilities in R

```
canonicForm(mc_abs)

## orange_abs
##  A  3 - dimensional discrete Markov Chain defined by the following states
##  S_1, S_2, S_3
##  The transition matrix  (by rows)  is defined as follows:
##     S_1 S_2 S_3
## S_1 1.0   0 0.0
## S_2 0.0   0 1.0
## S_3 0.5   0 0.5

meanAbsorptionTime(mc_abs)

## S_2 S_3
##   3   2

absorptionProbabilities(mc_abs)
```

Simulating a realisation with DTMCPack

```
library(DTMCPack)
set.seed(2740)
IC <- c(0, 1, 0) # Start in S_2 (black female)
sol <- DTMC(t(P_abs), IC, 10, trace = TRUE)

## NULL
```


State
3
2
1
2
4
6
8
10
Number of Iterations

Absorbing Markov chains

Spectral centrality in networks

PageRank and the Google matrix

Spectral centrality in networks

Degree and eigenvector centrality

Issue with directed graphs and Katz centrality

Why spectral centrality?

Combinatorial centrality measures on graphs are often based on counting shortest paths (such as closeness and betweenness centrality)

Computational bottleneck on massive networks:

- All-pairs shortest-path algorithms require $O(nm)$ or $O(n^3)$ operations
- For a social network or web graph with $n \sim 10^8$ vertices, $n^2 \sim 10^{16}$ is completely intractable

The spectral alternative:

- Express vertex importance via matrix-vector products $A\boldsymbol{x}$, eigenvectors $A\boldsymbol{x} = \lambda\boldsymbol{x}$ or linear systems $(\mathbb{I} - \alpha A)\boldsymbol{x} = \boldsymbol{b}$
- When the adjacency matrix A is **sparse** ($m = |E| \ll n^2$ non-zero entries), one sparse matrix-vector product $A\boldsymbol{x}$ takes only $O(m)$ time!

Convention for directed adjacency matrices

To remain consistent with our column-stochastic Markov chain convention $\boldsymbol{p}(t+1) = P\boldsymbol{p}(t)$ where column j records transitions *from* j *to* i, we use the following convention for a directed graph $\mathcal{G} = (V, E)$ on n vertices:

$$a_{ij} = \begin{cases} 1 & \text{if there is a directed arc } j \to i \text{ (from } j \text{ to } i\text{)} \\ 0 & \text{otherwise} \end{cases}$$

With this convention:

- **Row** i of A records arcs *entering* vertex i, so the **in-degree** of vertex i is $d_i^{\text{in}} = \sum_{j=1}^{n} a_{ij} = (A\mathbb{1}_n)_i$
- **Column** j of A records arcs *leaving* vertex j, so the **out-degree** of vertex j is $d_j^{\text{out}} = \sum_{i=1}^{n} a_{ij} = (\mathbb{1}_n^T A)_j$
- For an undirected graph, $A = A^T$ is symmetric and $d_i^{\text{in}} = d_i^{\text{out}} = d_i$

First-order measure – In-degree centrality

The simplest measure of prestige in a directed network is **in-degree centrality**:

$$\boldsymbol{d}^{\text{in}} = A\mathbb{1}_n, \qquad d_i^{\text{in}} = \sum_{j=1}^{n} a_{ij}$$

Each arc $j \to i$ is treated as one "vote" for the importance of vertex i

Fundamental limitation:

- In-degree treats all incoming links as equal
- Receiving a link from a major hub or authoritative page counts the exact same $(+1)$ as receiving a link from an isolated spam page
- We want a **recursive** notion of prestige: *a vertex is important if it is pointed to by other important vertices*

Eigenvector centrality

Instead of summing 1 for each incoming neighbour $j \to i$, we sum the *importance scores* x_j of those neighbours and require x_i to be proportional to that sum

Definition 145 (Eigenvector centrality)

Let $A = [a_{ij}] \in \mathcal{M}_n$ be the adjacency matrix of a graph or digraph (with $a_{ij} = 1$ if $j \to i$). The ***eigenvector centrality*** *(or* ***prestige score****) of vertex i is the ith entry of the non-negative eigenvector $\boldsymbol{x} \geq \mathbf{0}$ satisfying*

$$\lambda x_i = \sum_{j=1}^{n} a_{ij} x_j, \quad i = 1, \ldots, n \iff A\boldsymbol{x} = \lambda \boldsymbol{x}$$

associated with the dominant eigenvalue $\lambda = \rho(A)$

Notice that $x_i = \frac{1}{\lambda} \sum_{j \in \mathcal{N}^{\text{in}}(i)} x_j$ depends on both the **quantity** of in-neighbours $|\mathcal{N}^{\text{in}}(i)|$ and the **quality** x_j of each in-neighbour

Why Perron-Frobenius theory is essential here

For $\boldsymbol{x}$ to be a valid centrality ranking, its entries must be **non-negative** ($x_i \geq 0$) and uniquely determined (up to normalisation $\mathbb{1}_n^T \boldsymbol{x} = 1$)

Theorem 146 (Existence and uniqueness of eigenvector centrality)

Let $\mathcal{G} = (V, E)$ be a strongly connected digraph (or connected undirected graph) with adjacency matrix $A \in \mathcal{M}_n$. Then A is irreducible and non-negative, so by the Perron-Frobenius theorem (Unit 08):

1. *The spectral radius $\lambda_1 = \rho(A) > 0$ is a simple eigenvalue of A*
2. *There exists a unique eigenvector $\boldsymbol{x} > \mathbf{0}$ (the **Perron eigenvector**) satisfying*

$$A\boldsymbol{x} = \rho(A)\boldsymbol{x} \quad \text{and} \quad \mathbb{1}_n^T \boldsymbol{x} = \sum_{i=1}^{n} x_i = 1$$

3. *No other eigenvalue has a non-negative eigenvector*

Constructing a stochastic matrix from an adjacency matrix

Let $A = [a_{ij}] \in \mathcal{M}_n$ be the adjacency matrix of a simple undirected graph $\mathcal{G} = (V, E)$ with no isolated vertices, and let $D = \text{diag}(d_{11}, \ldots, d_{nn})$ be its degree matrix, with diagonal entries

$$d_{ii} = \sum_{j=1}^{n} a_{ji} = \sum_{j=1}^{n} a_{ij} > 0$$

Proposition 147 (Random walk transition matrix on an undirected graph)

*The matrix $P = AD^{-1}$ is **column-stochastic**, and defines the transition matrix of a random walk on $\mathcal{G}$ where a walker at vertex j moves to a uniformly chosen neighbour $i \in \mathcal{N}(j)$ with probability $1/d_{jj}$*

Proof of Proposition 147

Write the entries of the diagonal matrix D^{-1} as d_{ij}^{-1}, where $d_{ii}^{-1} = 1/d_{ii}$ for $1 \leq i \leq n$ and $d_{ij}^{-1} = 0$ for $i \neq j$

By matrix multiplication, the (i, j) entry of AD^{-1} is

$$(AD^{-1})_{ij} = \sum_{k=1}^{n} a_{ik} d_{kj}^{-1} = a_{ij} d_{jj}^{-1} = \frac{a_{ij}}{d_{jj}} \geq 0, \qquad i, j = 1, \ldots, n$$

Therefore, the sum of column j of AD^{-1} is

$$\begin{aligned} \sum_{i=1}^{n} a_{ij} d_{jj}^{-1} &= d_{jj}^{-1} \sum_{i=1}^{n} a_{ij} \\ &= d_{jj}^{-1} d_{jj} = 1 \end{aligned}$$

Hence every column sums to 1 and AD^{-1} is column-stochastic ■

Computing eigenvector centrality – The power method

For large networks, computing the full characteristic polynomial is impossible. Instead, we compute the dominant eigenpair $(\lambda_1, \boldsymbol{x})$ using the **power method**: Start with an initial positive vector $\boldsymbol{x}^{(0)} > \mathbf{0}$ (such as $\boldsymbol{x}^{(0)} = \frac{1}{n}\mathbb{1}_n$). For $k = 1, 2, \ldots$:

1. Multiply by A: $\boldsymbol{y}^{(k)} = A\boldsymbol{x}^{(k-1)}$
2. Normalise (using the ℓ_1 norm $\|\boldsymbol{y}^{(k)}\|_1 = \mathbb{1}_n^T \boldsymbol{y}^{(k)}$ or the maximum-magnitude entry $m(\boldsymbol{y}^{(k)})$):

$$\boldsymbol{x}^{(k)} = \frac{\boldsymbol{y}^{(k)}}{\|\boldsymbol{y}^{(k)}\|_1}$$

until $\|\boldsymbol{x}^{(k)} - \boldsymbol{x}^{(k-1)}\|_1 < \varepsilon$

Theorem 148 (Convergence of the power method)

If A is primitive (so $\lambda_1 = \rho(A) > |\lambda_2| \geq \cdots \geq |\lambda_n|$), then $\boldsymbol{x}^{(k)} \to \boldsymbol{x}$ as $k \to \infty$, and the error decays at the geometric rate

$$\|\boldsymbol{x}^{(k)} - \boldsymbol{x}\| = O\left(\left|\frac{\lambda_2}{\lambda_1}\right|^k\right)$$

Spectral centrality in networks

Degree and eigenvector centrality

Issue with directed graphs and Katz centrality

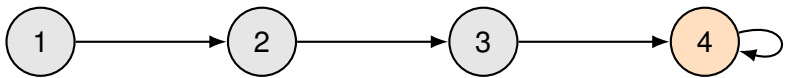
What goes wrong on general directed graphs?

Real-world directed networks (citation graphs, web graphs, DAGs) are rarely strongly connected! When $\mathcal{G}(A)$ is **reducible**, eigenvector centrality suffers from a severe pathology:

- Suppose vertex i is a **source node** (in-degree $d_i^{\text{in}} = 0$, so nobody points to i)
- Then row i of A is all zeros, so the eigenvector equation $A\boldsymbol{x} = \lambda \boldsymbol{x}$ with $\lambda > 0$ gives

$$\lambda x_i = \sum_{j=1}^{n} a_{ij} x_j = 0 \implies x_i = 0$$

- **Domino effect:** If vertex k only receives links from source nodes with $x_i = 0$, then $\lambda x_k = \sum_{j \to k} x_j = 0 \implies x_k = 0$ as well!



Here $x_1 = 0 \implies x_2 = 0 \implies x_3 = 0$, and only vertex 4 gets non-zero eigenvector centrality ($x_4 = 1$)!

Katz centrality (1953) – Baseline “free” importance

Leo Katz (1953) resolved the zero-propagation problem on directed graphs by giving every vertex a small baseline “free” importance $\beta > 0$, plus damped importance $\alpha > 0$ from its in-neighbours

Definition 149 (Katz centrality)

*Let $A \in \mathcal{M}_n$ be the adjacency matrix of a directed graph, $\alpha > 0$ be an **attenuation factor** (or damping parameter) and $\beta > 0$ be a baseline constant. The **Katz centrality** vector $\boldsymbol{x} \in \mathbb{R}^n$ is the solution to*

$$x_i = \alpha \sum_{j=1}^{n} a_{ij} x_j + \beta, \quad i = 1, \ldots, n \iff \boldsymbol{x} = \alpha A \boldsymbol{x} + \beta \mathbb{1}_n$$

Rearranging terms gives the linear system

$$(\mathbb{I}_n - \alpha A)\boldsymbol{x} = \beta \mathbb{1}_n$$

Since β only scales $\boldsymbol{x}$ by a positive constant, we typically set $\beta = 1$ (or normalise $\boldsymbol{x}$

Neumann series and walk-counting interpretation of Katz centrality

When is $\mathbb{I}_n - \alpha A$ guaranteed to be invertible with a strictly positive solution $\boldsymbol{x} > \mathbf{0}$?

Theorem 150 (Existence and walk-counting expansion of Katz centrality)

Let $A \in \mathcal{M}_n$ be a non-negative adjacency matrix and suppose $0 < \alpha < \frac{1}{\rho(A)}$ and $\beta > 0$. Then $\rho(\alpha A) = \alpha\rho(A) < 1$, so by Theorem 142 (the Neumann series), $\mathbb{I}_n - \alpha A$ is invertible and

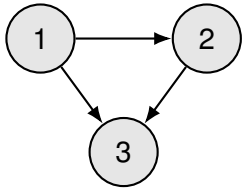
$$\boldsymbol{x} = \beta(\mathbb{I}_n - \alpha A)^{-1}\mathbb{1}_n = \beta\left(\sum_{k=0}^{\infty} \alpha^k A^k\right)\mathbb{1}_n = \beta\left(\mathbb{1}_n + \alpha A\mathbb{1}_n + \alpha^2 A^2\mathbb{1}_n + \cdots\right) > \mathbf{0}$$

Walk-counting interpretation:

- Recall from Unit 07 that $(A^k)_{ij}$ is the number of directed walks of length k from j to i
- Therefore $(A^k\mathbb{1}_n)_i = \sum_{j=1}^{n}(A^k)_{ij}$ is the total number of directed walks of length k from *any* vertex j terminating at vertex i
- Katz centrality sums walks of **all lengths** $k \geq 0$ ending at i, discounted by α^k:

Example – Katz centrality on a 3-vertex digraph

Consider the 3-vertex directed graph with arcs $1 \to 2$, $1 \to 3$ and $2 \to 3$:



With $a_{ij} = 1$ if $j \to i$, the adjacency matrix is strictly lower triangular:

$$A = \begin{pmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 1 & 1 & 0 \end{pmatrix}$$

Here $\rho(A) = 0$, so any $\alpha > 0$ satisfies $\rho(\alpha A) = 0 < 1$

Let us compute Katz centrality with $\alpha = 1/2$ and $\beta = 1$ by solving $(\mathbb{I}_3 - \frac{1}{2}A)\boldsymbol{x} = \mathbb{1}_3$:

$$\mathbb{I}_3 - \frac{1}{2}A = \begin{pmatrix} 1 & 0 & 0 \\ -1/2 & 1 & 0 \\ -1/2 & -1/2 & 1 \end{pmatrix} \implies \begin{cases} x_1 = 1 \\ x_2 = \frac{1}{2}x_1 + 1 = \frac{3}{2} \\ x_3 = \frac{1}{2}x_1 + \frac{1}{2}x_2 + 1 = \frac{1}{2} + \frac{3}{4} + 1 = \frac{9}{4} \end{cases}$$

Normalising $\boldsymbol{x} = (1, 3/2, 9/4)^T = \frac{1}{4}(4, 6, 9)^T$ by $\mathbb{1}_3^T \boldsymbol{x} = 19/4$ gives $\tilde{\boldsymbol{x}} = (\frac{4}{19}, \frac{6}{19}, \frac{9}{19})^T$, ranking $3 \succ 2 \succ 1$

Absorbing Markov chains

Spectral centrality in networks

PageRank and the Google matrix

PageRank and the Google matrix

Motivation and vote splitting

Obstacles on real web graphs and the Google matrix

Linear system formulation and examples

What makes an important webpage?

In the early days of the World Wide Web (mid-1990s), search engines ranked pages primarily by keyword frequency on the page itself, which was easily manipulated

In 1998, Sergey Brin, Larry Page, Rajeev Motwani and Terry Winograd introduced **PageRank**, the algorithm at the heart of Google

Core idea: Use the directed hyperlink structure of the Web to measure authority

- Webpages are vertices $V = \{1, \ldots, n\}$; a hyperlink from page j to page i is a directed arc $j \to i$
- A link $j \to i$ is a recommendation or "vote" from page j for the importance of page i
- **Random surfer model:** Imagine a web surfer who clicks links at random from page to page. The PageRank of page i is the long-run proportion of time the random surfer spends on page i

From Katz centrality to vote splitting

Why not simply use Katz or eigenvector centrality on the Web adjacency matrix A?

The "generous hub" flaw of Katz/eigenvector centrality:

- In Katz centrality ($x_i = \alpha \sum_{j \to i} x_j + \beta$), if an important directory page j links to 10,000 pages, *every single one* of those 10,000 pages receives αx_j importance!
- A page with more outgoing links exerts more total influence on the network

Brin and Page's vote-splitting principle:

- Each page j has a single unit of voting power equal to its own importance x_j
- If page j has $d_j^{\text{out}} > 0$ outgoing links, it splits its importance **equally** among them, passing x_j / d_j^{out} along each outgoing link:

$$x_i = \sum_{j \to i} \frac{x_j}{d_j^{\text{out}}}$$

The hyperlink transition matrix

Let $A = [a_{ij}] \in \mathcal{M}_n$ be the directed adjacency matrix ($a_{ij} = 1$ if page j links to page i, with no self-loops). Suppose first that every page has at least one outgoing link ($d_j^{\text{out}} = \sum_{i=1}^{n} a_{ij} \geq 1$)

Define the diagonal out-degree matrix $D^{\text{out}} = \text{diag}(d_1^{\text{out}}, \ldots, d_n^{\text{out}})$ and the **hyperlink matrix** $P = [p_{ij}] \in \mathcal{M}_n$:

$$p_{ij} = \begin{cases} \dfrac{1}{d_j^{\text{out}}} & \text{if page } j \text{ links to page } i \\ 0 & \text{otherwise} \end{cases} \qquad \Longleftrightarrow \qquad P = A(D^{\text{out}})^{-1}$$

Proposition 151

*If $d_j^{out} \geq 1$ for all $j = 1, \ldots, n$, then $P = A(D^{out})^{-1}$ is **column-stochastic**, and the vote-splitting equations $x_i = \sum_{j \to i} x_j / d_j^{out}$ take the fixed-point form*

$$P\boldsymbol{x} = \boldsymbol{x}$$

PageRank and the Google matrix

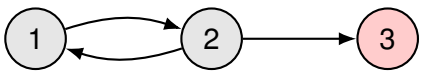
- Motivation and vote splitting
- **Obstacles on real web graphs and the Google matrix**
- Linear system formulation and examples

Two linear algebra obstacles on real web graphs

If we try to solve $P\boldsymbol{x} = \boldsymbol{x}$ directly on the real Web, two fatal problems occur:

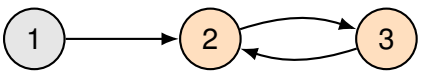
Obstacle 1: Dangling nodes ($d_j^{\text{out}} = 0$)

- Many pages (PDF files, images, dead-end pages) have **no outgoing links**: $d_j^{\text{out}} = 0$
- Column j of the raw hyperlink matrix is **0**, so probability mass "leaks" out of the network and P is not stochastic!



Obstacle 2: Reducibility and spider traps

- The Web is not strongly connected ($\mathcal{G}(P)$ is reducible) and contains **spider traps** (closed strongly connected components) and periodic cycles
- $P\boldsymbol{x} = \boldsymbol{x}$ may have multiple linearly independent solutions, assign $x_i = 0$ outside traps, or cause power iteration to oscillate!



Fixing Obstacle 1 – Handling dangling nodes

Before forming the Google matrix, we must ensure that P is genuinely column-stochastic ($\mathbb{1}_n^T P = \mathbb{1}_n^T$)

Whenever a vertex j is a **dangling node** ($d_j^{\text{out}} = 0$, giving a zero column in the raw hyperlink matrix), there are two standard conventions:

1. **Uniform teleportation from dead ends:** Replace the zero column j with the uniform probability vector $\frac{1}{n}\mathbb{1}_n = (1/n, \ldots, 1/n)^T$ (a random surfer reaching a dead-end page picks any page on the Web uniformly at random)
2. **Self-loop at dead ends:** Add a self-loop $j \to j$ so that $p_{jj} = 1$ (column j becomes $\boldsymbol{e}_j$)

In both conventions, every column of the resulting matrix P is a valid probability vector ($P \geq 0$ and $\mathbb{1}_n^T P = \mathbb{1}_n^T$). From now on, $P \in \mathcal{M}_n$ denotes this column-stochastic web transition matrix

Fixing Obstacle 2 – Teleportation and the Google matrix

To overcome reducibility, spider traps and periodicity, Brin and Page introduced a **damping factor** (or teleportation parameter) $\alpha \in (0, 1)$, typically chosen as $\alpha = 0.85$:

- With probability α, the random surfer follows an outgoing link on the current page (according to P)
- With probability $1 - \alpha$, the surfer gets bored and jumps (teleports) to a page chosen uniformly at random from all n pages (according to $\frac{1}{n}\mathbb{1}_n$)

Definition 152 (Google matrix)

*Let $P \in \mathcal{M}_n$ be a column-stochastic transition matrix and let $\alpha \in (0, 1)$ be a damping factor. The **Google matrix** is the $n \times n$ matrix*

$$G = \alpha P + (1 - \alpha)\frac{1}{n}\mathbb{1}_n\mathbb{1}_n^T = \alpha P + \frac{1 - \alpha}{n}\mathbb{J}_n$$

where $\mathbb{J}_n = \mathbb{1}_n\mathbb{1}_n^T \in \mathcal{M}_n$ is the $n \times n$ matrix of all ones

Existence, uniqueness and convergence of PageRank

Theorem 153 (PageRank theorem)

Let $P \in \mathcal{M}_n$ be a column-stochastic matrix and let $\alpha \in (0,1)$. Then the Google matrix

$$G = \alpha P + (1-\alpha)\frac{1}{n}\mathbb{1}_n\mathbb{1}_n^T$$

*is **column-stochastic** and **strictly positive** ($G > 0$), hence **primitive**. By Perron's theorem (Unit 08):*

1. *The spectral radius $\lambda_1 = \rho(G) = 1$ is a simple, strictly dominant eigenvalue of G (in fact, $|\lambda| \leq \alpha < 1$ for every other eigenvalue of G)*
2. *There exists a unique stationary probability vector $\boldsymbol{x} > \mathbf{0}$ (the **PageRank vector**) satisfying*
$$G\boldsymbol{x} = \boldsymbol{x} \quad \text{and} \quad \mathbb{1}_n^T\boldsymbol{x} = 1$$
3. *For any initial probability distribution $\boldsymbol{x}^{(0)}$, the power iteration $\boldsymbol{x}^{(k)} = G^k\boldsymbol{x}^{(0)}$ converges to $\boldsymbol{x}$ as $k \to \infty$ at the geometric rate $O(\alpha^k)$*

Proof of Theorem 153

1. Strict positivity ($G > 0$): Since $P \geq 0$ and $\alpha \in (0, 1)$, we have $\alpha P \geq 0$. Every entry of $\frac{1-\alpha}{n}\mathbb{J}_n$ equals $\frac{1-\alpha}{n} > 0$. Therefore, every entry of G satisfies

$$g_{ij} = \alpha p_{ij} + \frac{1-\alpha}{n} \geq \frac{1-\alpha}{n} > 0$$

so $G > 0$ is strictly positive (and therefore irreducible and primitive, as $G^1 > 0$)

2. Column-stochasticity ($\mathbb{1}_n^T G = \mathbb{1}_n^T$): Since $\mathbb{1}_n^T P = \mathbb{1}_n^T$ and $\mathbb{1}_n^T \mathbb{1}_n = n$, we compute

$$\mathbb{1}_n^T G = \alpha(\mathbb{1}_n^T P) + \frac{1-\alpha}{n}(\mathbb{1}_n^T \mathbb{1}_n)\mathbb{1}_n^T = \alpha \mathbb{1}_n^T + \frac{1-\alpha}{n} n \mathbb{1}_n^T = (\alpha + 1 - \alpha)\mathbb{1}_n^T = \mathbb{1}_n^T$$

Thus G is column-stochastic, which implies $\rho(G) = 1$

3. Application of Perron's theorem: Since $G > 0$ is a positive column-stochastic matrix, Perron's theorem guarantees that $\lambda_1 = \rho(G) = 1$ is simple and strictly dominant, with a unique positive eigenvector $\boldsymbol{x} > \mathbf{0}$ normalised by $\mathbb{1}_n^T \boldsymbol{x} = 1$, and $\lim_{k\to\infty} G^k \boldsymbol{x}^{(0)} = \boldsymbol{x}$ ■

PageRank and the Google matrix

- Motivation and vote splitting
- Obstacles on real web graphs and the Google matrix
- **Linear system formulation and examples**

Reformulating PageRank as a linear system

Although $G\boldsymbol{x} = \boldsymbol{x}$ is an eigenvalue problem for the eigenvalue $\lambda = 1$, we can use the normalisation condition $\mathbb{1}_n^T \boldsymbol{x} = 1$ to turn it into a **non-homogeneous linear system** $(\mathbb{I}_n - \alpha P)\boldsymbol{x} = \frac{1-\alpha}{n}\mathbb{1}_n$!

Theorem 154 (Linear system formulation of PageRank)

Let $P \in \mathcal{M}_n$ be column-stochastic and $\alpha \in (0, 1)$. A vector $\boldsymbol{x} \in \mathbb{R}^n$ with $\mathbb{1}_n^T \boldsymbol{x} = 1$ satisfies $G\boldsymbol{x} = \boldsymbol{x}$ if and only if it satisfies the linear system

$$(\mathbb{I}_n - \alpha P)\boldsymbol{x} = \frac{1-\alpha}{n}\mathbb{1}_n$$

Moreover, $\mathbb{I}_n - \alpha P$ is invertible and its inverse is given by the Neumann series $(\mathbb{I}_n - \alpha P)^{-1} = \sum_{k=0}^{\infty} \alpha^k P^k$, so the unique PageRank vector is

$$\boldsymbol{x} = \frac{1-\alpha}{n}(\mathbb{I}_n - \alpha P)^{-1}\mathbb{1}_n$$

Proof of Theorem 154

1. Equivalence with $G\boldsymbol{x} = \boldsymbol{x}$: For any vector $\boldsymbol{x}$ satisfying $\mathbb{1}_n^T \boldsymbol{x} = 1$, associativity of matrix multiplication gives

$$\mathbb{J}_n \boldsymbol{x} = (\mathbb{1}_n \mathbb{1}_n^T)\boldsymbol{x} = \mathbb{1}_n(\mathbb{1}_n^T \boldsymbol{x}) = \mathbb{1}_n(1) = \mathbb{1}_n$$

Substituting this into $G\boldsymbol{x} = \boldsymbol{x}$ gives

$$G\boldsymbol{x} = \boldsymbol{x} \iff \alpha P\boldsymbol{x} + \frac{1-\alpha}{n}\mathbb{1}_n\mathbb{1}_n^T\boldsymbol{x} = \boldsymbol{x} \iff \alpha P\boldsymbol{x} + \frac{1-\alpha}{n}\mathbb{1}_n = \boldsymbol{x} \iff (\mathbb{I}_n - \alpha P)\boldsymbol{x} = \frac{1-\alpha}{n}\mathbb{1}_n$$

2. Invertibility of $\mathbb{I}_n - \alpha P$: Since P is column-stochastic, $\rho(P) = 1$. Because $\alpha \in (0,1)$, we have $\rho(\alpha P) = \alpha\rho(P) = \alpha < 1$. By Theorem 142 (the Neumann series), $\mathbb{I}_n - \alpha P$ is invertible with $(\mathbb{I}_n - \alpha P)^{-1} = \sum_{k=0}^{\infty} \alpha^k P^k \geq \mathbf{0}$

3. Automatic normalisation ($\mathbb{1}_n^T \boldsymbol{x} = 1$): Left-multiplying $(\mathbb{I}_n - \alpha P)\boldsymbol{x} = \frac{1-\alpha}{n}\mathbb{1}_n$ by $\mathbb{1}_n^T$ and using $\mathbb{1}_n^T P = \mathbb{1}_n^T$ yields

$$\mathbb{1}_n^T(\mathbb{I}_n - \alpha P)\boldsymbol{x} = (1-\alpha)\mathbb{1}_n^T\boldsymbol{x} = \frac{1-\alpha}{n}\mathbb{1}_n^T\mathbb{1}_n = 1 - \alpha \implies \mathbb{1}_n^T\boldsymbol{x} = 1 \qquad \blacksquare$$

Unifying thread of Unit 09 – The resolvent $(\mathbb{I} - M)^{-1}$

Notice how all three topics in this unit are governed by the exact same matrix inverse (the **Neumann series**):

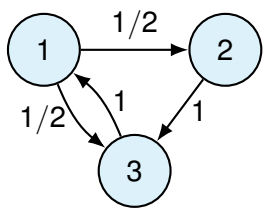
Problem	**Matrix** M	**Condition** $\rho(M) < 1$	**Key quant**
Absorbing Markov chains	Transient block Q	Absorption reachable	$N = (\mathbb{I}_m - Q$
Katz centrality	αA	$0 < \alpha < 1/\rho(A)$	$\boldsymbol{x} = \beta(\mathbb{I}_n - \alpha A$
PageRank	αP	$0 < \alpha < 1$ $(\rho(P) = 1)$	$\boldsymbol{x} = \frac{1-\alpha}{n}(\mathbb{I}_n - \alpha$

For hand calculations on small graphs ($n = 3$ or 4), solving the linear system $(\mathbb{I}_n - \alpha P)\boldsymbol{x} = \frac{1-\alpha}{n}\mathbb{1}_n$ is much faster than computing the dense matrix G and finding its nullspace!

Worked exam-style example 1 – 3-page web digraph

Consider a 3-page miniature web where:

- Page 1 links to Page 2 and Page 3 ($d_1^{\text{out}} = 2$)
- Page 2 links to Page 3 ($d_2^{\text{out}} = 1$)
- Page 3 links to Page 1 ($d_3^{\text{out}} = 1$)



The column-stochastic transition matrix $P = [p_{ij}]$ (where $p_{ij} = 1/d_j^{\text{out}}$ if $j \to i$) is

$$P = \begin{pmatrix} 0 & 0 & 1 \\ 1/2 & 0 & 0 \\ 1/2 & 1 & 0 \end{pmatrix}$$

With damping factor $\alpha = 1/2$, the Google matrix is

$$G = \frac{1}{2}P + \frac{1}{6}\mathbb{J}_3 = \begin{pmatrix} 1/6 & 1/6 & 2/3 \\ 5/12 & 1/6 & 1/6 \\ 5/12 & 2/3 & 1/6 \end{pmatrix}$$

Worked exam-style example 1 – Solving $(\mathbb{I}_3 - \alpha P)\boldsymbol{x} = \frac{1-\alpha}{3}\mathbb{1}_3$

Let us compute the PageRank vector $\boldsymbol{x}$ for $\alpha = 1/2$ by solving $(\mathbb{I}_3 - \frac{1}{2}P)\boldsymbol{x} = \frac{1}{6}\mathbb{1}_3$:

$$\mathbb{I}_3 - \frac{1}{2}P = \begin{pmatrix} 1 & 0 & -1/2 \\ -1/4 & 1 & 0 \\ -1/4 & -1/2 & 1 \end{pmatrix} \implies \begin{cases} x_1 - \frac{1}{2}x_3 = \frac{1}{6} & \text{(Row 1)} \\ -\frac{1}{4}x_1 + x_2 = \frac{1}{6} & \text{(Row 2)} \\ -\frac{1}{4}x_1 - \frac{1}{2}x_2 + x_3 = \frac{1}{6} & \text{(Row 3)} \end{cases}$$

Express x_2 and x_3 in terms of x_1 using Rows 1 and 2:

$$x_2 = \frac{1}{4}x_1 + \frac{1}{6}, \qquad x_3 = 2x_1 - \frac{1}{3}$$

Substitute into the normalisation condition $x_1 + x_2 + x_3 = 1$:

$$x_1 + \left(\frac{1}{4}x_1 + \frac{1}{6}\right) + \left(2x_1 - \frac{1}{3}\right) = 1 \implies \frac{13}{4}x_1 = \frac{7}{6} \implies x_1 = \frac{14}{39}$$

Then $x_2 = \frac{1}{4}\left(\frac{14}{39}\right) + \frac{1}{6} = \frac{10}{39}$ and $x_3 = 2\left(\frac{14}{39}\right) - \frac{13}{39} = \frac{15}{39}$, so

$$\boldsymbol{x} = \begin{pmatrix} 14/39 \\ 10/39 \\ 15/39 \end{pmatrix} \approx \begin{pmatrix} 0.359 \\ 0.256 \\ 0.385 \end{pmatrix} \implies \text{Ranking: Page 3} \succ \text{Page 1} \succ \text{Page 2}$$

Why Page 1 ranks almost as high as Page 3

Look closely at the ranking in our 3-page example:

- **In-degrees:** Page 3 has in-degree 2 (from 1 and 2), while Page 1 and Page 2 both have in-degree 1
- By raw in-degree, Page 1 and Page 2 would be tied!
- However, in PageRank, Page 1 receives the **undivided vote** of the most important page (Page 3, with $d_3^{\text{out}} = 1$), whereas Page 2 only receives **half a vote** from Page 1 ($d_1^{\text{out}} = 2$)
- Consequently, Page 1 ($x_1 = 14/39$) ranks far above Page 2 ($x_2 = 10/39$) and nearly matches Page 3 ($x_3 = 15/39$)

What if we use the standard Google damping factor $\alpha = 0.85$ on the same 3-page network?

$$(\mathbb{I}_3 - 0.85P)\boldsymbol{x} = \frac{0.15}{3}\mathbb{1}_3 \implies \boldsymbol{x} \approx \begin{pmatrix} 0.388 \\ 0.215 \\ 0.397 \end{pmatrix}$$

Worked exam-style example 2 – 4-page web with a dangling node

Suppose we add a dangling Page 4 ($d_4^{\text{out}} = 0$) to a 4-page web where $1 \to 2$, $1 \to 3$, $2 \to 3$, $3 \to 1$ and $2 \to 4$ (so $d_1^{\text{out}} = 2$, $d_2^{\text{out}} = 2$, $d_3^{\text{out}} = 1$ and $d_4^{\text{out}} = 0$):

$$P_{\text{raw}} = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 1/2 & 0 & 0 & 0 \\ 1/2 & 1/2 & 0 & 0 \\ 0 & 1/2 & 0 & 0 \end{pmatrix} \xrightarrow{\text{replace zero col 4 by } \frac{1}{4}\mathbb{1}_4} P = \begin{pmatrix} 0 & 0 & 1 & 1/4 \\ 1/2 & 0 & 0 & 1/4 \\ 1/2 & 1/2 & 0 & 1/4 \\ 0 & 1/2 & 0 & 1/4 \end{pmatrix}$$

Notice why we can simplify $(\mathbb{I}_4 - \alpha P)\boldsymbol{x} = \frac{1-\alpha}{4}\mathbb{1}_4$ even faster when column 4 is $\frac{1}{4}\mathbb{1}_4$:

- Write $P = P_{\text{raw}} + \frac{1}{4}\mathbb{1}_4 \boldsymbol{e}_4^T$, so $P\boldsymbol{x} = P_{\text{raw}}\boldsymbol{x} + \frac{x_4}{4}\mathbb{1}_4$
- Then $G\boldsymbol{x} = \boldsymbol{x} \iff (\mathbb{I}_4 - \alpha P_{\text{raw}})\boldsymbol{x} = \left(\frac{1-\alpha}{4} + \frac{\alpha x_4}{4}\right)\mathbb{1}_4 = c\mathbb{1}_4$, where P_{raw} is sparse!
- Thus we can solve $(\mathbb{I}_4 - \alpha P_{\text{raw}})\boldsymbol{y} = \mathbb{1}_4$ by sparse back-substitution and normalise $\boldsymbol{x} = \boldsymbol{y}/(\mathbb{1}_4^T \boldsymbol{y})$!

Computing PageRank at web scale – Sparse power iteration

For the real Web ($n \sim 10^{10}$ pages), how is PageRank computed in practice?

- **Direct Gaussian elimination** on $(\mathbb{I}_n - \alpha P)\boldsymbol{x} = \frac{1-\alpha}{n}\mathbb{1}_n$ costs $O(n^3)$ time and $O(n^2)$ memory due to fill-in, which is completely impossible for $n = 10^{10}$
- **Forming the dense Google matrix** G explicitly also requires n^2 entries because $\mathbb{J}_n$ is completely dense!
- **Matrix-free power iteration:** Starting from $\boldsymbol{x}^{(0)} = \frac{1}{n}\mathbb{1}_n$, for any probability vector $\boldsymbol{x}^{(k)}$ ($\mathbb{1}_n^T \boldsymbol{x}^{(k)} = 1$) we compute

$$\boldsymbol{x}^{(k+1)} = G\boldsymbol{x}^{(k)} = \alpha P\boldsymbol{x}^{(k)} + \frac{1-\alpha}{n}\mathbb{1}_n$$

Why is this fast?

- We **never** form G: each step only requires one sparse matrix-vector multiplication $P\boldsymbol{x}^{(k)}$ ($O(m)$ operations where $m \approx 10n$ is the number of hyperlinks) plus adding the scalar $\frac{1-\alpha}{n}$ to each entry!
- Since $|\lambda_2(G)| \leq \alpha = 0.85$, after $k = 50$ iterations the error is bounded by $0.85^{50} \approx 3 \times 10^{-4}$

R implementation – Checking our 3-page example

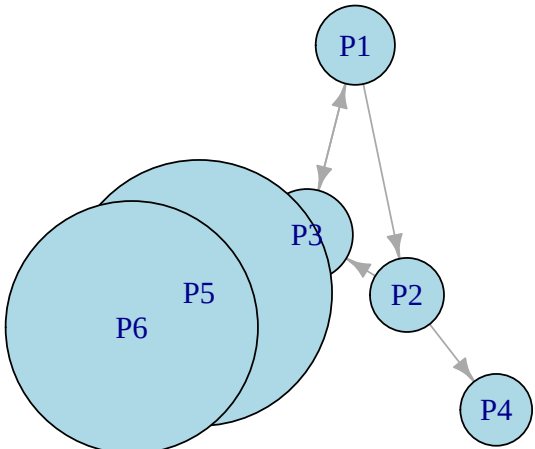
We verify our hand calculation for $\alpha = 0.5$ and $\alpha = 0.85$ in R:

```
P3 <- matrix(c(0,   0, 1,
               0.5, 0, 0,
               0.5, 1, 0), nrow = 3, byrow = TRUE)

# Solve (I - alpha * P) x = ((1 - alpha) / n) * 1 for alpha = 0.5
alpha <- 0.5
x_half <- solve(diag(3) - alpha * P3, rep((1 - alpha) / 3, 3))
MASS::fractions(x_half)

## [1] 14/39 10/39  5/13

# Compare with igraph::page_rank at alpha = 0.85
library(igraph)
g3 <- graph_from_edgelist(matrix(c(1,2, 1,3, 2,3, 3,1), ncol = 2, byrow = TR
page_rank(g3, damping = 0.85)$vector

## [1] 0.3877897 0.2148106 0.3973997
```

R example – 6-page web with a dangling node and spider trap

Consider a 6-page web graph where Page 4 is a dangling node ($d_4^{\text{out}} = 0$) and Pages $\{5, 6\}$ form a spider trap:

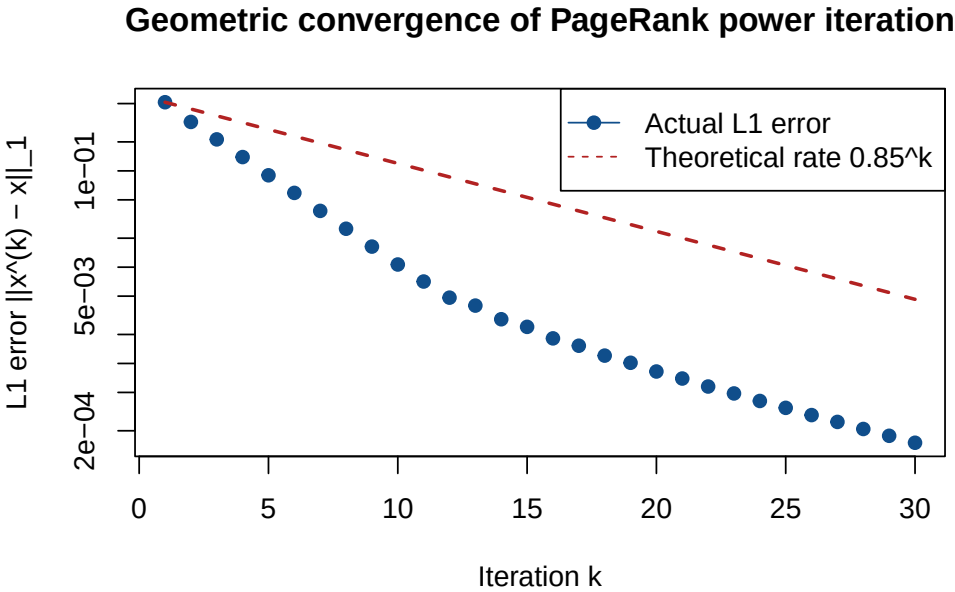
```
edges <- matrix(c(1,2, 1,3, 2,3, 3,1, 2,4, 3,5, 5,6, 6,5),
                ncol = 2, byrow = TRUE)
g6 <- graph_from_edgelist(edges, directed = TRUE)
V(g6)$name <- paste0("P", 1:6)
pr6 <- page_rank(g6, damping = 0.85)$vector
round(pr6, 4)

##     P1     P2     P3     P4     P5     P6
## 0.0728 0.0646 0.0920 0.0611 0.3653 0.3442

par(mar = c(0, 0, 0, 0))
plot(g6, vertex.size = 350 * pr6 + 18, vertex.color = "lightblue",
     vertex.label.cex = 1.1, edge.arrow.size = 0.7)
```


P1
P3
P5
P6
P2
P4

Power iteration vs linear system in R

We implement the dangling-node fix and sparse power iteration from scratch and plot convergence:

```
A6 <- as.matrix(as_adjacency_matrix(g6)) # row i -> col j in igraph, so tra
A6 <- t(A6)                              # now a_{ij} = 1 if j -> i
n <- nrow(A6)
out_deg <- colSums(A6)
P6 <- matrix(0, n, n)
for (j in 1:n) {
  if (out_deg[j] > 0) P6[, j] <- A6[, j] / out_deg[j] else P6[, j] <- rep(1
}
alpha <- 0.85
x_exact <- solve(diag(n) - alpha * P6, rep((1 - alpha) / n, n))

x_k <- rep(1 / n, n)
err <- numeric(30)
for (k in 1:30) {
```


Geometric convergence of PageRank power iteration
Actual L1 error
Theoretical rate 0.85^k
L1 error ||x^(k) − x||_1
2e−04
5e−03
1e−01
0
5
10
15
20
25
30
Iteration k

Summary of Unit 09

- **Absorbing Markov chains:** Reordering states so the r absorbing states precede the m transient states gives the canonical column-stochastic block form
$$P = \begin{pmatrix} \mathbb{I}_r & R \\ \mathbf{0}_{m,r} & Q \end{pmatrix}, \qquad \rho(Q) < 1$$
- **Fundamental matrix:** $N = (\mathbb{I}_m - Q)^{-1} = \sum_{k=0}^{\infty} Q^k$ counts expected visits to transient states, $\boldsymbol{t}^T = \mathbb{1}_m^T N$ gives expected absorption times and $B = RN$ gives absorption probabilities
- **Spectral centrality:** Eigenvector centrality ($A\boldsymbol{x} = \lambda \boldsymbol{x}$) requires strong connectivity; Katz centrality ($(\mathbb{I}_n - \alpha A)\boldsymbol{x} = \beta \mathbb{1}_n$ for $\alpha < 1/\rho(A)$) fixes directed acyclic and source nodes by adding baseline importance
- **PageRank:** Normalising votes by out-degree and adding teleportation yields the primitive column-stochastic Google matrix $G = \alpha P + \frac{1-\alpha}{n}\mathbb{1}_n\mathbb{1}_n^T$, whose unique stationary vector solves $(\mathbb{I}_n - \alpha P)\boldsymbol{x} = \frac{1-\alpha}{n}\mathbb{1}_n$

Symmetric graph matrices and spectral clustering

MATH 2740 – Mathematics of Data Science – Unit U10

Julien Arino
julien.arino@umanitoba.ca

Department of Mathematics @ University of Manitoba

Fall 2026

The University of Manitoba campuses are located on original lands of Anishinaabeg, Ininew, Anisininew, Dakota and Dene peoples, and on the National Homeland of the Red River Métis. We respect the Treaties that were made on these territories, we acknowledge the harms and mistakes of the past, and we dedicate ourselves to move forward in partnership with Indigenous communities in a spirit of Reconciliation and collaboration.

Outline

- Undirected graphs and similarity graphs
- The combinatorial graph Laplacian
- Spectral properties of the Laplacian
- Graph cuts, the Fiedler vector and spectral clustering

From directed rankings to undirected communities

In Units 08 and 09, we studied directed graphs, non-negative matrices and the Perron-Frobenius theorem

There, the dominant eigenvector $\boldsymbol{v}_1 > \mathbf{0}$ associated with $\rho(A)$ answered a *ranking* question: which vertices are most central or visited most often?

In this unit, we turn to **undirected graphs** and **similarity networks**, where the associated matrices are *real and symmetric*

The main question shifts from ranking individual vertices to **clustering**: how do we partition a graph (or a dataset) into cohesive communities with minimal connections between them?

Why real symmetric matrices change everything

When a matrix $M \in \mathcal{M}_n(\mathbb{R})$ is symmetric ($M^T = M$), the spectral theorem gives us three powerful tools that general directed graphs lack:

1. Every eigenvalue $\lambda_1, \ldots, \lambda_n$ of M is **real**

2. Eigenvectors from distinct eigenspaces are **mutually orthogonal**, and $\mathbb{R}^n$ has an **orthonormal basis of eigenvectors** $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n\}$

3. Eigenvalues and eigenvectors are characterised variationally through the **quadratic form** $\boldsymbol{x}^T M \boldsymbol{x}$ and the **Rayleigh quotient** $\dfrac{\boldsymbol{x}^T M \boldsymbol{x}}{\boldsymbol{x}^T \boldsymbol{x}}$

Roadmap of Unit 10

1. **Undirected graphs and similarity graphs:** turning point clouds $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n \in \mathbb{R}^p$ into symmetric adjacency and weight matrices

2. **The combinatorial graph Laplacian $L = D - A$:** degree matrix, incidence factorisation $L = B^T B$ and the quadratic form $\boldsymbol{x}^T L \boldsymbol{x} = \sum_{\{i,j\} \in E} a_{ij}(x_i - x_j)^2$

3. **Spectral properties of L:** positive semi-definiteness, $\lambda_1 = 0$ with eigenvector $\mathbb{1}$ and why the multiplicity of $\lambda = 0$ counts connected components

4. **Graph cuts, the Fiedler vector and spectral clustering:** expressing $\mathrm{cut}(S, \bar{S}) = \frac{1}{4} \boldsymbol{x}^T L \boldsymbol{x}$, spectral relaxation and partitioning with the Fiedler vector $\boldsymbol{v}_2$

Undirected graphs and similarity graphs

The combinatorial graph Laplacian

Spectral properties of the Laplacian

Graph cuts, the Fiedler vector and spectral clustering

Undirected simple graphs

Let $G = (V, E)$ be an undirected simple graph with vertex set $V = \{1, 2, \ldots, n\}$ and edge set E consisting of unordered pairs $\{i, j\}$ of distinct vertices

- Undirected means $\{i, j\} = \{j, i\}$: if i is connected to j, then j is connected to i
- Simple means there are no self-loops ($\{i, i\} \notin E$) and at most one edge between any pair of vertices

Definition 155 (Adjacency matrix of an undirected graph)

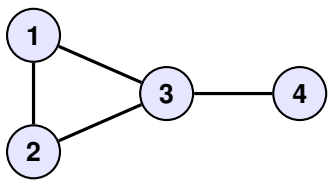
*The **adjacency matrix** of an undirected simple graph $G = (V, E)$ on n vertices is the matrix $A = [a_{ij}] \in \mathcal{M}_n(\{0, 1\})$ defined by*

$$a_{ij} = \begin{cases} 1 & \text{if } \{i, j\} \in E \\ 0 & \text{otherwise} \end{cases}$$

Symmetry of the adjacency matrix

Because $\{i,j\} \in E \iff \{j,i\} \in E$ and $\{i,i\} \notin E$, the adjacency matrix A of an undirected simple graph always satisfies

$$A^T = A \quad (a_{ij} = a_{ji} \ \ \forall i,j) \qquad \text{and} \qquad a_{ii} = 0 \quad (\forall i = 1, \ldots, n)$$



$$A = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} = A^T$$

Notice that the trace of A is $\text{tr}(A) = \sum_{i=1}^{n} a_{ii} = 0$, so the n real eigenvalues of A always sum to 0 (some are positive and some are negative)

Weighted similarity graphs

In data science, edges often carry a nonnegative weight measuring the **similarity** (or affinity) between vertices i and j

Definition 156 (Weighted similarity graph and weight matrix)

A **weighted undirected graph** $G = (V, E, W)$ has a symmetric weight matrix $W = [w_{ij}] \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$ satisfying

$$w_{ij} = w_{ji} \geq 0 \quad (\forall i, j = 1, \ldots, n) \qquad \text{and} \qquad w_{ii} = 0 \quad (\forall i = 1, \ldots, n)$$

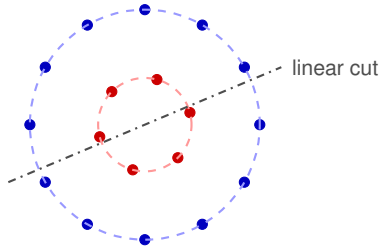
Here $w_{ij} > 0$ if $\{i, j\} \in E$ and $w_{ij} = 0$ if $\{i, j\} \notin E$

- Large w_{ij} means vertices i and j are **strongly connected** (highly similar)
- $w_{ij} = 0$ (or $w_{ij} \approx 0$) means vertices i and j are **unconnected** (dissimilar)
- An unweighted graph is simply the special case $W = A \in \mathcal{M}_n(\{0, 1\})$

Why turn a dataset into a similarity graph?

Suppose we are given n observations $\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_n \in \mathbb{R}^p$ and want to partition them into clusters

- Centroid-based methods (k-means) and linear projections (PCA) rely on **global Euclidean distances** $\|\boldsymbol{x}_i - \boldsymbol{x}_j\|_2$ and linear separating hyperplanes
- They work well when clusters are compact, convex clouds
- They fail completely when clusters have **non-convex geometry** (such as concentric rings, spirals or interlocking crescents), where points in the same cluster are far apart in $\mathbb{R}^p$ but connected through a chain of nearby neighbours



Representing the data as a **local similarity graph** turns non-convex geometric clustering into a **graph connectivity** problem

Constructing similarity graphs from data points

Given observations $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n \in \mathbb{R}^p$ with pairwise distances $d(\boldsymbol{x}_i, \boldsymbol{x}_j) = \|\boldsymbol{x}_i - \boldsymbol{x}_j\|_2$, there are three standard ways to build a symmetric similarity graph:

1. **ε-neighbourhood graph**: Fix a threshold $\varepsilon > 0$ and connect $i \neq j$ with an unweighted edge ($a_{ij} = 1$) if and only if

$$\|\boldsymbol{x}_i - \boldsymbol{x}_j\|_2 \leq \varepsilon$$

2. **k-nearest neighbour (k-NN) graph**: Connect i and j if $\boldsymbol{x}_i$ is among the k closest points to $\boldsymbol{x}_j$ *or* $\boldsymbol{x}_j$ is among the k closest points to $\boldsymbol{x}_i$ (symmetrised so $A^T = A$)
3. **Gaussian (RBF) similarity graph**: Connect all pairs $i \neq j$ with weight

$$w_{ij} = \exp\left(-\frac{\|\boldsymbol{x}_i - \boldsymbol{x}_j\|_2^2}{2\sigma^2}\right), \qquad w_{ii} = 0$$

where $\sigma > 0$ is the bandwidth (scale) parameter

Example: from 4 points in $\mathbb{R}^2$ to a similarity graph

Consider $n = 4$ points in $\mathbb{R}^2$: two nearby points $\boldsymbol{x}_1 = (0,0)^T, \boldsymbol{x}_2 = (0,1)^T$ on the left and two nearby points $\boldsymbol{x}_3 = (3,0)^T, \boldsymbol{x}_4 = (3,1)^T$ on the right

Pairwise Euclidean distances $\|\boldsymbol{x}_i - \boldsymbol{x}_j\|_2$:

$$\Delta = \begin{pmatrix} 0 & 1 & 3 & \sqrt{10} \\ 1 & 0 & \sqrt{10} & 3 \\ 3 & \sqrt{10} & 0 & 1 \\ \sqrt{10} & 3 & 1 & 0 \end{pmatrix}$$

With $\varepsilon = 1.5$, the **ε-neighbourhood graph** has adjacency matrix

$$A = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

With $\sigma = 1$, the **Gaussian similarity matrix** $w_{ij} = e^{-\|\boldsymbol{x}_i - \boldsymbol{x}_j\|_2^2/2}$ $(i \neq j)$ is

$$W \approx \begin{pmatrix} 0 & 0.607 & 0.011 & 0.007 \\ 0.607 & 0 & 0.007 & 0.011 \\ 0.011 & 0.007 & 0 & 0.607 \\ 0.007 & 0.011 & 0.607 & 0 \end{pmatrix}$$

- Within-cluster weights: $e^{-1/2} \approx 0.607$
- Between-cluster weights: $e^{-9/2} \approx 0.011$ and $e^{-10/2} \approx 0.007$

Role of the scale parameter ε, k or σ

In all three constructions, the scale parameter controls which pairs are treated as local neighbours:

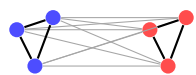
Scale too small

True clusters shatter into many isolated pieces

Well-chosen scale

Dense within clusters, sparse or zero between clusters

Scale too large



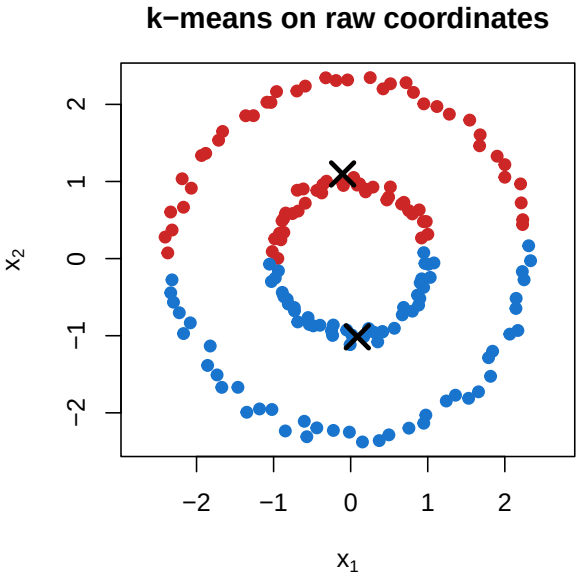
Shortcut edges merge distinct clusters together

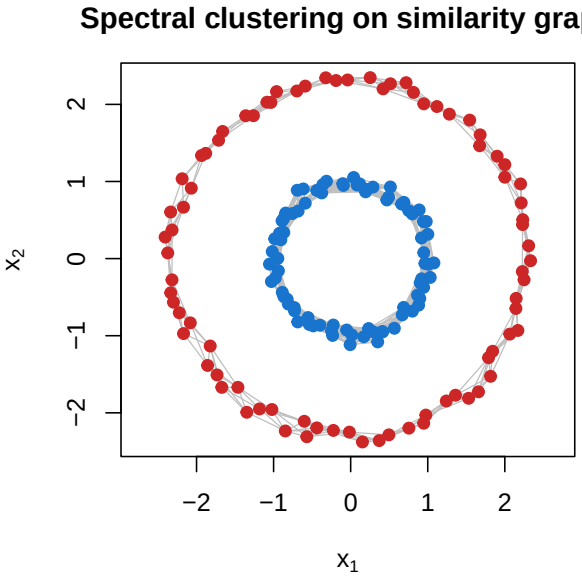
In what follows, we write $A = [a_{ij}]$ for the adjacency or similarity matrix (whether $a_{ij} \in \{0, 1\}$ or $a_{ij} = w_{ij} \geq 0$): all formulas hold identically in both cases

R illustration: two concentric rings

Let us generate $n = 160$ points on two concentric circles in $\mathbb{R}^2$ and compare k-means in $\mathbb{R}^2$ against an ε-neighbourhood graph

```
set.seed(2740)
n1 = 80; n2 = 80
theta1 = seq(0, 2 * pi, length.out = n1 + 1)[-(n1 + 1)]
theta2 = seq(0, 2 * pi, length.out = n2 + 1)[-(n2 + 1)]
X = rbind(
  cbind(1.0 * cos(theta1), 1.0 * sin(theta1)) + rnorm(2 * n1, sd = 0.06),
  cbind(2.3 * cos(theta2), 2.3 * sin(theta2)) + rnorm(2 * n2, sd = 0.06)
)
# epsilon-neighbourhood adjacency matrix (epsilon = 0.65)
dist_mat = as.matrix(dist(X))
A_rings = (dist_mat <= 0.65 & dist_mat > 0) * 1
```


k−means on raw coordinates
x2
x1
2
1
0
−1
−2
−2
−1
0
1
2


Spectral clustering on similarity graph
x2
x1
2
1
0
−1
−2
−2
−1
0
1
2

Undirected graphs and similarity graphs

The combinatorial graph Laplacian

Spectral properties of the Laplacian

Graph cuts, the Fiedler vector and spectral clustering

The combinatorial graph Laplacian

Definition and basic examples

The Laplacian quadratic form

Vertex degrees and the degree matrix

Definition 157 (Vertex degree and degree matrix)

Let $G = (V, E)$ be an undirected graph on n vertices with symmetric adjacency (or weight) matrix $A = [a_{ij}] \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$. The ***degree*** *of vertex i is the row sum*

$$d_i = \sum_{j=1}^{n} a_{ij} = (A\mathbb{1})_i \qquad (i = 1, \ldots, n)$$

where $\mathbb{1} = (1, 1, \ldots, 1)^T \in \mathbb{R}^n$. The ***degree matrix*** *of G is the diagonal matrix*

$$D = \operatorname{diag}(d_1, d_2, \ldots, d_n) = \begin{pmatrix} d_1 & 0 & \cdots & 0 \\ 0 & d_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & d_n \end{pmatrix}$$

The combinatorial graph Laplacian

Definition 158 (Combinatorial graph Laplacian)

Let $G = (V, E)$ be an undirected graph on n vertices with degree matrix D and adjacency (or weight) matrix A. The ***combinatorial graph Laplacian*** *(or simply* ***Laplacian matrix****) of G is the $n \times n$ matrix*

$$L = D - A$$

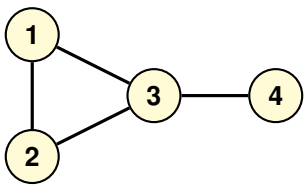
Entry-by-entry, $L = [\ell_{ij}]$ is given by

$$\ell_{ij} = \begin{cases} d_i & \text{if } i = j \\ -a_{ij} & \text{if } i \neq j \end{cases}$$

For a simple unweighted graph, $\ell_{ii} = d_i$, $\ell_{ij} = -1$ if $\{i, j\} \in E$ and $\ell_{ij} = 0$ otherwise

Example 1: Laplacian of a 4-vertex paw graph

Consider the 4-vertex graph $G = (V, E)$ below (a triangle $\{1, 2, 3\}$ with a pendant edge $\{3, 4\}$):



Degrees: $d_1 = 2$, $d_2 = 2$, $d_3 = 3$, $d_4 = 1$

$$D = \begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \qquad A = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

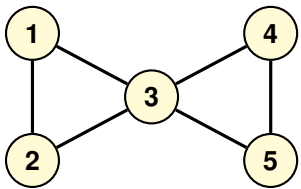
$$L = D - A = \begin{pmatrix} 2 & -1 & -1 & 0 \\ -1 & 2 & -1 & 0 \\ -1 & -1 & 3 & -1 \\ 0 & 0 & -1 & 1 \end{pmatrix}$$

Quick sanity check on L:

- L is symmetric ($L^T = L$), with positive degrees on the diagonal and -1 on edges
- Every row and every column of L sums to 0: e.g. row 3 has $(-1) + (-1) + 3 + (-1) = 0$

Example 2: Laplacian of a 5-vertex bowtie graph

Consider the 5-vertex bowtie graph formed by two triangles $\{1, 2, 3\}$ and $\{3, 4, 5\}$ sharing central vertex 3:



Degrees: $d_1 = d_2 = d_4 = d_5 = 2$, $d_3 = 4$

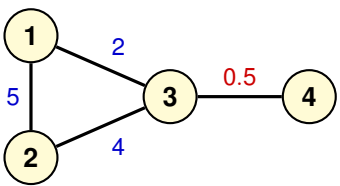
$$L = D - A = \begin{pmatrix} 2 & -1 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 \\ -1 & -1 & 4 & -1 & -1 \\ 0 & 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & -1 & 2 \end{pmatrix}$$

Notice how the structure of L reflects the graph topology:

- The upper-left 2×2 block $\{1, 2\}$ and lower-right 2×2 block $\{4, 5\}$ have no direct edges between them (0 in the off-diagonal blocks)
- All paths between $\{1, 2\}$ and $\{4, 5\}$ pass through vertex 3 (row and column 3)

Example 3: Laplacian of a weighted similarity graph

For a weighted graph $G = (V, E, W)$, the exact same recipe $L = D - W$ applies using weighted degrees $d_i = \sum_{j=1}^{n} w_{ij}$:



Weighted degrees:
$d_1 = 5 + 2 = 7,\ d_2 = 5 + 4 = 9$
$d_3 = 2 + 4 + 0.5 = 6.5,\ d_4 = 0.5$

$$W = \begin{pmatrix} 0 & 5 & 2 & 0 \\ 5 & 0 & 4 & 0 \\ 2 & 4 & 0 & 0.5 \\ 0 & 0 & 0.5 & 0 \end{pmatrix}$$

$$L = D - W = \begin{pmatrix} 7 & -5 & -2 & 0 \\ -5 & 9 & -4 & 0 \\ -2 & -4 & 6.5 & -0.5 \\ 0 & 0 & -0.5 & 0.5 \end{pmatrix}$$

Every row and column of L still sums to 0, and
$\mathrm{tr}(L) = \sum_{i=1}^{n} d_i = 2 \sum_{\{i,j\} \in E} w_{ij} = 23$

Connection with the oriented incidence matrix

We can also factor L as a Gram matrix $B^T B$ by assigning an arbitrary orientation to each edge of $G = (V, E)$

Definition 159 (Oriented edge-vertex incidence matrix)

Let $G = (V, E)$ have $n = |V|$ vertices and $m = |E|$ edges $e_1, \ldots, e_m$. Choose an arbitrary direction $e_k = (i, j)$ for each edge $(i \to j)$. The **oriented incidence matrix** $B = [b_{kv}] \in \mathcal{M}_{mn}(\{-1, 0, 1\})$ has one row per edge and one column per vertex:

$$b_{kv} = \begin{cases} +1 & \text{if } v = i \text{ (tail of } e_k) \\ -1 & \text{if } v = j \text{ (head of } e_k) \\ 0 & \text{otherwise} \end{cases}$$

Proposition 160 (Incidence factorisation of the Laplacian)

For any choice of edge orientations on an unweighted graph G, $L = B^T B$

Proof of Proposition 160 ($L = B^T B$) and worked check

Proof: The (u, v)-entry of $B^T B \in \mathcal{M}_n(\mathbb{R})$ is the dot product of columns u and v of B:

$$(B^T B)_{uv} = \sum_{k=1}^{m} b_{ku} b_{kv}$$

- If $u = v$: $b_{ku}^2 = (\pm 1)^2 = 1$ whenever edge e_k is incident to vertex u, and 0 otherwise, so $(B^T B)_{uu} = \sum_{k=1}^{m} b_{ku}^2 = d_u = \ell_{uu}$
- If $u \neq v$: the product $b_{ku} b_{kv}$ is non-zero only if edge e_k connects u and v, in which case one entry is $+1$ and the other is -1, so $b_{ku} b_{kv} = -1 = \ell_{uv}$ □

Check on the 4-vertex paw graph (orienting $e_1 = (1, 2), e_2 = (1, 3), e_3 = (2, 3), e_4 = (3, 4)$):

$$B = \begin{pmatrix} 1 & -1 & 0 & 0 \\ 1 & 0 & -1 & 0 \\ 0 & 1 & -1 & 0 \\ 0 & 0 & 1 & -1 \end{pmatrix} \implies B^T B = \begin{pmatrix} 2 & -1 & -1 & 0 \\ -1 & 2 & -1 & 0 \\ -1 & -1 & 3 & -1 \\ 0 & 0 & -1 & 1 \end{pmatrix} = L$$

Elementary matrix properties of L

From $L = D - A$ and $L = B^T B$, several useful matrix identities follow immediately:

1. **Trace of L:** By the handshaking lemma, the sum of the eigenvalues of L equals twice the total edge weight:

$$\sum_{k=1}^{n} \lambda_k = \operatorname{tr}(L) = \sum_{i=1}^{n} d_i = 2 \sum_{\{i,j\} \in E} a_{ij} \qquad (= 2|E| \text{ for an unweighted graph})$$

2. **Sum of rank-one edge matrices:** Each edge $e = \{i, j\} \in E$ contributes a 2×2 sub-Laplacian on vertices i and j. Letting $\boldsymbol{e}_1, \ldots, \boldsymbol{e}_n$ denote the standard basis of $\mathbb{R}^n$,

$$L = \sum_{\{i,j\} \in E} a_{ij} (\boldsymbol{e}_i - \boldsymbol{e}_j)(\boldsymbol{e}_i - \boldsymbol{e}_j)^T$$

3. **Weak diagonal dominance:** Every row i satisfies $|\ell_{ii}| = d_i = \sum_{j \neq i} a_{ij} = \sum_{j \neq i} |\ell_{ij}|$

The combinatorial graph Laplacian

Definition and basic examples

The Laplacian quadratic form

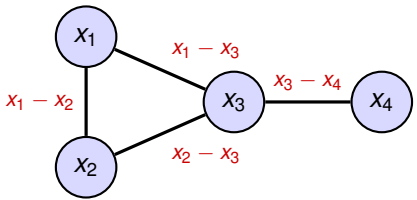
Vectors on $\mathbb{R}^n$ as functions on the vertices

How does L act on a vector $\boldsymbol{x} = (x_1, x_2, \ldots, x_n)^T \in \mathbb{R}^n$?

Think of $\boldsymbol{x} \in \mathbb{R}^n$ as assigning a real number (a "signal" or "potential") $x_i \in \mathbb{R}$ to each vertex $i \in V$:

- At vertex i, the ith entry of $L\boldsymbol{x}$ is

$$(L\boldsymbol{x})_i = d_i x_i - \sum_{j=1}^{n} a_{ij} x_j = \sum_{j=1}^{n} a_{ij}(x_i - x_j)$$



- Along each edge $e_k = (i, j)$, the kth entry of $B\boldsymbol{x}$ is the **edge difference**

$$(B\boldsymbol{x})_k = x_i - x_j$$

Multiplying on the left by $\boldsymbol{x}^T$ produces the fundamental identity of spectral graph theory

The Laplacian quadratic form

Theorem 161 (Laplacian quadratic form)

Let $G = (V, E)$ be an undirected graph on n vertices with symmetric weight matrix $A = [a_{ij}]$ ($a_{ij} = a_{ji} \geq 0$) and combinatorial Laplacian $L = D - A$. Then for every vector $\boldsymbol{x} = (x_1, \ldots, x_n)^T \in \mathbb{R}^n$,

$$\boldsymbol{x}^T L \boldsymbol{x} = \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} a_{ij}(x_i - x_j)^2 = \sum_{\{i,j\} \in E} a_{ij}(x_i - x_j)^2$$

In particular, for an unweighted simple graph ($a_{ij} \in \{0, 1\}$),

$$\boldsymbol{x}^T L \boldsymbol{x} = \sum_{\{i,j\} \in E} (x_i - x_j)^2$$

Proof of Theorem 161 (Method 1: direct algebraic expansion)

Let $\boldsymbol{x} = (x_1, \ldots, x_n)^T \in \mathbb{R}^n$. Using $L = D - A$ and $d_i = \sum_{j=1}^n a_{ij}$, we expand

$$\boldsymbol{x}^T L \boldsymbol{x} = \boldsymbol{x}^T D \boldsymbol{x} - \boldsymbol{x}^T A \boldsymbol{x} = \sum_{i=1}^n d_i x_i^2 - \sum_{i=1}^n \sum_{j=1}^n a_{ij} x_i x_j$$

$$= \sum_{i=1}^n \left(\sum_{j=1}^n a_{ij} \right) x_i^2 - \sum_{i=1}^n \sum_{j=1}^n a_{ij} x_i x_j$$

By symmetry $a_{ij} = a_{ji}$, relabelling dummy indices $i \leftrightarrow j$ gives $\sum_{i,j} a_{ij} x_i^2 = \sum_{i,j} a_{ji} x_j^2 = \sum_{i,j} a_{ij} x_j^2$, so we can split the first sum into two equal halves:

$$\boldsymbol{x}^T L \boldsymbol{x} = \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n a_{ij} x_i^2 - \sum_{i=1}^n \sum_{j=1}^n a_{ij} x_i x_j + \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n a_{ij} x_j^2$$

$$= \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n a_{ij} \left(x_i^2 - 2 x_i x_j + x_j^2 \right) = \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n a_{ij} (x_i - x_j)^2$$

Since each undirected edge $\{i, j\} \in E$ appears twice in the double sum (as (i, j)

Proof of Theorem 161 (Method 2: via the incidence matrix)

For an unweighted graph $G = (V, E)$ with $m = |E|$ edges, Proposition 160 gives $L = B^T B$, where $B \in \mathcal{M}_{mn}$ is the oriented incidence matrix

For any $\boldsymbol{x} \in \mathbb{R}^n$, the kth entry of $B\boldsymbol{x} \in \mathbb{R}^m$ corresponding to oriented edge $e_k = (i, j)$ is

$$(B\boldsymbol{x})_k = x_i - x_j$$

Therefore,

$$\boldsymbol{x}^T L\boldsymbol{x} = \boldsymbol{x}^T(B^T B)\boldsymbol{x} = (B\boldsymbol{x})^T(B\boldsymbol{x}) = \|B\boldsymbol{x}\|_2^2 = \sum_{k=1}^{m}(B\boldsymbol{x})_k^2 = \sum_{\{i,j\}\in E}(x_i - x_j)^2$$

(For a weighted graph with diagonal edge-weight matrix $W_E = \mathrm{diag}(w_{e_1}, \ldots, w_{e_m})$, writing $L = B^T W_E B = (W_E^{1/2} B)^T (W_E^{1/2} B)$ gives $\boldsymbol{x}^T L\boldsymbol{x} = \|W_E^{1/2} B\boldsymbol{x}\|_2^2 = \sum_{\{i,j\}\in E} w_{ij}(x_i - x_j)^2$)

□

Geometric interpretation: smoothness (Dirichlet energy) on a graph

The quadratic form $\boldsymbol{x}^T L \boldsymbol{x} = \sum_{\{i,j\} \in E} a_{ij}(x_i - x_j)^2$ is called the **Dirichlet energy** of the vector $\boldsymbol{x}$ on the graph G:

- Every edge $\{i, j\} \in E$ acts like a spring with stiffness a_{ij} that penalises the squared difference $(x_i - x_j)^2$ between its endpoints
- **Smooth vector** ($\boldsymbol{x}^T L \boldsymbol{x}$ small): adjacent vertices have nearly equal values ($x_i \approx x_j$ whenever a_{ij} is large)
- **Constant vector** ($\boldsymbol{x}^T L \boldsymbol{x} = 0$): $x_i = x_j$ across every edge of G
- **Rough / oscillating vector** ($\boldsymbol{x}^T L \boldsymbol{x}$ large): adjacent vertices have very different values (e.g. alternating signs $+1$ and -1 across edges)

Example: evaluating $\boldsymbol{x}^T L\boldsymbol{x}$ on the 4-vertex paw graph

Take the 4-vertex paw graph with edges $\{1,2\},\{1,3\},\{2,3\},\{3,4\}$ and compare two vectors of equal norm:

Case 1 (smooth): $\boldsymbol{x} = (1,1,0,-1)^T$

$$\begin{aligned}\boldsymbol{x}^T L\boldsymbol{x} &= (x_1-x_2)^2 + (x_1-x_3)^2 \\ &\quad + (x_2-x_3)^2 + (x_3-x_4)^2 \\ &= (1-1)^2 + (1-0)^2 \\ &\quad + (1-0)^2 + (0-(-1))^2 \\ &= 0+1+1+1 = 3\end{aligned}$$

Adjacent vertices $\{1,2\}$ share the same value 1, and values change gradually along the graph

Case 2 (oscillating): $\boldsymbol{y} = (1,-1,0,1)^T$

$$\begin{aligned}\boldsymbol{y}^T L\boldsymbol{y} &= (y_1-y_2)^2 + (y_1-y_3)^2 \\ &\quad + (y_2-y_3)^2 + (y_3-y_4)^2 \\ &= (1-(-1))^2 + (1-0)^2 \\ &\quad + (-1-0)^2 + (0-1)^2 \\ &= 4+1+1+1 = 7\end{aligned}$$

Adjacent vertices 1 and 2 jump from $+1$ to -1, producing a much larger energy

Undirected graphs and similarity graphs

The combinatorial graph Laplacian

Spectral properties of the Laplacian

Graph cuts, the Fiedler vector and spectral clustering

Fundamental spectral properties of the Laplacian

Theorem 162 (Fundamental spectral properties of L)

Let $G = (V, E)$ be an undirected graph on n vertices with symmetric nonnegative weight matrix $A = [a_{ij}]$ and combinatorial Laplacian $L = D - A$. Then:

1. *L is real and symmetric ($L^T = L$), so it has n real eigenvalues and an orthonormal basis of real eigenvectors $\{\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n\}$*
2. *L is **positive semi-definite** ($\boldsymbol{x}^T L \boldsymbol{x} \geq 0$ for all $\boldsymbol{x} \in \mathbb{R}^n$), so all eigenvalues of L are nonnegative and can be ordered in ascending order:*

$$0 \leq \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n$$

3. *The all-ones vector $\mathbb{1} = (1, 1, \ldots, 1)^T$ satisfies $L\mathbb{1} = \mathbf{0}$, so $\lambda_1 = 0$ is always the smallest eigenvalue of L with eigenvector $\boldsymbol{v}_1 = \mathbb{1}$ (or unit eigenvector $\frac{1}{\sqrt{n}}\mathbb{1}$)*

Proof of Theorem 162

1. **Symmetry and real spectrum:** Since D is diagonal ($D^T = D$) and A is symmetric ($A^T = A$),

$$L^T = (D - A)^T = D^T - A^T = D - A = L$$

By the spectral theorem for real symmetric matrices, L has n real eigenvalues and an orthonormal basis of real eigenvectors in $\mathbb{R}^n$

2. **Positive semi-definiteness and $\lambda_k \geq 0$:** By Theorem 161, for any $\boldsymbol{x} \in \mathbb{R}^n$,

$$\boldsymbol{x}^T L \boldsymbol{x} = \sum_{\{i,j\} \in E} a_{ij}(x_i - x_j)^2 \geq 0$$

since $a_{ij} \geq 0$ and $(x_i - x_j)^2 \geq 0$. If $L\boldsymbol{v}_k = \lambda_k \boldsymbol{v}_k$ with $\boldsymbol{v}_k \neq \mathbf{0}$, then

$$\lambda_k = \frac{\boldsymbol{v}_k^T (L\boldsymbol{v}_k)}{\boldsymbol{v}_k^T \boldsymbol{v}_k} = \frac{\boldsymbol{v}_k^T L \boldsymbol{v}_k}{\|\boldsymbol{v}_k\|_2^2} \geq 0$$

3. **Zero row sums ($L\mathbb{1} = \mathbf{0}$):** For each row $i = 1, \ldots, n$,

$$(L\mathbb{1})_i = \sum_{j=1}^{n} \ell_{ij} \cdot 1 = d_i - \sum_{j=1}^{n} a_{ij} = d_i - d_i = 0 \implies L\mathbb{1} = \mathbf{0} = 0 \cdot \mathbb{1}$$ □

Beware of the eigenvalue ordering convention!

In PCA (Unit 05) and Perron-Frobenius theory (Units 08–09), we ordered eigenvalues from *largest to smallest* because we wanted maximum variance or the dominant long-term state

For the graph Laplacian L, the universal convention is the **ascending order**

$$0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n$$

Why? Because for any unit eigenvector $\boldsymbol{v}_k$ ($\|\boldsymbol{v}_k\|_2 = 1$),

$$\lambda_k = \boldsymbol{v}_k^T L \boldsymbol{v}_k = \sum_{\{i,j\} \in E} a_{ij}(v_{ki} - v_{kj})^2$$

- **Smallest eigenvalues ($\lambda_1 = 0, \lambda_2, \ldots$)**: eigenvectors vary *least* across edges, revealing connected components and large-scale community structure
- **Largest eigenvalues ($\ldots, \lambda_n$)**: eigenvectors oscillate rapidly across edges

Multiplicity of $\lambda = 0$ and connected components

We know $\lambda_1 = 0$ is always an eigenvalue of L. When is $\lambda_2 = 0$ as well?

Theorem 163 (Multiplicity of 0 equals the number of connected components)

Let $G = (V, E)$ be an undirected graph with nonnegative edge weights and Laplacian L. The multiplicity of 0 *as an eigenvalue of L (that is, $\text{nullity}(L) = \dim(\text{null}(L))$) equals the number k of connected components $C_1, C_2, \ldots, C_k$ of G*

Moreover, the component indicator vectors $\mathbb{1}_{C_1}, \mathbb{1}_{C_2}, \ldots, \mathbb{1}_{C_k} \in \{0,1\}^n$, defined by

$$(\mathbb{1}_{C_r})_i = \begin{cases} 1 & \text{if } i \in C_r \\ 0 & \text{otherwise} \end{cases}$$

form an orthogonal basis of $\text{null}(L)$

Proof of Theorem 163 – Part 1: connected graph ($k = 1$)

Suppose first that G is connected ($k = 1$). We must show that $\text{null}(L) = \text{span}\{\mathbb{1}\}$, so $\dim(\text{null}(L)) = 1$

1. We already know $L\mathbb{1} = \mathbf{0}$, so $\text{span}\{\mathbb{1}\} \subseteq \text{null}(L)$
2. Conversely, let $\boldsymbol{x} \in \text{null}(L)$, so $L\boldsymbol{x} = \mathbf{0}$. Taking the dot product with $\boldsymbol{x}$ and applying Theorem 161 gives
$$0 = \boldsymbol{x}^T\mathbf{0} = \boldsymbol{x}^T L\boldsymbol{x} = \sum_{\{i,j\}\in E} a_{ij}(x_i - x_j)^2$$
3. For every edge $\{i, j\} \in E$, we have $a_{ij} > 0$ and $(x_i - x_j)^2 \geq 0$. A sum of nonnegative terms equals 0 if and only if every term is 0:
$$(x_i - x_j)^2 = 0 \implies x_i = x_j \quad \text{for every edge } \{i, j\} \in E$$
4. Since G is connected, any two vertices $u, v \in V$ are linked by a path $u = i_0, i_1, \ldots, i_m = v$ in G, so $x_u = x_{i_1} = \cdots = x_v$
5. Hence all entries of $\boldsymbol{x}$ are equal ($x_1 = \cdots = x_n = c$), i.e. $\boldsymbol{x} = c\,\mathbb{1} \in \text{span}\{\mathbb{1}\}$

□

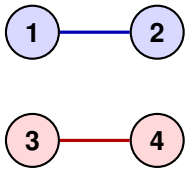
Proof of Theorem 163 – Part 2: k connected components ($k \geq 2$)

Now suppose G has k connected components $C_1, \ldots, C_k$. Reorder the vertices so that those in C_1 come first, followed by $C_2, \ldots, C_k$

1. Because there are no edges between different components, A, D and L are block-diagonal:
$$L = \begin{pmatrix} L_1 & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{0} & L_2 & \cdots & \mathbf{0} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \cdots & L_k \end{pmatrix}$$
where each L_r is the Laplacian of the connected subgraph on C_r
2. Writing $\boldsymbol{x} = (\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(k)})^T$ according to the blocks, $L\boldsymbol{x} = \mathbf{0} \iff L_r\boldsymbol{x}^{(r)} = \mathbf{0}$ for every $r = 1, \ldots, k$
3. By Part 1 applied to each connected component C_r, $L_r\boldsymbol{x}^{(r)} = \mathbf{0} \iff \boldsymbol{x}^{(r)} = c_r \mathbb{1}_{|C_r|}$, which in $\mathbb{R}^n$ means
$$\boldsymbol{x} = c_1\mathbb{1}_{C_1} + c_2\mathbb{1}_{C_2} + \cdots + c_k\mathbb{1}_{C_k}$$
4. Since $C_1, \ldots, C_k$ are disjoint, $\mathbb{1}_{C_r}^T \mathbb{1}_{C_s} = 0$ for $r \neq s$, so $\{\mathbb{1}_{C_1}, \ldots, \mathbb{1}_{C_k}\}$ is an orthogonal basis of $\text{null}(L)$ and $\dim(\text{null}(L)) = k$

Example: a disconnected graph on 4 vertices

Consider $G = (V, E)$ on $n = 4$ vertices with only two edges $\{1, 2\}$ and $\{3, 4\}$, so G has $k = 2$ connected components $C_1 = \{1, 2\}$ and $C_2 = \{3, 4\}$:



$$L = \left(\begin{array}{cc|cc} 1 & -1 & 0 & 0 \\ -1 & 1 & 0 & 0 \\ \hline 0 & 0 & 1 & -1 \\ 0 & 0 & -1 & 1 \end{array}\right) = \begin{pmatrix} L_1 & \mathbf{0} \\ \mathbf{0} & L_2 \end{pmatrix}$$

- Each 2×2 block $\begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}$ has trace 2 and determinant 0, so its eigenvalues are 0 and 2
- The spectrum of L is therefore $\lambda_1 = 0,\ \lambda_2 = 0,\ \lambda_3 = 2,\ \lambda_4 = 2$ (multiplicity of 0 is $k = 2$)
- The orthogonal indicator vectors for null(L) are

$$\mathbb{1}_{C_1} = (1, 1, 0, 0)^T \qquad \text{and} \qquad \mathbb{1}_{C_2} = (0, 0, 1, 1)^T$$

Counting connected components with $\lambda = 0$ in R

Let us build a graph on $n = 8$ vertices consisting of $k = 3$ disconnected components (a triangle $\{1, 2, 3\}$, a path $\{4, 5, 6\}$ and an edge $\{7, 8\}$) and check the eigenvalues of L:

```
G_comp = make_graph(edges = c(1,2, 2,3, 1,3,  4,5, 5,6,  7,8),
                    n = 8, directed = FALSE)
L_comp = as.matrix(laplacian_matrix(G_comp))
ev_comp = eigen(L_comp, symmetric = TRUE)
# Eigenvalues of L in ascending order
round(rev(ev_comp$values), 4)

## [1] 0 0 0 1 2 3 3 3

# Number of zero eigenvalues (up to numerical precision)
sum(abs(ev_comp$values) < 1e-10)

## [1] 3
```

Exactly three eigenvalues of L equal 0, matching the $k = 3$ connected

Spectrum of extreme graphs: empty vs complete graph K_n

Comparing two extreme graphs on n vertices illustrates how Laplacian eigenvalues measure connectivity:

1. Empty graph E_n ($|E| = 0$):

$$L = \mathbf{0}_n$$

- n isolated vertices ($k = n$ components)
- Spectrum of L:

$$\lambda_1 = \lambda_2 = \cdots = \lambda_n = 0$$

2. Complete graph K_n (all pairs connected):

$$L = (n-1)\mathbb{I}_n - (\mathbb{J}_n - \mathbb{I}_n) = n\mathbb{I}_n - \mathbb{1}\mathbb{1}^T$$

- For $\boldsymbol{v}_1 = \mathbb{1}$:
  $L\mathbb{1} = n\mathbb{1} - n\mathbb{1} = \mathbf{0} \implies \lambda_1 = 0$
- For any $\boldsymbol{x} \perp \mathbb{1}$ ($\mathbb{1}^T\boldsymbol{x} = 0$):

$$L\boldsymbol{x} = n\boldsymbol{x} - \mathbb{1}(\mathbb{1}^T\boldsymbol{x}) = n\boldsymbol{x}$$

- Spectrum of K_n: $\lambda_1 = 0$ and $\lambda_2 = \cdots = \lambda_n = n$

For every simple graph G on n vertices, its eigenvalues lie between these two extremes: $0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n \leq n$

What if G is "almost" disconnected?

Suppose G has two dense clusters C_1 and C_2 that are completely disconnected ($k = 2$):

- Then $\lambda_1 = 0$ and $\lambda_2 = 0$, and any vector in null(L) is constant on C_1 and constant on C_2:

$$\boldsymbol{v}_1 = (1, \ldots, 1,\ 1, \ldots, 1)^T, \qquad \boldsymbol{v}_2 = (1, \ldots, 1,\ -1, \ldots, -1)^T \quad (\text{if } |C_1| = |C_2|)$$

Now perturb G slightly by adding a single weak bridge edge between C_1 and C_2:

- The graph is now connected ($k = 1$), so $\lambda_1 = 0$ and $\lambda_2 > 0$
- Because eigenvalues and eigenvectors of symmetric matrices vary continuously under small perturbations, λ_2 will be **positive but very small** ($\lambda_2 \approx 0$)
- The associated eigenvector $\boldsymbol{v}_2 \perp \mathbb{1}$ will remain **nearly constant and positive on** C_1 and **nearly constant and negative on** C_2!

Next, we make this intuition completely rigorous via **graph cuts**

Undirected graphs and similarity graphs

The combinatorial graph Laplacian

Spectral properties of the Laplacian

Graph cuts, the Fiedler vector and spectral clustering

Graph cuts, the Fiedler vector and spectral clustering

Graph partitioning and cut size

Spectral relaxation and the Fiedler vector

Worked exam-style examples and R implementation

Partitions and the cut of a graph

Let $G = (V, E)$ be an undirected graph on n vertices with symmetric edge weights $a_{ij} \geq 0$

A **2-way partition** (or **bipartition**) of G splits V into two disjoint non-empty subsets S and $\bar{S} = V \setminus S$ ($S \cup \bar{S} = V$, $S \cap \bar{S} = \emptyset$)

Definition 164 (Cut of a partition)

*The **cut** between S and $\bar{S}$ is the total weight of edges with one endpoint in S and the other endpoint in $\bar{S}$:*

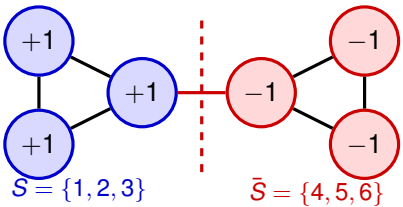
$$\text{cut}(S, \bar{S}) = \sum_{i \in S,\, j \in \bar{S}} a_{ij}$$

For an unweighted graph ($a_{ij} \in \{0, 1\}$), $\text{cut}(S, \bar{S})$ is simply the number of edges crossing between S and $\bar{S}$

Encoding a partition with a ± 1 indicator vector

To connect graph cuts to linear algebra, we represent any bipartition $(S, \bar{S})$ by a **cut indicator vector** $\boldsymbol{x} = (x_1, \ldots, x_n)^T \in \{-1, +1\}^n$:

$$x_i = \begin{cases} +1 & \text{if } i \in S \\ -1 & \text{if } i \in \bar{S} \end{cases}$$



Proposition 165 (Cut size as a quadratic form)

For any partition $(S, \bar{S})$ of V with indicator vector $\boldsymbol{x} \in \{-1, +1\}^n$,

$$\boldsymbol{x}^T L \boldsymbol{x} = 4 \operatorname{cut}(S, \bar{S})$$

or equivalently,

$$\operatorname{cut}(S, \bar{S}) = \frac{1}{4} \boldsymbol{x}^T L \boldsymbol{x}$$

Proof of Proposition 165 ($\boldsymbol{x}^T L \boldsymbol{x} = 4\,\mathrm{cut}(S, \bar{S})$)

Let $\boldsymbol{x} \in \{-1, +1\}^n$ be the cut indicator vector for $(S, \bar{S})$. For any edge $\{i, j\} \in E$, look at the squared difference $(x_i - x_j)^2$:

- **Both endpoints in** S ($i \in S, j \in S$): $x_i = +1$ and $x_j = +1$, so
$$(x_i - x_j)^2 = (1 - 1)^2 = 0$$
- **Both endpoints in** $\bar{S}$ ($i \in \bar{S}, j \in \bar{S}$): $x_i = -1$ and $x_j = -1$, so
$$(x_i - x_j)^2 = (-1 - (-1))^2 = 0$$
- **Endpoints in different sets** ($i \in S, j \in \bar{S}$ or $i \in \bar{S}, j \in S$): $x_i - x_j = \pm 2$, so
$$(x_i - x_j)^2 = (\pm 2)^2 = 4$$

Substituting into Theorem 161 ($\boldsymbol{x}^T L \boldsymbol{x} = \sum_{\{i,j\} \in E} a_{ij}(x_i - x_j)^2$), only the edges crossing between S and $\bar{S}$ survive, each multiplied by 4:

$$\boldsymbol{x}^T L \boldsymbol{x} = \sum_{i \in S,\, j \in \bar{S}} 4\, a_{ij} = 4 \sum_{i \in S,\, j \in \bar{S}} a_{ij} = 4\,\mathrm{cut}(S, \bar{S}) \qquad \square$$

Example: comparing two cuts on the 4-vertex paw graph

Recall the 4-vertex paw graph with Laplacian

$$L = \begin{pmatrix} 2 & -1 & -1 & 0 \\ -1 & 2 & -1 & 0 \\ -1 & -1 & 3 & -1 \\ 0 & 0 & -1 & 1 \end{pmatrix}$$

Let us compare two candidate partitions using $\text{cut}(S, \bar{S}) = \frac{1}{4}\boldsymbol{x}^T L \boldsymbol{x}$:

Partition A: $S_1 = \{1, 2, 3\}$, $\bar{S}_1 = \{4\}$

$$\boldsymbol{x}^{(1)} = (1, 1, 1, -1)^T$$

$$L\boldsymbol{x}^{(1)} = \begin{pmatrix} 0 \\ 0 \\ 2 \\ -2 \end{pmatrix} \implies (\boldsymbol{x}^{(1)})^T L \boldsymbol{x}^{(1)} = 4$$

$$\text{cut}(S_1, \bar{S}_1) = \frac{4}{4} = 1 \quad \text{(only edge } \{3, 4\}\text{)}$$

Partition B: $S_2 = \{1, 3\}$, $\bar{S}_2 = \{2, 4\}$

$$\boldsymbol{x}^{(2)} = (1, -1, 1, -1)^T$$

$$L\boldsymbol{x}^{(2)} = \begin{pmatrix} 2 \\ -4 \\ 4 \\ -2 \end{pmatrix} \implies (\boldsymbol{x}^{(2)})^T L \boldsymbol{x}^{(2)} = 12$$

$$\text{cut}(S_2, \bar{S}_2) = \frac{12}{4} = 3 \quad \text{(edges } \{1, 2\}, \{2, 3\},$$

Graph cuts, the Fiedler vector and spectral clustering

- Graph partitioning and cut size
- **Spectral relaxation and the Fiedler vector**
- Worked exam-style examples and R implementation

Why unconstrained minimum cut is not enough

Since $\text{cut}(S, \bar{S}) = \frac{1}{4}\boldsymbol{x}^T L \boldsymbol{x}$, can we cluster a graph simply by minimising $\boldsymbol{x}^T L \boldsymbol{x}$ over $\boldsymbol{x} \in \{-1, +1\}^n$?

1. **Trivial all-in-one solution:** Without constraints, the minimum of $\boldsymbol{x}^T L \boldsymbol{x}$ over $\{-1, +1\}^n$ is 0, achieved at $\boldsymbol{x} = +\mathbb{1}$ ($S = V, \bar{S} = \emptyset$) and $\boldsymbol{x} = -\mathbb{1}$ ($S = \emptyset, \bar{S} = V$)

2. **Unbalanced "whisker" cuts:** Even if we forbid $\boldsymbol{x} = \pm\mathbb{1}$, minimum cut often slices off a single degree-1 vertex (like vertex 4 in the paw graph) rather than splitting the graph into two meaningful communities of comparable size

To find balanced communities, we must add a **balance constraint** requiring $|S|$ and $|\bar{S}|$ to be equal (or comparable) in size

Encoding balanced bisection algebraically

Suppose n is even and we want a **balanced bisection** with $|S| = |\bar{S}| = n/2$

For any cut indicator vector $\boldsymbol{x} \in \{-1, +1\}^n$, notice two algebraic facts:

1. **Balance equals orthogonality to $\mathbb{1}$:** Since $x_i = +1$ on S and $x_i = -1$ on $\bar{S}$,

$$\mathbb{1}^T \boldsymbol{x} = \sum_{i=1}^{n} x_i = |S| - |\bar{S}|$$

Therefore,

$$|S| = |\bar{S}| \quad \Longleftrightarrow \quad \mathbb{1}^T \boldsymbol{x} = 0 \quad \Longleftrightarrow \quad \boldsymbol{x} \perp \mathbb{1}$$

2. **Fixed Euclidean norm:** Every vector $\boldsymbol{x} \in \{-1, +1\}^n$ has squared norm

$$\|\boldsymbol{x}\|_2^2 = \boldsymbol{x}^T \boldsymbol{x} = \sum_{i=1}^{n} x_i^2 = \sum_{i=1}^{n} (\pm 1)^2 = n$$

General balanced cuts: RatioCut for unequal cluster sizes

What if the two true clusters do not have the exact same size ($|S| \neq |\bar{S}|$)? We penalise small sets by minimising the **RatioCut** (Hagen & Kahng, 1992):

$$\text{RatioCut}(S, \bar{S}) = \text{cut}(S, \bar{S}) \left(\frac{1}{|S|} + \frac{1}{|\bar{S}|} \right) = \frac{n}{|S|\,|\bar{S}|} \text{cut}(S, \bar{S})$$

Define the weighted indicator vector $\boldsymbol{x} = (x_1, \ldots, x_n)^T$ by

$$x_i = \begin{cases} +\sqrt{|\bar{S}|/|S|} & \text{if } i \in S \\ -\sqrt{|S|/|\bar{S}|} & \text{if } i \in \bar{S} \end{cases}$$

Then one checks directly that:

1. $\mathbb{1}^T \boldsymbol{x} = |S|\sqrt{|\bar{S}|/|S|} - |\bar{S}|\sqrt{|S|/|\bar{S}|} = \sqrt{|S|\,|\bar{S}|} - \sqrt{|S|\,|\bar{S}|} = 0$ ($\boldsymbol{x} \perp \mathbb{1}$)
2. $\|\boldsymbol{x}\|_2^2 = |S|\frac{|\bar{S}|}{|S|} + |\bar{S}|\frac{|S|}{|\bar{S}|} = |\bar{S}| + |S| = n$, and $\boldsymbol{x}^T L \boldsymbol{x} = n\,\text{RatioCut}(S, \bar{S})$

From NP-hard discrete cut to continuous spectral relaxation

Both balanced minimum bisection and RatioCut lead to minimising $\boldsymbol{x}^T L \boldsymbol{x}$ subject to $\boldsymbol{x} \perp \mathbb{1}$, $\|\boldsymbol{x}\|_2^2 = n$ and a discrete two-value constraint on the entries x_i:

$$\min_{\substack{\boldsymbol{x} \in \{-1,+1\}^n \\ \boldsymbol{x}^T \mathbb{1} = 0}} \frac{1}{4} \boldsymbol{x}^T L \boldsymbol{x}$$

- **Computational barrier:** Checking all balanced partitions $\binom{n}{n/2}$ is **NP-hard** (for $n = 100$ vertices, $\binom{100}{50} \approx 1.0 \times 10^{29}$)
- **Spectral relaxation:** Relax the discrete constraint on x_i to allow *arbitrary real values* $\boldsymbol{x} \in \mathbb{R}^n$ that still satisfy the norm constraint $\boldsymbol{x}^T \boldsymbol{x} = n$ and the balance constraint $\boldsymbol{x}^T \mathbb{1} = 0$:

$$\min_{\substack{\boldsymbol{x} \in \mathbb{R}^n \\ \boldsymbol{x}^T \boldsymbol{x} = n,\ \boldsymbol{x}^T \mathbb{1} = 0}} \boldsymbol{x}^T L \boldsymbol{x} \quad \Longleftrightarrow \quad \min_{\substack{\boldsymbol{x} \in \mathbb{R}^n \setminus \{\mathbf{0}\} \\ \boldsymbol{x} \perp \mathbb{1}}} \frac{\boldsymbol{x}^T L \boldsymbol{x}}{\boldsymbol{x}^T \boldsymbol{x}}$$

Solving the relaxed problem via the Rayleigh quotient

Theorem 166 (Rayleigh quotient theorem for λ_2)

Let L be the Laplacian of an undirected graph G on n vertices with eigenvalues $0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n$ and orthonormal eigenvectors $\boldsymbol{v}_1 = \frac{1}{\sqrt{n}}\mathbb{1}, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n$. Then

$$\min_{\substack{\boldsymbol{x} \in \mathbb{R}^n \setminus \{\mathbf{0}\} \\ \boldsymbol{x} \perp \mathbb{1}}} \frac{\boldsymbol{x}^T L \boldsymbol{x}}{\boldsymbol{x}^T \boldsymbol{x}} = \lambda_2$$

and the minimum is attained when $\boldsymbol{x}$ is any non-zero scalar multiple of $\boldsymbol{v}_2$

In particular, if we scale $\boldsymbol{v}_2$ so that $\|\boldsymbol{x}\|_2^2 = n$, the optimal continuous vector is $\boldsymbol{x} = \sqrt{n}\,\boldsymbol{v}_2$, with objective value $\boldsymbol{x}^T L \boldsymbol{x} = n\lambda_2$ (giving the lower bound $\text{cut}(S, \bar{S}) \geq \frac{n}{4}\lambda_2$ for any balanced bisection!)

Proof of Theorem 166

Let $\{\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n\}$ be an orthonormal basis of eigenvectors of L with $L\boldsymbol{v}_k = \lambda_k \boldsymbol{v}_k$, where $0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n$ and $\boldsymbol{v}_1 = \frac{1}{\sqrt{n}}\mathbb{1}$

1. Expand any vector $\boldsymbol{x} \in \mathbb{R}^n \setminus \{\mathbf{0}\}$ in the orthonormal basis $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n\}$:

$$\boldsymbol{x} = c_1\boldsymbol{v}_1 + c_2\boldsymbol{v}_2 + \cdots + c_n\boldsymbol{v}_n, \qquad c_k = \boldsymbol{v}_k^T\boldsymbol{x}$$

2. The orthogonality constraint $\boldsymbol{x} \perp \mathbb{1}$ means $c_1 = \boldsymbol{v}_1^T\boldsymbol{x} = \frac{1}{\sqrt{n}}\mathbb{1}^T\boldsymbol{x} = 0$, so

$$\boldsymbol{x} = \sum_{k=2}^{n} c_k\boldsymbol{v}_k \qquad \text{and} \qquad \boldsymbol{x}^T\boldsymbol{x} = \sum_{k=2}^{n} c_k^2 > 0$$

3. Since $L\boldsymbol{x} = \sum_{k=2}^{n} c_k\lambda_k\boldsymbol{v}_k$ and the $\boldsymbol{v}_k$ are orthonormal,

$$\frac{\boldsymbol{x}^T L\boldsymbol{x}}{\boldsymbol{x}^T\boldsymbol{x}} = \frac{\sum_{k=2}^{n}\lambda_k c_k^2}{\sum_{k=2}^{n} c_k^2} \geq \frac{\sum_{k=2}^{n}\lambda_2 c_k^2}{\sum_{k=2}^{n} c_k^2} = \lambda_2$$

with equality when $c_2 \neq 0$ and $c_3 = \cdots = c_n = 0$, i.e. $\boldsymbol{x} = c_2\boldsymbol{v}_2$ □

Algebraic connectivity and the Fiedler vector

Miroslav Fiedler (1973) discovered the fundamental role of λ_2 and $\boldsymbol{v}_2$ in measuring how well-connected a graph is

Definition 167 (Algebraic connectivity and Fiedler vector)

Let $G = (V, E)$ be an undirected graph on n vertices with Laplacian eigenvalues $0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n$

- *The second smallest eigenvalue λ_2 is called the **algebraic connectivity** (or **Fiedler value**) of G*
- *An eigenvector $\boldsymbol{v}_2$ associated with λ_2 (orthogonal to $\mathbb{1}$) is called a **Fiedler vector** of G*

- By Theorem 163, $\lambda_2 > 0$ **if and only if** G **is connected**
- Because $\boldsymbol{v}_2 \perp \mathbb{1}$, the entries of $\boldsymbol{v}_2$ sum to 0 ($\sum_{i=1}^{n} v_{2i} = 0$), so $\boldsymbol{v}_2$ always has **both positive and negative entries**!

The 2-way spectral partitioning algorithm

How do we turn the continuous Fiedler vector $\boldsymbol{v}_2 = (v_{21}, \ldots, v_{2n})^T \in \mathbb{R}^n$ back into a discrete partition $(S, \bar{S})$? We round its entries by their signs!

Algorithm: 2-way unnormalised spectral clustering (Fiedler bisection)

Input: Undirected graph $G = (V, E)$ with symmetric adjacency/similarity matrix $A \in \mathcal{M}_n(\mathbb{R}_{\geq 0})$

1. Compute the degree matrix $D = \operatorname{diag}(A\mathbb{1})$ and the Laplacian $L = D - A$
2. Compute the second smallest eigenvalue λ_2 and associated Fiedler vector $\boldsymbol{v}_2 = (v_{21}, \ldots, v_{2n})^T$ of L
3. Partition the vertices according to the sign of v_{2i}:

$$S = \{i \in V : v_{2i} \geq 0\}, \qquad \bar{S} = \{i \in V : v_{2i} < 0\}$$

(Alternatively, split at the median of $\boldsymbol{v}_2$ if an exact $n/2 - n/2$ split is required)

Note: Because $-\boldsymbol{v}_2$ is also a Fiedler vector, flipping the sign of $\boldsymbol{v}_2$ simply swaps the labels $S \leftrightarrow \bar{S}$, which defines the *exact same* partition of V

Extension to k clusters and normalised Laplacians

What if a graph (or dataset) has $k > 2$ clusters, or vertices with wildly different degrees?

- **k-way spectral clustering:**
  1. Compute the first k eigenvectors $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_k$ of L associated with the k smallest eigenvalues $0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_k$
  2. Stack them as columns of the matrix $U = [\boldsymbol{v}_1 \;\; \boldsymbol{v}_2 \;\; \cdots \;\; \boldsymbol{v}_k] \in \mathcal{M}_{nk}(\mathbb{R})$
  3. Treat row i of U as the new embedding $\boldsymbol{y}_i \in \mathbb{R}^k$ of vertex i, and run k-means on $\{\boldsymbol{y}_1, \ldots, \boldsymbol{y}_n\} \subset \mathbb{R}^k$
- **Normalised graph Laplacians** (used when degrees d_i vary over orders of magnitude):

$$L_{\text{sym}} = D^{-1/2} L D^{-1/2} = \mathbb{I} - D^{-1/2} A D^{-1/2}, \qquad L_{\text{rw}} = D^{-1} L = \mathbb{I} - D^{-1} A$$

Notice that $D^{-1}A$ is the random-walk transition matrix on G! Spectral clustering with L_{sym} or L_{rw} relaxes the degree-normalised cut (Shi & Malik 2000; Ng, Jordan & Weiss 2001)

Choosing the number of clusters k: the eigengap heuristic

How do we choose the number of clusters k from the spectrum $0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n$?

- By Theorem 163, if G consists of k completely disconnected clusters, then

$$\lambda_1 = \lambda_2 = \cdots = \lambda_k = 0 \qquad \text{and} \qquad \lambda_{k+1} > 0$$

- When those k dense clusters are joined by only a few weak between-cluster edges, the first k eigenvalues remain **very close to** 0, while λ_{k+1} jumps sharply upward
- **Eigengap heuristic**: plot $\lambda_1, \lambda_2, \ldots, \lambda_m$ in ascending order and look for the index k with the largest **eigengap**

$$\Delta_k = \lambda_{k+1} - \lambda_k$$

For example, if $\lambda_1 = 0,\ \lambda_2 = 0.04,\ \lambda_3 = 0.07,\ \lambda_4 = 2.85,\ \lambda_5 = 3.10$, the large gap $\Delta_3 = \lambda_4 - \lambda_3 = 2.78$ strongly indicates $k = 3$ clusters

Graph cuts, the Fiedler vector and spectral clustering

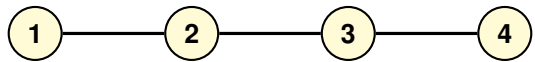
Graph partitioning and cut size

Spectral relaxation and the Fiedler vector

Worked exam-style examples and R implementation

Example 1: spectral bisection of the 4-vertex path P_4

Consider the 4-vertex path graph P_4:



Step 1 – Degree, adjacency and Laplacian matrices:

$$D = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad A = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \implies L = D - A = \begin{pmatrix} 1 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 1 \end{pmatrix}$$

Example 1 (continued): Fiedler vector and cut of P_4

Step 2 – Verify the Fiedler eigenpair $(\lambda_2, \boldsymbol{v}_2)$: Claim: $\lambda_2 = 2 - \sqrt{2} \approx 0.586$ and $\boldsymbol{v}_2 = \begin{pmatrix} 1, & \sqrt{2}-1, & 1-\sqrt{2}, & -1 \end{pmatrix}^T \approx \begin{pmatrix} 1, & 0.414, & -0.414, & -1 \end{pmatrix}^T$
Let us verify $L\boldsymbol{v}_2 = \lambda_2 \boldsymbol{v}_2$ directly:

$$L\boldsymbol{v}_2 = \begin{pmatrix} 1 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ \sqrt{2}-1 \\ 1-\sqrt{2} \\ -1 \end{pmatrix} = \begin{pmatrix} 2-\sqrt{2} \\ 3\sqrt{2}-4 \\ 4-3\sqrt{2} \\ \sqrt{2}-2 \end{pmatrix} = (2-\sqrt{2}) \begin{pmatrix} 1 \\ \sqrt{2}-1 \\ 1-\sqrt{2} \\ -1 \end{pmatrix} = \lambda_2 \boldsymbol{v}_2$$

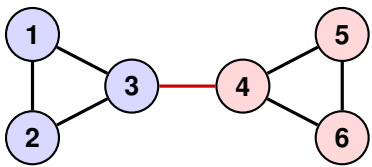
(since $(2-\sqrt{2})(\sqrt{2}-1) = 3\sqrt{2}-4$). Notice also that
$\mathbb{1}^T \boldsymbol{v}_2 = 1 + (\sqrt{2}-1) + (1-\sqrt{2}) - 1 = 0$

Step 3 – Partition by the signs of $\boldsymbol{v}_2$:

- $v_{21} = 1 > 0$ and $v_{22} = \sqrt{2} - 1 \approx 0.414 > 0 \implies S = \{1, 2\}$
- $v_{23} = 1 - \sqrt{2} \approx -0.414 < 0$ and $v_{24} = -1 < 0 \implies \bar{S} = \{3, 4\}$
- The spectral cut severing the middle edge $\{2, 3\}$ has $\text{cut}(S, \bar{S}) = 1$

Example 2: the 6-vertex barbell graph

Consider two triangles $C_1 = \{1, 2, 3\}$ and $C_2 = \{4, 5, 6\}$ joined by a single bridge edge $\{3, 4\}$:



Degrees: $d_1 = d_2 = d_5 = d_6 = 2$, $d_3 = d_4 = 3$

$$L = \left(\begin{array}{ccc|ccc} 2 & -1 & -1 & 0 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 & 0 \\ -1 & -1 & 3 & -1 & 0 & 0 \\ \hline 0 & 0 & -1 & 3 & -1 & -1 \\ 0 & 0 & 0 & -1 & 2 & -1 \\ 0 & 0 & 0 & -1 & -1 & 2 \end{array}\right)$$

Notice that L is nearly block-diagonal: the two 3×3 blocks are coupled only through $\ell_{34} = \ell_{43} = -1$ and the extra $+1$ on ℓ_{33} and ℓ_{44}

Example 2 (continued): exact Fiedler vector of the barbell

By symmetry between $\{1,2\}$ and $\{5,6\}$ and antisymmetry across the bridge $\{3,4\}$ (so that $\mathbb{1}^T \boldsymbol{v}_2 = 0$), we look for a Fiedler vector of the form

$$\boldsymbol{v}_2 = (1,\ 1,\ c,\ -c,\ -1,\ -1)^T$$

Substitute into $L\boldsymbol{v}_2 = \lambda \boldsymbol{v}_2$:

- **Row 1 (and Row 2):** $2(1) - 1(1) - 1(c) = 1 - c = \lambda \implies c = 1 - \lambda$
- **Row 3:** $-1(1) - 1(1) + 3c - 1(-c) = 4c - 2 = \lambda c$

Substitute $c = 1 - \lambda$ into $4c - 2 = \lambda c$:

$$4(1-\lambda) - 2 = \lambda(1-\lambda) \iff \lambda^2 - 5\lambda + 2 = 0 \implies \lambda_2 = \frac{5 - \sqrt{17}}{2} \approx 0.4384$$

Then $c = 1 - \lambda_2 = \frac{\sqrt{17}-3}{2} \approx 0.5616 > 0$, giving the Fiedler vector

$$\boldsymbol{v}_2 \approx (1,\ 1,\ 0.562,\ -0.562,\ -1,\ -1)^T$$

Splitting by sign (≥ 0 vs < 0) gives $S = \{1,2,3\}$ and $\bar{S} = \{4,5,6\}$, cutting only the bridge $\{3,4\}$!

Computing the barbell Fiedler vector in R

We can build the barbell Laplacian in R (either from $D - A$ or with `igraph::laplacian_matrix`) and inspect its eigenvalues and Fiedler vector:

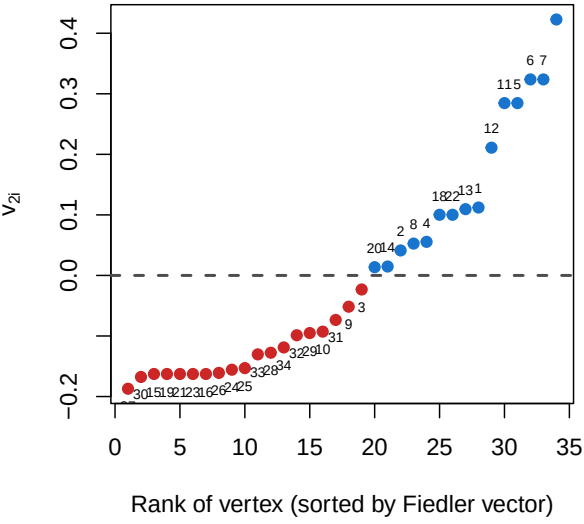
```
A_barbell = matrix(c(
  0, 1, 1, 0, 0, 0,
  1, 0, 1, 0, 0, 0,
  1, 1, 0, 1, 0, 0,
  0, 0, 1, 0, 1, 1,
  0, 0, 0, 1, 0, 1,
  0, 0, 0, 1, 1, 0
), nrow = 6, byrow = TRUE)
L_barbell = diag(rowSums(A_barbell)) - A_barbell
ev_b = eigen(L_barbell, symmetric = TRUE)
round(rev(ev_b$values), 4)          # eigenvalues in ascending order

## [1] 0.0000 0.4384 3.0000 3.0000 3.0000 4.5616

round(ev_b$vectors[, 5], 4)         # unit Fiedler vector v_2
```

Real-world benchmark: Zachary's Karate club network

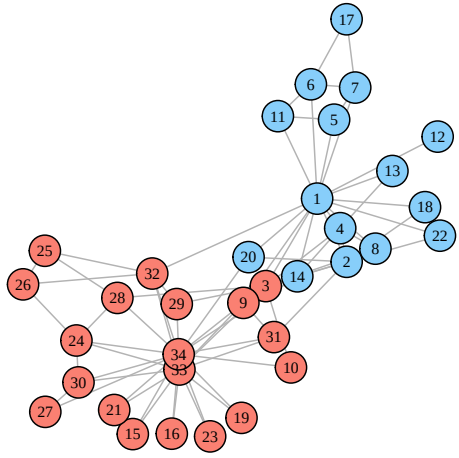
Zachary (1977) recorded social interactions among $n = 34$ members of a university karate club just before a dispute between the instructor (Mr Hi, vertex 1) and the club president (John A, vertex 34) split the club into two factions

```
G_karate = make_graph("Zachary")
L_karate = as.matrix(laplacian_matrix(G_karate))
ev_k = eigen(L_karate, symmetric = TRUE)
n_k = vcount(G_karate)
# Smallest three eigenvalues of L
round(ev_k$values[n_k:(n_k - 2)], 4)

## [1] 0.0000 0.4685 0.9092

# Fiedler vector v_2 (oriented so vertex 1 is positive)
v2_k = ev_k$vectors[, n_k - 1]
if (v2_k[1] < 0) v2_k = -v2_k
table(Sign_of_v2 = ifelse(v2_k >= 0, "Group 1 (+)", "Group 2 (-)"))
```

Sorted entries of the Fiedler vector



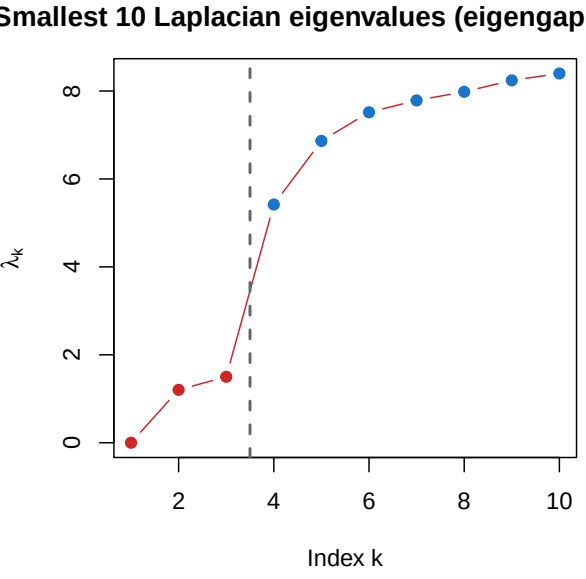
Karate club partitioned by sign(v2)

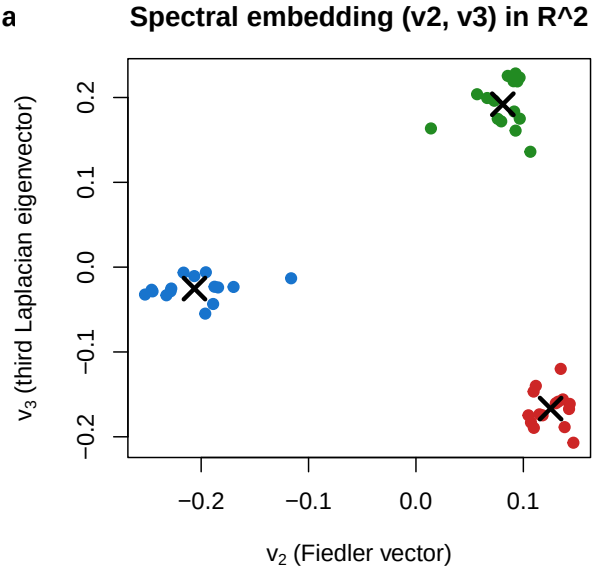


k-way spectral clustering in R ($k = 3$ communities)

Suppose a network has $k = 3$ dense communities of 15 vertices each ($n = 45$), with high within-community edge probability ($p_{\text{in}} = 0.7$) and low between-community probability ($p_{\text{out}} = 0.03$):

```
set.seed(2740)
pm = matrix(0.03, nrow = 3, ncol = 3); diag(pm) = 0.70
G_sbm = sample_sbm(45, pref.matrix = pm, block.sizes = c(15, 15, 15))
L_sbm = as.matrix(laplacian_matrix(G_sbm))
ev_sbm = eigen(L_sbm, symmetric = TRUE)
# Smallest 6 eigenvalues of L (note the jump after lambda_3!)
round(rev(ev_sbm$values)[1:6], 3)

## [1] 0.000 1.203 1.500 5.418 6.865 7.517

# Spectral embedding using v_2 and v_3, followed by k-means (k = 3)
U_sbm = ev_sbm$vectors[, c(44, 43)]
km_sbm = kmeans(U_sbm, centers = 3, nstart = 20)
table(True_block = rep(1:3, each = 15), Spectral_cluster = km_sbm$cluster)
```


Smallest 10 Laplacian eigenvalues (eigengap a
λk
0
2
4
6
8
2
4
6
8
10
Index k


Spectral embedding (v2, v3) in R^2
v3 (third Laplacian eigenvector)
0.2
0.1
0.0
−0.1
−0.2
−0.2
−0.1
0.0
0.1
v2 (Fiedler vector)

Summary of Unit 10

- For an undirected graph $G = (V, E)$ with symmetric adjacency/weight matrix A and degree matrix $D = \mathrm{diag}(A\mathbb{1})$, the **combinatorial Laplacian** is $L = D - A = B^T B$
- The **Laplacian quadratic form** measures smoothness across edges:

$$\boldsymbol{x}^T L \boldsymbol{x} = \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} a_{ij}(x_i - x_j)^2 = \sum_{\{i,j\} \in E} a_{ij}(x_i - x_j)^2 \geq 0$$

- L is real, symmetric and positive semi-definite with eigenvalues $0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n$ and $L\mathbb{1} = \mathbf{0}$; the multiplicity of $\lambda = 0$ equals the number of connected components of G
- For any ± 1 cut indicator vector $\boldsymbol{x} \in \{-1, +1\}^n$, $\mathrm{cut}(S, \bar{S}) = \frac{1}{4} \boldsymbol{x}^T L \boldsymbol{x}$
- Relaxing balanced minimum cut ($\boldsymbol{x} \perp \mathbb{1}$, $\|\boldsymbol{x}\|_2^2 = n$) to $\mathbb{R}^n$ yields the **Fiedler value** λ_2 and **Fiedler vector** $\boldsymbol{v}_2$, whose signs partition G into two communities

Vector space models, sparse text matrices and NMF

MATH 2740 – Mathematics of Data Science – Unit U11

Julien Arino
julien.arino@umanitoba.ca

Department of Mathematics @ University of Manitoba

Fall 2026

The University of Manitoba campuses are located on original lands of Anishinaabeg, Ininew, Anisininew, Dakota and Dene peoples, and on the National Homeland of the Red River Métis. We respect the Treaties that were made on these territories, we acknowledge the harms and mistakes of the past, and we dedicate ourselves to move forward in partnership with Indigenous communities in a spirit of Reconciliation and collaboration.

Outline

- Text tokenisation and the vector space model
- Document-term matrices and TF-IDF scaling
- Angles and cosine similarity in high dimensions
- Non-negative matrix factorisation (NMF)

Why represent text using linear algebra?

- Much of the data generated in science, industry and daily life consists of unstructured text: books, articles, clinical notes and web pages
- Computers cannot perform calculus or geometry directly on strings of characters
- In this unit, we turn discrete text corpora into **sparse non-negative data matrices** and apply three core tools of linear algebra:
  1. **Diagonal matrix scaling** (TF-IDF) to normalise rows (documents) and reweight columns (terms)
  2. **Inner products and angles** (cosine similarity) to measure semantic similarity in high dimensions
  3. **Constrained low-rank factorisation** (Non-negative Matrix Factorisation via Alternating Least Squares) to discover latent topics

Text tokenisation and the vector space model

Document-term matrices and TF-IDF scaling

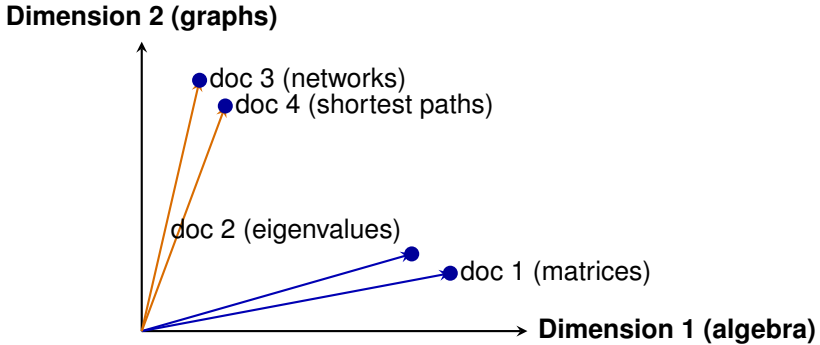
Angles and cosine similarity in high dimensions

Non-negative matrix factorisation (NMF)

The central idea: from words to vectors

- The fundamental challenge in natural language processing (**NLP**) and information retrieval is to translate discrete symbolic data (**words** and **documents**) into a numerical format
- Why? Mathematical models are built on vector spaces, inner products and matrix factorisations rather than dictionary definitions
- The **vector space model** (**VSM**), introduced in information retrieval by Salton, Wong and Yang (1975), represents documents and words as vectors in $\mathbb{R}^p$ so that the geometry of the space reflects semantic relationships
- Documents or words with similar meaning are mapped to vectors pointing in similar directions in $\mathbb{R}^p$

A visual metaphor for the vector space model



- Vectors for documents on linear algebra point in nearly the same direction
- Vectors for documents on graph theory cluster along a different direction
- The angle between a linear algebra document and a graph theory document is large

What is a token?

- A **token** is the fundamental discrete unit of text processed by a model: it can be a word, a word stem, a sub-word or a character
- Converting a raw character string into a sequence of tokens is called **tokenisation**
- For example, after lower-casing and punctuation removal, the sentence “The cat sat on the mat” is tokenised as

$$[\texttt{"the"}, \texttt{"cat"}, \texttt{"sat"}, \texttt{"on"}, \texttt{"the"}, \texttt{"mat"}]$$

- Each unique token in our collection is assigned an integer feature index $j \in \{1, \ldots, p\}$ from a fixed vocabulary, yielding an integer sequence such as $[5, 1, 4, 3, 5, 2]$

Vocabulary and standard basis (one-hot) vectors

Definition 167 (Vocabulary and tokenisation)

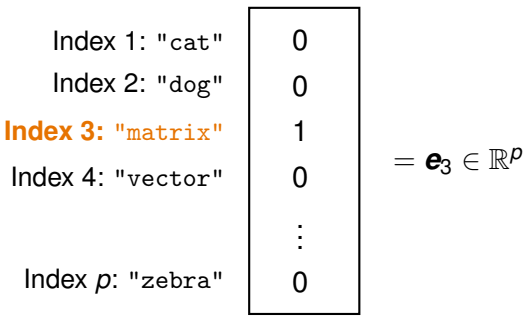
*A **vocabulary** $V = \{w_1, w_2, \ldots, w_p\}$ is an ordered set of $p = |V|$ distinct tokens (serving as the p features). **Tokenisation** maps a raw document d to a finite sequence of tokens $(t_1, t_2, \ldots, t_L)$ with each $t_\ell \in V$*

Definition 168 (One-hot representation of a token)

*Given a vocabulary $V = \{w_1, \ldots, w_p\}$ of size p, the **one-hot vector** associated with token $w_j \in V$ is the j-th standard basis vector $\boldsymbol{e}_j \in \mathbb{R}^p$, whose k-th entry is*

$$(\boldsymbol{e}_j)_k = \delta_{jk} = \begin{cases} 1 & \text{if } k = j \\ 0 & \text{if } k \neq j \end{cases}$$

Visualising one-hot vectors and their limitations



- The set $\{\boldsymbol{e}_1, \ldots, \boldsymbol{e}_p\}$ is the standard orthonormal basis of $\mathbb{R}^p$
- For any two distinct tokens $w_j \neq w_k$:

$$\boldsymbol{e}_j \bullet \boldsymbol{e}_k = 0 \quad \text{and} \quad \|\boldsymbol{e}_j - \boldsymbol{e}_k\|_2 = \sqrt{2}$$

- One-hot vectors alone treat every pair of distinct words as equally unrelated: `"matrix"` is as far from `"vector"` as it is from `"cat"`

From tokens to documents: the bag-of-words model

How do we represent a whole document d_i ($i \in \{1, \ldots, n\}$) as a single feature vector in $\mathbb{R}^p$?

Definition 169 (Bag-of-words document vector)

*Let $V = \{w_1, \ldots, w_p\}$ be a vocabulary of size p and let d_i be a tokenised document $(t_1, t_2, \ldots, t_{L_i})$. The **bag-of-words** (**BoW**) vector of document d_i is*

$$\boldsymbol{x}_i = \sum_{\ell=1}^{L_i} \boldsymbol{e}_{\text{idx}(t_\ell)} = \sum_{j=1}^{p} f_{ij} \boldsymbol{e}_j = \begin{pmatrix} f_{i1} \\ f_{i2} \\ \vdots \\ f_{ip} \end{pmatrix} \in \mathbb{R}^p_{\geq 0}$$

where $f_{ij} \in \mathbb{N}_0$ is the number of times term w_j occurs in document d_i

Example – Constructing bag-of-words vectors

Suppose our vocabulary has $p = 5$ terms:

$$V = \{w_1 = \text{"}\texttt{cat}\text{"},\ w_2 = \text{"}\texttt{dog}\text{"},\ w_3 = \text{"}\texttt{mat}\text{"},\ w_4 = \text{"}\texttt{sat}\text{"},\ w_5 = \text{"}\texttt{the}\text{"}\}$$

Consider two short documents (after lower-casing):

- d_1: `"the cat sat on the mat"` (ignoring `"on"` $\notin V$)
- d_2: `"the dog sat on the dog mat"`

Summing the standard basis vectors gives their bag-of-words vectors in $\mathbb{R}^5_{\geq 0}$:

$$\boldsymbol{x}_1 = 1\boldsymbol{e}_1 + 0\boldsymbol{e}_2 + 1\boldsymbol{e}_3 + 1\boldsymbol{e}_4 + 2\boldsymbol{e}_5 = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 1 \\ 2 \end{pmatrix}, \qquad \boldsymbol{x}_2 = 0\boldsymbol{e}_1 + 2\boldsymbol{e}_2 + 1\boldsymbol{e}_3 + 1\boldsymbol{e}_4 + 2\boldsymbol{e}_5 = \begin{pmatrix} 0 \\ 2 \\ 1 \\ 1 \\ 2 \end{pmatrix}$$

What bag-of-words preserves and what it discards

- Because vector addition in $\mathbb{R}^p$ is commutative ($\boldsymbol{e}_j + \boldsymbol{e}_k = \boldsymbol{e}_k + \boldsymbol{e}_j$), the bag-of-words map completely discards **word order** and grammar
- For example, “dog bites man” and “man bites dog” produce the exact same bag-of-words vector $\boldsymbol{x} \in \mathbb{R}^p_{\geq 0}$
- Despite this simplification, bag-of-words vectors are remarkably effective for **topic classification**, **document clustering** and **search**:
  - Documents about linear algebra frequently contain `"matrix"`, `"rank"` and `"eigenvalue"`
  - Documents about graph theory frequently contain `"vertex"`, `"edge"` and `"path"`

Sparsity and non-negativity in high dimensions

Definition 170 (Support and number of non-zero entries)

For a vector $\boldsymbol{x} = (x_1, \ldots, x_p)^T \in \mathbb{R}^p$, its **support** is the index set of non-zero entries

$$\text{supp}(\boldsymbol{x}) = \{j \in \{1, \ldots, p\} : x_j \neq 0\}$$

and we write $\|\boldsymbol{x}\|_0 = |\text{supp}(\boldsymbol{x})|$ for the number of non-zero entries of $\boldsymbol{x}$

- Note that $\|\boldsymbol{x}\|_0$ is *not* a true norm (since $\|\alpha\boldsymbol{x}\|_0 = \|\boldsymbol{x}\|_0$ for $\alpha \neq 0$), but it measures **sparsity**
- In realistic text corpora, the vocabulary size p is large (10^4 to 10^5 terms), whereas a single document uses only a few hundred distinct words
- Thus $\|\boldsymbol{x}_i\|_0 \ll p$: every document vector $\boldsymbol{x}_i \in \mathbb{R}^p_{\geq 0}$ is a **high-dimensional, sparse, non-negative vector** (98% to 99.9% of its entries are zero)

Vocabulary pre-processing as feature selection

In practice, before forming document vectors $\boldsymbol{x}_i \in \mathbb{R}^p_{\geq 0}$, raw tokens undergo two standard pre-processing steps:

1. **Stop-word removal**: discarding high-frequency function words (`"the"`, `"and"`, `"of"`, `"to"`) that carry little topical meaning
2. **Stemming or lemmatisation**: collapsing inflected forms (`"matrices"`, `"matrix"` $\rightarrow$ `"matrix"`; `"computing"`, `"computed"` $\rightarrow$ `"comput"`) into a single vocabulary feature

- **Linear algebra view**: Removing $p - p'$ stop words from a vocabulary of size p is equivalent to left-multiplying each document vector $\boldsymbol{x}_i \in \mathbb{R}^p_{\geq 0}$ by a **feature-selection matrix** $P \in \mathcal{M}_{p',p}(\{0,1\})$ obtained by deleting the stop-word rows of $\mathbb{I}_p$:

$$\boldsymbol{x}_i^{\text{reduced}} = P\boldsymbol{x}_i \in \mathbb{R}^{p'}_{\geq 0} \quad \Longleftrightarrow \quad (\boldsymbol{x}_i^{\text{reduced}})^T = \boldsymbol{x}_i^T P^T \in \mathbb{R}^{1\times p'}_{\geq 0}$$

Text tokenisation and the vector space model

Document-term matrices and TF-IDF scaling

Angles and cosine similarity in high dimensions

Non-negative matrix factorisation (NMF)

Document-term matrices and TF-IDF scaling

The document-term data matrix

TF-IDF as diagonal matrix multiplication

The document-term data matrix

Definition 171 (Document-term data matrix)

*Given a corpus $D = \{d_1, \ldots, d_n\}$ of n documents (observations) and a vocabulary $V = \{w_1, \ldots, w_p\}$ of p terms (features), the **data matrix** (called in NLP and information retrieval the **document-term matrix**) is the non-negative matrix*

$$X = [x_{ij}] = \begin{pmatrix} \boldsymbol{x}_1^T \\ \boldsymbol{x}_2^T \\ \vdots \\ \boldsymbol{x}_n^T \end{pmatrix} = \begin{bmatrix} \boldsymbol{c}_1 & \boldsymbol{c}_2 & \cdots & \boldsymbol{c}_p \end{bmatrix} \in \mathcal{M}_{np}(\mathbb{R}_{\geq 0})$$

where:

- *Row $i \in \{1, \ldots, n\}$ is the row vector $\boldsymbol{x}_i^T \in \mathbb{R}_{\geq 0}^{1 \times p}$ of document (observation) d_i*
- *Column $j \in \{1, \ldots, p\}$ is the column vector $\boldsymbol{c}_j \in \mathbb{R}_{\geq 0}^{n}$ of vocabulary term (feature) w_j*
- *Entry $x_{ij} \geq 0$ is the weight (e.g., raw frequency f_{ij}) of term w_j in document d_i*

Example – A toy corpus of $n = 4$ documents and $p = 6$ terms

Consider $n = 4$ short documents across two mathematical themes, tracked over the vocabulary $V = \{\texttt{matrix}, \texttt{vector}, \texttt{eigenvalue}, \texttt{graph}, \texttt{network}, \texttt{data}\}$ ($p = 6$):

- **Document d_1 (linear algebra)**: mentions `matrix` (3 times), `vector` (2 times), `eigenvalue` (2 times) and `data` (4 times)
- **Document d_2 (linear algebra)**: mentions `matrix` (2 times), `vector` (3 times) and `data` (3 times)
- **Document d_3 (graph theory)**: mentions `graph` (3 times), `network` (2 times) and `data` (5 times)
- **Document d_4 (graph theory)**: mentions `vector` (1 time), `graph` (2 times), `network` (3 times) and `data` (4 times)

Example – Explicit 4×6 document-term matrix X

Assembling the document vectors $\boldsymbol{x}_1^T, \boldsymbol{x}_2^T, \boldsymbol{x}_3^T, \boldsymbol{x}_4^T \in \mathbb{R}_{\geq 0}^{1\times 6}$ as rows gives $X \in \mathcal{M}_{46}(\mathbb{R}_{\geq 0})$:

Doc	matrix	vector	eigen	graph	network	data
d_1	3	2	2	0	0	4
d_2	2	3	0	0	0	3
d_3	0	0	0	3	2	5
d_4	0	1	0	2	3	4

$$\Longrightarrow X = \begin{pmatrix} 3 & 2 & 2 & 0 & 0 & 4 \\ 2 & 3 & 0 & 0 & 0 & 3 \\ 0 & 0 & 0 & 3 & 2 & 5 \\ 0 & 1 & 0 & 2 & 3 & 4 \end{pmatrix}$$

- Notice the near-block structure in columns 1–5 separating $\{d_1, d_2\}$ from $\{d_3, d_4\}$
- In contrast, column 6 (`data`) has large entries across **all four rows**, obscuring the two topics!

Dual geometric views of $X \in \mathcal{M}_{np}(\mathbb{R}_{\geq 0})$

Every document-term data matrix $X \in \mathcal{M}_{np}(\mathbb{R}_{\geq 0})$ admits two complementary geometric interpretations:

1. Row view (document space $\mathbb{R}^p_{\geq 0}$)

- Each transposed row $\boldsymbol{x}_i \in \mathbb{R}^p_{\geq 0}$ ($i = 1, \ldots, n$) represents a **document** (observation) as a point in p-dimensional term space
- Used to compare, cluster or classify documents

2. Column view (term space $\mathbb{R}^n_{\geq 0}$)

- Each column $\boldsymbol{c}_j \in \mathbb{R}^n_{\geq 0}$ ($j = 1, \ldots, p$) represents a **term** (feature) via its occurrence profile across the n documents
- Two words that co-occur in the same documents have similar column vectors

Document-term matrices and TF-IDF scaling

The document-term data matrix

TF-IDF as diagonal matrix multiplication

Why raw term counts fail

Using raw frequency counts $x_{ij} = f_{ij}$ directly in X suffers from two major defects:

1. **Common words dominate rare discriminative words**
   - Words like `"the"`, `"is"` or `"data"` appear in almost every document with large counts, dominating Euclidean norms and inner products
   - Rare domain terms like `"eigenvalue"` or `"isomorphism"` occur fewer times but carry far more information about a document's topic

2. **Document length distorts vector magnitude**
   - A 20-page article and a 1-paragraph abstract on the exact same topic have proportional term frequencies, but the longer document has a much larger norm $\|\boldsymbol{x}_i\|_2$

Term frequency and inverse document frequency (TF-IDF)

Definition 172 (Document frequency, IDF and TF-IDF)

Let $X = [f_{ij}] \in \mathcal{M}_{np}(\mathbb{R}_{\geq 0})$ be a raw document-term count matrix for n documents and p terms:

1. *The* ***term frequency*** tf_{ij} *is the raw count f_{ij} (or a scaled variant such as $f_{ij} / \sum_{k=1}^{p} f_{ik}$)*
2. *The* ***document frequency*** df_j *of term w_j is the number of documents in which w_j appears:*

$$\mathrm{df}_j = \left|\{i \in \{1, \ldots, n\} : f_{ij} > 0\}\right| = \|\boldsymbol{c}_j\|_0 \in \{1, \ldots, n\}$$

3. *The* ***inverse document frequency*** *of term w_j is*

$$\mathrm{idf}_j = \log\left(\frac{n}{\mathrm{df}_j}\right) \geq 0$$

4. *The* ***TF-IDF weight*** *of term w_j in document d_i is* $\mathrm{tfidf}_{ij} = \mathrm{tf}_{ij} \cdot \mathrm{idf}_j$

How $\text{idf}_j = \log(n/\text{df}_j)$ reweights vocabulary terms

Assume every vocabulary term $w_j \in V$ appears in at least one document ($1 \leq \text{df}_j \leq n$):

Term behaviour	df_j	$\text{idf}_j = \log(n/\text{df}_j)$	**Effect on column j of X**
Appears in all documents	n	$\log(1) = 0$	Zeroed out completely
Appears in half of documents	$n/2$	$\log(2) \approx 0.693$	Moderately weighted
Appears in 1 document only	1	$\log(n)$ (maximal)	Strongly amplified

- Note: throughout this course, $\log(\cdot)$ denotes the natural logarithm $\ln(\cdot)$ (changing the base of the logarithm only multiplies all idf_j by a positive constant)
- In practice, when out-of-corpus queries might have $\text{df}_j = 0$, smoothed variants such as $\log\left(\frac{n+1}{\text{df}_j+1}\right) + 1$ are also used

TF-IDF and length normalisation as diagonal matrix scaling

Theorem 173 (Row and column scaling via diagonal matrices)

Let $X = [x_{ij}] \in \mathcal{M}_{np}(\mathbb{R}_{\geq 0})$ be a document-term data matrix with non-zero rows $\boldsymbol{x}_1^T, \ldots, \boldsymbol{x}_n^T$. Define the two diagonal matrices

$$D_{IDF} = \operatorname{diag}(\text{idf}_1, \ldots, \text{idf}_p) \in \mathcal{M}_p(\mathbb{R}_{\geq 0}), \qquad D_{len}^{-1} = \operatorname{diag}\left(\frac{1}{\|\boldsymbol{x}_1^{TF\text{-}IDF}\|_2}, \ldots, \frac{1}{\|\boldsymbol{x}_n^{TF\text{-}IDF}\|_2} \right) \in \mathcal{M}$$

where $(\boldsymbol{x}_i^{TF\text{-}IDF})^T = \boldsymbol{x}_i^T D_{IDF}$ is the i-th row of XD_{IDF}. Then:

1. ***Right-multiplication*** *$X_{TF\text{-}IDF} = XD_{IDF}$ scales the **columns** (terms/features) of X: entry (i,j) is $x_{ij}\,\text{idf}_j$*
2. ***Left-multiplication*** *$D_{len}^{-1}A$ scales the **rows** (documents/observations) of A: entry (i,j) is $a_{ij}/\|\boldsymbol{x}_i^{TF\text{-}IDF}\|_2$*
3. *The combined matrix $\hat{X} = D_{len}^{-1}XD_{IDF} \in \mathcal{M}_{np}(\mathbb{R}_{\geq 0})$ has entries $\hat{x}_{ij} = \frac{x_{ij}\,\text{idf}_j}{\|\boldsymbol{x}_i^{TF\text{-}IDF}\|_2}$ and unit-norm document rows $\|\hat{\boldsymbol{x}}_i\|_2 = 1$*

Proof of Theorem 173

Let $D_{\text{IDF}} = [c_{kj}] \in \mathcal{M}_p$ with $c_{jj} = \text{idf}_j$ and $c_{kj} = 0$ for $k \neq j$. By definition of matrix multiplication, the (i, j) entry of XD_{IDF} is

$$(XD_{\text{IDF}})_{ij} = \sum_{k=1}^{p} x_{ik} c_{kj} = x_{ij} c_{jj} = x_{ij} \, \text{idf}_j$$

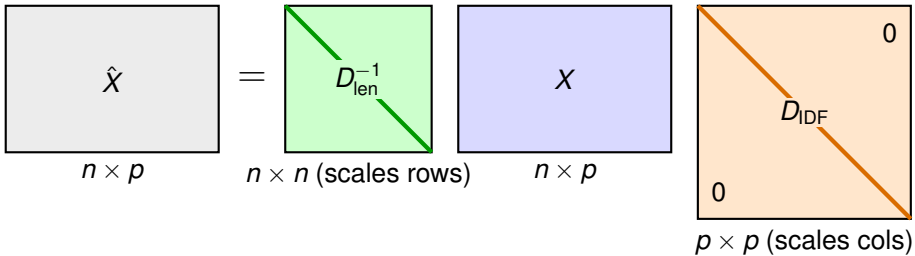
Thus column j of X is multiplied by the scalar idf_j

Similarly, let $D_{\text{len}}^{-1} = [d_{i\ell}] \in \mathcal{M}_n$ with $d_{ii} = 1/\|\boldsymbol{x}_i^{\text{TF-IDF}}\|_2$ and $d_{i\ell} = 0$ for $\ell \neq i$. For any matrix $A = [a_{\ell j}] \in \mathcal{M}_{np}$, the (i, j) entry of $D_{\text{len}}^{-1} A$ is

$$(D_{\text{len}}^{-1} A)_{ij} = \sum_{\ell=1}^{n} d_{i\ell} a_{\ell j} = d_{ii} a_{ij} = \frac{a_{ij}}{\|\boldsymbol{x}_i^{\text{TF-IDF}}\|_2}$$

Setting $A = XD_{\text{IDF}}$ gives $\hat{x}_{ij} = (D_{\text{len}}^{-1} XD_{\text{IDF}})_{ij} = \frac{x_{ij} \, \text{idf}_j}{\|\boldsymbol{x}_i^{\text{TF-IDF}}\|_2}$ □

Visualising $\hat{X} = D_{\text{len}}^{-1} X D_{\text{IDF}}$



- Notice the fundamental pattern from elementary linear algebra:
  - **Left-multiplying** by a diagonal matrix scales the **rows** (the n documents / observations)
  - **Right-multiplying** by a diagonal matrix scales the **columns** (the p vocabulary terms / features)

Example – Computing XD_{IDF} by hand (Part 1)

Consider a corpus of $n = 4$ documents over $p = 4$ terms with raw document-term matrix $X \in \mathcal{M}_{44}(\mathbb{R}_{\geq 0})$:

Doc	w_1 (matrix)	w_2 (eigenvalue)	w_3 (graph)	w_4 (data)
d_1	3	2	0	5
d_2	2	0	0	4
d_3	0	0	1	6
d_4	0	0	3	5

$$\implies X = \begin{pmatrix} 3 & 2 & 0 & 5 \\ 2 & 0 & 0 & 4 \\ 0 & 0 & 1 & 6 \\ 0 & 0 & 3 & 5 \end{pmatrix}$$

Step 1: Count non-zero entries in each column to find df_j and $\text{idf}_j = \log(4/\text{df}_j)$:

- Column 1 (w_1): appears in $d_1, d_2 \implies \text{df}_1 = 2 \implies \text{idf}_1 = \log(4/2) = \log 2$
- Column 2 (w_2): appears in d_1 only
  $\implies \text{df}_2 = 1 \implies \text{idf}_2 = \log(4/1) = \log 4 = 2\log 2$
- Column 3 (w_3): appears in $d_3, d_4 \implies \text{df}_3 = 2 \implies \text{idf}_3 = \log(4/2) = \log 2$
- Column 4 (w_4): appears in
  $d_1, d_2, d_3, d_4 \implies \text{df}_4 = 4 \implies \text{idf}_4 = \log(4/4) = \log 1 = 0$

Example – Computing XD_{IDF} by hand (Part 2)

Step 2: Form D_{IDF} and right-multiply X:

$$D_{\text{IDF}} = \operatorname{diag}(\log 2,\ 2\log 2,\ \log 2,\ 0) = (\log 2)\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

Multiplying each column of X by its diagonal entry gives

$$X_{\text{TF-IDF}} = XD_{\text{IDF}} = (\log 2)\begin{pmatrix} 3 & 4 & 0 & 0 \\ 2 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 3 & 0 \end{pmatrix} \approx \begin{pmatrix} 2.08 & 2.77 & 0 & 0 \\ 1.39 & 0 & 0 & 0 \\ 0 & 0 & 0.69 & 0 \\ 0 & 0 & 2.08 & 0 \end{pmatrix}$$

- Column 4 (`data`) is annihilated because it carries zero discriminative information across this corpus, whereas the rare term `eigenvalue` in Column 2 has its relative weight doubled!

Example – Row normalisation $D_{\text{len}}^{-1} X D_{\text{IDF}}$ (Part 3)

Step 3: Left-multiply $X_{\text{TF-IDF}}$ by D_{len}^{-1} to normalise the document rows in ℓ_2 norm:

- The Euclidean norms of the four rows of $X_{\text{TF-IDF}} = X D_{\text{IDF}}$ are:

$$\|\boldsymbol{x}_1^{\text{TF-IDF}}\|_2 = (\log 2)\sqrt{3^2 + 4^2} = 5 \log 2, \quad \|\boldsymbol{x}_2^{\text{TF-IDF}}\|_2 = (\log 2)\sqrt{2^2 + 0^2} = 2 \log 2$$
$$\|\boldsymbol{x}_3^{\text{TF-IDF}}\|_2 = (\log 2)\sqrt{1^2} = \log 2, \qquad \|\boldsymbol{x}_4^{\text{TF-IDF}}\|_2 = (\log 2)\sqrt{3^2} = 3 \log 2$$

- Left-multiplying by $D_{\text{len}}^{-1} = (\log 2)^{-1} \text{diag}(1/5,\ 1/2,\ 1,\ 1/3)$ cancels $\log 2$ and yields unit-norm document rows:

$$\hat{X} = D_{\text{len}}^{-1} X D_{\text{IDF}} = \begin{pmatrix} 3/5 & 4/5 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} = \begin{pmatrix} 0.6 & 0.8 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

Text tokenisation and the vector space model

Document-term matrices and TF-IDF scaling

Angles and cosine similarity in high dimensions

Non-negative matrix factorisation (NMF)

Why Euclidean distance fails for sparse document vectors

Suppose we want to measure how similar two documents $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^p_{\geq 0}$ are. Why not use the Euclidean distance $\|\boldsymbol{x} - \boldsymbol{y}\|_2$?

- Consider three documents over $V = \{\texttt{matrix}, \texttt{vector}, \texttt{graph}\}$ ($p = 3$):

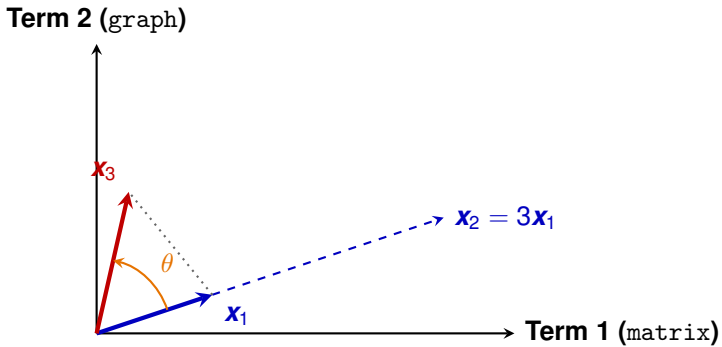
$$\boldsymbol{x}_1 = \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}, \qquad \boldsymbol{x}_2 = \begin{pmatrix} 5 \\ 5 \\ 0 \end{pmatrix} = 5\boldsymbol{x}_1, \qquad \boldsymbol{x}_3 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$$

- Documents d_1 and d_2 have the **exact same word proportions** (pure linear algebra), whereas d_3 shares **no words at all** with d_1
- Yet their Euclidean distances are:

$$\|\boldsymbol{x}_1 - \boldsymbol{x}_2\|_2 = \left\| \begin{pmatrix} -4 \\ -4 \\ 0 \end{pmatrix} \right\|_2 = \sqrt{32} \approx 5.66 \quad > \quad \|\boldsymbol{x}_1 - \boldsymbol{x}_3\|_2 = \left\| \begin{pmatrix} 1 \\ 1 \\ -1 \end{pmatrix} \right\|_2 = \sqrt{3} \approx 1.73$$

- Euclidean distance judges d_1 to be much closer to the unrelated short document d_3 than to a longer copy of itself!

Direction matters more than magnitude



- Scaling a document vector by a positive constant ($\boldsymbol{x}_2 = \alpha \boldsymbol{x}_1$ with $\alpha > 0$) changes its length $\|\boldsymbol{x}\|_2$ but preserves its **direction**
- Therefore, semantic similarity between documents should be measured by the **angle** θ between their vectors

Cosine similarity

Definition 174 (Cosine similarity)

Let $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^p$ be two non-zero vectors. The ***cosine similarity*** *between $\boldsymbol{x}$ and $\boldsymbol{y}$ is the cosine of the angle $\theta \in [0, \pi]$ between them:*

$$\text{sim}(\boldsymbol{x}, \boldsymbol{y}) = \cos(\theta) = \frac{\boldsymbol{x} \bullet \boldsymbol{y}}{\|\boldsymbol{x}\|_2 \, \|\boldsymbol{y}\|_2} = \frac{\sum_{j=1}^{p} x_j y_j}{\sqrt{\sum_{j=1}^{p} x_j^2} \sqrt{\sum_{j=1}^{p} y_j^2}}$$

Equivalently, in terms of the ℓ_2-normalised unit vectors $\hat{\boldsymbol{x}} = \frac{\boldsymbol{x}}{\|\boldsymbol{x}\|_2}$ and $\hat{\boldsymbol{y}} = \frac{\boldsymbol{y}}{\|\boldsymbol{y}\|_2}$,

$$\text{sim}(\boldsymbol{x}, \boldsymbol{y}) = \hat{\boldsymbol{x}} \bullet \hat{\boldsymbol{y}} = \hat{\boldsymbol{x}}^T \hat{\boldsymbol{y}}$$

Fundamental properties of cosine similarity

Proposition 175 (Properties of cosine similarity)

Let $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^p$ be non-zero vectors. Then:

1. ***Positive scale invariance**: For any scalars $\alpha > 0$ and $\beta > 0$,*

$$\text{sim}(\alpha \boldsymbol{x}, \beta \boldsymbol{y}) = \text{sim}(\boldsymbol{x}, \boldsymbol{y})$$

2. ***General bounds**: By the Cauchy-Schwarz inequality, $\text{sim}(\boldsymbol{x}, \boldsymbol{y}) \in [-1, 1]$, with $\text{sim}(\boldsymbol{x}, \boldsymbol{y}) = 1 \iff \boldsymbol{y} = c\boldsymbol{x}$ for some $c > 0$*
3. ***Non-negative orthant bounds**: If $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^p_{\geq 0}$ are non-negative vectors, then*

$$0 \leq \text{sim}(\boldsymbol{x}, \boldsymbol{y}) \leq 1$$

Furthermore, for $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^p_{\geq 0}$,

$$\text{sim}(\boldsymbol{x}, \boldsymbol{y}) = 0 \iff \text{supp}(\boldsymbol{x}) \cap \text{supp}(\boldsymbol{y}) = \emptyset$$

Proof of Proposition 175 (Part 1: scale invariance)

Let $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^p \setminus \{\mathbf{0}\}$ and let $\alpha, \beta > 0$. Using the bilinearity of the dot product and the positive homogeneity of the Euclidean norm ($\|\alpha\boldsymbol{x}\|_2 = |\alpha| \, \|\boldsymbol{x}\|_2 = \alpha\|\boldsymbol{x}\|_2$ since $\alpha > 0$), we compute

$$\begin{aligned}
\mathrm{sim}(\alpha\boldsymbol{x}, \beta\boldsymbol{y}) &= \frac{(\alpha\boldsymbol{x}) \bullet (\beta\boldsymbol{y})}{\|\alpha\boldsymbol{x}\|_2 \, \|\beta\boldsymbol{y}\|_2} \\
&= \frac{\alpha\beta(\boldsymbol{x} \bullet \boldsymbol{y})}{(\alpha\|\boldsymbol{x}\|_2)(\beta\|\boldsymbol{y}\|_2)} \\
&= \frac{\cancel{\alpha\beta}(\boldsymbol{x} \bullet \boldsymbol{y})}{\cancel{\alpha\beta}\|\boldsymbol{x}\|_2 \, \|\boldsymbol{y}\|_2} \\
&= \mathrm{sim}(\boldsymbol{x}, \boldsymbol{y})
\end{aligned}$$

(Note: if $\alpha < 0$ and $\beta > 0$, then $|\alpha| = -\alpha$, so $\mathrm{sim}(\alpha\boldsymbol{x}, \beta\boldsymbol{y}) = -\mathrm{sim}(\boldsymbol{x}, \boldsymbol{y})$) □

Proof of Proposition 175 (Part 2: non-negative vectors)

Assume $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^p_{\geq 0} \setminus \{\mathbf{0}\}$, so $x_j \geq 0$ and $y_j \geq 0$ for every $j \in \{1, \ldots, p\}$

1. Since the product of two non-negative real numbers is non-negative, $x_j y_j \geq 0$ for all $j = 1, \ldots, p$. Summing over j yields

$$\boldsymbol{x} \bullet \boldsymbol{y} = \sum_{j=1}^{p} x_j y_j \geq 0$$

   Dividing by $\|\boldsymbol{x}\|_2 \|\boldsymbol{y}\|_2 > 0$ and combining with Cauchy-Schwarz gives $0 \leq \text{sim}(\boldsymbol{x}, \boldsymbol{y}) \leq 1$

2. Because each summand $x_j y_j$ is non-negative, a sum of non-negative terms $\sum_{j=1}^{p} x_j y_j$ equals 0 if and only if **every individual term is zero**:

$$\text{sim}(\boldsymbol{x}, \boldsymbol{y}) = 0 \iff \sum_{j=1}^{p} x_j y_j = 0 \iff \forall j \in \{1, \ldots, p\},\ x_j y_j = 0$$

   Since $x_j y_j = 0 \iff (x_j = 0 \text{ or } y_j = 0)$, there is no index j with both $x_j > 0$ and $y_j > 0$, which means $\text{supp}(\boldsymbol{x}) \cap \text{supp}(\boldsymbol{y}) = \emptyset$ □

Interpretation of Proposition 175 for text

- Because bag-of-words and TF-IDF vectors lie in the non-negative orthant $\mathbb{R}^p_{\geq 0}$, two documents can **never** have negative cosine similarity ($\theta \in [0, \pi/2]$)
- $\text{sim}(\boldsymbol{x}_i, \boldsymbol{x}_k) = 0$ ($\theta = \pi/2$, orthogonal vectors) occurs **if and only if the two documents share no vocabulary words in common** (after any zero-IDF stop words are removed)
- Moreover, for ℓ_2-normalised vectors $\hat{\boldsymbol{x}} = \boldsymbol{x}/\|\boldsymbol{x}\|_2$ and $\hat{\boldsymbol{y}} = \boldsymbol{y}/\|\boldsymbol{y}\|_2$ on the unit sphere S^{p-1}, Euclidean distance and cosine similarity are monotonically related:

$$\|\hat{\boldsymbol{x}} - \hat{\boldsymbol{y}}\|_2^2 = \|\hat{\boldsymbol{x}}\|_2^2 + \|\hat{\boldsymbol{y}}\|_2^2 - 2(\hat{\boldsymbol{x}} \bullet \hat{\boldsymbol{y}}) = 2\,(1 - \text{sim}(\boldsymbol{x}, \boldsymbol{y}))$$

All-pairs cosine similarity via the Gram matrix

How do we compute the cosine similarity between **all pairs** of n documents in a data matrix $X \in \mathcal{M}_{np}$ with rows $\boldsymbol{x}_1^T, \ldots, \boldsymbol{x}_n^T$ at once?

Theorem 176 (Cosine similarity Gram matrix)

Let $X \in \mathcal{M}_{np}$ have non-zero document rows $\boldsymbol{x}_1^T, \ldots, \boldsymbol{x}_n^T \in \mathbb{R}^{1\times p}$ and let

$$\hat{X} = D_{norm}^{-1} X = \begin{pmatrix} \hat{\boldsymbol{x}}_1^T \\ \hat{\boldsymbol{x}}_2^T \\ \vdots \\ \hat{\boldsymbol{x}}_n^T \end{pmatrix} \in \mathcal{M}_{np} \quad where \quad D_{norm}^{-1} = \text{diag}\left(\frac{1}{\|\boldsymbol{x}_1\|_2}, \ldots, \frac{1}{\|\boldsymbol{x}_n\|_2} \right)$$

Then the symmetric ***Gram matrix***

$$S = \hat{X}\hat{X}^T \in \mathcal{M}_n(\mathbb{R})$$

has (i,j) entry equal to the pairwise cosine similarity $s_{ij} = \hat{\boldsymbol{x}}_i^T \hat{\boldsymbol{x}}_j = \text{sim}(\boldsymbol{x}_i, \boldsymbol{x}_j)$, with $s_{ii} = 1$ along the diagonal

Example – Pairwise cosine similarity by hand (Part 1)

Consider $n = 3$ sparse document vectors in $\mathbb{R}^5_{\geq 0}$ ($p = 5$):

$$\boldsymbol{x}_1 = \begin{pmatrix} 3 \\ 4 \\ 0 \\ 0 \\ 0 \end{pmatrix}, \qquad \boldsymbol{x}_2 = \begin{pmatrix} 6 \\ 8 \\ 0 \\ 0 \\ 0 \end{pmatrix}, \qquad \boldsymbol{x}_3 = \begin{pmatrix} 0 \\ 2 \\ 0 \\ 2 \\ 1 \end{pmatrix}$$

Step 1: Compute the Euclidean norm of each document vector:

$$\|\boldsymbol{x}_1\|_2 = \sqrt{3^2 + 4^2 + 0 + 0 + 0} = \sqrt{25} = 5$$
$$\|\boldsymbol{x}_2\|_2 = \sqrt{6^2 + 8^2 + 0 + 0 + 0} = \sqrt{100} = 10$$
$$\|\boldsymbol{x}_3\|_2 = \sqrt{0 + 2^2 + 0 + 2^2 + 1^2} = \sqrt{9} = 3$$

Example – Pairwise cosine similarity by hand (Part 2)

Step 2: Compute pairwise dot products (only shared non-zero indices contribute!):

$$\begin{aligned}
\boldsymbol{x}_1 \bullet \boldsymbol{x}_2 &= 3(6) + 4(8) = 18 + 32 = 50 \\
\boldsymbol{x}_1 \bullet \boldsymbol{x}_3 &= 4(2) = 8 \\
\boldsymbol{x}_2 \bullet \boldsymbol{x}_3 &= 8(2) = 16
\end{aligned}$$

Step 3: Divide by norms to form the Gram matrix $S = \hat{X}\hat{X}^T$:

$$\text{sim}(\boldsymbol{x}_1, \boldsymbol{x}_2) = \frac{50}{5 \times 10} = 1, \quad \text{sim}(\boldsymbol{x}_1, \boldsymbol{x}_3) = \frac{8}{5 \times 3} = \frac{8}{15}, \quad \text{sim}(\boldsymbol{x}_2, \boldsymbol{x}_3) = \frac{16}{10 \times 3} = \frac{8}{15}$$

$$\implies S = \hat{X}\hat{X}^T = \begin{pmatrix} 1 & 1 & 8/15 \\ 1 & 1 & 8/15 \\ 8/15 & 8/15 & 1 \end{pmatrix} \approx \begin{pmatrix} 1.000 & 1.000 & 0.533 \\ 1.000 & 1.000 & 0.533 \\ 0.533 & 0.533 & 1.000 \end{pmatrix}$$

Text tokenisation and the vector space model

Document-term matrices and TF-IDF scaling

Angles and cosine similarity in high dimensions

Non-negative matrix factorisation (NMF)

Non-negative matrix factorisation (NMF)

SVD vs NMF on non-negative data

Topic modelling interpretation of W and H

Alternating least squares (ALS)

Recall from Unit 04: low-rank approximation via SVD

In Unit 04, we saw that any matrix $X \in \mathcal{M}_{np}(\mathbb{R})$ admits a Singular Value Decomposition $X = U\Sigma V^T = \sum_{r=1}^{\text{rank}(X)} \sigma_r \boldsymbol{u}_r \boldsymbol{v}_r^T$

- By the **Eckart-Young theorem**, the best rank-k approximation to X in the Frobenius norm $\|A\|_F = \sqrt{\sum_{i=1}^{n} \sum_{j=1}^{p} a_{ij}^2}$ is the **truncated SVD**:

$$X_k = \sum_{r=1}^{k} \sigma_r \boldsymbol{u}_r \boldsymbol{v}_r^T = U_k \Sigma_k V_k^T$$

- Applied to a document-term matrix X, truncated SVD is known in information retrieval as **Latent Semantic Analysis** (**LSA**, Deerwester et al. 1990)

Why truncated SVD is awkward on non-negative data

Many real-world data matrices are inherently **non-negative** ($x_{ij} \geq 0$):

- Document-term word count and TF-IDF matrices
- Pixel intensities in greyscale or colour images
- Audio spectrograms, chemical concentrations and gene expression counts

Why does truncated SVD struggle with interpretability on $X \in \mathcal{M}_{np}(\mathbb{R}_{\geq 0})$?

- By the Perron-Frobenius theorem, the first singular vectors $\boldsymbol{u}_1, \boldsymbol{v}_1$ can be chosen non-negative
- However, U_k and V_k have **orthonormal columns**: for every $r \geq 2$,

$$\boldsymbol{u}_r \bullet \boldsymbol{u}_1 = 0 \quad \text{and} \quad \boldsymbol{v}_r \bullet \boldsymbol{v}_1 = 0$$

- As proved in Proposition 175, a non-zero vector orthogonal to a strictly positive vector $\boldsymbol{v}_1$ **must contain both positive and negative entries**!

Subtractive cancellation vs additive parts

- Because $\boldsymbol{u}_2, \ldots, \boldsymbol{u}_k$ and $\boldsymbol{v}_2, \ldots, \boldsymbol{v}_k$ contain negative numbers, SVD reconstructs a document $\boldsymbol{x}_i \approx \sum_{r=1}^{k} (\sigma_r u_{ir}) \boldsymbol{v}_r$ through **subtractive cancellation**:
  - What does it mean for a "topic" vector $\boldsymbol{v}_2$ to contain $-0.54 \times$ "`matrix`"?
  - What does it mean to subtract 0.3 of Topic 2 from a document so that negative word counts cancel out?
- For example, on our 4×6 document-term matrix X, the first two right singular vectors (term-topic directions) in V_2 are

$$\boldsymbol{v}_1 = \begin{pmatrix} 0.30 \\ 0.30 \\ 0.11 \\ 0.32 \\ 0.31 \\ 0.78 \end{pmatrix} \geq \mathbf{0}, \qquad \boldsymbol{v}_2 = \begin{pmatrix} \mathbf{-0.54} \\ \mathbf{-0.47} \\ \mathbf{-0.24} \\ 0.45 \\ 0.45 \\ 0.14 \end{pmatrix} \notin \mathbb{R}^6_{\geq 0}$$

- To obtain directly interpretable topics (or parts of an image), we drop the orthogonality constraint and instead enforce **non-negativity** on both factors!

Non-negative Matrix Factorisation (NMF)

Definition 177 (Non-negative Matrix Factorisation (NMF))

Given a non-negative data matrix $X \in \mathcal{M}_{np}(\mathbb{R}_{\geq 0})$ and a target rank $k < \min(n, p)$, ***Non-negative Matrix Factorisation*** *(**NMF**, Paatero & Tapper 1994; Lee & Seung 1999) seeks two non-negative matrices*

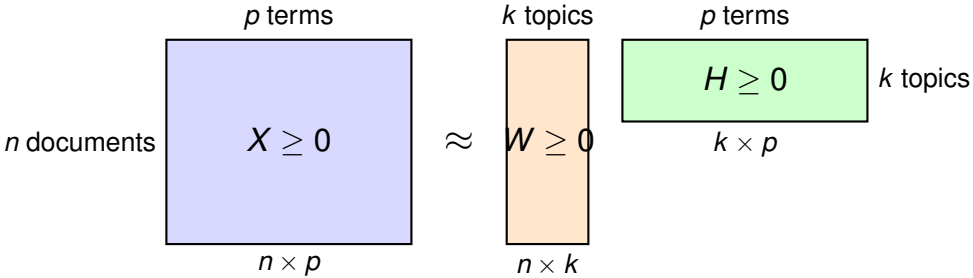
$$W \in \mathcal{M}_{nk}(\mathbb{R}_{\geq 0}) \quad and \quad H \in \mathcal{M}_{kp}(\mathbb{R}_{\geq 0})$$

such that $X \approx WH$, by solving the constrained optimisation problem

$$\min_{W \in \mathcal{M}_{nk}(\mathbb{R}_{\geq 0}),\ H \in \mathcal{M}_{kp}(\mathbb{R}_{\geq 0})} \|X - WH\|_F^2 = \sum_{i=1}^{n} \sum_{j=1}^{p} \left(x_{ij} - \sum_{r=1}^{k} w_{ir} h_{rj} \right)^2$$

subject to $w_{ir} \geq 0$ and $h_{rj} \geq 0$ for all i, r, j

Visualising the low-rank NMF decomposition $X \approx WH$



- When $k \ll \min(n, p)$, NMF compresses np entries into $k(n + p)$ non-negative parameters while preserving an **additive, parts-based** structure

Comparison: truncated SVD vs NMF

Property	**Truncated SVD ($U_k \Sigma_k V_k^T$)**	**NMF (WH)**
Structural constraint	Orthogonality ($U_k^T U_k = V_k^T V_k = \mathbb{I}_k$)	Non-negativity ($W \geq$
Entries of factors	Positive and negative ($\in \mathbb{R}$)	Non-negative only ($\in$
Reconstruction	Subtractive cancellation	Purely additive parts
Frobenius error $\|X - \hat{X}\|_F$	Globally minimal for rank k	Higher or equal ($\geq \|$
Optimisation problem	Exact via eigendecomposition	Non-convex; iterative
Nested factors across k	Yes (first k cols stay fixed)	No (recomputed for e

Non-negative matrix factorisation (NMF)

- SVD vs NMF on non-negative data
- **Topic modelling interpretation of W and H**
- Alternating least squares (ALS)

Row-wise view: documents as additive mixtures of topics

Write $W = \begin{bmatrix} \boldsymbol{w}_1^T \\ \vdots \\ \boldsymbol{w}_n^T \end{bmatrix} \in \mathcal{M}_{nk}(\mathbb{R}_{\geq 0})$ in terms of its rows $\boldsymbol{w}_i^T \in \mathbb{R}_{\geq 0}^{1\times k}$ and

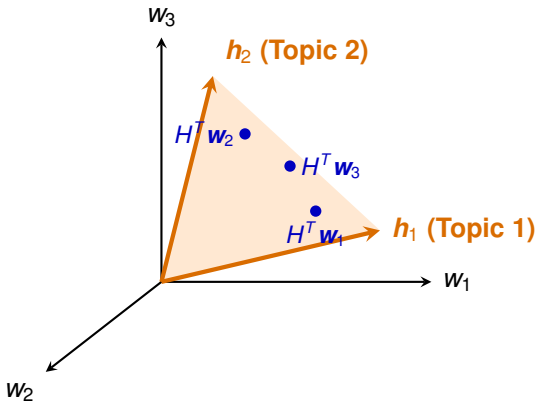
$H = \begin{bmatrix} \boldsymbol{h}_1^T \\ \vdots \\ \boldsymbol{h}_k^T \end{bmatrix} \in \mathcal{M}_{kp}(\mathbb{R}_{\geq 0})$ in terms of its rows $\boldsymbol{h}_r^T \in \mathbb{R}_{\geq 0}^{1\times p}$

By standard row-wise matrix multiplication, the i-th row of $X \approx WH$ is $\boldsymbol{x}_i^T \approx \boldsymbol{w}_i^T H = \sum_{r=1}^{k} w_{ir} \boldsymbol{h}_r^T$, or in column-vector form:

$$\boldsymbol{x}_i \approx H^T \boldsymbol{w}_i = w_{i1}\boldsymbol{h}_1 + w_{i2}\boldsymbol{h}_2 + \cdots + w_{ik}\boldsymbol{h}_k = \sum_{r=1}^{k} w_{ir}\boldsymbol{h}_r$$

- **Rows of H ($\boldsymbol{h}_r^T \in \mathbb{R}_{\geq 0}^{1\times p}$, the dictionary / topic basis)**:
  - Row $\boldsymbol{h}_r^T$ gives the weight of each of the p vocabulary words in **topic r**
  - The largest entries of $\boldsymbol{h}_r$ identify the signature keywords of topic r
- **Rows of W ($\boldsymbol{w}_i^T \in \mathbb{R}_{\geq 0}^{1\times k}$, the document topic encodings)**:
  - Entry $w_{ir} \geq 0$ tells how strongly document d_i expresses topic r

Geometric interpretation: the conical hull of H^T



- In unconstrained linear algebra, $\{H^T \boldsymbol{w} : \boldsymbol{w} \in \mathbb{R}^k\}$ is the k-dimensional subspace span$(\boldsymbol{h}_1, \ldots, \boldsymbol{h}_k)$
- In NMF, because $\boldsymbol{w}_i \in \mathbb{R}^k_{\geq 0}$ has non-negative weights $w_{ir} \geq 0$, the reconstructed documents $H^T \boldsymbol{w}_i$ must lie inside the **conical hull** (or simplicial cone) generated by the rows of H:

$$\text{cone}(H^T) = \left\{ \sum_{r=1}^{k} w_r \boldsymbol{h}_r : w_1, \ldots, w_k \geq 0 \right\}$$

Rank-one outer product expansion and polysemy

Just as the SVD writes $X_k = \sum_{r=1}^{k} \sigma_r \boldsymbol{u}_r \boldsymbol{v}_r^T$ as a sum of k rank-one matrices, NMF decomposes WH into k **non-negative rank-one layers**:

$$WH = \sum_{r=1}^{k} \boldsymbol{u}_r \boldsymbol{h}_r^T$$

where $\boldsymbol{u}_r \in \mathbb{R}_{\geq 0}^{n}$ is the r-th column of W and $\boldsymbol{h}_r^T \in \mathbb{R}_{\geq 0}^{1 \times p}$ is the r-th row of H

- Each outer product $M_r = \boldsymbol{u}_r \boldsymbol{h}_r^T \in \mathcal{M}_{np}(\mathbb{R}_{\geq 0})$ displays the contribution of **topic r alone** across all n documents and all p words
- Because rows of H do not have to be orthogonal, a single word w_j with multiple meanings (**polysemy**, e.g., `"degree"` in polynomial algebra vs vertex degree in graphs) can have positive weight $h_{rj} > 0$ in **more than one topic row**!

Example – Interpreting W and H (Part 1)

Suppose a corpus of $n = 4$ documents over $p = 5$ terms $\{\texttt{matrix}, \texttt{vector}, \texttt{rank}, \texttt{graph}, \texttt{vertex}\}$ has document-term matrix $X \in \mathcal{M}_{45}(\mathbb{R}_{\geq 0})$ and rank-2 NMF gives $X \approx WH$:

$$X = \begin{pmatrix} 6 & 3 & 3 & 0 & 0 \\ 4 & 2 & 2 & 0 & 0 \\ 0 & 0 & 0 & 6 & 3 \\ 2 & 1 & 1 & 4 & 2 \end{pmatrix}, \qquad W = \begin{pmatrix} 3 & 0 \\ 2 & 0 \\ 0 & 3 \\ 1 & 2 \end{pmatrix}, \qquad H = \begin{pmatrix} 2 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 2 & 1 \end{pmatrix}$$

(where rows $1, \ldots, 4$ of X and W are d_1, d_2, d_3, d_4 while columns $1, \ldots, 5$ of X and H are $\texttt{matrix}, \texttt{vector}, \texttt{rank}, \texttt{graph}, \texttt{vertex}$)

Questions:

1. How should we label Topic 1 ($\boldsymbol{h}_1^T$) and Topic 2 ($\boldsymbol{h}_2^T$) from H?
2. How should we characterise documents d_1, d_2, d_3, d_4 from W?
3. What is the reconstruction error $\|X - WH\|_F^2$?

Example – Interpreting W and H (Part 2)

1. **Labelling topics from the rows of H:**
   - Row $\boldsymbol{h}_1^T = (2, 1, 1, 0, 0)$ has non-zero weights only on `matrix`, `vector` and `rank` $\implies$ **Topic 1 is Linear Algebra**
   - Row $\boldsymbol{h}_2^T = (0, 0, 0, 2, 1)$ has non-zero weights only on `graph` and `vertex` $\implies$ **Topic 2 is Graph Theory**

2. **Classifying documents from the rows of W:**
   - $\boldsymbol{w}_1^T = (3, 0)$ and $\boldsymbol{w}_2^T = (2, 0)$ load purely on Topic 1 (Linear Algebra)
   - $\boldsymbol{w}_3^T = (0, 3)$ loads purely on Topic 2 (Graph Theory)
   - $\boldsymbol{w}_4^T = (1, 2)$ is a **multi-topic document**: $\boldsymbol{x}_4 = 1\boldsymbol{h}_1 + 2\boldsymbol{h}_2$ ($1/3$ Linear Algebra, $2/3$ Graph Theory, e.g., spectral graph theory!)

3. **Reconstruction error**: Multiplying WH reproduces X exactly, so $\|X - WH\|_F^2 = 0$ and $\text{rank}(X) = 2$

Non-negative matrix factorisation (NMF)

SVD vs NMF on non-negative data

Topic modelling interpretation of W and H

Alternating least squares (ALS)

Why computing NMF is an optimisation challenge

Consider the NMF objective function

$$f(W,H) = \|X - WH\|_F^2 = \sum_{i=1}^{n}\sum_{j=1}^{p}\left(x_{ij} - \sum_{r=1}^{k} w_{ir}h_{rj}\right)^2$$

1. **Non-uniqueness (scaling and permutation ambiguity)**:
   - For any permutation matrix $P \in \mathcal{M}_k$ and any positive diagonal matrix $D = \text{diag}(d_1, \ldots, d_k)$ with $d_r > 0$, setting $\tilde{W} = WPD \geq 0$ and $\tilde{H} = D^{-1}P^T H \geq 0$ gives $\tilde{W}\tilde{H} = WH$

2. **Non-convexity jointly in** (W, H):
   - Because unknown parameters w_{ir} and h_{rj} are multiplied together in WH, $f(W,H)$ is **not convex** in (W,H) simultaneously and can have multiple local minima

The key linear algebra insight: bi-convexity

Although $f(W,H) = \|X - WH\|_F^2$ is non-convex jointly in (W,H), it is **bi-convex** (convex in H when W is fixed and convex in W when H is fixed)!

1. **Hold** $W \in \mathcal{M}_{nk}(\mathbb{R}_{\geq 0})$ **fixed**: Denoting the j-th columns of X and H by $\boldsymbol{c}_j \in \mathbb{R}^n_{\geq 0}$ and $\boldsymbol{h}_{\cdot j} \in \mathbb{R}^k_{\geq 0}$,

$$\|X - WH\|_F^2 = \sum_{j=1}^{p} \|\boldsymbol{c}_j - W\boldsymbol{h}_{\cdot j}\|_2^2$$

   Each term $\|\boldsymbol{c}_j - W\boldsymbol{h}_{\cdot j}\|_2^2$ is a standard **linear least-squares problem** $W\boldsymbol{h}_{\cdot j} \approx \boldsymbol{c}_j$ of the exact form studied in Unit 03 and Unit 04!

2. **Hold** $H \in \mathcal{M}_{kp}(\mathbb{R}_{\geq 0})$ **fixed**: Using $\|A\|_F^2 = \|A^T\|_F^2$ and $(WH)^T = H^T W^T$,

$$\|X - WH\|_F^2 = \|X^T - H^T W^T\|_F^2 = \sum_{i=1}^{n} \|\boldsymbol{x}_i - H^T \boldsymbol{w}_i\|_2^2$$

   This is again a linear least-squares problem for the rows $\boldsymbol{w}_i^T$ of W (columns $\boldsymbol{w}_i$ of W^T) with design matrix $H^T \in \mathcal{M}_{pk}$!

Deriving the projected least-squares update for H

Proposition 178 (Projected least-squares step for H)

Suppose $W \in \mathcal{M}_{nk}(\mathbb{R}_{\geq 0})$ is fixed and has full column rank ($\operatorname{rank}(W) = k$*, so $W^T W \in \mathcal{M}_k$ is invertible). Then:*

1. *For each column $j \in \{1, \dots, p\}$, the unconstrained least-squares problem $\min_{\boldsymbol{h}_{\cdot j} \in \mathbb{R}^k} \|\boldsymbol{c}_j - W\boldsymbol{h}_{\cdot j}\|_2^2$ has the unique normal-equations solution*

$$\boldsymbol{h}_{\cdot j}^{LS} = (W^T W)^{-1} W^T \boldsymbol{c}_j$$

2. *Stacking all p columns side by side into $H_{LS} = [\boldsymbol{h}_{\cdot 1}^{LS} \cdots \boldsymbol{h}_{\cdot p}^{LS}] \in \mathcal{M}_{kp}(\mathbb{R})$ gives the matrix formula*

$$H_{LS} = (W^T W)^{-1} W^T X$$

3. *In projected **Alternating Least Squares** (**ALS**), we enforce non-negativity by clipping negative entries to zero entrywise:*

$$H \leftarrow \max(0, H_{LS}) \quad i.e., \quad h_{rj} = \max\big(0, (H_{LS})_{rj}\big)$$

Deriving the projected least-squares update for W

Proposition 179 (Projected least-squares step for W)

Suppose $H \in \mathcal{M}_{kp}(\mathbb{R}_{\geq 0})$ is fixed and has full row rank ($\operatorname{rank}(H) = k$, so $HH^T \in \mathcal{M}_k$ is invertible). Then:

1. *Minimising $\|X - WH\|_F^2 = \|X^T - H^T W^T\|_F^2$ without sign constraints yields the normal equations for W^T:*

$$(H^T)^T (H^T) W_{LS}^T = (H^T)^T X^T \iff (HH^T) W_{LS}^T = HX^T$$

2. *Solving for W_{LS}^T and taking the transpose (using $((HH^T)^{-1})^T = (HH^T)^{-1}$) gives*

$$W_{LS}^T = (HH^T)^{-1} HX^T \iff W_{LS} = XH^T (HH^T)^{-1}$$

3. *Clipping negative entries to zero entrywise gives the projected ALS update:*

$$W \leftarrow \max(0, W_{LS})$$

The Alternating Least Squares (ALS) algorithm for NMF

Putting Propositions 178 and 179 together yields the **Projected Alternating Least Squares** (**ALS**) algorithm (Paatero & Tapper 1994; Berry et al. 2007):

1. **Input**: Non-negative data matrix $X \in \mathcal{M}_{np}(\mathbb{R}_{\geq 0})$ and target rank $k < \min(n, p)$
2. **Initialise**: Choose a non-negative initial matrix $W^{(0)} \in \mathcal{M}_{nk}(\mathbb{R}_{\geq 0})$ with $\operatorname{rank}(W^{(0)}) = k$ (e.g., using positive random numbers)
3. **Repeat for** $t = 0, 1, 2, \ldots$ **until convergence**:
   - 3.1 **Update** H **(fixing** $W^{(t)}$**)**: Solve $(W^{(t)T} W^{(t)}) H_{\text{LS}} = W^{(t)T} X$ and project:

$$H^{(t+1)} = \max\Big(0,\ (W^{(t)T} W^{(t)})^{-1} W^{(t)T} X\Big)$$

   - 3.2 **Update** W **(fixing** $H^{(t+1)}$**)**: Solve $W_{\text{LS}}(H^{(t+1)} H^{(t+1)T}) = X H^{(t+1)T}$ and project:

$$W^{(t+1)} = \max\Big(0,\ X H^{(t+1)T} (H^{(t+1)} H^{(t+1)T})^{-1}\Big)$$

   - 3.3 *(Optional)* Normalise each row of $H^{(t+1)}$ (or column of $W^{(t+1)}$) to unit ℓ_2 or ℓ_1 norm to prevent scaling drift

Example – One ALS update step by hand (Part 1)

Let $X \in \mathcal{M}_{33}(\mathbb{R}_{\geq 0})$ and let the initial rank-2 factor matrix $W^{(0)} \in \mathcal{M}_{32}(\mathbb{R}_{\geq 0})$ be

$$X = \begin{pmatrix} 3 & 0 & 2 \\ 0 & 3 & 3 \\ 0 & 3 & 1 \end{pmatrix}, \qquad W^{(0)} = \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{pmatrix}$$

Let us compute the first ALS update $H^{(1)} = \max\big(0, (W^{(0)T} W^{(0)})^{-1} W^{(0)T} X\big)$ by hand

Step 1: Compute $W^{(0)T} W^{(0)} \in \mathcal{M}_2$ and its inverse:

$$W^{(0)T} W^{(0)} = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$$

Since $\det(W^{(0)T} W^{(0)}) = 2(2) - 1(1) = 3 \neq 0$, the 2×2 inverse formula gives

$$(W^{(0)T} W^{(0)})^{-1} = \frac{1}{3} \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}$$

Example – One ALS update step by hand (Part 2)

Step 2: Compute $W^{(0)T}X \in \mathcal{M}_{23}$:

$$W^{(0)T}X = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{pmatrix} \begin{pmatrix} 3 & 0 & 2 \\ 0 & 3 & 3 \\ 0 & 3 & 1 \end{pmatrix} = \begin{pmatrix} 3 & 3 & 5 \\ 0 & 6 & 4 \end{pmatrix}$$

Step 3: Compute the unconstrained least-squares matrix $H_{\mathbf{LS}}^{(1)}$:

$$H_{\mathrm{LS}}^{(1)} = (W^{(0)T}W^{(0)})^{-1}W^{(0)T}X = \frac{1}{3}\begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}\begin{pmatrix} 3 & 3 & 5 \\ 0 & 6 & 4 \end{pmatrix} = \frac{1}{3}\begin{pmatrix} 6 & 0 & 6 \\ -3 & 9 & 3 \end{pmatrix} = \begin{pmatrix} 2 & 0 \\ \mathbf{-1} & 3 \end{pmatrix}$$

Step 4: Project onto $\mathbb{R}_{\geq 0}$ by clipping negative entries to zero:

$$H^{(1)} = \max\left(0, H_{\mathrm{LS}}^{(1)}\right) = \begin{pmatrix} 2 & 0 & 2 \\ \mathbf{0} & 3 & 1 \end{pmatrix} \in \mathcal{M}_{23}(\mathbb{R}_{\geq 0})$$

Notice how unconstrained least squares produced -1 in entry $(2,1)$, which the projection step clips to 0!

Example – Evaluating the reconstruction $W^{(0)}H^{(1)}$ (Part 3)

Step 5: Compute the rank-2 approximation $\hat{X}^{(1)} = W^{(0)}H^{(1)}$ and its Frobenius error:

$$\hat{X}^{(1)} = W^{(0)}H^{(1)} = \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 2 & 0 & 2 \\ 0 & 3 & 1 \end{pmatrix} = \begin{pmatrix} 2 & 0 & 2 \\ 2 & 3 & 3 \\ 0 & 3 & 1 \end{pmatrix}$$

Comparing $\hat{X}^{(1)}$ with $X = \begin{pmatrix} 3 & 0 & 2 \\ 0 & 3 & 3 \\ 0 & 3 & 1 \end{pmatrix}$, the residual matrix is

$$X - W^{(0)}H^{(1)} = \begin{pmatrix} 1 & 0 & 0 \\ -2 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} \implies \|X - W^{(0)}H^{(1)}\|_F^2 = 1^2 + (-2)^2 = 5$$

- Columns 2 and 3 of X are reconstructed **with zero error** on the very first half-step; the next half-step updates $W^{(1)} = \max\big(0, XH^{(1)T}(H^{(1)}H^{(1)T})^{-1}\big)$ to reduce the error on column 1 even further!

Remark: exact NNLS and multiplicative updates

- **Projected ALS vs exact Non-Negative Least Squares (NNLS)**:
  - Solving unconstrained least squares and clipping $\max(0, \cdot)$ is fast (requiring only a $k \times k$ linear system $W^T W$ per iteration) and works well when k is small and X is sparse
  - A more accurate variant solves each subproblem $\min_{\boldsymbol{w}_i \geq \mathbf{0}} \|\boldsymbol{x}_i - H^T \boldsymbol{w}_i\|_2^2$ and $\min_{\boldsymbol{h}_{\cdot j} \geq \mathbf{0}} \|\boldsymbol{c}_j - W \boldsymbol{h}_{\cdot j}\|_2^2$ using an exact active-set **NNLS** solver (Lawson & Hanson 1974), guaranteeing that $\|X - W^{(t)} H^{(t)}\|_F^2$ decreases monotonically at every step

- **Lee & Seung (1999) multiplicative updates**:
  - Another popular algorithm updates entries by elementwise multiplication:

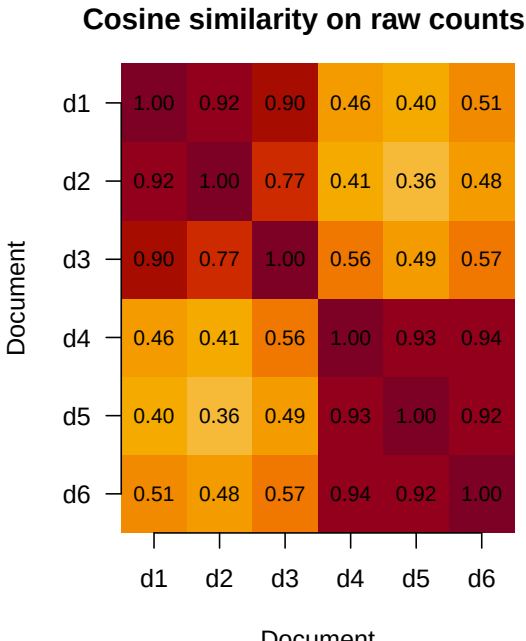
$$h_{rj} \leftarrow h_{rj} \frac{(W^T X)_{rj}}{(W^T W H)_{rj}}, \qquad w_{ir} \leftarrow w_{ir} \frac{(X H^T)_{ir}}{(W H H^T)_{ir}}$$

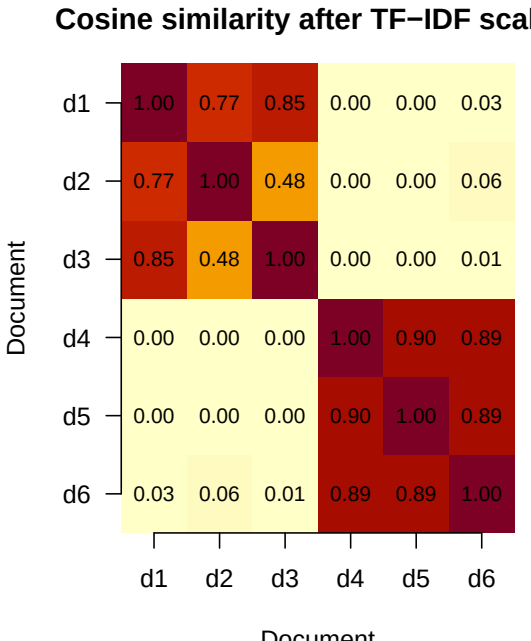
  - Because all factors on the right-hand side are non-negative, W and H automatically remain non-negative without clipping while $\|X - WH\|_F^2$ is guaranteed not to increase

R demonstration – TF-IDF and cosine similarity

Let us build a 6×8 document-term matrix in R with $n = 6$ documents (rows: 3 on linear algebra, 3 on graphs) and $p = 8$ terms (columns) and compute $\hat{X} = D_{\text{len}}^{-1} X D_{\text{IDF}}$ and $S = \hat{X}\hat{X}^T$:

```
terms = c("matrix", "vector", "eigen", "rank", "graph", "vertex", "edge", "c
X = matrix(c(
  4, 3, 2, 1, 0, 0, 0, 5,  # d1: linear algebra
  3, 4, 0, 2, 0, 0, 0, 4,  # d2: linear algebra
  2, 1, 3, 2, 0, 0, 0, 6,  # d3: linear algebra
  0, 0, 0, 0, 4, 3, 2, 5,  # d4: graph theory
  0, 0, 0, 0, 3, 2, 4, 4,  # d5: graph theory
  0, 1, 0, 0, 2, 4, 3, 5   # d6: graph theory (mentions "vector" once)
), nrow = 6, ncol = 8, byrow = TRUE, dimnames = list(paste0("d", 1:6), terms


# Right-multiplication by diagonal IDF matrix: X %*% D_IDF
df_j = colSums(X > 0)
idf_j = log(nrow(X) / df_j)
```


Cosine similarity on raw counts X
d1 1.00 0.92 0.90 0.46 0.40 0.51
d2 0.92 1.00 0.77 0.41 0.36 0.48
d3 0.90 0.77 1.00 0.56 0.49 0.57
d4 0.46 0.41 0.56 1.00 0.93 0.94
d5 0.40 0.36 0.49 0.93 1.00 0.92
d6 0.51 0.48 0.57 0.94 0.92 1.00
Document
d1 d2 d3 d4 d5 d6
Document


Cosine similarity after TF−IDF scaling
d1 1.00 0.77 0.85 0.00 0.00 0.03
d2 0.77 1.00 0.48 0.00 0.00 0.06
d3 0.85 0.48 1.00 0.00 0.00 0.01
d4 0.00 0.00 0.00 1.00 0.90 0.89
d5 0.00 0.00 0.00 0.90 1.00 0.89
d6 0.03 0.06 0.01 0.89 0.89 1.00
Document
d1 d2 d3 d4 d5 d6
Document

R demonstration – Projected ALS for NMF

We now run 30 iterations of projected Alternating Least Squares with rank $k = 2$ on our TF-IDF matrix:

```
set.seed(2740)
k = 2
W = matrix(runif(nrow(X_tfidf) * k, 0.1, 1), nrow = nrow(X_tfidf), ncol = k)
H = matrix(0, nrow = k, ncol = ncol(X_tfidf))
err_history = numeric(30)

for (iter in 1:30) {
  # Update H by least squares and clip negatives to 0
  H = pmax(0, solve(t(W) %*% W + 1e-8 * diag(k), t(W) %*% X_tfidf))
  # Update W by least squares and clip negatives to 0
  W = pmax(0, t(solve(H %*% t(H) + 1e-8 * diag(k), H %*% t(X_tfidf))))
  # Normalise rows of H and rescale columns of W accordingly
  h_norms = sqrt(rowSums(H^2))
  H = diag(1 / h_norms) %*% H
```

R demonstration – Inspecting the learned W and H factors

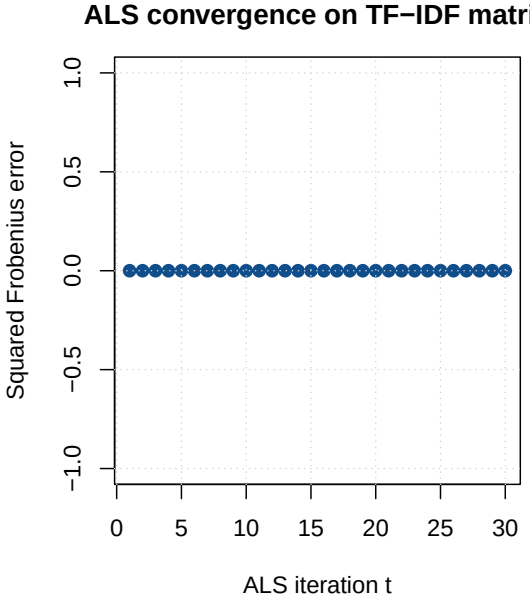
```
round(W, 2)
```

```
##    Topic_1 Topic_2
## d1    0.92    0.27
## d2    0.42    0.80
## d3    0.36    0.82
## d4    0.66    0.93
## d5    0.33    0.84
## d6    0.70    0.41
```

```
round(H, 3)
```

```
##  [1] 1.922 0.000 1.002 0.019 1.269 0.211 0.144 0.680 0.312 1.360 1.173 0.
## [13] 0.402 1.167 0.000 0.000
```

```
## Error in ‘H[, 8:1]‘:
## !  incorrect number of dimensions
```


ALS convergence on TF−IDF matrix
Squared Frobenius error
1.0
0.5
0.0
−0.5
−1.0
0
5
10
15
20
25
30
ALS iteration t

Summary of Unit U11

- **Tokenisation and bag-of-words**: A vocabulary $V = \{w_1, \ldots, w_p\}$ maps each token to $\boldsymbol{e}_j \in \mathbb{R}^p$ and each document d_i to a sparse count vector $\boldsymbol{x}_i = \sum_{j=1}^{p} x_{ij}\boldsymbol{e}_j \in \mathbb{R}^p_{\geq 0}$ with $\|\boldsymbol{x}_i\|_0 \ll p$
- **Document-term matrices and TF-IDF scaling**: Stacking document vectors as rows forms $X \in \mathcal{M}_{np}(\mathbb{R}_{\geq 0})$ where TF-IDF with length normalisation is the two-sided diagonal matrix product

$$\hat{X} = D_{\text{len}}^{-1} X D_{\text{IDF}}$$

- **Cosine similarity in high dimensions**: Semantic similarity is measured by the scale-invariant angle $\text{sim}(\boldsymbol{x}, \boldsymbol{y}) = \frac{\boldsymbol{x} \bullet \boldsymbol{y}}{\|\boldsymbol{x}\|_2 \|\boldsymbol{y}\|_2} \in [0, 1]$ on $\mathbb{R}^p_{\geq 0}$ whose pairwise values form the Gram matrix $S = \hat{X}\hat{X}^T \in \mathcal{M}_n(\mathbb{R})$
- **Non-negative Matrix Factorisation (NMF)**: Approximating $X \approx WH$ with $W \in \mathcal{M}_{nk}(\mathbb{R}_{\geq 0})$ and $H \in \mathcal{M}_{kp}(\mathbb{R}_{\geq 0})$ avoids the subtractive cancellation of SVD, yielding additive topic rows $\boldsymbol{h}_r^T$ and document encodings $\boldsymbol{w}_i^T$ computed via projected **Alternating Least Squares (ALS)**

Neural networks as composed affine and spectral operators

MATH 2740 – Mathematics of Data Science – Unit U12

Julien Arino
julien.arino@umanitoba.ca

Department of Mathematics @ University of Manitoba

Fall 2026

The University of Manitoba campuses are located on original lands of Anishinaabeg, Ininew, Anisininew, Dakota and Dene peoples, and on the National Homeland of the Red River Métis. We respect the Treaties that were made on these territories, we acknowledge the harms and mistakes of the past, and we dedicate ourselves to move forward in partnership with Indigenous communities in a spirit of Reconciliation and collaboration.

Outline

From single neurons to affine matrix layers

Why non-linearity is essential: collapsing linear networks

Linear autoencoders and subspace projections (bridge to PCA)

Graph convolutional networks: combining left and right matrix multiplication

Why neural networks belong in a linear algebra course

Modern machine learning and artificial intelligence are often presented through the lens of multivariable calculus and black-box software libraries

At their structural core, however, neural networks are **compositions of matrix operators** interleaved with simple coordinate-wise non-linearities

In this unit, we unify the major threads of the course:

- **Affine maps and hyperplanes** (Unit 06): a single neuron is an affine functional followed by a threshold or activation function
- **Matrix multiplication and rank bounds**: forward propagation is batch matrix multiplication and non-linearity prevents rank collapse
- **SVD and PCA** (Units 04 and 05): linear autoencoders learn orthogonal projections onto principal subspaces
- **Graph matrices and Markov chains** (Units 08 to 10): graph convolutional networks combine left-multiplication by normalised adjacency matrices with right-multiplication by weight matrices

From single neurons to affine matrix layers

Why non-linearity is essential: collapsing linear networks

Linear autoencoders and subspace projections (bridge to PCA)

Graph convolutional networks: combining left and right matrix multiplication

From single neurons to affine matrix layers

Single artificial neuron and the perceptron

Multi-layer feedforward networks (MLPs) in vector and batch matrix form

Biological inspiration and the McCulloch-Pitts-Rosenblatt neuron

Artificial neural networks *(ANNs) are computing systems inspired by biological neural networks. A collection of connected nodes called* ***artificial neurons*** *receive real-valued signals along weighted edges (synapses), aggregate them via an affine combination and emit an output through a non-linear* ***activation function***

- McCulloch and Pitts (1943) introduced threshold logic units for Boolean computation
- Rosenblatt (1957) introduced the **perceptron** for pattern classification (such as recognising letters or digits from a pixel grid called the *retina*)
- Mathematically, a single artificial neuron is a direct generalisation of the linear classifiers and separating hyperplanes studied in Unit 06

The single artificial neuron as an affine functional

Definition 180 (Single artificial neuron)

*Given an input feature vector $\boldsymbol{x} = (x_1, \ldots, x_p)^T \in \mathbb{R}^p$, a **single artificial neuron** is parameterised by a **weight vector** $\boldsymbol{w} = (w_1, \ldots, w_p)^T \in \mathbb{R}^p$ and a scalar **bias** $b \in \mathbb{R}$. It computes:*

1. *An **affine pre-activation** (or **logit**):*

$$z = \boldsymbol{w}^T \boldsymbol{x} + b = \sum_{j=1}^{p} w_j x_j + b \in \mathbb{R}$$

2. *A scalar **activation** (output):*

$$a = \sigma(z) = \sigma\left(\boldsymbol{w}^T \boldsymbol{x} + b\right) \in \mathbb{R}$$

*where $\sigma : \mathbb{R} \to \mathbb{R}$ is a chosen **activation function***

Absorbing the bias via augmented coordinates

In classical perceptron notation, a threshold θ is used so the neuron fires when $\boldsymbol{w}^T\boldsymbol{x} > \theta$; setting $b = -\theta$ gives $\boldsymbol{w}^T\boldsymbol{x} + b > 0$

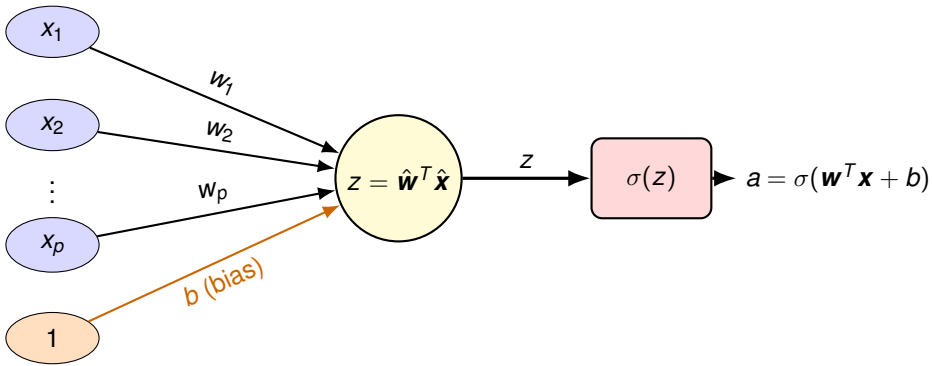
Just as in homogeneous coordinates, we can absorb the bias b into the weight vector by introducing a constant "dummy" input feature $x_0 = 1$:

$$\hat{\boldsymbol{x}} = \begin{pmatrix} \boldsymbol{x} \\ 1 \end{pmatrix} = \begin{pmatrix} x_1 \\ \vdots \\ x_p \\ 1 \end{pmatrix} \in \mathbb{R}^{p+1}, \qquad \hat{\boldsymbol{w}} = \begin{pmatrix} \boldsymbol{w} \\ b \end{pmatrix} = \begin{pmatrix} w_1 \\ \vdots \\ w_p \\ b \end{pmatrix} \in \mathbb{R}^{p+1}$$

Then the affine pre-activation becomes a pure inner product in $\mathbb{R}^{p+1}$:

$$z = \boldsymbol{w}^T\boldsymbol{x} + b = \hat{\boldsymbol{w}}^T\hat{\boldsymbol{x}}$$

Diagram of a single neuron with augmented bias input



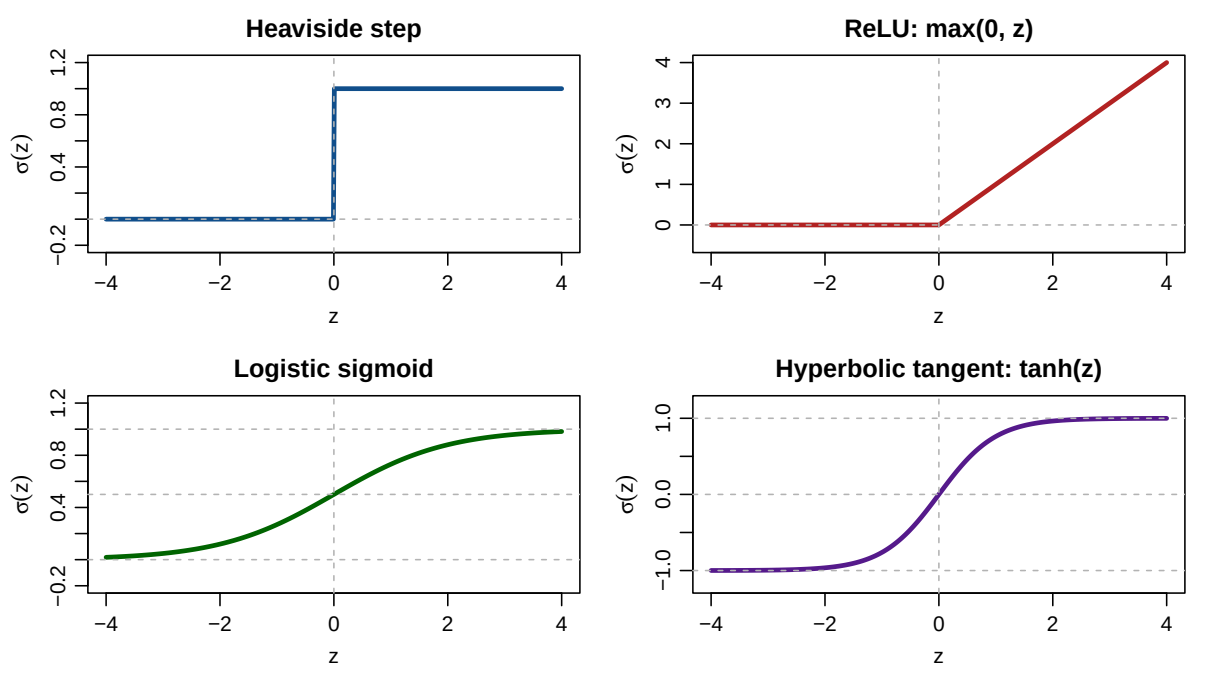
Geometrically, the locus $\{\boldsymbol{x} \in \mathbb{R}^p : \boldsymbol{w}^T \boldsymbol{x} + b = 0\}$ is an **affine hyperplane** with normal vector $\boldsymbol{w}$, partitioning $\mathbb{R}^p$ into two half-spaces $\{\boldsymbol{w}^T \boldsymbol{x} + b > 0\}$ and $\{\boldsymbol{w}^T \boldsymbol{x} + b < 0\}$

Common scalar activation functions

Definition 181 (Component-wise activation functions)

When applied to a vector $\boldsymbol{z} \in \mathbb{R}^p$ or matrix $Z \in \mathcal{M}_{np}$, a scalar activation function $\sigma : \mathbb{R} \to \mathbb{R}$ acts **entrywise**: $\left[\sigma(Z)\right]_{ij} = \sigma(z_{ij})$. Standard choices include:

- **Heaviside step (threshold)**: $H(z) = \begin{cases} 0 & \text{if } z \leq 0 \\ 1 & \text{if } z > 0 \end{cases}$
- **Rectified linear unit (ReLU)**: $\mathrm{ReLU}(z) = \max(0, z) = \begin{cases} 0 & \text{if } z \leq 0 \\ z & \text{if } z > 0 \end{cases}$
- **Logistic sigmoid**: $\sigma_{\mathrm{sig}}(z) = \dfrac{1}{1 + e^{-z}} \in (0, 1)$
- **Hyperbolic tangent**: $\tanh(z) = \dfrac{e^z - e^{-z}}{e^z + e^{-z}} = 2\sigma_{\mathrm{sig}}(2z) - 1 \in (-1, 1)$


Heaviside step
σ(z)
z
ReLU: max(0, z)
σ(z)
z
Logistic sigmoid
σ(z)
z
Hyperbolic tangent: tanh(z)
σ(z)
z

Single-layer perceptrons on Boolean logic gates: OR and AND

Consider binary inputs $\boldsymbol{x} = (x_1, x_2)^T \in \{0, 1\}^2$ with the Heaviside step activation $H(z)$

Logical OR gate:

x_1	x_2	$z = x_1 + x_2 - 0.5$	$H(z)$
0	0	-0.5	0
1	0	$+0.5$	1
0	1	$+0.5$	1
1	1	$+1.5$	1

Solved by $\boldsymbol{w} = (1, 1)^T$ and $b = -0.5$

Logical AND gate:

x_1	x_2	$z = x_1 + x_2 - 1.5$	$H(z)$
0	0	-1.5	0
1	0	-0.5	0
0	1	-0.5	0
1	1	$+0.5$	1

Solved by $\boldsymbol{w} = (1, 1)^T$ and $b = -1.5$

In both cases, a single line $x_1 + x_2 + b = 0$ separates the class-0 points from the class-1 points in $\mathbb{R}^2$

The perceptron and Widrow-Hoff learning rule in vector form

How does a single neuron learn its augmented weight vector $\hat{\boldsymbol{w}} = (\boldsymbol{w}^T, b)^T \in \mathbb{R}^{p+1}$ from labelled training pairs $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$ with $y_i \in \{0, 1\}$?

In **supervised error-correction learning** (Widrow-Hoff / delta rule), when input $\hat{\boldsymbol{x}}_i = (\boldsymbol{x}_i^T, 1)^T$ produces output $\hat{y}_i = H\big((\hat{\boldsymbol{w}}^{(t)})^T \hat{\boldsymbol{x}}_i\big)$, we update the weight vector by

$$\hat{\boldsymbol{w}}^{(t+1)} = \hat{\boldsymbol{w}}^{(t)} + \eta\big(y_i - \hat{y}_i\big)\hat{\boldsymbol{x}}_i$$

where $\eta > 0$ is the **learning rate**

Geometric meaning via inner products:

- If $\hat{y}_i = y_i$ (correct prediction), $\hat{\boldsymbol{w}}^{(t+1)} = \hat{\boldsymbol{w}}^{(t)}$ (no change)
- If $y_i = 1$ but $\hat{y}_i = 0$ (false negative), $\hat{\boldsymbol{w}}^{(t+1)} = \hat{\boldsymbol{w}}^{(t)} + \eta\hat{\boldsymbol{x}}_i$, so the new pre-activation on $\hat{\boldsymbol{x}}_i$ strictly increases:

$$\big(\hat{\boldsymbol{w}}^{(t+1)}\big)^T \hat{\boldsymbol{x}}_i = \big(\hat{\boldsymbol{w}}^{(t)}\big)^T \hat{\boldsymbol{x}}_i + \eta\|\hat{\boldsymbol{x}}_i\|_2^2 > \big(\hat{\boldsymbol{w}}^{(t)}\big)^T \hat{\boldsymbol{x}}_i$$

tilting the hyperplane normal $\hat{\boldsymbol{w}}$ towards $\hat{\boldsymbol{x}}_i$!

Worked trace: two perceptron updates to learn OR

Let us trace the update rule $\hat{\boldsymbol{w}}^{(t+1)} = \hat{\boldsymbol{w}}^{(t)} + \eta(y_i - \hat{y}_i)\hat{\boldsymbol{x}}_i$ with $\eta = 0.1$ and augmented inputs $\hat{\boldsymbol{x}} = (x_1, x_2, 1)^T$, starting from $\hat{\boldsymbol{w}}^{(0)} = (0, 0, 0)^T$:

1. **Step 1**: Present $\boldsymbol{x} = (1, 1)^T$, so $\hat{\boldsymbol{x}} = (1, 1, 1)^T$ with target $y = 1$. Since $\left(\hat{\boldsymbol{w}}^{(0)}\right)^T \hat{\boldsymbol{x}} = 0 \leq 0$, we predict $\hat{y} = 0$ (error $y - \hat{y} = 1$):

$$\hat{\boldsymbol{w}}^{(1)} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} + 0.1(1 - 0) \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 0.1 \\ 0.1 \\ 0.1 \end{pmatrix}$$

2. **Step 2**: Present $\boldsymbol{x} = (0, 0)^T$, so $\hat{\boldsymbol{x}} = (0, 0, 1)^T$ with target $y = 0$. Since $\left(\hat{\boldsymbol{w}}^{(1)}\right)^T \hat{\boldsymbol{x}} = 0.1 > 0$, we predict $\hat{y} = 1$ (error $y - \hat{y} = -1$):

$$\hat{\boldsymbol{w}}^{(2)} = \begin{pmatrix} 0.1 \\ 0.1 \\ 0.1 \end{pmatrix} + 0.1(0 - 1) \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0.1 \\ 0.1 \\ 0 \end{pmatrix}$$

With $\boldsymbol{w} = (0.1, 0.1)^T$ and $b = 0$, $H(0.1x_1 + 0.1x_2)$ classifies all four OR inputs correctly!

Why a single neuron fails on XOR (Minsky and Papert, 1969)

Now consider the **exclusive OR (XOR)** truth table on $\{0,1\}^2$:

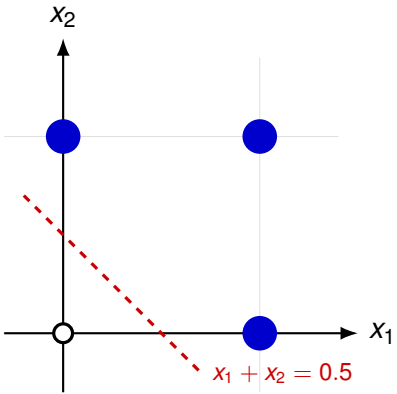
$$(0,0) \mapsto 0, \qquad (1,0) \mapsto 1, \qquad (0,1) \mapsto 1, \qquad (1,1) \mapsto 0$$

Suppose a single neuron with weights $\boldsymbol{w} = (w_1, w_2)^T$ and bias b could represent XOR via $H(\boldsymbol{w}^T \boldsymbol{x} + b)$. Then the four inputs require:

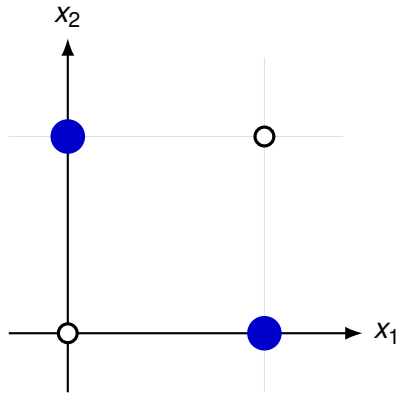
$$\begin{aligned}
(0,0) \mapsto 0 &\implies b \leq 0 \\
(1,0) \mapsto 1 &\implies w_1 + b > 0 \\
(0,1) \mapsto 1 &\implies w_2 + b > 0 \\
(1,1) \mapsto 0 &\implies w_1 + w_2 + b \leq 0
\end{aligned}$$

Adding the second and third inequalities gives $w_1 + w_2 + 2b > 0$. Since $b \leq 0$, this implies $w_1 + w_2 + b \geq w_1 + w_2 + 2b > 0$, contradicting $w_1 + w_2 + b \leq 0$!

Linear separability: OR vs XOR in $\mathbb{R}^2$



OR: linearly separable



XOR: not linearly separable

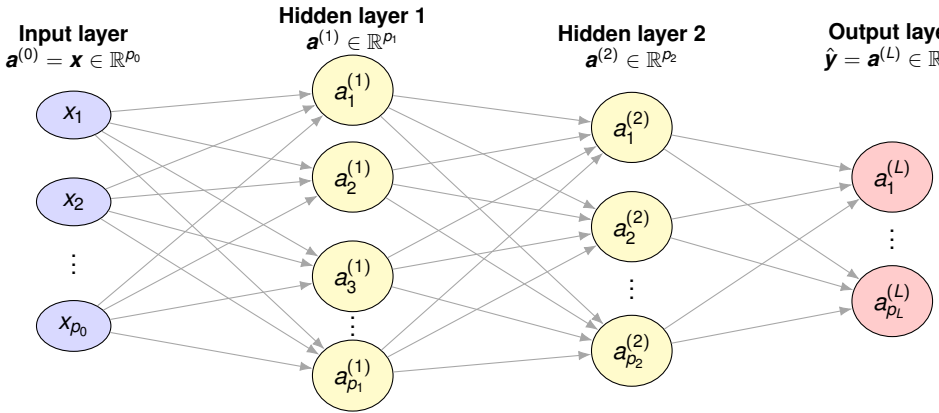
To solve non-linearly separable problems, we must **compose** multiple layers: a hidden layer first maps the inputs into a new coordinate space where they become linearly separable

From single neurons to affine matrix layers

Single artificial neuron and the perceptron

Multi-layer feedforward networks (MLPs) in vector and batch matrix form

Architecture of a multi-layer feedforward network (MLP)



Single-vector formulation of an MLP layer

Definition 182 (Affine neural network layer – vector form)

An L-layer ***multi-layer perceptron (MLP)*** *has layer widths* $p_0, p_1, \ldots, p_L \in \mathbb{N}_+$ *(where* $p_0 = p$ *is the number of input features). Given a single input column vector* $\boldsymbol{a}^{(0)} = \boldsymbol{x} \in \mathbb{R}^{p_0}$*, each layer* $\ell \in \{1, \ldots, L\}$ *maps* $\boldsymbol{a}^{(\ell-1)} \in \mathbb{R}^{p_{\ell-1}}$ *to* $\boldsymbol{a}^{(\ell)} \in \mathbb{R}^{p_\ell}$ *via*

$$\boldsymbol{z}^{(\ell)} = W^{(\ell)} \boldsymbol{a}^{(\ell-1)} + \boldsymbol{b}^{(\ell)} \in \mathbb{R}^{p_\ell}, \qquad \boldsymbol{a}^{(\ell)} = \sigma_\ell\big(\boldsymbol{z}^{(\ell)}\big) \in \mathbb{R}^{p_\ell}$$

where:

- $W^{(\ell)} = \big[w_{ij}^{(\ell)}\big] \in \mathcal{M}_{p_\ell, p_{\ell-1}}$ *is the* ***weight matrix*** *of layer* ℓ *(*p_ℓ *rows for outputs,* $p_{\ell-1}$ *columns for inputs)*
- $\boldsymbol{b}^{(\ell)} \in \mathbb{R}^{p_\ell}$ *is the* ***bias vector*** *of layer* ℓ
- $\sigma_\ell : \mathbb{R}^{p_\ell} \to \mathbb{R}^{p_\ell}$ *is the component-wise activation function of layer* ℓ

From single vectors to batch matrices

In data science (Units 04 and 05), a dataset of n observations with $p_0 = p$ features is stored as a data matrix $X \in \mathcal{M}_{n,p_0}$ whose **rows** are transposed observation vectors (in deep learning, a subset of n observations is called a **batch** or **mini-batch**, often denoted B, and X is called the **batch input matrix**):

$$X = \begin{pmatrix} \boldsymbol{x}_1^T \\ \boldsymbol{x}_2^T \\ \vdots \\ \boldsymbol{x}_n^T \end{pmatrix} \in \mathcal{M}_{n,p_0}$$

For observation $i \in \{1, \ldots, n\}$, the column vector pre-activation in layer 1 is $\boldsymbol{z}_i^{(1)} = W^{(1)}\boldsymbol{x}_i + \boldsymbol{b}^{(1)} \in \mathbb{R}^{p_1}$. Taking transposes to store outputs as **rows** gives:

$$\left(\boldsymbol{z}_i^{(1)}\right)^T = \boldsymbol{x}_i^T \left(W^{(1)}\right)^T + \left(\boldsymbol{b}^{(1)}\right)^T \in \mathcal{M}_{1,p_1}$$

How do we stack all n rows simultaneously into a single matrix equation?

Batch matrix formulation and rank-1 bias broadcasting

Definition 183 (Batch matrix forward propagation)

Given a data matrix (batch input matrix) $A^{(0)} = X \in \mathcal{M}_{n,p_0}$ whose n rows are input observations, layer $\ell \in \{1, \ldots, L\}$ computes the pre-activation matrix $Z^{(\ell)} \in \mathcal{M}_{n,p_\ell}$ and activation matrix $A^{(\ell)} \in \mathcal{M}_{n,p_\ell}$ via

$$Z^{(\ell)} = A^{(\ell-1)}\left(W^{(\ell)}\right)^T + \mathbb{1}_n\left(\boldsymbol{b}^{(\ell)}\right)^T \in \mathcal{M}_{n,p_\ell}, \qquad A^{(\ell)} = \sigma_\ell\left(Z^{(\ell)}\right) \in \mathcal{M}_{n,p_\ell}$$

where $\mathbb{1}_n = (1, 1, \ldots, 1)^T \in \mathbb{R}^n$ is the all-ones column vector

Why does $\mathbb{1}_n\left(\boldsymbol{b}^{(\ell)}\right)^T$ appear? Because the outer product of $\mathbb{1}_n \in \mathcal{M}_{n1}$ and $\left(\boldsymbol{b}^{(\ell)}\right)^T \in \mathcal{M}_{1,p_\ell}$ is a **rank-1 matrix** that replicates the bias row vector across all n observations:

$$\mathbb{1}_n\left(\boldsymbol{b}^{(\ell)}\right)^T = \begin{pmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{pmatrix} \begin{pmatrix} b_1^{(\ell)} & b_2^{(\ell)} & \cdots & b_{p_\ell}^{(\ell)} \end{pmatrix} = \begin{pmatrix} b_1^{(\ell)} & b_2^{(\ell)} & \cdots & b_{p_\ell}^{(\ell)} \\ b_1^{(\ell)} & b_2^{(\ell)} & \cdots & b_{p_\ell}^{(\ell)} \\ \vdots & \vdots & \ddots & \vdots \\ b_1^{(\ell)} & b_2^{(\ell)} & & b_{p_\ell}^{(\ell)} \end{pmatrix} \in \mathcal{M}_{n,p_\ell}$$

Block matrix view: absorbing the bias column into $A^{(\ell-1)}$

Just as for a single vector, we can also express the batch affine transformation as a single block matrix product by appending a column of ones $\mathbb{1}_n$ to $A^{(\ell-1)}$ and the bias column $\boldsymbol{b}^{(\ell)}$ to $W^{(\ell)}$:

$$\hat{A}^{(\ell-1)} = \left[\, A^{(\ell-1)} \;\middle|\; \mathbb{1}_n \,\right] \in \mathcal{M}_{n,p_{\ell-1}+1}, \qquad \hat{W}^{(\ell)} = \left[\, W^{(\ell)} \;\middle|\; \boldsymbol{b}^{(\ell)} \,\right] \in \mathcal{M}_{p_\ell,p_{\ell-1}+1}$$

By block matrix multiplication,

$$\hat{A}^{(\ell-1)}\big(\hat{W}^{(\ell)}\big)^T = \left[\, A^{(\ell-1)} \;\middle|\; \mathbb{1}_n \,\right] \begin{bmatrix} \big(W^{(\ell)}\big)^T \\ \big(\boldsymbol{b}^{(\ell)}\big)^T \end{bmatrix} = A^{(\ell-1)}\big(W^{(\ell)}\big)^T + \mathbb{1}_n\big(\boldsymbol{b}^{(\ell)}\big)^T = Z^{(\ell)}$$

Both views are algebraically identical: the rank-1 outer product $\mathbb{1}_n\big(\boldsymbol{b}^{(\ell)}\big)^T$ is simply the contribution of the constant column $\mathbb{1}_n$!

Dimension tracking and parameter counting

In software (such as R or PyTorch), shape mismatches are the most common source of bugs. Tracking dimensions systematically is essential

Proposition 184 (Dimension and parameter count of an L-layer MLP)

For a batch of n observations passing through an MLP with widths $p_0, p_1, \ldots, p_L$:

- *At layer $\ell \in \{1, \ldots, L\}$:*

$$\underbrace{A^{(\ell-1)}}_{n \times p_{\ell-1}} \underbrace{\left(W^{(\ell)}\right)^T}_{p_{\ell-1} \times p_\ell} + \underbrace{\mathbb{1}_n}_{n \times 1} \underbrace{\left(\boldsymbol{b}^{(\ell)}\right)^T}_{1 \times p_\ell} = \underbrace{Z^{(\ell)}}_{n \times p_\ell} \xrightarrow{\sigma_\ell} \underbrace{A^{(\ell)}}_{n \times p_\ell}$$

- *The total number of trainable scalar parameters (weights and biases) is*

$$P = \sum_{\ell=1}^{L} \left(p_\ell p_{\ell-1} + p_\ell\right) = \sum_{\ell=1}^{L} p_\ell \left(p_{\ell-1} + 1\right)$$

*Notice that P is **independent of the number of observations** n*

Example: dimension tracking and parameter counting

Problem: Consider a 3-layer MLP with input dimension $p_0 = 4$, hidden widths $p_1 = 5$ and $p_2 = 3$ and output dimension $p_3 = 2$, evaluated on a batch of $n = 10$ observations. Track all matrix dimensions and count the total number of trainable parameters

Layer ℓ	$A^{(\ell-1)}$	$W^{(\ell)}$	$\left(W^{(\ell)}\right)^T$	$\boldsymbol{b}^{(\ell)}$	$\mathbb{1}_n\left(\boldsymbol{b}^{(\ell)}\right)^T$	$Z^{(\ell)}, A^{(\ell)}$
$\ell = 1$	10×4	5×4	4×5	5×1	10×5	10×5
$\ell = 2$	10×5	3×5	5×3	3×1	10×3	10×3
$\ell = 3$	10×3	2×3	3×2	2×1	10×2	10×2

Total trainable scalar parameters:

- Layer 1: $p_1 p_0 + p_1 = 5 \times 4 + 5 = 25$
- Layer 2: $p_2 p_1 + p_2 = 3 \times 5 + 3 = 18$
- Layer 3: $p_3 p_2 + p_3 = 2 \times 3 + 2 = 8$
- **Total:** $P = 25 + 18 + 8 = 51$ parameters

Example by hand: solving XOR with a 2-layer ReLU network

Problem: Evaluate a 2-layer network ($p_0 = 2, p_1 = 2, p_2 = 1$) on all $n = 4$ points of $\{0,1\}^2$ simultaneously, with ReLU hidden activation $\sigma_1(z) = \max(0, z)$ and linear output $\sigma_2(z) = z$:

$$X = \begin{pmatrix} 0 & 0 \\ 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix} \in \mathcal{M}_{42}, \quad W^{(1)} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \in \mathcal{M}_{22}, \quad \boldsymbol{b}^{(1)} = \begin{pmatrix} 0 \\ -1 \end{pmatrix} \in \mathbb{R}^2$$

$$W^{(2)} = \left(\boldsymbol{w}^{(2)}\right)^T = \begin{pmatrix} 1 & -2 \end{pmatrix} \in \mathcal{M}_{12}, \quad b^{(2)} = 0 \in \mathbb{R}$$

Step 1 – Hidden pre-activations $Z^{(1)} = X\left(W^{(1)}\right)^T + \mathbb{1}_4\left(\boldsymbol{b}^{(1)}\right)^T$:

$$Z^{(1)} = \begin{pmatrix} 0 & 0 \\ 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix} \begin{pmatrix} 0 & -1 \end{pmatrix} = \begin{pmatrix} 0 & 0 \\ 1 & 1 \\ 1 & 1 \\ 2 & 2 \end{pmatrix} + \begin{pmatrix} 0 & -1 \\ 0 & -1 \\ 0 & -1 \\ 0 & -1 \end{pmatrix} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{pmatrix}$$

Example by hand: hidden activations and output layer

Step 2 – Apply ReLU entrywise $A^{(1)} = \max\left(0, Z^{(1)}\right)$:

$$A^{(1)} = \max\left(0, \begin{pmatrix} 0 & -1 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{pmatrix}\right) = \begin{pmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{pmatrix} \in \mathcal{M}_{42}$$

Step 3 – Output layer $\hat{\boldsymbol{y}} = Z^{(2)} = A^{(1)}\left(W^{(2)}\right)^T + \mathbb{1}_4 b^{(2)}$:

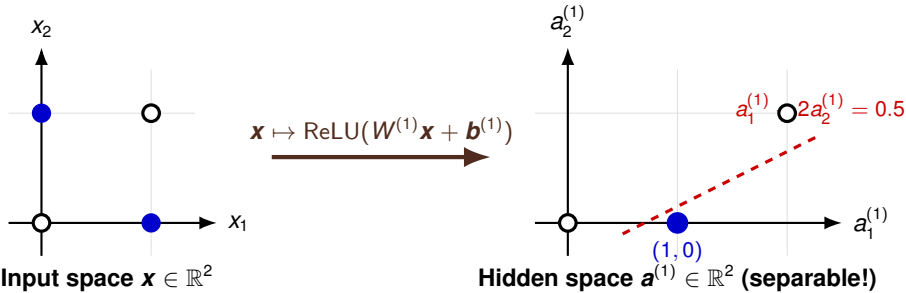
$$\hat{\boldsymbol{y}} = \begin{pmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ -2 \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 0(1) + 0(-2) \\ 1(1) + 0(-2) \\ 1(1) + 0(-2) \\ 2(1) + 1(-2) \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \end{pmatrix}$$

The 2-layer ReLU network reproduces the exact XOR truth table on all four points!

Geometric interpretation: how the hidden layer untangles XOR

Look at the rows of the hidden activation matrix $A^{(1)}$ as coordinates in the hidden space $\mathbb{R}^2$:

$$\boldsymbol{x}_1 = (0,0)^T \mapsto \boldsymbol{a}_1^{(1)} = (0,0)^T, \quad \boldsymbol{x}_2 = (1,0)^T \text{ and } \boldsymbol{x}_3 = (0,1)^T \mapsto (1,0)^T, \quad \boldsymbol{x}_4 = (1,1)^T \mapsto$$



Verification in R: batch matrix forward pass for XOR

```
X = matrix(c(0, 0,  1, 0,  0, 1,  1, 1), nrow = 4, byrow = TRUE)
W1 = matrix(c(1, 1,  1, 1), nrow = 2, byrow = TRUE)
b1 = c(0, -1);  one_n = rep(1, nrow(X))
Z1 = X %*% t(W1) + one_n %*% t(b1)
A1 = pmax(0, Z1)
y_hat = A1 %*% c(1, -2)

## Error in `A1 %*% c(1, -2)`:
## !  non-conformable arguments

cbind(X, A1 = A1, y_hat = as.vector(y_hat))

## Error:
## !  object 'y_hat' not found
```

From single neurons to affine matrix layers

Why non-linearity is essential: collapsing linear networks

Linear autoencoders and subspace projections (bridge to PCA)

Graph convolutional networks: combining left and right matrix multiplication

What if all activation functions are linear?

Suppose we build a deep neural network with L layers, but we omit the non-linear activations by setting every activation function to the identity map:

$$\sigma_\ell(\boldsymbol{z}) = \boldsymbol{z} \qquad \forall \ell \in \{1, \ldots, L\}$$

Such an architecture is called a **deep linear network**

At each layer ℓ, the transformation is purely affine:

$$\boldsymbol{a}^{(\ell)} = W^{(\ell)} \boldsymbol{a}^{(\ell-1)} + \boldsymbol{b}^{(\ell)}$$

What is the expressive power of composing L affine transformations in sequence?

Collapse of composed affine maps

Theorem 185 (Collapse of deep linear networks)

*Let $f_\ell(\boldsymbol{u}) = W^{(\ell)}\boldsymbol{u} + \boldsymbol{b}^{(\ell)}$ be affine maps with $W^{(\ell)} \in \mathcal{M}_{p_\ell, p_{\ell-1}}$ and $\boldsymbol{b}^{(\ell)} \in \mathbb{R}^{p_\ell}$ for $\ell = 1, \ldots, L$. Their composition $f = f_L \circ f_{L-1} \circ \cdots \circ f_1 : \mathbb{R}^{p_0} \to \mathbb{R}^{p_L}$ is a **single affine map***

$$f(\boldsymbol{x}) = W\boldsymbol{x} + \boldsymbol{b}$$

where the effective weight matrix $W \in \mathcal{M}_{p_L, p_0}$ and effective bias $\boldsymbol{b} \in \mathbb{R}^{p_L}$ are given by

$$W = W^{(L)} W^{(L-1)} \cdots W^{(1)}, \qquad \boldsymbol{b} = \sum_{\ell=1}^{L-1} \left(W^{(L)} \cdots W^{(\ell+1)} \right) \boldsymbol{b}^{(\ell)} + \boldsymbol{b}^{(L)}$$

In batch matrix form on $X \in \mathcal{M}_{n, p_0}$, the L-layer linear network collapses to

$$A^{(L)} = XW^T + \mathbb{1}_n \boldsymbol{b}^T$$

Proof of Theorem 185 (collapse of composed affine maps)

We proceed by induction on the number of layers $L \geq 1$

Base case ($L = 2$): For two layers, substituting $\boldsymbol{a}^{(1)} = W^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)}$ into layer 2 gives

$$\begin{aligned}\boldsymbol{a}^{(2)} &= W^{(2)}\boldsymbol{a}^{(1)} + \boldsymbol{b}^{(2)} \\ &= W^{(2)}\big(W^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)}\big) + \boldsymbol{b}^{(2)} \\ &= \big(W^{(2)}W^{(1)}\big)\boldsymbol{x} + \big(W^{(2)}\boldsymbol{b}^{(1)} + \boldsymbol{b}^{(2)}\big)\end{aligned}$$

which is a single affine map with $W = W^{(2)}W^{(1)}$ and $\boldsymbol{b} = W^{(2)}\boldsymbol{b}^{(1)} + \boldsymbol{b}^{(2)}$

Inductive step: Suppose after $L-1$ layers that $\boldsymbol{a}^{(L-1)} = \widetilde{W}\boldsymbol{x} + \tilde{\boldsymbol{b}}$ with $\widetilde{W} = W^{(L-1)} \cdots W^{(1)}$. Applying layer L yields

$$\boldsymbol{a}^{(L)} = W^{(L)}\big(\widetilde{W}\boldsymbol{x} + \tilde{\boldsymbol{b}}\big) + \boldsymbol{b}^{(L)} = \big(W^{(L)}\widetilde{W}\big)\boldsymbol{x} + \big(W^{(L)}\tilde{\boldsymbol{b}} + \boldsymbol{b}^{(L)}\big) = W\boldsymbol{x} + \boldsymbol{b} \quad \square$$

Rank bottleneck bound for deep linear networks

Not only does a deep linear network collapse into a single matrix $W = W^{(L)} \cdots W^{(1)}$, the rank of W is choked by the narrowest layer in the network

Theorem 186 (Rank bottleneck bound)

Let $W = W^{(L)} W^{(L-1)} \cdots W^{(1)} \in \mathcal{M}_{p_L, p_0}$ *with* $W^{(\ell)} \in \mathcal{M}_{p_\ell, p_{\ell-1}}$ *for* $\ell = 1, \ldots, L$. *Then*

$$\operatorname{rank}(W) \leq \min_{1 \leq \ell \leq L} \operatorname{rank}\big(W^{(\ell)}\big) \leq \min_{0 \leq \ell \leq L} p_\ell$$

Before proving Theorem 186, recall the fundamental definitions for $M \in \mathcal{M}_{np}$:

$$\operatorname{range}(M) = \big\{M\boldsymbol{x} : \boldsymbol{x} \in \mathbb{R}^p\big\} \subseteq \mathbb{R}^n, \qquad \operatorname{null}(M) = \big\{\boldsymbol{x} \in \mathbb{R}^p : M\boldsymbol{x} = \boldsymbol{0}\big\} \subseteq \mathbb{R}^p$$

and the **rank-nullity theorem**:
$\operatorname{rank}(M) + \operatorname{nullity}(M) = \dim \operatorname{range}(M) + \dim \operatorname{null}(M) = p$

Proof of Theorem 186 – part 1: subspace inclusions

Let $A \in \mathcal{M}_{np}$ and $B \in \mathcal{M}_{pq}$ so that $AB \in \mathcal{M}_{nq}$. We prove two fundamental inclusions:

1. **Range inclusion** $\text{range}(AB) \subseteq \text{range}(A)$: Let $\boldsymbol{y} \in \text{range}(AB)$. Then there exists $\boldsymbol{x} \in \mathbb{R}^q$ such that

$$\boldsymbol{y} = (AB)\boldsymbol{x} = A(B\boldsymbol{x})$$

Setting $\boldsymbol{u} = B\boldsymbol{x} \in \mathbb{R}^p$, we have $\boldsymbol{y} = A\boldsymbol{u} \in \text{range}(A)$. Thus $\text{range}(AB) \subseteq \text{range}(A)$, which implies

$$\text{rank}(AB) = \dim \text{range}(AB) \leq \dim \text{range}(A) = \text{rank}(A)$$

2. **Nullspace inclusion** $\text{null}(B) \subseteq \text{null}(AB)$: Let $\boldsymbol{x} \in \text{null}(B)$, so $B\boldsymbol{x} = \boldsymbol{0}$. Multiplying on the left by A gives

$$(AB)\boldsymbol{x} = A(B\boldsymbol{x}) = A\boldsymbol{0} = \boldsymbol{0} \implies \boldsymbol{x} \in \text{null}(AB)$$

Thus $\text{null}(B) \subseteq \text{null}(AB)$, so $\text{nullity}(B) \leq \text{nullity}(AB)$

Proof of Theorem 186 – part 2: rank of a matrix product

3. **Combining with the rank-nullity theorem**: Since $B \in \mathcal{M}_{pq}$ and $AB \in \mathcal{M}_{nq}$ both have q columns, the rank-nullity theorem gives

$$\text{rank}(AB) = q - \text{nullity}(AB) \leq q - \text{nullity}(B) = \text{rank}(B)$$

Combining parts 1 and 2 yields the fundamental inequality for any compatible matrices A and B:

$$\text{rank}(AB) \leq \min\big(\text{rank}(A), \text{rank}(B)\big)$$

4. **Induction over L layers**: Applying $\text{rank}(AB) \leq \min(\text{rank}(A), \text{rank}(B))$ inductively to $W = W^{(L)} W^{(L-1)} \cdots W^{(1)}$ gives

$$\text{rank}(W) \leq \min_{1 \leq \ell \leq L} \text{rank}\big(W^{(\ell)}\big)$$

Finally, since $W^{(\ell)} \in \mathcal{M}_{p_\ell, p_{\ell-1}}$, its rank cannot exceed its number of rows p_ℓ or columns $p_{\ell-1}$, so $\text{rank}(W^{(\ell)}) \leq \min(p_\ell, p_{\ell-1})$, which gives $\text{rank}(W) \leq \min_{0 \leq \ell \leq L} p_\ell$ □

Consequences of linear collapse and the rank bottleneck

Theorem 185 and Theorem 186 reveal two critical design principles of neural networks:

1. **Depth without non-linearity is futile**: A 100-layer linear network cannot represent any function that a single 1-layer affine map $f(\boldsymbol{x}) = W\boldsymbol{x} + \boldsymbol{b}$ cannot already represent. In particular, a 100-layer linear network **still cannot solve XOR**!
2. **Narrow hidden layers create permanent rank bottlenecks**: Suppose a 3-layer linear network has architecture $\mathbb{R}^{10} \xrightarrow{W^{(1)}} \mathbb{R}^{2} \xrightarrow{W^{(2)}} \mathbb{R}^{50} \xrightarrow{W^{(3)}} \mathbb{R}^{10}$. Even though layer 2 expands to 50 neurons and the output has dimension 10,

$$\operatorname{rank}\left(W^{(3)}W^{(2)}W^{(1)}\right) \leq \min(10, 2, 50, 10) = 2$$

Every output vector $f(\boldsymbol{x})$ is confined to at most a 2-dimensional affine subspace of $\mathbb{R}^{10}$!

Non-linear activations σ_ℓ bend the coordinate hyperplanes at each layer, allowing the network to construct piecewise-linear (for ReLU) or smooth non-linear decision boundaries

Example by hand: collapsing a 2-layer linear network

Problem: Consider a 2-layer linear network mapping $\mathbb{R}^3 \to \mathbb{R}^2 \to \mathbb{R}^3$ ($\sigma_1 = \sigma_2 = \text{id}$) with

$$W^{(1)} = \begin{pmatrix} 1 & 2 & -1 \\ 0 & 1 & 1 \end{pmatrix}, \quad \boldsymbol{b}^{(1)} = \begin{pmatrix} 1 \\ -1 \end{pmatrix}, \quad W^{(2)} = \begin{pmatrix} 2 & 1 \\ -1 & 0 \\ 1 & 1 \end{pmatrix}, \quad \boldsymbol{b}^{(2)} = \begin{pmatrix} 0 \\ 2 \\ 1 \end{pmatrix}$$

Find the collapsed affine map $f(\boldsymbol{x}) = W\boldsymbol{x} + \boldsymbol{b}$ and determine $\text{rank}(W)$

Solution: By Theorem 185,

$$W = W^{(2)}W^{(1)} = \begin{pmatrix} 2 & 1 \\ -1 & 0 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 2 & -1 \\ 0 & 1 & 1 \end{pmatrix} = \begin{pmatrix} 2 & 5 & -1 \\ -1 & -2 & 1 \\ 1 & 3 & 0 \end{pmatrix}, \quad \boldsymbol{b} = W^{(2)}\boldsymbol{b}^{(1)} + \boldsymbol{b}^{(2)} =$$

Example by hand: verifying the rank bottleneck

Although $W \in \mathcal{M}_3$ is a 3×3 matrix, Theorem 186 guarantees that because the hidden layer has width $p_1 = 2$,

$$\text{rank}(W) \leq \min(3, 2, 3) = 2 < 3$$

so W **cannot be invertible** ($\det(W) = 0$)

Indeed, inspecting the rows R_1, R_2, R_3 of $W = \begin{pmatrix} 2 & 5 & -1 \\ -1 & -2 & 1 \\ 1 & 3 & 0 \end{pmatrix}$, we see that

$$R_1 + R_2 = (2 - 1,\ 5 - 2,\ -1 + 1) = (1, 3, 0) = R_3$$

Since R_1 and R_2 are not scalar multiples of each other, $\text{rank}(W) = 2$

How ReLU breaks the rank bound: In our XOR example, the pre-activation matrix $Z^{(1)} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{pmatrix}$ had column 2 equal to (column 1) $- \mathbb{1}_4$, so no linear combination with $\mathbb{1}_4$ could yield $\boldsymbol{y} = (0, 1, 1, 0)^T$. Applying ReLU zeroed out only $\begin{pmatrix} 0 & 0 \end{pmatrix}$

From single neurons to affine matrix layers

Why non-linearity is essential: collapsing linear networks

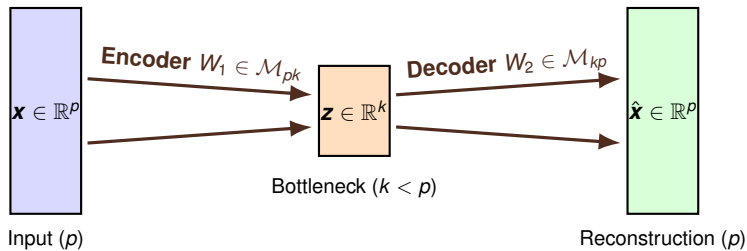
Linear autoencoders and subspace projections (bridge to PCA)

Graph convolutional networks: combining left and right matrix multiplication

When a rank bottleneck is a feature, not a bug!

In supervised classification, an unintended rank bottleneck restricts the expressive power of a linear network

In **unsupervised representation learning** and **dimensionality reduction** (Units 04 and 05), however, we **deliberately** introduce a narrow hidden layer of width $k < p$ to force the network to compress p-dimensional data into k latent coordinates!



Linear autoencoders

Definition 187 (Linear autoencoder)

*Let $\widetilde{X} = C_n X \in \mathcal{M}_{np}$ be a centred data matrix (n observations in $\mathbb{R}^p$, column means zero) and let $1 \leq k < p$ be the bottleneck dimension. A **linear autoencoder** consists of:*

1. *An **encoder matrix** $W_1 \in \mathcal{M}_{pk}$ mapping data to a low-dimensional **latent code***
$$Z = \widetilde{X} W_1 \in \mathcal{M}_{nk}$$
2. *A **decoder matrix** $W_2 \in \mathcal{M}_{kp}$ mapping the latent code back to a reconstruction*
$$\hat{X} = Z W_2 = \widetilde{X} W_1 W_2 \in \mathcal{M}_{np}$$

The autoencoder is trained to minimise the squared Frobenius reconstruction error:

$$\min_{W_1 \in \mathcal{M}_{pk},\ W_2 \in \mathcal{M}_{kp}} \left\| \widetilde{X} - \widetilde{X} W_1 W_2 \right\|_F^2 = \min_{W_1 \in \mathcal{M}_{pk},\ W_2 \in \mathcal{M}_{kp}} \sum_{i=1}^{n} \left\| \tilde{\boldsymbol{x}}_i - W_2^T W_1^T \tilde{\boldsymbol{x}}_i \right\|_2^2$$

Bridge to SVD (Unit 04) and PCA (Unit 05)

Let us connect the linear autoencoder problem directly to the Eckart-Young theorem (Unit 04) and principal component analysis (Unit 05)

- Recall that the centred data matrix $\widetilde{X} \in \mathcal{M}_{np}$ has singular value decomposition

$$\widetilde{X} = U\Sigma V^T = \sum_{j=1}^{r} \sigma_j \boldsymbol{u}_j \boldsymbol{v}_j^T, \qquad \sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$$

  where the right singular vectors $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_p \in \mathbb{R}^p$ are the orthonormal eigenvectors of the sample covariance matrix $S_X = \frac{1}{n-1}\widetilde{X}^T\widetilde{X}$ (*i.e.*, the **principal axes**)

- Let $V_k = [\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k] \in \mathcal{M}_{pk}$ collect the top k principal axes. In Unit 05, the rank-k PCA reconstruction of $\widetilde{X}$ was shown to be

$$\widetilde{X}_k = U_k \Sigma_k V_k^T = (\widetilde{X} V_k) V_k^T = \widetilde{X}(V_k V_k^T)$$

  where $P_k = V_k V_k^T \in \mathcal{M}_p$ is the **orthogonal projector** onto $\text{span}\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\}$

Optimal linear autoencoders recover the PCA subspace

Theorem 188 (Baldi and Hornik, 1989 – Eckart-Young corollary)

Let $\widetilde{X} \in \mathcal{M}_{np}$ have singular values $\sigma_1 \geq \cdots \geq \sigma_p \geq 0$ and right singular vectors $V = [\boldsymbol{v}_1, \ldots, \boldsymbol{v}_p] \in \mathcal{M}_p$. For any $k < \text{rank}(\widetilde{X})$, consider

$$\min_{W_1 \in \mathcal{M}_{pk},\, W_2 \in \mathcal{M}_{kp}} \|\widetilde{X} - \widetilde{X} W_1 W_2\|_F^2$$

1. *The minimum reconstruction error is* $\displaystyle\sum_{j=k+1}^{p} \sigma_j^2$
2. *When $\sigma_k > \sigma_{k+1}$, the global minimum on* $\text{range}(\widetilde{X}^T)$ *is achieved when*

$$P = W_1 W_2 = V_k V_k^T$$

is the orthogonal projection matrix onto the top-k principal subspace $\text{span}\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\}$*, achieved by*

Proof of Theorem 188 (Baldi-Hornik / Eckart-Young corollary)

1. **Lower bound via the rank bottleneck bound and Eckart-Young**: For any $W_1 \in \mathcal{M}_{pk}$ and $W_2 \in \mathcal{M}_{kp}$, Theorem 186 implies

$$\text{rank}(W_1 W_2) \leq \min(p, k, p) = k \implies \text{rank}(\widetilde{X} W_1 W_2) \leq \text{rank}(W_1 W_2) \leq k$$

Hence $\hat{X} = \widetilde{X} W_1 W_2 \in \mathcal{M}_{np}$ is a matrix of rank at most k. By the Eckart-Young theorem (Unit 04), the best rank-k approximation to $\widetilde{X}$ in the Frobenius norm is the truncated SVD $\widetilde{X}_k = U_k \Sigma_k V_k^T$, so for all W_1, W_2:

$$\|\widetilde{X} - \widetilde{X} W_1 W_2\|_F^2 \geq \|\widetilde{X} - \widetilde{X}_k\|_F^2 = \sum_{j=k+1}^{p} \sigma_j^2$$

2. **Attaining the bound**: Let $C \in \text{GL}_k(\mathbb{R})$ be any invertible $k \times k$ matrix and set $W_1 = V_k C \in \mathcal{M}_{pk}$ and $W_2 = C^{-1} V_k^T \in \mathcal{M}_{kp}$. Then

$$W_1 W_2 = (V_k C)(C^{-1} V_k^T) = V_k (C C^{-1}) V_k^T = V_k \mathbb{I}_k V_k^T = V_k V_k^T$$

Since $\widetilde{X} V_k = U \Sigma V^T V_k = U_k \Sigma_k$, we obtain
$\widetilde{X} W_1 W_2 = \widetilde{X} V_k V_k^T = U_k \Sigma_k V_k^T = \widetilde{X}_k$, achieving the exact Eckart-Young minimum $\sum_{j=k+1}^{p} \sigma_j^2$.

Subspace equivalence vs basis non-uniqueness

Notice a subtle and important distinction in Theorem 188:

- **What is unique**: The product $P = W_1 W_2 = V_k V_k^T \in \mathcal{M}_p$ is the unique orthogonal projector onto the k-dimensional principal subspace $\mathcal{S}_k = \text{span}\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\}$ (satisfying $P^2 = P$ and $P^T = P$)
- **What is not unique (gauge freedom)**: Because $W_1 = V_k C$ and $W_2 = C^{-1} V_k^T$ give the exact same product $W_1 W_2 = V_k V_k^T$ for **any** invertible $C \in \text{GL}_k(\mathbb{R})$, a linear autoencoder without extra constraints learns a basis for the principal subspace $\mathcal{S}_k$, not necessarily the orthogonal principal axes V_k themselves
- **PCA corresponds to choosing** $C = \mathbb{I}_k$: Setting $W_1 = V_k$ and $W_2 = V_k^T$ imposes the orthogonality constraint $W_1^T W_1 = \mathbb{I}_k$ and ties the decoder weights to the transpose of the encoder ($W_2 = W_1^T$)

Orthogonal projector properties and Pythagorean error decomposition

Let us verify that $P_k = V_k V_k^T \in \mathcal{M}_p$ is an **orthogonal projection matrix**:

- **Symmetric**: $P_k^T = \left(V_k V_k^T\right)^T = \left(V_k^T\right)^T V_k^T = V_k V_k^T = P_k$
- **Idempotent**: Since the columns of V_k are orthonormal ($V_k^T V_k = \mathbb{I}_k$),

$$P_k^2 = \left(V_k V_k^T\right)\left(V_k V_k^T\right) = V_k \left(V_k^T V_k\right) V_k^T = V_k \mathbb{I}_k V_k^T = P_k$$

Every centred data matrix $\widetilde{X} \in \mathcal{M}_{np}$ decomposes into two mutually orthogonal components:

$$\widetilde{X} = \underbrace{\widetilde{X} P_k}_{\text{projection onto } \mathcal{S}_k} + \underbrace{\widetilde{X}(\mathbb{I}_p - P_k)}_{\text{residual in } \mathcal{S}_k^{\perp}}$$

Because $P_k(\mathbb{I}_p - P_k) = P_k - P_k^2 = 0$, the squared Frobenius norm splits by Pythagoras:

$$\|\widetilde{X} - \widetilde{X} P_k\|_F^2 = \|\widetilde{X}\|_F^2 - \|\widetilde{X} P_k\|_F^2 = \sum_{j=1}^{p} \sigma_j^2 - \sum_{j=1}^{k} \sigma_j^2 = \sum_{j=k+1}^{p} \sigma_j^2$$

Example by hand: rank-1 linear autoencoder vs PCA

Consider the centred 4×2 data matrix $\widetilde{X} \in \mathcal{M}_{42}$ and bottleneck dimension $k = 1$:

$$\widetilde{X} = \begin{pmatrix} 3 & 4 \\ -3 & -4 \\ 3 & 4 \\ -3 & -4 \end{pmatrix}, \qquad \widetilde{X}^T\widetilde{X} = \begin{pmatrix} 36 & 48 \\ 48 & 64 \end{pmatrix} = 100 \begin{pmatrix} 0.36 & 0.48 \\ 0.48 & 0.64 \end{pmatrix}$$

- The unit eigenvector of $\widetilde{X}^T\widetilde{X}$ for $\lambda_1 = \sigma_1^2 = 100$ is $\boldsymbol{v}_1 = \begin{pmatrix} 0.6 \\ 0.8 \end{pmatrix}$ (and $\sigma_2^2 = 0$ for $\boldsymbol{v}_2 = \begin{pmatrix} -0.8 \\ 0.6 \end{pmatrix}$)
- The PCA choice ($C = 1$) uses $W_1 = \boldsymbol{v}_1 = \begin{pmatrix} 0.6 \\ 0.8 \end{pmatrix}$ and $W_2 = \boldsymbol{v}_1^T = \begin{pmatrix} 0.6 & 0.8 \end{pmatrix}$
- Any scaled pair $\widetilde{W}_1 = c\boldsymbol{v}_1 = \begin{pmatrix} 1.2 \\ 1.6 \end{pmatrix}$ and $\widetilde{W}_2 = c^{-1}\boldsymbol{v}_1^T = \begin{pmatrix} 0.3 & 0.4 \end{pmatrix}$ (with $c = 2$) gives the **exact same orthogonal projector**:

$$P = W_1 W_2 = \widetilde{W}_1 \widetilde{W}_2 = \boldsymbol{v}_1 \boldsymbol{v}_1^T = \begin{pmatrix} 0.36 & 0.48 \\ 0.48 & 0.64 \end{pmatrix}, \qquad \hat{X} = \widetilde{X}P = \widetilde{X}$$

R demonstration: training a linear autoencoder recovers PCA

Generate a centred 3D dataset ($n = 150, p = 3$) and train a linear autoencoder ($k = 2$) by minimising $\|\widetilde{X} - \widetilde{X} W_1 W_2\|_F^2$ from random initial weights:

```
set.seed(2740); n = 150; p = 3; k = 2
M_mix = matrix(c(3, 2, 1, 0, 1.5, 1, 0, 0, 0.3), nrow = 3, byrow = TRUE)
X_tilde = scale(matrix(rnorm(n * p), n, p) %*% M_mix, center = TRUE, scale =
V_k = svd(X_tilde)$v[, 1:k];  P_pca = V_k %*% t(V_k)

ae_loss = function(th) {
  W1 = matrix(th[1:6], p, k);  W2 = matrix(th[7:12], k, p)
  sum((X_tilde - X_tilde %*% W1 %*% W2)^2)
}
opt = optim(rnorm(12, sd = 0.5), ae_loss, method = "BFGS",
            control = list(maxit = 500, reltol = 1e-12))
W1_hat = matrix(opt$par[1:6], p, k);  W2_hat = matrix(opt$par[7:12], k, p)
P_ae = W1_hat %*% W2_hat
```

Comparing the learned projector $W_1 W_2$ with $V_k V_k^T$

Even though W_1 and V_k differ by an invertible change of basis $C \in \mathrm{GL}_2(\mathbb{R})$, their product $W_1 W_2$ matches the orthogonal PCA projector $V_k V_k^T$ to high precision:

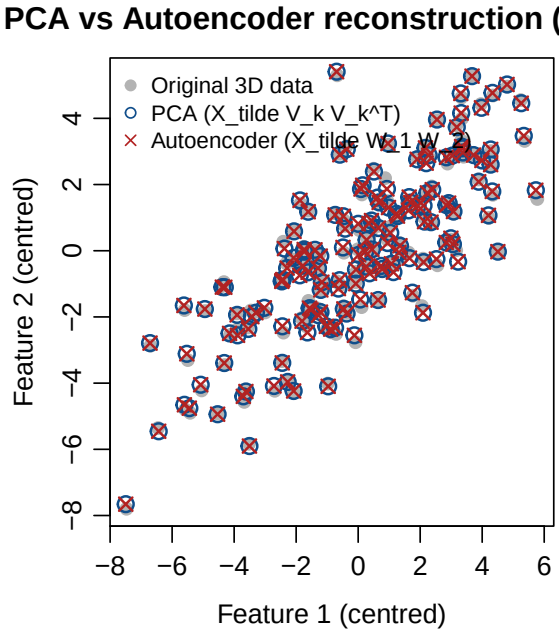
```
round(P_pca, 4)
```

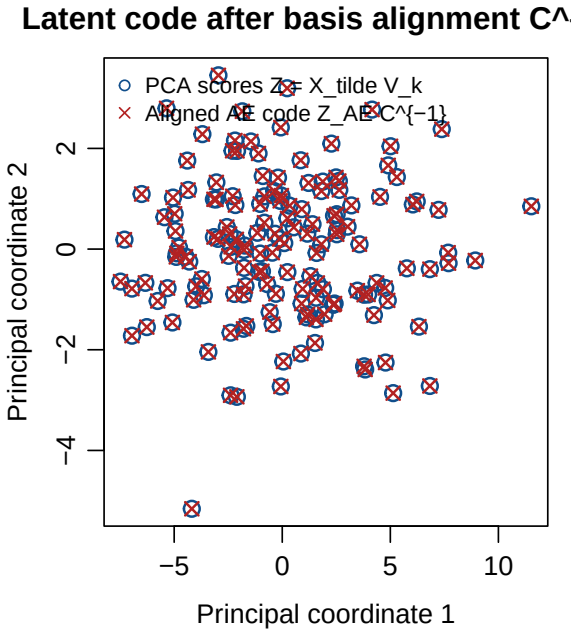
```
##         [,1]   [,2]    [,3]
## [1,]  0.9897 0.0570 -0.0832
## [2,]  0.0570 0.6838  0.4615
## [3,] -0.0832 0.4615  0.3265
```

```
round(P_ae, 4)
```

```
##         [,1]   [,2]    [,3]
## [1,]  0.9897 0.0570 -0.0832
## [2,]  0.0570 0.6838  0.4615
## [3,] -0.0832 0.4615  0.3265
```

```
max(abs(P_pca - P_ae))
```


PCA vs Autoencoder reconstruction (x
Original 3D data
PCA (X_tilde V_k V_k^T)
Autoencoder (X_tilde W_1 W_2)
Feature 2 (centred)
Feature 1 (centred)
−8
−6
−4
−2
0
2
4
6


Latent code after basis alignment C^{
PCA scores Z = X_tilde V_k
Aligned AE code Z_AE C^{−1}
Principal coordinate 2
Principal coordinate 1
−5
0
5
10

From single neurons to affine matrix layers

Why non-linearity is essential: collapsing linear networks

Linear autoencoders and subspace projections (bridge to PCA)

Graph convolutional networks: combining left and right matrix multiplication

Graph convolutional networks: combining left and right matrix multiplication

Left-multiplication vs right-multiplication on X

Self-loops, degree normalisation and the GCN layer

What when the rows of X are connected by a graph?

In a standard MLP or autoencoder, the rows of $X \in \mathcal{M}_{np}$ are treated as independent observations

In many applications (Units 08 to 10), however, the n rows of $X \in \mathcal{M}_{np}$ are the **vertices of a graph** $G = (V, E)$ with adjacency matrix $A \in \mathcal{M}_n$ (in graph machine learning, the data matrix X is called the **vertex feature matrix** or **node feature matrix**):

- **Citation networks**: vertices are scientific papers, rows $\boldsymbol{x}_i^T \in \mathbb{R}^p$ are word-frequency vectors and edges are citations
- **Social and epidemiological networks**: vertices are individuals or cities, features are demographic variables and edges are contact or mobility links
- **Molecular graphs**: vertices are atoms, features are atomic properties and edges are chemical bonds

How can a neural network layer use **both** the vertex features $X \in \mathcal{M}_{np}$ and the graph structure $A \in \mathcal{M}_n$?

The fundamental linear algebra duality: left vs right multiplication

Given a data matrix (vertex feature matrix) $X \in \mathcal{M}_{np}$ (n vertices in rows, p features in columns), consider what happens when we multiply X on the **right** versus on the **left**:

Property 189 (Right-multiplication mixes features; left-multiplication mixes vertices)

Let $X \in \mathcal{M}_{np}$ have rows $\boldsymbol{x}_1^T, \ldots, \boldsymbol{x}_n^T \in \mathbb{R}^{1\times p}$ and columns $\boldsymbol{c}_1, \ldots, \boldsymbol{c}_p \in \mathbb{R}^n$:

1. ***Right-multiplication by a weight matrix** $W \in \mathcal{M}_{pq}$:*

$$(XW)_{i\bullet} = \boldsymbol{x}_i^T W \in \mathbb{R}^{1\times q} \qquad (i = 1, \ldots, n)$$

   *Each vertex i transforms its **own features** independently of all other vertices (this is a standard MLP layer, completely ignoring the graph G)*

2. ***Left-multiplication by a graph operator** $S \in \mathcal{M}_n$:*

$$(SX)_{i\bullet} = \sum_{j=1}^{n} s_{ij} \boldsymbol{x}_j^T \in \mathbb{R}^{1\times p} \qquad (i = 1, \ldots, n)$$

Visualising the two-sided sandwich product SXW

Combining left-multiplication by a graph operator $S \in \mathcal{M}_n$ and right-multiplication by a feature weight matrix $W \in \mathcal{M}_{pq}$ produces the **sandwich product** $SXW \in \mathcal{M}_{nq}$:

$$\underbrace{S}_{\substack{n\times n \\ \text{mixes rows} \\ \text{(graph neighbours)}}} \quad \underbrace{X}_{\substack{n\times p \\ \text{vertex features} \\ \text{(data matrix)}}} \quad \underbrace{W}_{\substack{p\times q \\ \text{mixes columns} \\ \text{(trainable weights)}}} = \underbrace{Z}_{\substack{n\times q \\ \text{updated vertex} \\ \text{representations}}}$$

By associativity of matrix multiplication, $(SX)W = S(XW)$:

- Viewing it as $(SX)W$: first **aggregate/smooth features** across graph neighbours ($SX \in \mathcal{M}_{np}$), then **project features** using trainable weights W
- Viewing it as $S(XW)$: first **project each vertex's features** into $\mathbb{R}^q$ ($XW \in \mathcal{M}_{nq}$), then **aggregate the projected vectors** across graph neighbours via S

Graph convolutional networks: combining left and right matrix multiplication

Left-multiplication vs right-multiplication on X

Self-loops, degree normalisation and the GCN layer

Why raw adjacency multiplication AX has two flaws

Suppose we set $S = A$, where $A = [a_{ij}] \in \mathcal{M}_n$ is the unweighted adjacency matrix of an undirected simple graph G ($a_{ij} = 1$ if $i \sim j$ and 0 otherwise). Then row i of AX is

$$(AX)_{i\bullet} = \sum_{j=1}^{n} a_{ij} \boldsymbol{x}_j^T = \sum_{j \in \mathcal{N}(i)} \boldsymbol{x}_j^T$$

where $\mathcal{N}(i) = \{j \in V : i \sim j\}$ is the neighbourhood of vertex i

Using raw A as our left-multiplier has **two structural flaws**:

1. **Forgetting a vertex's own features**: Because a simple graph has no self-loops ($a_{ii} = 0$), $(AX)_{i\bullet}$ sums the neighbours' features $\boldsymbol{x}_j^T$ ($j \in \mathcal{N}(i)$) but **completely discards** vertex i's own feature vector $\boldsymbol{x}_i^T$!
2. **Exploding/vanishing scales across vertex degrees**: A high-degree hub with $d_i = 1000$ sums 1000 vectors, while a leaf vertex with $d_i = 1$ takes only 1 vector. Across multiple layers, feature magnitudes explode at hubs!

Fix 1 and Fix 2: self-loops and degree normalisation

We fix both flaws using standard graph matrix operations from Units 08 to 10:

1. **Fix 1 – Add self-loops to every vertex**: Define the **augmented adjacency matrix**

$$\tilde{A} = A + \mathbb{I}_n \in \mathcal{M}_n$$

   Now $\tilde{a}_{ii} = 1$, so $(\tilde{A}X)_{i\bullet} = \boldsymbol{x}_i^T + \sum_{j \in \mathcal{N}(i)} \boldsymbol{x}_j^T$ includes vertex i itself

2. **Fix 2 – Normalise by the augmented degree matrix**: Let $D = \text{diag}(d_1, \ldots, d_n)$ be the degree matrix of G, where $d_i = \sum_{j=1}^n a_{ij} = |\mathcal{N}(i)|$. The degree matrix of $\tilde{A}$ is the **augmented degree matrix**

$$\tilde{D} = D + \mathbb{I}_n = \text{diag}(d_1 + 1, d_2 + 1, \ldots, d_n + 1) \in \mathcal{M}_n$$

   Since $d_i \geq 0$, $\tilde{d}_{ii} = d_i + 1 \geq 1 > 0$, so $\tilde{D}$ is **always invertible**, even if G has isolated vertices!

Two canonical normalised graph smoothing operators

Using $\tilde{A} = A + \mathbb{I}_n$ and $\tilde{D} = D + \mathbb{I}_n$, there are two standard choices for the smoothing operator $S \in \mathcal{M}_n$:

Definition 190 (Random-walk and symmetric normalised graph operators)

1. **Random-walk (row-normalised) propagation matrix**:

$$S_{\text{rw}} = \tilde{D}^{-1}\tilde{A}, \qquad \left(S_{\text{rw}}\right)_{ij} = \frac{\tilde{a}_{ij}}{d_i + 1}$$

Here S_{rw} is **row-stochastic** ($S_{\text{rw}}\mathbb{1}_n = \mathbb{1}_n$) and row i of $S_{\text{rw}}X$ is the **exact arithmetic average** of vertex i and its neighbours:

$$\left(\tilde{D}^{-1}\tilde{A}X\right)_{i\bullet} = \frac{1}{d_i + 1}\boldsymbol{x}_i^T + \sum_{j \in \mathcal{N}(i)} \frac{1}{d_i + 1}\boldsymbol{x}_j^T$$

2. **Symmetric normalised propagation matrix** (Kipf and Welling, 2017):

The graph convolutional network (GCN) layer

Definition 191 (Graph convolutional network layer)

Given a graph $G = (V, E)$ with augmented adjacency matrix $\tilde{A} = A + \mathbb{I}_n$, augmented degree matrix $\tilde{D} = D + \mathbb{I}_n$ and propagation operator $S \in \{\tilde{D}^{-1}\tilde{A},\ \tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}\}$, an L-layer ***Graph Convolutional Network (GCN)*** *with layer widths $p_0, p_1, \ldots, p_L$ sets $H^{(0)} = X \in \mathcal{M}_{n,p_0}$ (where $p_0 = p$) and updates vertex representations at layer $\ell \in \{1, \ldots, L\}$ via*

$$H^{(\ell)} = \sigma_\ell\big(SH^{(\ell-1)}W^{(\ell)}\big) \in \mathcal{M}_{n,p_\ell}$$

where $W^{(\ell)} \in \mathcal{M}_{p_{\ell-1},p_\ell}$ is the trainable feature weight matrix of layer ℓ and σ_ℓ is a component-wise activation function

Notice that if the graph has **no edges** ($A = 0$), then $\tilde{A} = \tilde{D} = \mathbb{I}_n$, so $S = \mathbb{I}_n$ and the GCN layer reduces to a standard MLP layer $H^{(\ell)} = \sigma_\ell\big(H^{(\ell-1)}W^{(\ell)}\big)$!

Multi-hop aggregation and over-smoothing (link to Markov chains)

What happens as we stack L GCN layers?

- In layer 1, vertex i averages features from its 1-hop neighbourhood $\mathcal{N}(i) \cup \{i\}$
- In layer 2, vertex i averages layer-1 features from its neighbours, which already contain information from *their* neighbours – aggregating the 2-**hop neighbourhood**!
- In a linear GCN ($\sigma_\ell = \mathrm{id}$), k layers compute $H^{(k)} = S_{\mathrm{rw}}^k X W^{(1)} \dots W^{(k)} = \left(\tilde{D}^{-1}\tilde{A}\right)^k XW$

Connection to Unit 08 (Markov chains) – the over-smoothing phenomenon:

- Because $S_{\mathrm{rw}} = \tilde{D}^{-1}\tilde{A}$ is the transition matrix of a regular Markov chain on a connected graph (self-loops make it aperiodic!), as $k \to \infty$,

$$\lim_{k\to\infty} \left(\tilde{D}^{-1}\tilde{A}\right)^k = \mathbb{1}_n \pi^T \quad \text{(a rank-1 matrix)}, \qquad \pi_j = \frac{d_j + 1}{\sum_{m=1}^{n}(d_m + 1)}$$

- Thus $\left(\tilde{D}^{-1}\tilde{A}\right)^k X \to \mathbb{1}_n(\pi^T X)$: every row becomes **identical**, washing out all local differences between vertices! Hence GCNs in practice are kept shallow ($L = 2$ or 3)

Brief remark: Transformers and self-attention as dynamic graph operators

In large language models (Transformers), a sequence of n tokens is represented by a data matrix (token embedding matrix) $X \in \mathcal{M}_{np}$. A **self-attention head** projects X using three weight matrices $W_Q, W_K \in \mathcal{M}_{p,d_k}$ and $W_V \in \mathcal{M}_{p,d_v}$:

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V$$

and computes the output matrix

$$\text{Attention}(X) = \underbrace{\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)}_{S(X)\in\mathcal{M}_n} XW_V$$

where softmax is applied row-wise so every row of $S(X)$ is non-negative and sums to 1

Linear algebra unification: Self-attention is also a sandwich product $S(X)XW_V$! Instead of a fixed graph operator $S = \tilde{D}^{-1}\tilde{A}$, the row-stochastic matrix $S(X) \in \mathcal{M}_n$ is a **learned, data-dependent adjacency matrix** computed dynamically from the pairwise inner products $QK^T = XW_QW_K^TX^T$

Mini-example: self-attention as a row-stochastic left multiplier

Consider a 2-token sequence ("river", "bank") with 2×2 embedding matrix $X = \begin{pmatrix} 0.5 & 0.2 \\ 0.8 & 0.6 \end{pmatrix}$ and $W_Q = W_K = W_V = \mathbb{I}_2$ ($d_k = 2$), so $Q = K = V = X$:

1. **Pairwise dot-product score matrix** $QK^T = XX^T$:

$$\frac{QK^T}{\sqrt{2}} = \frac{1}{\sqrt{2}} \begin{pmatrix} 0.5 & 0.2 \\ 0.8 & 0.6 \end{pmatrix} \begin{pmatrix} 0.5 & 0.8 \\ 0.2 & 0.6 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 0.29 & 0.52 \\ 0.52 & 1.00 \end{pmatrix} \approx \begin{pmatrix} 0.205 & 0.368 \\ 0.368 & 0.707 \end{pmatrix}$$

2. **Row-wise softmax produces a** 2×2 **row-stochastic matrix** $S(X)$:

$$S(X) = \mathrm{softmax}\left(\frac{QK^T}{\sqrt{2}}\right) \approx \begin{pmatrix} 0.46 & 0.54 \\ 0.42 & 0.58 \end{pmatrix}$$

3. **Left-multiplying** $V = X$ **forms convex combinations of token rows**:

$$\boldsymbol{z}_{\text{river}}^T = 0.46 \begin{pmatrix} 0.5 & 0.2 \end{pmatrix} + 0.54 \begin{pmatrix} 0.8 & 0.6 \end{pmatrix} = \begin{pmatrix} 0.662 & 0.416 \end{pmatrix}$$

Example by hand: 1-layer GCN forward pass on a 3-vertex path

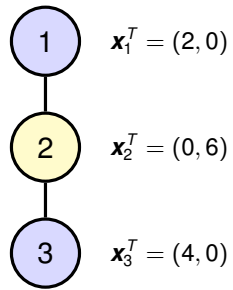
Problem: Consider the 3-vertex path graph $1 - 2 - 3$ with vertex feature matrix $X \in \mathcal{M}_{32}$ and weight matrix $W \in \mathcal{M}_{22}$:

$$X = \begin{pmatrix} 2 & 0 \\ 0 & 6 \\ 4 & 0 \end{pmatrix}, \qquad W = \begin{pmatrix} 2 & -1 \\ -1 & 1 \end{pmatrix}$$

Compute the 1-layer row-normalised ReLU GCN output

$$H = \mathrm{ReLU}\left(\tilde{D}^{-1}\tilde{A}XW\right)$$

by hand



Example by hand: step 1 – graph matrices A, $\tilde{A}$, $\tilde{D}^{-1}$ and S_{rw}

For the path graph $1 - 2 - 3$, the adjacency matrix A and augmented adjacency matrix $\tilde{A} = A + \mathbb{I}_3$ are

$$A = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}, \qquad \tilde{A} = A + \mathbb{I}_3 = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 1 \end{pmatrix}$$

The vertex degrees are $d_1 = 1$, $d_2 = 2$ and $d_3 = 1$, so $\tilde{D} = D + \mathbb{I}_3$, $\tilde{D}^{-1}$ and $S_{\text{rw}} = \tilde{D}^{-1}\tilde{A}$ are

$$\tilde{D} = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 2 \end{pmatrix}, \quad \tilde{D}^{-1} = \begin{pmatrix} \frac{1}{2} & 0 & 0 \\ 0 & \frac{1}{3} & 0 \\ 0 & 0 & \frac{1}{2} \end{pmatrix}, \quad S_{\text{rw}} = \tilde{D}^{-1}\tilde{A} = \begin{pmatrix} \frac{1}{2} & \frac{1}{2} & 0 \\ \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ 0 & \frac{1}{2} & \frac{1}{2} \end{pmatrix}$$

Example by hand: step 2 – left-multiplying X and right-multiplying W

Neighbourhood feature averaging (left-multiplication by $\tilde{D}^{-1}\tilde{A}$):

$$\tilde{D}^{-1}\tilde{A}X = \begin{pmatrix} \frac{1}{2} & \frac{1}{2} & 0 \\ \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ 0 & \frac{1}{2} & \frac{1}{2} \end{pmatrix} \begin{pmatrix} 2 & 0 \\ 0 & 6 \\ 4 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 3 \\ 2 & 2 \\ 2 & 3 \end{pmatrix}$$

Notice how vertex 2 (which initially had feature $x_{21} = 0$) acquired a positive first feature $(2 + 0 + 4)/3 = 2$ from its neighbours 1 and 3!

Feature projection and ReLU activation (right-multiplication by W, then $\max(0, \cdot)$):

$$Z = \left(\tilde{D}^{-1}\tilde{A}X\right)W = \begin{pmatrix} 1 & 3 \\ 2 & 2 \\ 2 & 3 \end{pmatrix} \begin{pmatrix} 2 & -1 \\ -1 & 1 \end{pmatrix} = \begin{pmatrix} -1 & 2 \\ 2 & 0 \\ 1 & 1 \end{pmatrix}, \qquad H = \mathrm{ReLU}(Z) = \begin{pmatrix} 0 & 2 \\ 2 & 0 \\ 1 & 1 \end{pmatrix}$$

Verification in R: 3-vertex GCN hand calculation

```
A = matrix(c(0, 1, 0,  1, 0, 1,  0, 1, 0), nrow = 3, byrow = TRUE)
A_tilde = A + diag(3)
S_rw = diag(1 / rowSums(A_tilde)) %*% A_tilde
X = matrix(c(2, 0,  0, 6,  4, 0), nrow = 3, byrow = TRUE)
W = matrix(c(2, -1,  -1, 1), nrow = 2, byrow = TRUE)
SX = S_rw %*% X;  Z = SX %*% W;  H = pmax(0, Z)
cbind(SX = SX, Z = Z, H = H)
```

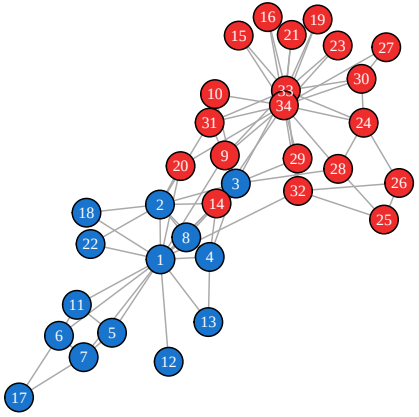
```
##                H
## [1,] 1 3 -1 2 0
## [2,] 2 2  2 0 2
## [3,] 2 3  1 1 1
```

R demonstration: 2-layer GCN on Zachary's Karate Club graph

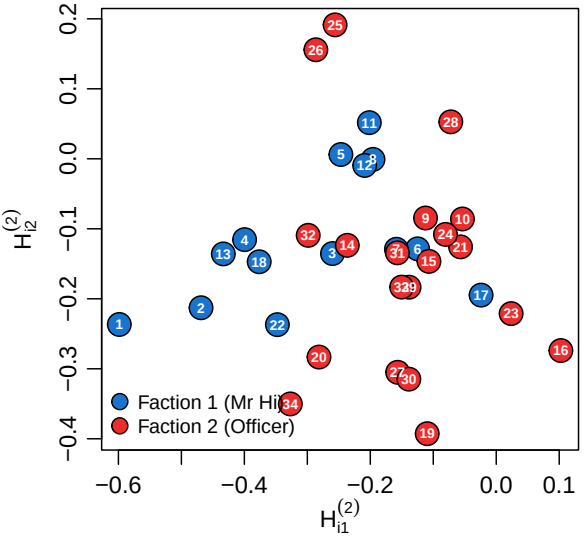
Kipf and Welling (2017) showed on Zachary's Karate Club network ($n = 34$ vertices in 2 factions) that **even with random untrained weights** $W^{(1)}, W^{(2)}$, a 2-layer GCN $H^{(2)} = \tanh\big(S_{\text{sym}}\ \tanh(S_{\text{sym}} X W^{(1)})\ W^{(2)}\big)$ with $X = \mathbb{I}_{34}$ separates the two communities in $\mathbb{R}^2$:

```
G_karate = make_graph("Zachary")
A_k = as.matrix(as_adjacency_matrix(G_karate));  n_k = nrow(A_k)
A_tilde_k = A_k + diag(n_k)
D_inv_sqrt = diag(1 / sqrt(rowSums(A_tilde_k)))
S_sym = D_inv_sqrt %*% A_tilde_k %*% D_inv_sqrt
set.seed(42)
W1_k = matrix(rnorm(n_k * 8, sd = 0.6), n_k, 8)
W2_k = matrix(rnorm(8 * 2, sd = 0.6), 8, 2)
H1_k = tanh(S_sym %*% diag(n_k) %*% W1_k)
H2_k = tanh(S_sym %*% H1_k %*% W2_k)
```

Zachary's Karate Club graph (2 factions)



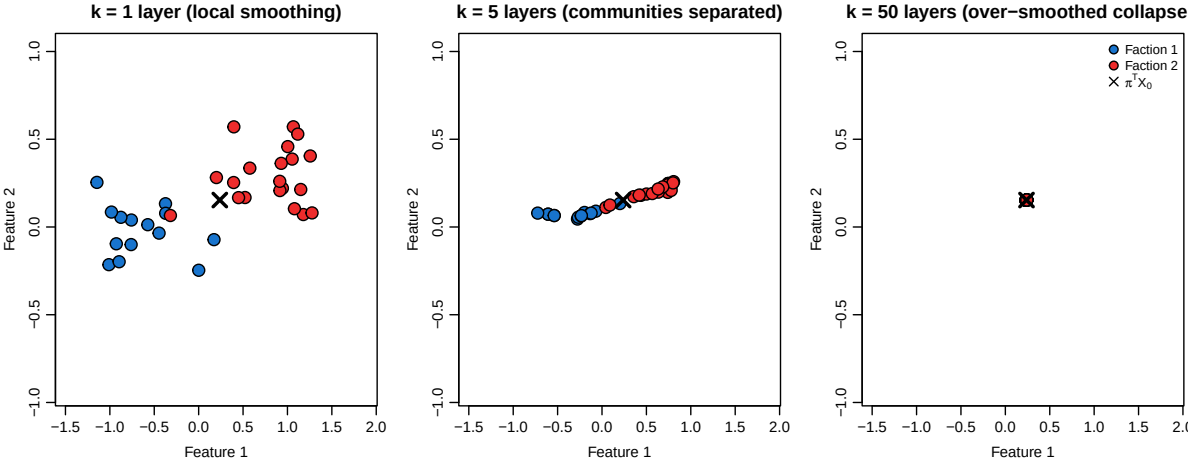
2−layer GCN embedding in R^2 (untrained



R demonstration: visualising Markov chain over-smoothing as k grows

Applying $S_{\text{rw}} = \tilde{D}^{-1}\tilde{A}$ for $k = 1, 5, 50$ steps to initial 2D features $X_0 \in \mathcal{M}_{34,2}$ converges to the rank-1 stationary average $\mathbb{1}_n(\pi^T X_0)$:

```
S_rw_k = diag(1 / rowSums(A_tilde_k)) %*% A_tilde_k;  set.seed(7)
X0_k = cbind(ifelse(faction == 1, -1, 1) + rnorm(n_k, sd = 0.4),
             rnorm(n_k, sd = 0.6))
mat_pow = function(M, k) Reduce(`%*%`, replicate(k, M, simplify = FALSE))
X_step1  = S_rw_k %*% X0_k;  X_step5 = mat_pow(S_rw_k, 5) %*% X0_k
X_step50 = mat_pow(S_rw_k, 50) %*% X0_k
pi_stat = rowSums(A_tilde_k) / sum(A_tilde_k)
rbind(Row1_k50 = X_step50[1, ], Stat_avg = as.vector(t(pi_stat) %*% X0_k))

##                [,1]      [,2]
## Row1_k50  0.2353291 0.1530728
## Stat_avg  0.2382113 0.1534625
```


k = 1 layer (local smoothing)
k = 5 layers (communities separated)
k = 50 layers (over−smoothed collapse!)
Faction 1
Faction 2
$\pi^T X_0$
Feature 1
Feature 2
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0
−1.0
−0.5
0.0
0.5
1.0

Summary: neural networks through the lens of linear algebra

Architecture	**Layer operator on $X \in \mathcal{M}_{np}$**	**Core linear algebra mechanism**
Single neuron	$\sigma\big(X\boldsymbol{w} + b\mathbb{1}_n\big)$	Affine hyperplane $\boldsymbol{w}^T\boldsymbol{x} + b = 0$ (Unit 06)
MLP layer	$\sigma\big(XW^T + \mathbb{1}_n\boldsymbol{b}^T\big)$	Right-multiplication + rank-1 bias broadcast
Deep linear net	$X\big(W^{(L)}\cdots W^{(1)}\big)^T + \mathbb{1}_n\boldsymbol{b}^T$	Collapses to 1 matrix; $\operatorname{rank}(W) \le \min_\ell p_\ell$
Linear autoencoder	$\hat{X} = \widetilde{X}W_1W_2 \;\; (k < p)$	Optimal $W_1W_2 = V_kV_k^T$ is PCA (Units 04–05)
GCN layer	$\sigma\big(\tilde{D}^{-1}\tilde{A}XW\big)$	Left graph smoothing $\times$ right feature projection
Self-attention	$\operatorname{softmax}\big(\frac{XW_QW_K^TX^T}{\sqrt{d_k}}\big)XW_V$	Data-dependent row-stochastic left multiplier

Modern deep learning architectures are built by composing the exact matrix factorisations, subspace projections and spectral graph operators developed throughout MATH 2740